React Handbook — Vol 1

core React — components, state, forms, effects, state management, hooks, styling, routing, performance (Ch 1–10)

10 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Why React, and Setting Up**
   - 1.1 What React is
   - 1.2 Declarative UI
   - 1.3 Rendering, the virtual DOM and reconciliation (brief)
   - 1.4 SPA vs MPA vs SSR (and SSG)
   - 1.5 Setting up a React project
   - 1.6 Project structure
   - 1.7 Your first component
   - 1.8 JSX
   - Chapter 1 TL;DR
2. **Chapter 2 — Components and Props**
   - 2.1 Function components
   - 2.2 Props
   - 2.3 The `children` prop
   - 2.4 Composition
   - 2.5 Pure components
   - 2.6 Keeping components small
   - 2.7 Conditional rendering
   - 2.8 Rendering lists and keys
   - 2.9 Putting it together — a product catalog
   - Chapter 2 TL;DR
3. **Chapter 3 — State and Events**
   - 3.1 Responding to events
   - 3.2 State: a component's memory — `useState`
   - 3.3 State as a snapshot
   - 3.4 Batching
   - 3.5 Updater functions — queueing updates
   - 3.6 Updating objects and arrays immutably
   - 3.7 Lifting state up
   - 3.8 Derived state — choosing the minimal state
   - 3.9 Controlled inputs (basics)
   - 3.10 State is tied to a position in the tree
   - 3.11 Putting it together — a to-do app
   - Chapter 3 TL;DR
4. **Chapter 4 — Forms and Actions**
   - 4.1 How a form submission works
   - 4.2 Controlled vs uncontrolled inputs
   - 4.3 Validation
   - 4.4 Form Actions — `<form action={fn}>`
   - 4.5 `useActionState` — the result of the last submission
   - 4.6 `useFormStatus` — status of the parent form
   - 4.7 `useOptimistic` — instant feedback, automatic rollback
   - 4.8 Server Functions and progressive enhancement (overview)
   - 4.9 Form libraries — React Hook Form and Zod (overview)
   - 4.10 Putting it together — an RSVP form
   - Chapter 4 TL;DR
5. **Chapter 5 — Effects and Refs**
   - 5.1 Where side effects belong
   - 5.2 `useEffect` basics
   - 5.3 Cleanup — undo what setup did
   - 5.4 StrictMode's double run (development only)
   - 5.5 Dependencies in depth
   - 5.6 You might not need an Effect
   - 5.7 Fetching data in Effects and race conditions
   - 5.8 `useRef` for values
   - 5.9 Refs to DOM nodes
   - 5.10 Passing refs to your own components
   - 5.11 `useLayoutEffect` — before the browser paints
   - 5.12 Putting it together — a city search
   - Chapter 5 TL;DR
6. **Chapter 6 — State Management**
   - 6.1 Kinds of state — and where each belongs
   - 6.2 `useReducer` — consolidate update logic
   - 6.3 Context — pass data deeply without props
   - 6.4 Context performance caveats
   - 6.5 Reducer + context — a store with only React
   - 6.6 When to reach for a library
   - 6.7 Redux Toolkit
   - 6.8 Zustand
   - 6.9 TanStack Query — server state
   - 6.10 Choosing — comparison and decision flow
   - 6.11 Putting it together — a bookshop
   - Chapter 6 TL;DR
7. **Chapter 7 — Custom Hooks and the Rules of Hooks**
   - 7.1 What a Hook really is
   - 7.2 The Rules of Hooks
   - 7.3 Custom Hooks — reusing stateful logic
   - 7.4 `useLocalStorage` — state that survives a reload
   - 7.5 `useDebounce` — wait until the user stops
   - 7.6 `useFetch` — loading, error, success, and no races
   - 7.7 Hooks that take callbacks — `useEffectEvent`
   - 7.8 `useId` — IDs for accessibility
   - 7.9 `useSyncExternalStore` — subscribing to outside data
   - 7.10 `useTransition` — non-urgent updates
   - 7.11 `useDeferredValue` — let part of the UI lag behind
   - 7.12 Putting it together — a user search
   - Chapter 7 TL;DR
8. **Chapter 8 — Styling React Apps**
   - 8.1 The styling landscape
   - 8.2 Plain CSS files
   - 8.3 Inline styles — the `style` prop
   - 8.4 CSS Modules
   - 8.5 Composing class names from props
   - 8.6 Tailwind CSS
   - 8.7 CSS-in-JS
   - 8.8 Theming and dark mode
   - 8.9 Component libraries
   - 8.10 Choosing an approach

- Chapter 8 TL;DR

9. **Chapter 9 — Routing and Data Fetching**
   - 9.1 Client-side routing
   - 9.2 React Router v8 and its three modes
   - 9.3 Routes, links and URL parameters (declarative mode)
   - 9.4 Nested routes and layouts
   - 9.5 Loaders — data before render (data mode)
   - 9.6 Actions, `<Form>` and fetchers
   - 9.7 Middleware, lazy routes and framework mode
   - 9.8 TanStack Query basics
   - 9.9 Query keys, pagination and dependent queries
   - 9.10 Mutations, invalidation and optimistic updates
   - 9.11 Suspense and `use()`
   - 9.12 Loading and error states that users like
   - 9.13 Putting it together — loaders + TanStack Query
   - Chapter 9 TL;DR
10. **Chapter 10 — Performance**
   - 10.1 The render cycle: trigger → render → commit
   - 10.2 When components re-render
   - 10.3 Measure first: DevTools Profiler and `<Profiler>`
   - 10.4 `memo` — skipping a child's re-render
   - 10.5 `useCallback` and `useMemo` for stable props
   - 10.6 `useMemo` for expensive calculations
   - 10.7 React Compiler
   - 10.8 Code splitting with `lazy` and `<Suspense>`
   - 10.9 List virtualization
   - 10.10 Common performance bugs
   - 10.11 A performance checklist
   - Chapter 10 TL;DR

Chapter 1 — Why React, and Setting Up

What problem React solves, how it updates the screen, and how to start a project — plus the JSX rules every later chapter assumes.

All code is in `code/`, run with **React 19.3** (current as of September 2026) on Node 24 (**Node.js** = a runtime that runs JavaScript outside the browser). `.jsx` files are **compiled** (translated to plain JavaScript) by **esbuild** (a fast compiler/bundler), rendered into **jsdom** (a browser-like DOM inside Node), and driven with `@testing-library/react` (clicks buttons, reads what a user sees). Run everything with `bash tools_run_js.sh react` from the repository root.

1.1 What React is

What it is

React is a JavaScript **library** (ready-made code that *your* program calls) for building **user interfaces** out of **components**. A *component* is a JavaScript function that returns a description of a piece of screen — a button, a card, a whole page. Small components combine into bigger ones.

Term	Meaning
DOM (Document Object Model)	The browser's live tree of objects for the page (`<div>`, `<button>` …). Changing it changes what you see.
Component	A function whose name starts with a capital letter and that returns JSX.
JSX	HTML-like syntax inside JavaScript (`<h1>Hi</h1>`) that compiles to plain function calls (§1.8).
Props	Inputs passed to a component, like function arguments (Chapter 2).
State	Data a component remembers between renders, e.g. "how many likes" (Chapter 3).
Render	React calling your component to find out what the UI should look like.
Hook	A function starting with `use` (e.g. `useState`) that lets a component use React features.

Library, not framework

React does one job: turn components + data into UI and keep it up to date. Routing (which page for which URL), data fetching and **build tooling** (turning source files into optimised browser files) come from other **packages** (installable code bundles, usually from npm) or from a **framework** built on React — a fuller toolkit that sets your app's structure and calls *your* code (Next.js, React Router, Expo — §1.5).

Package	What it does
`react`	Components, JSX runtime, Hooks — platform independent
`react-dom`	Renders React to a browser DOM (`react-dom/client`) or to HTML strings/streams on a server (`react-dom/server`)
`react-native`	Renders the same kind of components to native iOS/Android views (separate project)

A short history

Year	Event
2011–2012	Jordan Walke builds React at Facebook; it runs the News Feed, then Instagram.
May 2013	Open-sourced at JSConf US. JSX was controversial ("HTML in my JavaScript?").
2015	React Native brings the model to mobile apps.
Feb 2019	React 16.8 introduces **Hooks**; function components become standard.
Mar 2022	React 18: concurrent rendering (React can pause and resume rendering) and automatic batching (several state updates → one re-render).
Dec 5, 2024	**React 19**: Actions, `useActionState`, `useOptimistic`, `use`, `ref` as a normal prop, Server Components stable.
Feb 14, 2025	Create React App (CRA) deprecated for new apps.
Oct 2025	React 19.2; React Compiler 1.0 (automatic **memoization** — reusing a computed result while inputs are unchanged); Meta announces React will move to an independent React Foundation hosted by the Linux Foundation.
Sep 9, 2026	React 19.3 (`<ViewTransition>`, refs on `<Fragment>`) — the version used in this book.

Why it matters

- **Reusable pieces** — a `<Button>` written once, used in 200 places, fixed in one file.
- **Predictable updates** — describe the UI for the current data; React keeps the screen in sync (§1.2).
- **Huge ecosystem** (routers, forms, component kits, testing) and jobs; used by Meta (Facebook, Instagram, WhatsApp Web, Threads), Netflix, Airbnb, Discord, Shopify.
- **Many targets** — browser, server HTML (SSR, §1.4), mobile (React Native), even terminals and PDFs via community **renderers**.

When not to use it

Mostly static sites (blog, docs, landing page) → plain HTML/CSS or a static-site generator plus a sprinkle of JS; a tiny widget on a server-rendered page → vanilla JS; hard JS-size limits (slow devices/networks) → server-rendered HTML with minimal JS. React's runtime (react + react-dom) adds roughly 60–70 KB of **gzipped** (compressed for transfer) JavaScript — Figure 1.1's whole **bundle** (the combined JS file built from your code and packages) is 69 KB gzipped. Small for an app, large for a single dropdown.

Common mistakes

- Thinking React *is* the whole app — you still need routing, data fetching and a build tool, which is why react.dev recommends starting with a framework (§1.5).
- Learning from old tutorials: class components (the pre-2019 class-based style), `componentDidMount` (a class method that ran after the component appeared) and CRA are legacy. This book uses function components and Hooks only.

TL;DR

React = a UI library: components (functions returning JSX) + React DOM to put them in the browser. Routing, data and build come from the ecosystem or a framework.

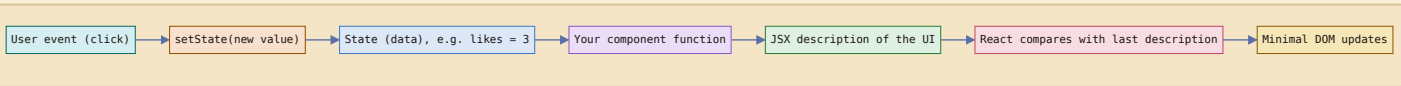
1.2 Declarative UI

What it is

- **Imperative**: step-by-step commands. "Find the label, set its text to 3 likes; find the button; if likes ≥ 3 change its text to Popular!"

- **Declarative**: describe the *result* for any data. "The label says *N like(s)*; the button says *Popular!* when N ≥ 3." React works out the steps — like telling a taxi "the station" rather than every turn.

The core idea is **UI = f(state)**: the screen is a function of the current data. When data changes, React calls your function again and updates the page to match.



The problem it solves

Imperatively, every **event handler** (function run on a click, key press…) must update *every* piece of screen that depends on the changed data; forget one line and the screen is inconsistent, and bugs grow with (state × places-that-show-it).

How it works — the same button both ways

`code/01_imperative_vs_declarative.jsx`

```jsx
// ---- Imperative: YOU write every step that changes the page ----
let likes = 0;
const btn = document.createElement('button');
const label = document.createElement('span');
label.textContent = 'No likes yet';
btn.textContent = 'Like';
btn.addEventListener('click', () => {
  likes++;                                        // 1. change the data
  label.textContent = `${likes} like(s)`;         // 2. remember to update the label
  btn.textContent = likes >= 3 ? 'Popular!' : 'Like'; // 3. ...and the button
});

// ---- Declarative: you describe WHAT the UI looks like for a given state ----
function LikeButton() {
  const [likes, setLikes] = useState(0);
  return (
    <div>
      <button onClick={() => setLikes(likes + 1)}>{likes >= 3 ? 'Popular!' : 'Like'}</button>{' '}
      <span>{likes === 0 ? 'No likes yet' : `${likes} like(s)`}</span>
    </div>
  );
}
// ... both are clicked three times, then printed
```

Output:

```
imperative: <button>Popular!</button> <span>3 like(s)</span>
declarative: <div><button>Popular!</button> <span>3 like(s)</span></div>
```

Same HTML; the difference is *where the knowledge lives*: the React handler only changes data (`setLikes`) and the JSX alone decides what is shown. (`useState`: Chapter 3.)

Trade-offs

	Imperative DOM code	Declarative React
Small widget	Shortest code, no dependency	Library + build-step overhead
Growing UI	Handlers get tangled; easy to forget an update	Each piece described once; scales well
Performance	You can hand-optimise every step	React does diffing work; fast enough for almost all UIs, with tools for hot spots
Debugging	Trace which handler changed what	Inspect state and props in React DevTools (§1.5)

In the interview

> **30-second answer:** "React is declarative: I describe what the UI should look like for the current state, and React works out the DOM operations. When state changes, React re-runs my component, compares the new output with the previous one, and applies only the differences. That removes a whole class of bugs where some part of the screen isn't updated."

- *Is React fully declarative?* — The UI description is. Event handlers ("on click, set state") and escape hatches — refs (hold a value or DOM node without re-rendering) and Effects (code run after rendering to sync with something outside React) (Chapter 5) — are imperative.
- *What does UI = f(state) mean?* — Same state and props → same JSX; the screen is derived from data, not edited by hand.

TL;DR

Imperative = list the steps; declarative = describe the result. React components describe the result for the current state and React applies the steps.

1.3 Rendering, the virtual DOM and reconciliation (brief)

What it is

- A **React element** is a small, plain object describing one piece of UI: `{ type: 'h1', props: { children: 'Hi' } }`. JSX creates these (§1.8).
- The element tree your components return is often called the **virtual DOM**: a lightweight *description* of the page in JS memory, not the real DOM. (react.dev talks about the *render tree*, but interviews say "virtual DOM".)
- **Reconciliation** is comparing the new element tree with the previous one to find the smallest set of real DOM changes.

How it works — three steps on every update

1. **Trigger** — the initial `root.render(<App />)`, or a state update (`setLikes`).
2. **Render** — React *calls your components* to get a new element tree and diffs it with the previous one. Pure calculation; nothing on screen changes.
3. **Commit** — React applies only the differences to the real DOM. The browser then paints.



A general tree diff is roughly O(n³) (**Big-O notation** describes how work grows with input size *n*: 1,000 elements could need ~a billion steps). React's **heuristics** (fast rules of thumb that are almost always right) make it about O(n) — work grows in step with the number of elements:

Rule	Consequence
Different type at the same position (`<div>` → `<section>`, or `<Profile>` → `<Settings>`)	React throws away the old subtree (DOM *and* component state) and builds a new one.
Same type at the same position	React keeps the DOM node and state, and updates only changed attributes/text.
Lists	Items are matched by their `key` prop, so React can tell "moved" from "changed" (§2.8).

Since React 16 the engine is **Fiber**: it splits rendering into small units that can pause and resume (the basis of React 18+ *concurrent rendering*).

Code — watching what React actually changes

`code/04_reconciliation.jsx` renders a clock, records every DOM **mutation** with a `MutationObserver` (a browser API that reports DOM changes), then renders again with a new `time` prop.

```jsx
function Clock({ time }) {
  return (
    <div className="clock">
      <h1>Hello, world!</h1>
      <p>It is {time}.</p>
    </div>
  );
}
await act(() => root.render(<Clock time="10:00:00" />));
// ... start observing, then:
await act(() => root.render(<Clock time="10:00:01" />));
```

Output:

```
DOM changes made by React:
  characterData on <p>: "10:00:00" -> "10:00:01"
same <h1> node kept? true
same <p> node kept?  true
HTML now: <div class="clock"><h1>Hello, world!</h1><p>It is 10:00:01.</p></div>
```

The whole `Clock` function ran again, yet exactly **one** text node changed. (`act()` from `react` makes React finish pending work before the next line; used only in tests and scripts.)

Common mistakes

- "Re-render = React rewrites the DOM." No: *re-render* means your function is called again; the DOM changes only where output differs.
- "The virtual DOM makes React faster than hand-written DOM code." Not necessarily — perfectly hand-tuned code does less work. The benefit is *good enough* performance automatically, with declarative code.
- Defining a component *inside* another: a new function (= new *type*) every render, so state resets (§1.7).

In the interview

30-second answer: "When state changes, React re-renders: it calls the components to build a new tree of elements, diffs it against the previous tree, and commits only the differences to the DOM. The diff is O(n) thanks to two heuristics — elements of different types produce different trees, and keys identify list items across renders."

- *Render vs commit?* — Render = calling components to compute the new tree (pure, may be repeated or discarded); commit = mutating the DOM and running layout effects/refs (layout effects run after DOM changes but before paint; Chapter 5).
- *After `setState`, which functions run?* — The owning component and, by default, all its children; only changed DOM nodes are updated.
- *Why can changing a wrapper from `<div>` to `<section>` lose input text?* — Different type at the same position → subtree destroyed and recreated, including state.

TL;DR

Trigger → render (call components, build element tree) → commit (apply minimal DOM diff). Same type at same position = keep and update; different type = replace; lists matched by key.

1.4 SPA vs MPA vs SSR (and SSG)

What it is

Strategies for **where and when HTML is produced**.

Term	Full name	How the page is produced
MPA	Multi-Page Application	Every navigation asks the server for a new HTML page (classic PHP/Rails/Django).
SPA	Single-Page Application	One almost-empty HTML file + a JS bundle; JavaScript builds every screen in the browser and navigates without full reloads. Also called **CSR** — client-side rendering.
SSR	Server-Side Rendering	The server runs your components per request and sends finished HTML; the browser then **hydrates** it (React attaches event handlers to the existing HTML).
SSG	Static Site Generation	Like SSR but once at **build time**; HTML files are served from a CDN (Content Delivery Network — servers worldwide that **cache** copies close to users).

How it works — see the difference in HTML

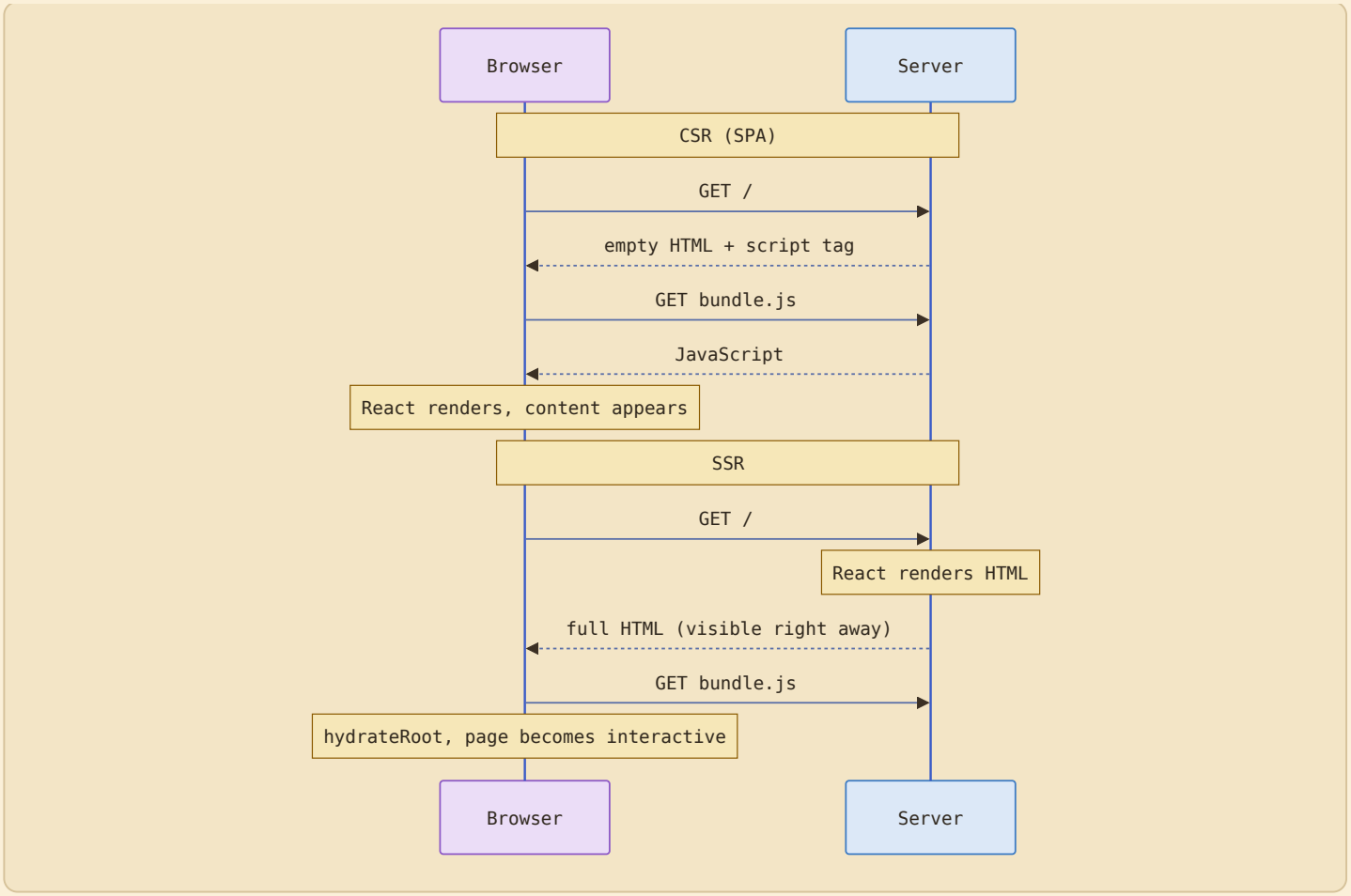
`code/05_csr_vs_ssr.jsx` renders one component both ways.

```
function App() {
  const [n, setN] = useState(0);
  return <main><h1>Shop</h1><button onClick={() => setN(n + 1)}>Cart: {n}</button></main>;
}
const html = renderToString(<App />);                  // SSR: component -> HTML string
document.getElementById('root').innerHTML = html;      // what the browser receives
button.click();                                        // before hydration
await act(() => { hydrateRoot(document.getElementById('root'), <App />); });
await act(() => button.click());                       // after hydration
```

Output:

```
CSR first HTML : <div id="root"></div>  (+ <script src="bundle.js">)
SSR first HTML : <div id="root"><main><h1>Shop</h1><button>Cart: <!-- -->0</button></main></div>
click before hydration -> Cart: 0 (nothing happens yet)
click after hydration  -> Cart: 1 | same DOM node reused: true
```

CSR's first HTML is empty — users and search-engine **crawlers** see nothing until JS runs. SSR's has the content (`<!-- -->` separates adjacent text pieces so hydration can match them), but it is *not interactive* until hydration finishes, and hydration **reuses** the server's DOM nodes.



Trade-offs

	MPA	SPA / CSR	SSR	SSG
First content shown	Fast	Slow (wait for JS)	Fast	Fastest (from CDN)
Navigation between pages	Full reload	Instant, app-like	App-like after hydration	App-like after hydration
SEO (search-engine optimisation) / link previews	Good	Weaker (crawlers must run JS)	Good	Good
Server needed	Yes	No — static hosting	Yes (Node or edge runtime near the user)	No — static hosting
Fresh per-request data	Yes	Via API calls	Yes	Only at build (or revalidation — rebuilding a page after a set time)
Typical examples	Wikipedia, classic admin tools	Figma-style tools, dashboards behind login	E-commerce, news	Docs, blogs, marketing pages

Modern frameworks mix these **per route**: static marketing page, server-rendered product page, client-rendered dashboard. React 19 **Server Components** (run only on the server, send no JS) go further — Chapter 13. (Servers, CDNs and caching in general: System Design Fundamentals.)

When not to use SSR: an internal tool behind a login where SEO does not matter and the tab stays open all day (a plain SPA is simpler and cheaper to host), or when you cannot run a server (choose CSR or SSG).

Common mistakes

- Using browser-only APIs (`window`, `localStorage`) during render in an SSR app: the server has no `window`, so it crashes or the HTML differs from the client (a **hydration mismatch**).

- Assuming SSR makes the page interactive sooner. It makes content *visible* sooner; interactivity still waits for JS + hydration.

In the interview

> **30-second answer:** "An SPA ships an empty shell and renders everything in the browser — simple hosting, app-like navigation, but slow first paint and weaker SEO. SSR renders HTML on the server per request, so content appears immediately, then hydration attaches React to it. SSG does the same at build time for pages that rarely change. Frameworks like Next.js let you choose per route."

- *What is hydration?* — React running in the browser over server-generated HTML, reusing the existing DOM and attaching event handlers and state.
- *What causes a hydration mismatch?* — Different output on server and client, e.g. `Date.now()`, `Math.random()`, or `window` checks during render.

TL;DR

MPA = server page per navigation; SPA/CSR = JS builds everything in the browser; SSR = server HTML + hydration; SSG = HTML at build time. Pick per route.

1.5 Setting up a React project

What you need first

- **Node.js** current LTS — Long-Term Support release (Vite needs 20.19+ or 22.12+; check `node -v`) — runs the dev server and build tools; your React code still runs in the browser.
- **npm** (ships with Node) or pnpm / yarn / bun — installs packages listed in `package.json` (check `npm -v`).
- An editor (usually VS Code) with ESLint (a **linter** that flags likely bugs) and Prettier (formatter).
- **React Developer Tools** extension (Chrome, Firefox, Edge) — inspect the component tree, props and state; profile renders.

Option A — start with a framework (react.dev's recommendation for new apps)

A framework bundles React with routing, data loading, server rendering and a build pipeline.

Framework	Command	Good for
Next.js (App Router)	`npx create-next-app@latest`	Full-stack web apps, SSR/SSG, Server Components; maintained by Vercel
React Router v8 (framework mode)	`npx create-react-router@latest`	Apps that grow from an SPA to full stack; built on Vite, maintained by Shopify
Expo	`npx create-expo-app@latest`	iOS, Android and web from one codebase (React Native)

react.dev also lists TanStack Start and RedwoodSDK. `npx` runs a package's command once without a global install; `@latest` picks the newest version.

Option B — build from scratch with Vite

For a plain client-side app (learning, dashboards, embedding in an existing site) where you add routing and data fetching yourself. React docs list **Vite**, **Parcel** and **Rsbuild**; Vite is the most common. (`react-ts` uses **TypeScript** — JavaScript plus type annotations checked before the code runs.)

```
# (not run here — needs network) create, install, start
npm create vite@latest my-react-app -- --template react   # or react-ts for TypeScript
cd my-react-app
npm install          # download react, react-dom, vite, plugins into node_modules/
npm run dev          # start the dev server -> http://localhost:5173
```

Command	What happens
`npm create vite@latest …`	Runs the `create-vite` scaffolder, which copies a template. The extra `--` passes `--template react` through npm. Templates include `react`, `react-ts`, and `react-compiler` / `react-compiler-ts` (pre-configured React Compiler).
`npm install`	Installs dependencies from `package.json` into `node_modules/`, writes `package-lock.json` (the **lockfile**: exact versions).
`npm run dev`	Starts Vite's **dev server**: serves native ES modules (files using `import`/`export`, loaded directly by the browser), compiles JSX on the fly, and uses **HMR** (Hot Module Replacement — edits appear in milliseconds without losing component state).
`npm run build`	Production bundle in `dist/`: minified, tree-shaken (unused code removed), hashed file names (change with content, so browsers cache safely).
`npm run preview`	Serves `dist/` locally to test the production build.

Equivalents: `pnpm create vite`, `yarn create vite`, `bun create vite`.

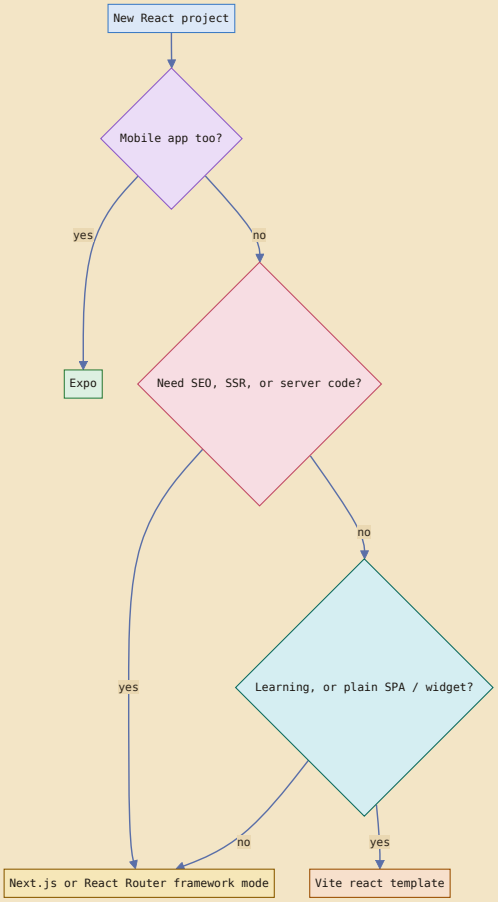
What about Create React App?

CRA was the standard starter from 2016. It was deprecated for new apps on **14 February 2025** (maintenance mode only): no active maintainers, slow (webpack-based), and no path to routing, data fetching or SSR. Tutorials using `npx create-react-app` are outdated.

Adding React to an existing page

Add a build step that bundles your component, put `<div id="cart-widget"></div>` in the HTML, and call `createRoot(document.getElementById('cart-widget')).render(<Cart />)`. A page can have several roots.

Choosing



How this book runs React without a browser

Examples are compiled with esbuild (the same kind of compiler Vite uses; Vite 8 uses Oxc, earlier versions esbuild), rendered into jsdom and driven with Testing Library; screenshots come from the same code run in **headless** Chrome (no visible window, script-controlled). Real projects test React the same way (Vitest/Jest + jsdom + Testing Library — Chapter 12).

Common mistakes

- Running `npm run dev` outside the project folder (no `package.json` → "Missing script: dev").
- Double-clicking `index.html`: it loads `/src/main.jsx`, which a browser cannot run directly; use the dev server or build first.
- Committing `node_modules/` to git; commit `package.json` and the lockfile instead.
- Mixing package managers (`npm` and `yarn` lockfiles in one repo).

In the interview

> **30-second answer:** "For a new production app, react.dev recommends a framework — Next.js, React Router, or Expo for native — because you get routing, data loading and SSR out of the box. For a pure client-side app or learning, I'd use Vite with the react or react-ts template. CRA is deprecated since February 2025."

- *Why Vite over CRA/webpack?* — The dev server serves native ES modules and compiles only requested files, so startup and HMR stay fast as the app grows; production is still fully bundled. CRA bundled everything with webpack before showing anything.
- *What is HMR?* — The dev server swaps the changed module in the running page, keeping state, instead of reloading.

TL;DR

Framework (Next.js / React Router / Expo) for real apps; `npm create vite@latest app -- --template react` for SPAs and learning; CRA is deprecated. `npm run dev` / `build` / `preview`.

1.6 Project structure

What it is

A new Vite React project (current `template-react`; the linter config name varies by version):

```
my-react-app/
├── index.html          ← the single HTML page; contains <div id="root"> and loads src/main.jsx
├── package.json        ← name, scripts (dev/build/preview/lint), dependencies
├── package-lock.json   ← exact installed versions (after npm install)
├── vite.config.js      ← Vite settings; registers @vitejs/plugin-react (JSX, Fast Refresh)
├── public/             ← files copied as-is (favicon, robots.txt), served from /
├── src/
│   ├── main.jsx        ← entry point: finds #root and renders <App />
│   ├── App.jsx         ← the top-level component you edit first
│   ├── App.css, index.css
│   └── assets/         ← images imported from code (get hashed file names on build)
└── node_modules/       ← installed packages (never edit, never commit)
```

How it works — from `index.html` to pixels

1. The browser loads `index.html`: an empty `<div id="root"></div>` and `<script type="module" src="/src/main.jsx">`.
2. `main.jsx` calls **`createRoot(domNode)`**, creating a React **root** — React's handle on that element.
3. `root.render(<App />)` renders your tree into it; React now owns everything inside `#root`.
4. **`<StrictMode>`** (development only) calls each component twice per render and runs every Effect's setup → cleanup → setup once extra on **mount** (first appearance), exposing **impure** code (changes outside things, or differs for the same input) and missing **cleanups** early (§2.5, Chapter 5). No effect in production.

`code/vite-app/src/main.jsx` (same shape as the template):

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

`code/vite-app/src/App.jsx`:

```
import { useState } from 'react';

function Welcome({ name }) {
  return <h1>Welcome to React, {name}!</h1>;
}

export default function App() {
  const [count, setCount] = useState(0);
  return (
    <div className="card">
      <Welcome name="Sam" />
      <p>This page was built from components.</p>
      <button onClick={() => setCount((c) => c + 1)}>count is {count}</button>
    </div>
  );
}
```

`code/06_first_app.jsx` mounts the **same** `App` into jsdom exactly as `main.jsx` does (`createRoot` + `<StrictMode>`), then clicks twice. Output:

```
after first render : Welcome to React, Sam!This page was built from components.count is 0
after two clicks   : count is 2
```

The same `main.jsx` bundled and opened in Chrome (`code/first-app.html`):

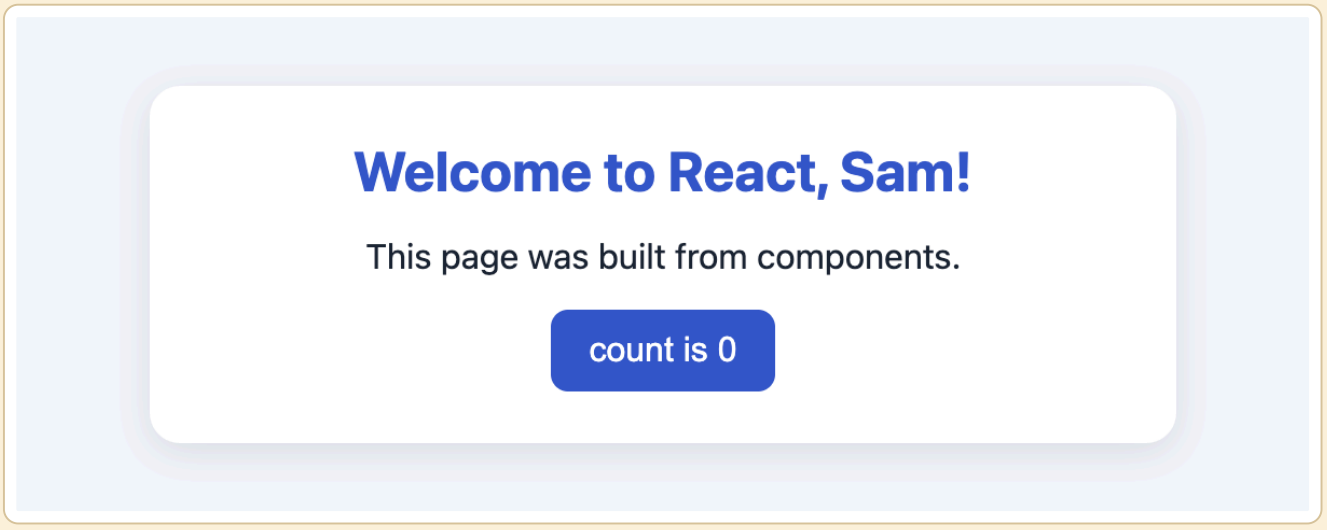


Figure 1.1 — `App` from `vite-app/src` running in a real browser.

`package.json` in brief

```
{
  "type": "module",
  "scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview" },
  "dependencies": { "react": "^19.x", "react-dom": "^19.x" },
  "devDependencies": { "vite": "…", "@vitejs/plugin-react": "…" }
}
```

`dependencies` ship in your app; `devDependencies` are only needed to build it. `^19.x` = "any 19.* version" (semantic versioning MAJOR.MINOR.PATCH, where only a MAJOR bump may break you — see the JavaScript book,

`../../../javascript/chapters/10-modules-and-tooling/README.md`).

How to grow the structure

Start flat; add folders as patterns appear — e.g. `src/components/`, `src/features/<feature>/` (components + hooks + tests), `src/hooks/`, `src/lib/` (non-React helpers); one exported component per file (Chapter 14).

Common mistakes

- Calling `createRoot` twice on the same element (React warns); create once, call `render` as needed.
- Editing DOM inside `#root` by hand (`innerHTML = …`) — React overwrites it or gets confused.
- Removing `<StrictMode>` to "fix" doubled console logs; the doubles reveal real bugs.

TL;DR

`index.html` (#root) → `main.jsx` (`createRoot(...).render(<StrictMode><App/></StrictMode>)`) → `App.jsx`. `public/` is copied as-is; `node_modules/` is never committed; StrictMode checks are dev-only.

1.7 Your first component

What it is

A **component** is a JavaScript function that (1) has a name starting with a **capital letter** (`Profile`, not `profile`) and (2) returns **JSX** (or `null` to render nothing).

```
function Profile() {
  return <img src="https://i.imgur.com/MK3eW3As.jpg" alt="Katherine Johnson" />;
}

export default function Gallery() {
  return (
    <section>
      <h1>Amazing scientists</h1>
      <Profile />
      <Profile />
      <Profile />
    </section>
  );
}
```

`Gallery` is a **parent** rendering three **child** `Profile`s; together they form the **component tree** that React DevTools shows. (Runnable version: `01_function_components.jsx`, §2.1.)

Why the capital letter? JSX uses it to decide the `type`: lowercase `<section>` → the string `'section'` (an HTML tag); `<Profile />` → the variable `Profile`. A lowercase component is treated as an unknown HTML tag.

Exporting and importing

Style	Export	Import
Default (one per file)	`export default function Button() {}`	`import Button from './Button.jsx'` (any name)
Named (many per file)	`export function Button() {}`	`import { Button } from './Button.jsx'` (exact name)

Teams usually pick one style; named exports make renames and editor auto-imports more reliable.

Rules

- **Define components at module top level, never inside another component** — an inner definition is a new function (= new *type*) every render, so React destroys its state and DOM (§1.3).
- **Be pure while rendering** (§2.5); keep components small (extract any JSX with its own meaning).

Common mistakes

Mistake	Symptom	Fix
`function profile()` (lowercase)	Renders `<profile>` tag, nothing shown	Capitalise
Missing `return` or a line break after `return`	Renders nothing (`return` alone returns `undefined`)	Wrap multi-line JSX in `( … )` starting on the `return` line
Nested component definitions	Inputs lose focus on every keystroke	Move the inner function to module level
Calling a component as a function `{Profile()}`	Hooks inside break rules, no own identity in the tree	Use `<Profile />`

TL;DR

Component = capitalised function returning JSX, defined at top level, exported default or named, used as `<Name />`.

1.8 JSX

What it is

JSX (JavaScript XML) lets you write markup inside JavaScript. It is *not* HTML and not a string: the compiler (esbuild, Babel, SWC, Oxc) turns each tag into a function call returning a React element object. React works without JSX, but almost nobody does.

`code/02_jsx_is_javascript.jsx`:

```
import { createElement, isValidElement } from 'react';

const fromJsx = <h1 className="title">Hello, {'Ada'}!</h1>;
const byHand = createElement('h1', { className: 'title' }, 'Hello, ', 'Ada', '!');

const show = (el) => JSON.stringify({ type: el.type, props: el.props });
console.log('JSX         ->', show(fromJsx));
console.log('createElement->', show(byHand));
console.log('is a React element?', isValidElement(fromJsx));
console.log('typeof element:', typeof fromJsx, '| frozen (dev):', Object.isFrozen(fromJsx));
```

Output:

```
JSX         -> {"type":"h1","props":{"className":"title","children":["Hello, ","Ada","!"]}}
createElement-> {"type":"h1","props":{"className":"title","children":["Hello, ","Ada","!"]}}
is a React element? true
typeof element: object | frozen (dev): true
```

JSX is a nicer `createElement(type, props, ...children)`. Since React 17 the compiler uses the **automatic JSX runtime** (inserts `import { jsx } from 'react/jsx-runtime'`), so modern files need no `import React from 'react'`. Elements are **immutable** (frozen in development): to change the UI you render a new element.

The rules

`code/03_jsx_rules.jsx`:

```jsx
const user = { name: 'Grace Hopper', avatar: 'https://i.imgur.com/yXOvdOSs.jpg', online: true };
const size = 64;

function ProfileHeader() {
  // Rule 1: return ONE root element. A Fragment (<>...</>) groups without adding a DOM node.
  return (
    <>
      {/* Rule 2: attributes are camelCase; class -> className, for -> htmlFor */}
      <label htmlFor="bio" className="label">Bio</label>
      {/* Rule 3: every tag must be closed, even <img /> and <input /> */}
      <img src={user.avatar} alt={user.name} width={size} height={size} />
      <input id="bio" defaultValue="Wrote the first compiler" />
      {/* Rule 4: curly braces hold any JavaScript EXPRESSION */}
      <h2>{user.name.toUpperCase()} ({user.online ? 'online' : 'offline'})</h2>
      {/* Rule 5: style takes an object; property names are camelCase */}
      <p style={{ backgroundColor: 'black', color: 'pink', fontSize: 14 }}>{2 + 3} papers</p>
    </>
  );
}
```

Output (the HTML React produced):

```html
<label for="bio" class="label">Bio</label>
<img alt="Grace Hopper" width="64" height="64" src="https://i.imgur.com/yXOvdOSs.jpg">
<input id="bio" value="Wrote the first compiler">
<h2>GRACE HOPPER (online)</h2>
<p style="background-color: black; color: pink; font-size: 14px;">5 papers</p>
```

#	Rule	Why	Example
1	**One root element**	A function returns one value; JSX becomes one object	Wrap in `<div>` or a **Fragment** `<>…</>` (no extra DOM node)
2	**Close every tag**	JSX is stricter than HTML	` `, `<img />`, `<li>…</li>`
3	**camelCase attributes**	Attributes become JS object keys; `class`/`for` are reserved words	`className`, `htmlFor`, `onClick`, `tabIndex`, `strokeWidth`. `aria-*` and `data-*` keep dashes
4	**`{}` for JavaScript expressions**	Anything producing a value: variables, calls, ternaries, `&&`, `.map()`	`{user.name}`, `{count * 2}`, `{formatDate(d)}`
5	**`style` is an object**	`{{ … }}` = braces for JS + an object literal	`style={{ fontSize: 14 }}` — numbers get `px` for most properties

Details that trip beginners:

- **Quotes vs braces:** `alt="text"` is a string literal; `alt={name}` a JS value. `src="{avatar}"` is the literal text `{avatar}`.
- **Expressions, not statements:** no `if`/`for`/`const` inside `{}`; use a ternary, `&&`, `.map()`, or a variable computed above the `return` (§2.7).
- **Comments:** `{/* like this */}`.
- **What renders nothing:** `null`, `undefined`, `true`, `false`, `''`. But `0` and `NaN` *do* render (the `&&` trap, §2.7).
- **Objects cannot be rendered:** `{user}` throws "Objects are not valid as a React child" — render `{user.name}`. Arrays of elements are fine.
- **Keyed fragments:** `<>` takes no props; for a `key` in a list use `<Fragment key={id}>` (imported from `react`).
- **Safety:** text in `{}` is **escaped** (`<` becomes harmless text), so `{"<script>…"}` shows as text — protection against XSS (cross-site scripting). Raw HTML needs `dangerouslySetInnerHTML={{ __html: … }}`; use only with **sanitised** content.
- `defaultValue` sets an input's *initial* value without React controlling it; `value` makes it a **controlled input** (value always from state) (§3.9).

Trade-offs

vs template languages (Vue/Angular/Svelte): JSX is full JavaScript inside markup (no new loop/if syntax) and type-checks with TypeScript like any code; directives (`v-for`, `@if`) are more restricted but easier for designers to read and need dedicated tooling. Both need a compile step in practice.

In the interview

> **30-second answer:** "JSX is syntax sugar: the compiler turns `<h1 className='x'>Hi</h1>` into a `jsx('h1', {className:'x', children:'Hi'})` call that returns a plain, immutable element object. The rules come from that: one root because a function returns one value, `className` because `class` is reserved, braces for expressions only, and style as an object."

- *What does `<Card title="x" />` compile to?* — `jsx(Card, { title: 'x' })`, returning `{ type: Card, props: { title: 'x' } }`.
- *Why no `import React` any more?* — The automatic JSX runtime (React 17+) imports `jsx` from `react/jsx-runtime` for you.
- *Is JSX safe from XSS?* — Values in braces are escaped; the risks are `dangerouslySetInnerHTML`, user-supplied URLs in `href` (React 19 replaces a `javascript:` URL with one that throws "React has blocked a javascript: URL as a security precaution"; still validate URLs) and third-party HTML.
- *`<div class="box">`?* — Must be `className`; React still sets it but logs "Invalid DOM property `class`. Did you mean `className`?".
- *Fragment vs div?* — A fragment groups children without an extra DOM node, keeping flex/grid/table layouts and HTML validity intact.

TL;DR

JSX = `createElement` calls → immutable element objects. One root (use `<>`), close all tags, camelCase attributes (`className`, `htmlFor`), `{expressions}`, `style={{…}}`, text is auto-escaped.

Chapter 1 TL;DR

Idea	Remember
React	Library for UIs built from components; `react-dom` renders to the browser
Declarative	Describe UI for the current state; React applies the DOM changes
Update cycle	Trigger → render (call components) → commit (minimal DOM diff)
Diff rules	Same type = keep + update; different type = replace; lists by key
Rendering strategies	CSR/SPA, SSR (+ hydration), SSG, MPA — frameworks mix them per route
Setup	Framework (Next.js / React Router / Expo) or Vite (`--template react`); CRA deprecated Feb 2025
Entry point	`createRoot(document.getElementById('root')).render(<StrictMode><App/></StrictMode>)`
Components	Capitalised functions returning JSX, defined at module top level
JSX	One root, closed tags, `className`, `{expression}`, `style={{}}`, fragments

Next: Chapter 2 — Components and Props.

Chapter 2 — Components and Props

How to write components, feed them data with **props**, nest them with `children`, keep them pure and small, and decide *what* to render with conditions and lists — ending with a small product catalog that uses every idea.

Code: `code/` (React 19.3, run in jsdom with Testing Library — see §1.5).

2.1 Function components

What it is

A **function component** is a plain function that takes one argument (the **props** object) and returns JSX (§1.8). Its name **must start with a capital letter**. React calls it whenever it needs to know what that part of the screen should look like.

> Older code uses **class components** (`class X extends React.Component` with `render()` and **lifecycle methods** that React calls at fixed moments, e.g. after mount or before removal). They still work but are legacy: Hooks, Server Components and the React Compiler all target function components. You only *read* classes in old code — and for **error boundaries** (components that catch a crash in their children and show a fallback UI; Chapter 14).

How it works

`code/01_function_components.jsx`:

```jsx
function Profile() {
  return <img src="https://i.imgur.com/MK3eW3As.jpg" alt="Katherine Johnson" />;
}

function Gallery() {
  return (
    <section>
      <h1>Amazing scientists</h1>
      <Profile />
      <Profile />
    </section>
  );
}

// Lowercase name: JSX treats <profile /> as an unknown HTML tag, NOT as the function below.
function profile() {
  return <img alt="never shown" />;
}
// ... render(<Gallery />) and render(<div><profile /></div>)
```

Output:

```
Gallery      : <section><h1>Amazing scientists</h1><img alt="Katherine Johnson" src="https://i.imgur.com/MK3eW3As.jpg">
<img alt="Katherine Johnson" src="https://i.imgur.com/MK3eW3As.jpg"></section>
<profile />  : <div><profile></profile></div>
React warned : The tag <profile> is unrecognized in this browser. If you meant to render a React component, start its
name with an uppercase letter.
```

1. `render(<Gallery />)` creates `{ type: Gallery }`; `type` is a function, so React **calls** `Gallery(props)`.
2. `Gallery` returns elements including two `{ type: Profile }`; React calls `Profile` for each.
3. When only strings (`'section'`, `'img'`) remain, React creates DOM nodes.
4. `<profile />` became the *string* `'profile'` → a meaningless `<profile>` tag plus a warning.

The components themselves never appear in the HTML. The component tree lives in React (visible in DevTools); the DOM contains only **host elements** (built-in tags like `<section>`, `<img>`).

Rules (§1.7): capitalised; returns JSX, a string/number, an array, or `null`; declared at module top level; used as `<Profile />`, not `Profile()` (a direct call gets no place in the tree, and its Hooks would belong to the caller).

TL;DR

A component is a capitalised function `(props) => JSX`. React calls it; only the host elements it returns reach the DOM.

2.2 Props

What it is

Props (*properties*) are the inputs a parent passes to a child, written like HTML attributes: `<Avatar person={ada} size={50} />`. React collects them into **one object** passed as the first argument. Any value can be a prop: strings, numbers, booleans, objects, arrays, functions, even JSX.

Props make one component reusable with different data and are the channel of **one-way data flow**: data moves *down* from parent to child (and *up* by passing a function as a prop — Chapter 3).

How it works

1. **Pass**: strings in quotes, anything else in braces. A bare attribute (`admin`) means `true`.
2. **Read**: usually **destructure** (unpack named fields into variables) in the parameter list: `function Avatar({ person, size })`.
3. **Default**: `{ size = 100 }` — applies only when the prop is missing or `undefined` (not `null` or `0`).
4. **Forward** unused props with rest/spread (`...rest` collects remaining fields; `{...rest}` in JSX spreads them back as props): `function Button({ variant, ...rest }) { return <button {...rest} /> }`.

`code/02_props.jsx`:

```jsx
function getImageUrl(person, size = 's') {
  return `https://i.imgur.com/${person.imageId}${size}.jpg`;
}

// Destructure props in the parameter list; `size = 100` is a default used when size is undefined.
function Avatar({ person, size = 100 }) {
  return <img className="avatar" src={getImageUrl(person)} alt={person.name} width={size} height={size} />;
}

function Profile() {
  return (
    <div>
      <Avatar person={{ name: 'Lin Lanying', imageId: '1bX5QH6' }} size={50} />
      <Avatar person={{ name: 'Aklilu Lemma', imageId: 'OKS67lh' }} />
    </div>
  );
}

// Forwarding: pass every prop through with spread syntax.
function Button({ variant = 'primary', ...rest }) {
  return <button className={`btn btn-${variant}`} {...rest} />;
}
render(<Button type="submit" disabled aria-label="Save">Save</Button>);

// Props are read-only: in development React freezes the props object.
function Mutator(props) {
  try { props.title = 'changed'; } catch (e) { console.log('mutating props ->', e.constructor.name + ':', e.message); }
  return <h2>{props.title}</h2>;
}

// What a component actually receives: ONE object.
function Spy(props) { console.log('props object:', JSON.stringify(props)); return null; }
render(<Spy name="Ada" age={36} admin tags={['math', 'code']} />);
```

Output:

```
<div>
<img class="avatar" alt="Lin Lanying" width="50" height="50" src="https://i.imgur.com/1bX5QH6s.jpg">
<img class="avatar" alt="Aklilu Lemma" width="100" height="100" src="https://i.imgur.com/OKS67lhs.jpg">
</div>

forwarded   : <button class="btn btn-primary" type="submit" disabled="" aria-label="Save">Save</button>
mutating props -> TypeError: Cannot assign to read only property 'title' of object '#<Object>'
props object: {"name":"Ada","age":36,"admin":true,"tags":["math","code"]}
```

- `person={{ … }}`: outer braces = "JavaScript here", inner = object literal (like `style={{…}}`).
- The second `Avatar` had no `size`, so the default `100` applied; `admin` became `true`.
- `children` ("Save") was forwarded by `{...rest}` too — it is just another prop.
- In development React **freezes** props (`Object.freeze`), so assigning threw. Production does not freeze, but mutating props is still a bug — the parent owns that data.

Props are a snapshot, and change only from above

Each render gets a fresh props object; when the parent re-renders with new values, the child is called again. You never "update" your own props — data that changes with interaction is **state** (Chapter 3).

	Props	State
Owned by	The parent	The component itself
Changed by	The parent re-rendering with new values	The component calling its setter (`setX`)
Purpose	Configure a component from outside	Remember something that changes with interaction
Analogy	Arguments to a function	A function's private memory between calls

Special props

Prop	Behaviour
`key`	Identifies list items (§2.8). **Not** passed to your component: `props.key` is `undefined`.
`children`	Whatever you nest between the tags (§2.3).
`ref`	A handle to a DOM node or value. In React 19 a normal prop for function components; `forwardRef` is no longer needed and will be deprecated (§5.10).

React 19 removed `propTypes` (runtime type warnings) and `defaultProps` for function components; use destructuring defaults, and TypeScript for types (Chapter 11).

Trade-offs — spreading props

`{...props}` forwarding	Listing props explicitly
Short; thin wrappers pass through any native attribute (`aria-*` accessibility hints for screen readers, `onClick`)	Clear which props the component supports
Can pass unknown props to the DOM (React warns) or override your own (later wins)	More typing when wrapping native elements

Rule of thumb: spread in thin wrappers around a native element (`Button`, `Input`); list explicitly in feature components.

Common mistakes

- `size="50"` passes a **string**; use `size={50}`.
- `function Avatar(person, size)` — forgot the destructuring braces; `size` is the second parameter (`undefined`), not a prop.
- Copying a prop into state "to change it" — the copy ignores later parent updates (§3.8).
- `defaultProps` on a function component in React 19 — ignored.

In the interview

> **30-second answer:** "Props are the read-only inputs a parent passes to a child, collected into one object. They make components reusable and implement one-way data flow: data goes down via props, and events go up via **callback props** (functions the parent passes down for the child to call). A component never mutates its props; if it needs changing data it uses state."

- *Why are props read-only?* — The parent owns the data; a mutating child would leave parent and siblings **stale** (out of date) or inconsistent, and React would not know to re-render.
- *Defaults?* — Destructuring default `({ size = 100 })`; used when `size` is missing or `undefined`.
- *Is `key` a prop?* — Written like one, but React consumes it; the component cannot read it.
- *`<Toggle on />` passes?* — `{ on: true }`.

TL;DR

Props = one read-only object from the parent. Destructure with defaults, spread to forward, never mutate. Changing data belongs in state.

2.3 The `children` prop

What it is

Whatever you put *between* a component's tags arrives as `props.children`, so **wrapper** components — cards, panels, **modals** (dialogs over the page), layouts — need not know what they contain.

How it works

`code/03_children_composition.jsx` (first part):

```jsx
function Card({ title, children }) {
  return (
    <div className="card">
      {title && <h3>{title}</h3>}
      <div className="card-body">{children}</div>
    </div>
  );
}

// children can be anything renderable: text, elements, arrays, or nothing.
function Show({ children }) { return <span>[{children}]</span>; }
render(<Show>hi</Show>); render(<Show>{['a', 'b']}</Show>); render(<Show />);
```

Output (last three lines of the program):

```
text   : <span>[hi]</span>
array  : <span>[ab]</span>
none   : <span>[]</span>
```

`children` may be a string, one element, an array, or `undefined`; render `{children}` and let React handle the shape. (The `Children` utility for counting/mapping children is filed under react.dev's *legacy APIs* because it is fragile; prefer passing data as props.)

TL;DR

Nested JSX arrives as `props.children`; render it with `{children}` to build generic wrappers.

2.4 Composition

What it is

Composition = building complex UIs by *combining* components (passing components into components) instead of **inheritance** (a class extending a parent and overriding parts). React has no component inheritance;

composition covers every use case.

How it works — slots

When a layout has several "holes", pass JSX through **named props** (*slots*) as well as `children`:

```
function PageLayout({ header, sidebar, children }) {
  return (
    <div className="page">
      <header>{header}</header>
      <aside>{sidebar}</aside>
      <main>{children}</main>
    </div>
  );
}

render(
  <PageLayout header={<h1>Dashboard</h1>} sidebar={<nav>Menu</nav>}>
    <Card title="Revenue">
      <p>$12,400 this week</p>
    </Card>
    <Card>
      <p>No title here</p>
    </Card>
  </PageLayout>,
);
```

Output (the program prints one tag per line; grouped here):

```
<div class="page">
<header><h1>Dashboard</h1></header>
<aside><nav>Menu</nav></aside>
<main>
<div class="card"><h3>Revenue</h3><div class="card-body"><p>$12,400 this week</p></div></div>
<div class="card"><div class="card-body"><p>No title here</p></div></div>
</main>
</div>
```

The second card has no `<h3>`: `title` was `undefined`, so `{title && …}` rendered nothing (§2.7).

Why it matters — composition beats prop drilling

Prop drilling = passing a prop through components that don't use it to reach a deep child (`App → Page → Sidebar → UserMenu → Avatar`, only `Avatar` needs `user`). Fixes:

1. **Composition (try first):** `App` creates `<Avatar user={user} />` itself and passes it down as `children` or a slot; the middle components just place `{children}`.
2. **Context** (§6.3) — a value any descendant can read without props — when many distant components need the same data (theme, current user, **locale** = language and regional formats).

Composition vs inheritance

Need	OOP inheritance idea	React composition
A special kind of dialog	`class WelcomeDialog extends Dialog`	`<Dialog title="Welcome">…</Dialog>` — configure via props
Shared behaviour	Base class methods	A custom Hook (Chapter 7) or a wrapper component
Varying inner content	Override a method	`children` or a slot prop

Real world: UI kits (MUI, Radix/shadcn) expose `Card`, `Dialog`, `Tabs` with `children` and slots (`title`, `actions`); routers render the matched page into an **outlet** slot (Chapter 9).

In the interview

30-second answer: "Composition means building UIs by nesting components and passing JSX as `children` or slot props. React recommends it over inheritance. It also solves most prop drilling: the top component

creates the deep child itself and passes it through as children, so intermediate components never touch the data."

- *When would you use context instead?* — When many components at different depths need the same value (theme, auth, locale) and threading it through children would be awkward.
- *Can a prop be a component?* — Yes: an element (`icon={<Star />}`) or a component type (`as={Link}`), rendered as `<Icon />` after capitalising the variable.

TL;DR

Combine small components; pass JSX via `children` and slot props. Composition replaces inheritance and is the first cure for prop drilling.

2.5 Pure components

What it is

A **pure function** (1) returns the same result for the same inputs and (2) changes nothing outside itself (no **side effects**). React assumes every component is pure **during rendering**: same props, state and context → same JSX, and no modification of pre-existing variables, objects, the DOM, or anything else.

React may call a component **more than once, in any order, or throw a render away** (concurrent rendering, Suspense — fallback while part of the UI waits, Chapter 9 — StrictMode checks, server rendering). Side effects in render then produce wrong results. Purity also enables **caching**: with unchanged inputs, React (and the React Compiler) can skip the call.

How it works — StrictMode exposes impurity

`<StrictMode>` (development only) renders each component **twice** and keeps the second result. Pure components give identical output; impure ones show the bug.

`code/04_purity.jsx`:

```jsx
let guest = 0;
function ImpureCup() {
  guest = guest + 1;              // BAD: changes a variable outside the component during render
  return <h2>Tea cup for guest #{guest}</h2>;
}

function PureCup({ guest }) {     // GOOD: output depends only on props
  return <h2>Tea cup for guest #{guest}</h2>;
}

// Local mutation is fine: the array is created during this render.
function TeaGathering() {
  const cups = [];
  for (let i = 1; i <= 3; i++) cups.push(<PureCup key={i} guest={i} />);
  return <>{cups}</>;
}
```

Output:

```
impure, no StrictMode: Tea cup for guest #1Tea cup for guest #2Tea cup for guest #3
impure, StrictMode   : Tea cup for guest #2Tea cup for guest #4Tea cup for guest #6
pure,   StrictMode   : Tea cup for guest #1Tea cup for guest #2Tea cup for guest #3
local mutation OK    : Tea cup for guest #1Tea cup for guest #2Tea cup for guest #3
```

Without StrictMode the impure version *looks* right, so the bug would hide until some re-render; the double call (each cup rendered twice, shared counter bumped twice) makes it obvious: 2, 4, 6.

Where side effects belong

Allowed during render	Not allowed during render	Put it here instead
Reading props, state, context	Changing variables/objects that existed before the render	Local variables, or state
Creating and mutating **local** variables/arrays	Mutating props or state objects	Create new objects (§3.6)
Calculating derived values	DOM manipulation, `fetch`, timers, subscriptions, analytics logging	**Event handlers** (first choice) or **Effects** — `useEffect`, code run after rendering (Chapter 5)
	`Math.random()`, `Date.now()` for output (non-deterministic)	Compute once in a `useState` initialiser, a handler, or an Effect

Event handlers need not be pure: they run on a user action, not during rendering.

Common mistakes

- "Removing StrictMode fixes my doubled logs." It hides the symptom; fix the impurity.
- Sorting a prop **in place**: `items.sort()` mutates the parent's array. Use `items.toSorted()` or `[...items].sort()`.
- Pushing to a **module-level** array (top of file, shared by every render) during render — a render counter, a cache — works once, breaks on re-render.

In the interview

> **30-second answer:** "A React component must be pure while rendering: same props, state and context in, same JSX out, and no mutation of anything that existed before the render. React relies on this because it may render a component several times or discard a render. Side effects go in event handlers, or in Effects when they must sync with an **external system** (anything React does not control: the network, a timer, a browser API, a third-party widget). StrictMode double-renders in development to catch violations."

- *Is `console.log` in render impure?* — Technically, but harmless; DevTools dims the second StrictMode log. Avoid anything that changes app data.
- *Can a component mutate a local array?* — Yes — "local mutation" of values created in the same render is fine.
- *Where should `fetch` go?* — An event handler, or an Effect / data library (Chapters 5, 9).

TL;DR

Render = pure calculation. Same inputs → same JSX; mutate only what you created in this render; side effects go to event handlers or Effects. StrictMode renders twice to catch mistakes.

2.6 Keeping components small

What it is

A guideline: each component has **one clear job**; extract pieces when it grows several responsibilities.

When to extract

Signal	Example	Extract
A block of JSX has its own name in conversation	"the price", "the stock label"	`Price`, `StockLabel`
The same markup appears twice	Two identical badge spans	`Badge`
A section has its own logic	Stock thresholds	`StockLabel` with its `if`s
The file is hard to review (a few hundred lines)	A 400-line `Dashboard`	Sub-components per panel
A part needs its own state	An expandable row	`Row` with local state

How: cut the JSX block into a new capitalised top-level function; every parent variable it used becomes a **prop**; replace the block with `<NewComponent prop={value} />`; move it to its own file once a second file uses it.

Trade-off: too big = hard to read, test and reuse, many reasons to change; too small = many files, jumping around to understand one screen, props threaded through thin layers. A useful split is **presentational** components (props in, UI out — `Price`, `Badge`) vs **feature** components (know the data, wire things — `Catalog`); container/presentational patterns: Chapter 14.

TL;DR

One job per component. Extract when JSX has a name, repeats, carries its own logic, or needs its own state — its free variables become props.

2.7 Conditional rendering

What it is

Showing different JSX depending on props or state, using ordinary JavaScript: `if`, the ternary `cond ? a : b`, logical `a && b` (gives `b` only when `a` is truthy), and variables.

How it works — four techniques

`code/05_conditional_rendering.jsx`:

```jsx
// 1. if / early return (statements are allowed ABOVE the JSX)
function Item({ name, isPacked }) {
  if (isPacked) return <li className="item">{name} ✅</li>;
  return <li className="item">{name}</li>;
}
// 2. ternary inside JSX
function Item2({ name, isPacked }) {
  return <li className="item">{isPacked ? <del>{name}</del> : name}</li>;
}
// 3. && : render something or nothing
function Item3({ name, isPacked }) {
  return <li className="item">{name} {isPacked && '✅'}</li>;
}
// 4. store JSX in a variable
function Item4({ name, isPacked }) {
  let content = name;
  if (isPacked) content = <strong>{name} (packed)</strong>;
  return <li className="item">{content}</li>;
}
// returning null renders nothing at all
function Hidden({ show }) { return show ? <p>visible</p> : null; }

// The trap: a number on the left of && is rendered when it is 0.
function Inbox({ messages }) {
  return <p>{messages.length && `You have ${messages.length} messages`}</p>;
}
function InboxFixed({ messages }) {
  return <p>{messages.length > 0 && `You have ${messages.length} messages`}</p>;
}
```

Output:

```
if/return : <ul><li class="item">Space suit ✅</li><li class="item">Photo</li></ul>
ternary   : <ul><li class="item"><del>Helmet</del></li></ul>
&&        : <ul><li class="item">Helmet ✅</li><li class="item">Map </li></ul>
variable  : <ul><li class="item"><strong>Helmet (packed)</strong></li></ul>
null      : ""
0 && bug  : <p>0</p>
fixed     : <p></p>
```

Which to use

Technique	Best for	Watch out
`if` + early `return`	Whole component differs (loading / error / empty states)	Duplicate wrapper markup
`cond ? a : b`	Choosing between two small pieces inline	Nested ternaries become unreadable → extract a component
`cond && a`	Show-or-nothing	Left side `0` or `NaN` is rendered — make it a real boolean (`> 0`, `!!x`, `Boolean(x)`)
Variable (`let content`)	Several conditions building one piece	Slightly more lines
`return null`	Component decides to hide itself	The *parent* often should decide instead

Why `0` shows: `a && b` returns `a` when `a` is **falsy** (`false`, `0`, `NaN` — "Not a Number" — `''`, `null`, `undefined`). `false`, `null`, `undefined` render nothing, but `0` is a number and numbers render (`''` gives an invisible empty text node).

Identity: `isPacked ? <del>{name}</del> : name` yields different *types* at one position, so React replaces the DOM there (§1.3). If both branches are the same component (`cond ? <Counter a /> : <Counter b />`), React keeps the same **instance** and its state — see §3.10 for how `key` changes that.

In the interview

- *Why does `{count && <Badge />}` show "0"?* — `&&` returns its falsy left operand; 0 is renderable. Use `count > 0 &&`.
- *How do you render nothing?* — Return `null` (or `false`/`undefined` inside JSX).

TL;DR

`if`/early return for big branches, `? :` for either-or, `&&` for show-or-nothing (with a boolean on the left!), `null` for nothing.

2.8 Rendering lists and keys

What it is

Turn an **array of data** into an **array of elements** with `map()` (and `filter()` to select). Each element needs a `key`: a string or number identifying that item among its siblings.

How it works

`code/06_lists_and_keys.jsx`:

```
const people = [
  { id: 0, name: 'Creola Katherine Johnson', profession: 'mathematician' },
  { id: 1, name: 'Mario José Molina-Pasquel Henríquez', profession: 'chemist' },
  { id: 2, name: 'Mohammad Abdus Salam', profession: 'physicist' },
  { id: 3, name: 'Percy Lavon Julian', profession: 'chemist' },
];

function ChemistList() {
  const chemists = people.filter((p) => p.profession === 'chemist');
  return <ul>{chemists.map((p) => <li key={p.id}>{p.name}</li>)}</ul>;
}

// Missing key -> React logs a warning in development.
render(<ul>{people.map((p) => <li>{p.name}</li>)}</ul>);
```

Output:

```
<ul>
<li>Mario José Molina-Pasquel Henríquez</li>
<li>Percy Lavon Julian</li>
</ul>

warning: Each child in a list should have a unique "key" prop. See https://react.dev/link/warning-keys for more
information.
```

Arrow functions (`(args) => result`): `p => <li>…</li>` returns implicitly; with a body `p => { return <li>…</li>; }` you must write `return`, or the list is empty.

Why keys matter

On re-render React matches old list items to new ones by key (§1.3). With a stable ID it knows "the Bread row moved down"; with the **array index** it thinks "row 0 changed its text from Bread to Eggs", so any **state** in the row (input text, component state, **focus**, animations) stays at the old position.

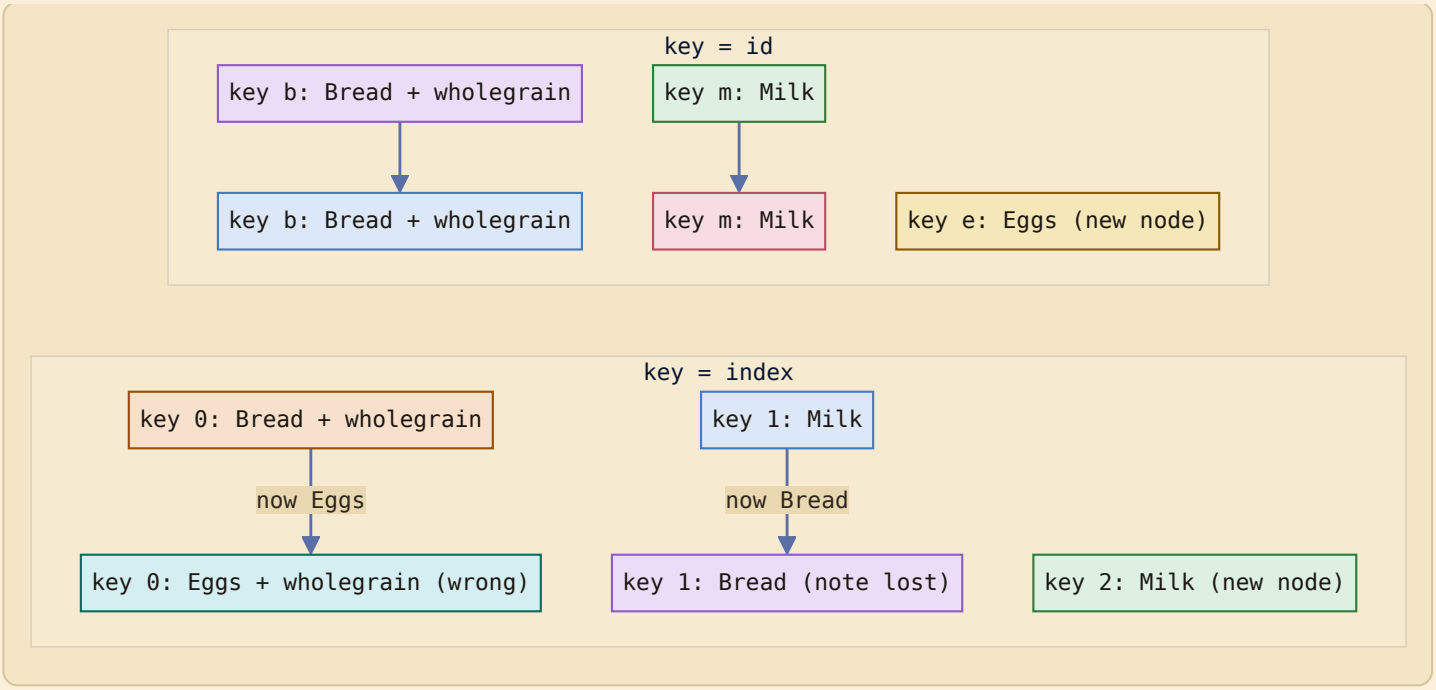
The program renders rows each holding an **uncontrolled** `<input>` (value lives in the DOM, not React state), types *wholegrain* into Bread's input, then inserts Eggs at the top:

```
function NoteList({ items, useIndexKey }) {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={useIndexKey ? index : item.id}>
          {item.label}: <input aria-label={`note for ${item.label}`} defaultValue="" />
        </li>
      ))}
    </ul>
  );
}
// type 'wholegrain' into Bread's input, then rerender with Eggs prepended
```

Output:

```
key=index after prepend -> Eggs"wholegrain"  Bread""  Milk""
key=id    after prepend -> Eggs""  Bread"wholegrain"  Milk""
```

With index keys the note jumped to **Eggs** — real data corruption. With ID keys it stayed with Bread.



Rules for keys

1. **Unique among siblings** (not globally); different lists can reuse keys.
2. **Stable**: same item, same key every render. Use IDs from your data (database IDs, or **UUIDs** — random IDs practically guaranteed unique — created *when the item is created*).
3. **Never generate during render**: `key={Math.random()}` / `key={crypto.randomUUID()}` → new keys each render → every item **remounts** (destroyed and recreated): slow, loses focus and state.
4. **Index only** for static lists: never reordered, filtered or inserted into (except at the end), and stateless items.
5. Put the key on the **outermost element returned from** `map` — `<ProductCard key=… />`, not the `<li>` inside `ProductCard`.
6. The key is not passed to the component; pass the ID separately if needed (`<Row key={id} id={id} />`).
7. One item rendering several siblings → `<Fragment key={id}>…</Fragment>` (`<>` cannot take a key).

Common mistakes

Mistake	Symptom
Missing key	Console warning; index used implicitly → same bugs as index keys
Index key on a sortable/filterable list (or after deleting the first item)	Inputs, checkboxes or expanded state attach to the wrong row (everything shifts)
`Math.random()` key	Inputs lose focus on each keystroke, poor performance
Duplicate keys	Warning "Encountered two children with the same key"; items duplicated or dropped
Key inside the child component instead of on the element in `map`	Warning persists

Keys also **reset** a component outside lists: `<Profile key={userId} />` gives each user a fresh instance (§3.10).

In the interview

30-second answer: "`map` data to elements and give each a key that is stable and unique among siblings, usually the item's ID. React uses keys to match items between renders, so it can move DOM nodes and keep state with the right item. Index keys break when the list is reordered, filtered or prepended — state stays at the position instead of following the item. Random keys are worse: everything **remounts** every render."

- *When is index as key fine?* — Static lists with no per-item state (e.g. a fixed footer menu).
- *Globally unique?* — No, only among siblings of the same parent.
- *Can I read `props.key`?* — No; pass the value under another prop name.

> **TL;DR**

`data.map(item => <X key={item.id} … />)`. Keys: unique among siblings, stable, from data, never random, index only for static lists.

2.9 Putting it together — a product catalog

`code/catalog.jsx` combines small presentational components (`Price`, `Badge`, `StockLabel`), a card composing them, a `Section` wrapper using `children`, conditional rendering, and a filtered keyed list.

```jsx
// Price formats with Intl.NumberFormat; Badge = <span className={`badge badge-${tone}`}>{children}</span>;
// Section = <section><h2>{title}</h2>{children}</section>  (full file: code/catalog.jsx)
export function StockLabel({ stock }) {
  if (stock === 0) return <Badge tone="red">Out of stock</Badge>;
  if (stock < 5) return <Badge tone="amber">Only {stock} left</Badge>;
  return null;                                  // plenty in stock: show nothing
}

export function ProductCard({ product }) {
  const { name, price, stock, rating, tags } = product;
  return (
    <article className="product" aria-label={name}>
      <h3>{name}</h3>
      <div className="badges">
        {tags.includes('new') && <Badge tone="blue">New</Badge>}
        {tags.includes('sale') && <Badge tone="green">Sale</Badge>}
        <StockLabel stock={stock} />
      </div>
      <p><Price amount={price} /> · ★ {rating.toFixed(1)}</p>
      <button disabled={stock === 0}>{stock === 0 ? 'Sold out' : 'Add to cart'}</button>
    </article>
  );
}

export function Catalog({ products, minRating = 0 }) {
  const visible = products.filter((p) => p.rating >= minRating);
  if (visible.length === 0) return <p>No products match.</p>;
  return (
    <Section title={`Products (${visible.length})`}>
      <div className="grid">
        {visible.map((p) => <ProductCard key={p.id} product={p} />)}
      </div>
    </Section>
  );
}
```

`code/07_product_catalog.jsx` renders `<Catalog products={PRODUCTS} />`, reads the screen like a user (or test) — by **roles** (what a screen reader announces an element as) such as `article`, `heading`, `button` — then `rerender`s with `minRating={4.5}` and `{5}`:

```
Products (4)
  Mechanical keyboard  |  $89.00 | badges: New               | button: Add to cart
  USB-C hub            |  $35.00 | badges: Out of stock      | button: Sold out (disabled)
  27" monitor          | $249.00 | badges: Sale, Only 3 left | button: Add to cart
  Laptop stand         |  $42.00 | badges: -                 | button: Add to cart

minRating=4.5 -> Products (2) [ 'Mechanical keyboard', '27" monitor' ]
minRating=5   -> No products match.
```

The same components in the browser (`code/catalog.html`, styles in `catalog.css`):

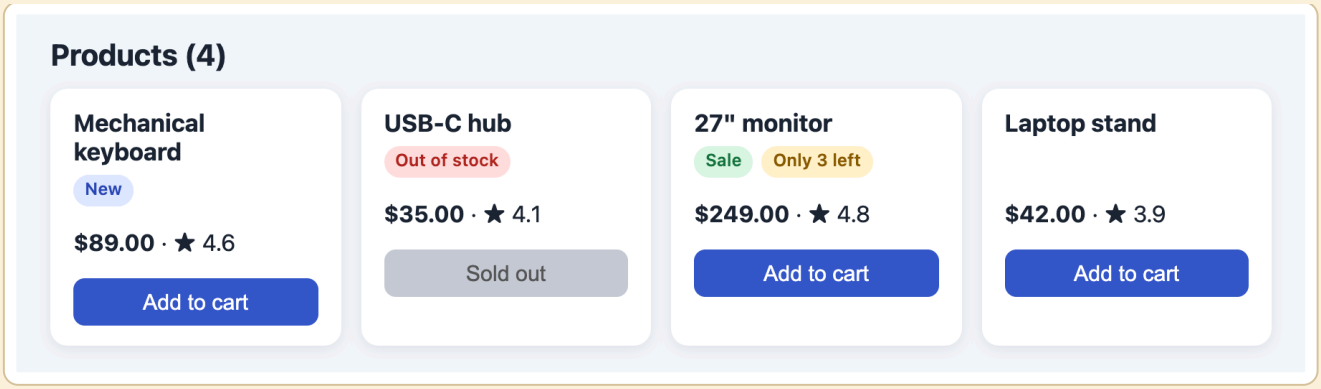


Figure 2.1 — `Catalog` rendered in Chrome. Every badge and button state comes from props and conditional rendering.

`rerender` with a new `minRating` re-ran `Catalog`; nothing "updates" itself — components just describe the UI for their props. The buttons do nothing yet: that needs **state and events** (Chapter 3).

Chapter 2 TL;DR

Idea	Remember
Component	Capitalised function `(props) => JSX`, top-level, used as `<X />`
Props	One read-only object; destructure with defaults; strings in quotes, other values in braces
Forwarding	`({ a, ...rest }) => <button {...rest} />`; later props override earlier
`children`	Nested JSX; enables wrappers; slots = JSX in named props
Composition	Combine components instead of inheriting; first fix for prop drilling
Purity	Same inputs → same JSX, no outside mutation during render; StrictMode renders twice in dev
Small components	One job each; free variables become props
Conditions	`if`/return, `? :`, `&&` (boolean on the left — the `0` trap), `null`
Lists	`map` + `key` from stable IDs; index only for static lists; never random

Next: Chapter 3 — State and Events.

Chapter 3 — State and Events

Chapter 2's catalog could *show* data but not *react* to the user. **Event handlers** respond to clicks and typing, and **state** lets a component remember values between **renders** (each time React calls your component). Most beginner bugs — "my counter only goes up by one", "the screen doesn't update", "the note moved to the wrong row" — come from the ideas here, so each is shown with a runnable program.

Code: `code/` — React 19.3 in jsdom; user actions simulated with `fireEvent` (dispatches one DOM event) and `@testing-library/user-event` (simulates a real user: **focus** — which element receives keyboard input — key presses, clicks).

3.1 Responding to events

What it is

An **event** is something that happens in the browser — a click, a key press, a form submission. An **event handler** is a function React runs when it happens, passed as a prop named `on` + event name in camelCase: `onClick`, `onChange`, `onSubmit`, `onKeyDown`, `onPointerMove`, `onFocus`, …

How it works

1. Define a function inside the component (so it can read props and state); convention `handle` + event, e.g. `handleClick`.
2. Pass the **function itself** — `onClick={handleClick}` or an inline arrow `onClick={() => …}`.
3. React calls it with an **event object** `e`: `e.target`, `e.type`, `e.key` (keyboard), `e.preventDefault()`, `e.stopPropagation()`.

`code/02_events.jsx`:

```
// 1. Pass the function; do not call it.
function Right() { return <button onClick={() => console.log('  Right: clicked')}>right</button>; }
function Wrong() {
  const handleClick = () => console.log('  Wrong: ran during RENDER, not on click');
  return <button onClick={handleClick()}>wrong</button>; // calls it now; onClick receives undefined
}

// 2. Passing arguments + custom event props (onX / handleX naming).
function Toolbar({ onPlay, onUpload }) {
  return (
    <div>
      <button onClick={() => onPlay('Kiki')}>Play movie</button>
      <button onClick={onUpload}>Upload image</button>
    </div>
  );
}
function App() {
  function handlePlay(title) { console.log(`  Playing ${title}`); }
  return <Toolbar onPlay={handlePlay} onUpload={(e) => console.log('  Uploading... event type =', e.type)} />;
}

// 3. Propagation: events bubble from child to parent; stopPropagation halts it.
function Panel({ stop }) {
  return (
    <div onClick={() => console.log('    parent <div> onClick')}>
      <button onClick={(e) => { if (stop) e.stopPropagation(); console.log('    child <button> onClick'); }}>
        {stop ? 'stop' : 'bubble'}
      </button>
    </div>
  );
}

// 4. preventDefault: stop the browser's default action (a form submit reloads the page).
function Signup() {
  return (
    <form onSubmit={(e) => { e.preventDefault(); console.log('  submitted without reload; defaultPrevented =',
e.defaultPrevented); }}>
      <input aria-label="email" />
      <button>Send</button>
    </form>
  );
}
```

Output:

```
rendering <Wrong/>:
  Wrong: ran during RENDER, not on click
clicking wrong:
clicking right:
  Right: clicked
  Playing Kiki
  Uploading... event type = click
click "bubble":
    child <button> onClick
    parent <div> onClick
click "stop":
    child <button> onClick
  submitted without reload; defaultPrevented = true
```

- **`onClick={handleClick()}` is a bug**: it runs *during rendering* and the button receives `undefined`, so clicking does nothing. If the function sets state you get an **infinite loop** (React stops it with "Too many re-renders"). Pass `handleClick`, or wrap in an arrow for arguments: `onClick={() => onPlay('Kiki')}`.
- **Custom components define their own event props** (`onPlay`, `onUpload`) — plain function props named `on…` by convention. The child decides *when* to call; the parent decides *what* happens. This is how data flows **up**.
- **Bubbling (propagation)**: the click fired on the button, then its parent `<div>`; `e.stopPropagation()` stopped it at the button. All React events bubble except `onScroll`. To catch an event on the way *down* before children see it, use the **capture phase** version (root → target, before bubbling back up), e.g. `onClickCapture`.
- **`e.preventDefault()`** stops the browser's built-in behaviour (here the page reload of a form submit). It does *not* stop bubbling.

React's event object

React passes a **SyntheticEvent**: a cross-browser-consistent wrapper around the **native** browser event with the same interface (`target`, `currentTarget`, `preventDefault` …); `e.nativeEvent` gives the original. React attaches one **listener** per event type at the root container and dispatches to your handlers (**event delegation**), so you never call `addEventListener` for normal UI events.

Common mistakes

Mistake	Fix
`onClick={handle()}`	`onClick={handle}` or `onClick={() => handle(id)}`
`onclick` (lowercase)	`onClick` — React warns about the unknown prop
Click handlers on a `<div>`	Use `<button>` — keyboard and screen-reader support for free
`stopPropagation` to fix a design problem	Usually the parent should get an explicit callback instead
No `preventDefault` on `onSubmit`	The page reloads and state is lost

In the interview

> **30-second answer:** "Handlers are functions passed as `on…` props — pass the function, don't call it. React gives them a SyntheticEvent that mirrors the native one. Events bubble, so `stopPropagation` stops parents from seeing them, and `preventDefault` stops browser defaults like form submission. Components expose their own `onSomething` props so parents decide what happens."

- *Can handlers have side effects?* — Yes: handlers are *the* place for side effects (set state, fetch, log). Only rendering must be pure.
- *`e.target` vs `e.currentTarget`?* — `target` is where the event started; `currentTarget` is the element whose handler is running.

TL;DR

`onEvent={fn}` — pass, don't call. Arrow to pass arguments. Custom `onX` props send events up. `stopPropagation` = stop bubbling; `preventDefault` = stop browser default.

3.2 State: a component's memory — `useState`

What it is

State is data a component remembers between renders and that changes with interaction: input text, the open tab, cart items. Add it with the `useState` **Hook** (a **Hook** is a React function starting with `use` that taps into React features):

```
const [count, setCount] = useState(0);
//     value  setter          initial value (used on the first render only)
```

`useState` returns a pair (named via array destructuring): the **current value** and a **setter**. Calling the setter (1) stores the new value and (2) asks React to **re-render** the component.

The problem it solves

`code/01_local_variable_vs_state.jsx`:

```
function BrokenCounter() {
  let count = 0;                          // re-created as 0 on every render
  renders++;
  return <button onClick={() => { count = count + 1; console.log('  inside handler count =', count); }}>Broken: {count}
</button>;
}

function Counter() {
  const [count, setCount] = useState(0);  // React keeps this value between renders
  return <button onClick={() => setCount(count + 1)}>Working: {count}</button>;
}
// click each twice; then render two <Counter />s and click only the first
```

Output:

```
  inside handler count = 1
  inside handler count = 2
screen shows: Broken: 0 | renders: 1
screen shows: Working: 2
two instances: Working: 1 | Working: 0
```

The local variable fails twice: changing it does **not trigger a render** (renders stayed 1), and `let count = 0` would reset it anyway. `useState` **persists** the value and its setter **triggers** a render. The last line shows **state is local to each instance**.

How it works

1. First render: `useState(0)` returns `[0, setCount]`; React stores `0` for *this instance at this position in the tree*.
2. Click → `setCount(1)` → React schedules a re-render.
3. Second render: `useState(0)` ignores the `0` and returns `[1, setCount]`; React diffs and commits.

React matches `useState` calls to stored values **by call order**, hence the rule: call Hooks only at the **top level** of a component or **custom Hook** (your own `use…` function) — never inside `if`, loops or nested functions (§7.2).

Details worth knowing

- **Several state variables** are fine; group values that always change together into one object. Name them `[thing, setThing]`.
- **Lazy initial state**: `useState(createInitialTodos)` (the function, not its result) runs an expensive initialiser only on the first render; `useState(createInitialTodos())` calls it every render and discards the result.
- **Setter identity is stable** across renders — safe to pass down.
- **Same value → no re-render**: if the new value is identical (`Object.is` — "same value / same object"), React skips re-rendering the component and its children (§3.6).

When not to use state

Value	Where it belongs
Never changes	A constant outside the component
Comes from the parent	Props (don't copy into state — §3.8)
Can be computed from props/state	A variable in render (§3.8)
Must persist but must **not** re-render (timer ID, DOM node)	`useRef` (§5.8)
Shared by many distant components	Lift up (§3.7), then context / a store (Chapter 6)
Server data from an API (a server interface you request data from)	A data library such as TanStack Query (§6.9, Chapter 9)

In the interview

> **30-second answer:** "`useState` gives a component memory. It returns the current value and a setter; calling the setter stores the new value and schedules a re-render. Local variables don't work because they reset each render and changing them doesn't trigger one. State is private to each component instance and identified by the Hook's call order, which is why Hooks must be called unconditionally at the top level."

- *`useState(fn)` vs `useState(fn())`?* — `fn` is called once for the initial value; `fn()` runs every render.
- *When is the argument of `useState(x)` used?* — Only on the first render.
- *Shared between two instances?* — No; each instance has its own.

TL;DR

`const [v, setV] = useState(initial)`. State persists between renders, is private per instance, and the setter triggers a re-render. Top-level calls only.

3.3 State as a snapshot

What it is

Each render gets the state values **for that render**, like a photograph. Everything created during it — JSX *and event handlers* — sees those fixed values. A setter does not change the variable you hold; it requests a *new* render with a new snapshot.

How it works

`code/03_state_as_snapshot.jsx`:

```jsx
function Counter() {
  const [number, setNumber] = useState(0);
  return (
    <>
      <h1>{number}</h1>
      <button onClick={() => {
        setNumber(number + 1);   // setNumber(0 + 1)
        setNumber(number + 1);   // setNumber(0 + 1)
        setNumber(number + 1);   // setNumber(0 + 1)
        console.log('  right after the three calls, number =', number);
      }}>+3</button>
    </>
  );
}

// Even a delayed callback sees the snapshot from the render it was created in.
function Delayed() {
  const [n, setN] = useState(0);
  return (
    <p>
      n is <output>{n}</output>{' '}
      <button onClick={() => { setN(n + 5); setTimeout(() => console.log('  timeout fired, n =', n), 10); }}>+5
later</button>
    </p>
  );
}
```

Output:

```
  right after the three calls, number = 0
heading after one click: 1
  timeout fired, n = 0
screen shows n = 5 (the timeout logged the old snapshot)
```

In the render where `number` is 0, the handler's `number` is the constant `0`: three `setNumber(number + 1)` = three `setNumber(1)`, the log still sees 0, and React re-renders once with 1. The timeout (`setTimeout(fn, ms)`

runs `fn` once after a delay) shows the snapshot is permanent: 10 ms later, with the screen at 5, the closure still sees `n = 0` — plain JavaScript **closures** (a function remembers the variables in scope where it was created; JavaScript book, `../../../javascript/chapters/04-functions-scope-and-closures/README.md`).

Why: handlers are predictable. Click "Send to Alice", then quickly switch the recipient to Bob — the message still goes to Alice, captured at click time.

Common mistakes

- Reading state right after setting it → use the value you computed: `const next = number + 1; setNumber(next); send(next);`.
- Expecting three `setX(x + 1)` to add 3 → updater functions (§3.5).
- **Stale closures**: a timer or **subscription** (long-lived "tell me when X changes" registration) created once keeps reading an old snapshot. Use an updater, or refs/Effects (Chapter 5).

TL;DR

State inside a render is a constant snapshot. Setters request a new render; they never change the variable you are holding.

3.4 Batching

What it is

Batching: React collects all state updates made during one event and performs **a single re-render** after the handler finishes — better performance (five setters = one render) and no half-updated screens (new name with old avatar).

Since **React 18** batching is **automatic everywhere**: React handlers *and* `setTimeout`, **promises** (values that arrive later, used with `.then`/`await`), `fetch` callbacks and native listeners. (Before 18, only React event handlers.) React never batches *across* separate user events: two clicks = two renders.

How it works — counting renders

`code/04_batching_and_updaters.jsx` (component in §3.5) counts renders per button:

```
replace x3       -> number =  1 | re-renders caused: 1
updater x3       -> number =  4 | re-renders caused: 1
+5 then n=>n+1   -> number = 10 | re-renders caused: 1
+5, n=>n+1, 42   -> number = 42 | re-renders caused: 1
in timeout       -> number = 43 | re-renders caused: 1
```

Every row caused **one** re-render — including "replace x3" (four setter calls on two state variables) and "in timeout" (two setters inside `setTimeout`).

Opting out: when the DOM must update **synchronously** before the next line (e.g. scroll to a just-added item), `flushSync(() => setX(v))` from `react-dom` forces it. Use sparingly — it hurts performance.

TL;DR

All setter calls in one event → one re-render (everywhere since React 18). `flushSync` opts out.

3.5 Updater functions — queueing updates

What it is

Pass a setter a **function** that receives the **pending** state and returns the next one: `setNumber(n => n + 1)`. React queues it (first in, first out) and applies the queue in order on the next render.

How it works

`code/04_batching_and_updaters.jsx`:

```
function Counter() {
  const [number, setNumber] = useState(0);
  const [clicks, setClicks] = useState(0);
  renders++;
  return (
    <>
      <h1>{number}</h1>
      <button onClick={() => { setNumber(number + 1); setNumber(number + 1); setNumber(number + 1); setClicks(clicks +
1); }}>replace x3</button>
      <button onClick={() => { setNumber((n) => n + 1); setNumber((n) => n + 1); setNumber((n) => n + 1); }}>updater
x3</button>
      <button onClick={() => { setNumber(number + 5); setNumber((n) => n + 1); }}>+5 then n=&gt;n+1</button>
      <button onClick={() => { setNumber(number + 5); setNumber((n) => n + 1); setNumber(42); }}>+5, n=&gt;n+1,
42</button>
      <button onClick={() => setTimeout(() => { setNumber((n) => n + 1); setClicks((c) => c + 1); }, 0)}>in
timeout</button>
    </>
  );
}
```

(Output in §3.4: 1, 4, 10, 42, 43.)

A value in the queue means "replace with this"; a function means "compute from the previous result":

Button (starting number)	Queue	Processing	Result
replace x3 (0)	`1`, `1`, `1`	replace 1, replace 1, replace 1	**1**
updater x3 (1)	`n=>n+1` ×3	1→2→3→4	**4**
+5 then n=>n+1 (4)	`9`, `n=>n+1`	replace 9, then 9+1	**10**
+5, n=>n+1, 42 (10)	`15`, `n=>n+1`, `42`	15 → 16 → replace 42	**42**
in timeout (42)	`n=>n+1`	42+1	**43**

When to use which

Plain value `setX(v)`	Updater `setX(prev => …)`
New value doesn't depend on the old (`setName('Ada')`, `setOpen(false)`) — most setters	New value computed from the old and several updates may queue (`+1` twice)
	Inside timers/subscriptions created earlier (avoids stale snapshots)
	Handlers passed to memoised children (no dependency on current state)

Name the parameter after the state's first letter (`n`, `c`) or `prev`. Updaters must be **pure** — StrictMode calls them twice in development.

Common mistakes

- `setCount(count++)` — mutates the snapshot variable and passes the *old* value. Use `count + 1`.
- Side effects inside an updater (`setX(x => { fetch(…); return x + 1; })`) — it may run twice.

In the interview

30-second answer: "Setters queue updates. A plain value replaces the state; an updater function receives the pending value, so three `n => n + 1` calls add 3 while three `setN(n + 1)` calls add 1 because they all read the same snapshot. React processes the queue during the next render, and all of it is batched into one render."

- `setN(n + 5); setN(m => m + 1)` *from 0?* — 6.
- `setN(m => m + 1); setN(n + 5)` *from 0?* — 5 (the last replace wins).

TL;DR

`setX(v)` replaces; `setX(prev => next)` computes from the queued value. Queue processed in order at the next render.

3.6 Updating objects and arrays immutably

What it is

State may hold objects and arrays, but treat them as **immutable**: to change one, create a **new** object/array and pass it to the setter. **Mutation** = changing an existing object in place (`obj.x = 1`, `arr.push(…)`).

Why it matters

1. **React only notices new references** (a **reference** points to where an object lives; two variables can share one). The setter compares with `Object.is`; the same object — even with changed insides — looks unchanged, so React may skip the render.
2. **Snapshots and history**: old renders keep pointing at old objects, so undo/redo and time-travel debugging work.
3. **Optimisations** (`memo`, the React Compiler) rely on "new reference = changed".

Objects

`code/05_updating_objects.jsx`:

```
const [person, setPerson] = useState({
  name: 'Niki de Saint Phalle',
  artwork: { title: 'Blue Nana', city: 'Hamburg' },
});
// ...
<button onClick={() => { person.name = 'MUTATED'; setPerson(person); }}>mutate</button>
<button onClick={() => setPerson({ ...person, name: 'Niki' })}>copy name</button>
<button onClick={() => setPerson({ ...person, artwork: { ...person.artwork, city: 'Berlin' } })}>copy nested
city</button>
```

Output:

```
initial          -> "Niki de Saint Phalle — Blue Nana (Hamburg)" | renders so far: 1
mutate + set same -> "Niki de Saint Phalle — Blue Nana (Hamburg)" | renders so far: 1
copy name        -> "Niki — Blue Nana (Hamburg)" | renders so far: 2
copy nested city  -> "Niki — Blue Nana (Berlin)" | renders so far: 3
```

- **mutate**: the object changed, but the same reference was passed → no re-render → stale screen (and the change pops up later whenever something else re-renders).
- **copy name**: **spread** `{ ...person, name }` copies top-level properties into a new object, then overrides `name`.
- **nested**: spread is *shallow*; copy each level on the path (new `person` containing a new `artwork`).

Arrays — the recipe table

`code/06_updating_arrays.jsx`:

```
const add = () => setItems([...items, { id: nextId++, title: 'Aurora', seen: false }]);          // add to end
const addFirst = () => setItems([{ id: nextId++, title: 'Fjords', seen: false }, ...items]);     // add to start
const insertAt1 = () => setItems([...items.slice(0, 1), { id: nextId++, title: 'Glacier', seen: false },
...items.slice(1)]);
const remove = (id) => setItems(items.filter((it) => it.id !== id));                              // remove
const toggle = (id) => setItems(items.map((it) => (it.id === id ? { ...it, seen: !it.seen } : it))); // update one
const sortAZ = () => setItems(items.toSorted((a, b) => a.title.localeCompare(b.title)));          // sort a copy
const reverse = () => setItems([...items].reverse());                                             // copy, then reverse
const immerToggle = (id) => setItems(produce((draft) => {                                         // Immer: "mutate" a
draft
  const it = draft.find((d) => d.id === id); it.seen = !it.seen;
}));
```

Output:

```
initial                 -> [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army
add                     -> [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army, [ ] Aurora
add first               -> [ ] Fjords, [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army, [ ] Aurora
insert at 1             -> [ ] Fjords, [ ] Glacier, [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army, [ ]
Aurora
remove Lunar Landscape  -> [ ] Fjords, [ ] Glacier, [ ] Big Bellies, [x] Terracotta Army, [ ] Aurora
toggle Big Bellies      -> [ ] Fjords, [ ] Glacier, [x] Big Bellies, [x] Terracotta Army, [ ] Aurora
immer toggle Aurora     -> [ ] Fjords, [ ] Glacier, [x] Big Bellies, [x] Terracotta Army, [x] Aurora
sort                    -> [x] Aurora, [x] Big Bellies, [ ] Fjords, [ ] Glacier, [x] Terracotta Army
reverse                 -> [x] Terracotta Army, [ ] Glacier, [ ] Fjords, [x] Big Bellies, [x] Aurora
original array untouched: [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army
```

Goal	Avoid (mutates)	Use (returns a new array)
Add	`push`, `unshift`	`[...arr, x]`, `[x, ...arr]`
Insert at i	`splice(i, 0, x)`	`[...arr.slice(0, i), x, ...arr.slice(i)]` or `arr.toSpliced(i, 0, x)`
Remove	`splice`, `pop`, `shift`	`arr.filter(item => item.id !== id)`
Replace / update one item	`arr[i] = x`, `item.done = true`	`arr.map(item => item.id === id ? { ...item, done: true } : item)` or `arr.with(i, x)`
Sort / reverse	`sort()`, `reverse()`	`toSorted()`, `toReversed()` (ES2023), or copy first: `[...arr].sort()`

`[...arr]` copies the **array**, not the objects inside: `copy[0].seen = true` still mutates the shared object — which is why the toggle recipe also spreads the item.

Immer — write "mutations" safely

Immer gives you a **draft** — a **proxy** (stand-in that intercepts reads and writes) recording your changes — and produces new immutable state with structural sharing (unchanged parts reused). Above, `produce(draft => { … })` is passed to the setter as an updater. In components, `use-immer` wraps it:

```
// (use-immer — not run; same idea as produce above)
import { useImmer } from 'use-immer';
const [person, updatePerson] = useImmer({ name: 'Niki', artwork: { city: 'Hamburg' } });
updatePerson(draft => { draft.artwork.city = 'Berlin'; });
```

Redux Toolkit uses Immer internally (§6.7). Mutating the draft is fine; mutating real state is not.

Trade-off: manual spread = no dependency and explicit, but verbose for nested data; Immer = a few KB, reads like normal mutation even deep, but proxied drafts are slightly harder to debug. Also consider **flattening** nested state (normalise by ID) — updates get simpler either way.

Common mistakes

- `arr.push(x); setArr(arr)` → no re-render.
- `items.sort()` in render or before setting → mutates state.
- Copying only the outer level of nested data, then mutating inside.

In the interview

30-second answer: "State must be treated as immutable: create new objects and arrays instead of mutating. React decides whether something changed by reference (`Object.is`), so mutating and setting the same object may not re-render and breaks snapshots and memoisation. Use spread for objects, `map` / `filter` / `toSorted` / `with` for arrays, copy every level for nested updates, or use Immer to write mutation-style code on a draft."

- *Is `[...arr]` a deep copy?* — No, shallow: elements are shared.
- *Update `user.address.zip`?* — `setUser({ ...user, address: { ...user.address, zip } })`.
- *How does Immer stay efficient?* — Structural sharing: only changed paths get new objects.

TL;DR

Never mutate state. New object: `{ ...obj, key }`; new array: spread, `filter`, `map`, `toSorted`, `with`, `toSpliced`. Copy each nested level, or use Immer.

3.7 Lifting state up

What it is

When two components must **share or coordinate** state, move it to their **closest common parent** and pass it down as props, with callbacks to change it — **lifting state up**. The children become **controlled** (driven by props) instead of **uncontrolled** (driven by their own state).

How it works — an accordion

`code/07_lifting_state_up.jsx`. An **accordion** is a stack of panels where clicking a header expands it; requirement: only one open at a time.

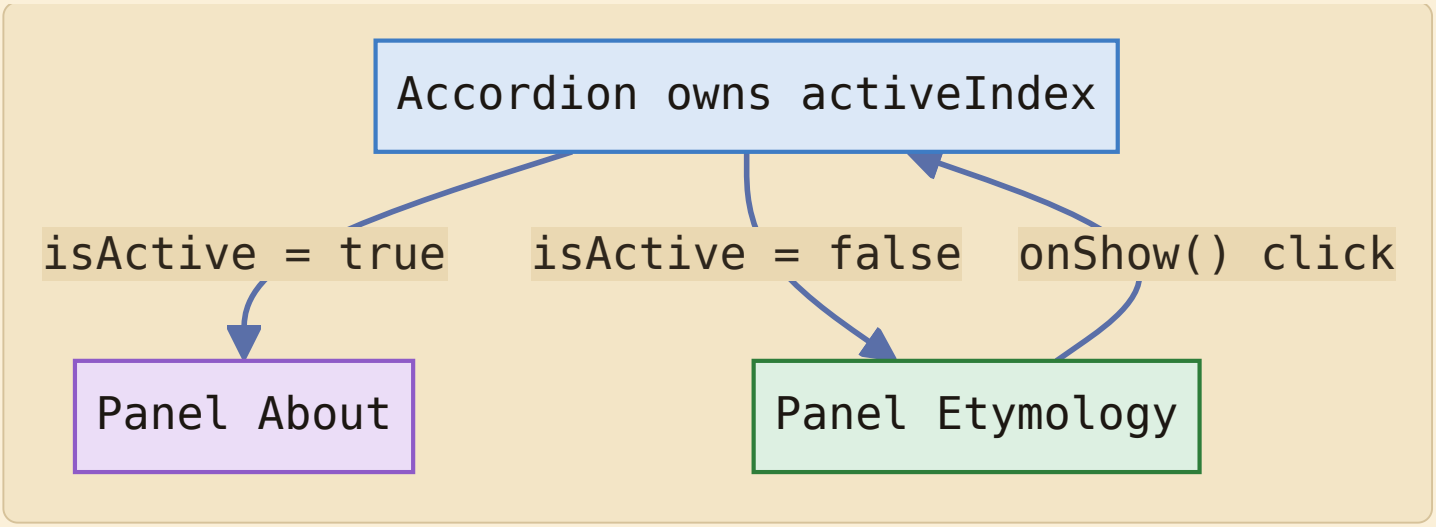
```
// Before: each Panel owns isActive -> both can be open at once.
function PanelLocal({ title, children }) {
  const [isActive, setIsActive] = useState(false);
  // ...
}

// After: Panel is controlled by props; the parent owns activeIndex.
function Panel({ title, isActive, onShow, children }) {
  return (
    <section>
      <h3>{title}</h3>
      {isActive ? <p>{children}</p> : <button onClick={onShow}>Show {title}</button>}
    </section>
  );
}
function Accordion() {
  const [activeIndex, setActiveIndex] = useState(0);
  return (
    <>
      <Panel title="About" isActive={activeIndex === 0} onShow={() => setActiveIndex(0)}>Almaty is Kazakhstan's largest
city.</Panel>
      <Panel title="Etymology" isActive={activeIndex === 1} onShow={() => setActiveIndex(1)}>The name comes from the
word for apple.</Panel>
    </>
  );
}
```

Output:

```
local state : open panels after clicking both = 2
lifted state: initially open = Almaty is Kazakhstan's largest city.
lifted state: after "Show Etymology" open panels = 1 -> The name comes from the word for apple.
```

Steps: (1) **remove** state from the children (`isActive` becomes a prop); (2) **pass** it from the common parent (`isActive={activeIndex === 0}`); (3) **add** state to the parent and pass a callback (`onShow`) so children can request changes.



Single source of truth

Each piece of state has **one owner**; others get it via props. Same data stored in two places and "kept in sync" → lift it to one owner.

Trade-offs

Lift up	Keep local
Siblings stay in sync; parent can reset/control	Simpler; state changes re-render less of the tree
Parent re-renders on every change (usually fine)	Parent cannot coordinate or read it
Too high → prop drilling (then composition §2.4 or context §6.3)	

Rule: keep state **as low as possible, as high as necessary**.

In the interview

> **30-second answer:** "When siblings need the same state, move it to their closest common parent, pass the value down as props and a callback to change it. The children become controlled components. It gives one source of truth; if lifting goes too high and causes prop drilling, use composition or context."

TL;DR

Shared state → closest common parent. Pass value + `onChange`-style callback down. One owner per piece of state.

3.8 Derived state — choosing the minimal state

What it is

Derived state is any value you can **calculate** from props or other state. Compute it as a normal variable during render — **not** with `useState`. Keep state **minimal** (store a birth date, not an age).

How it works

`code/08_derived_state.jsx` — three classic mistakes and fixes:

```
// 1. Redundant state: fullName can be computed from first + last.
const fullName = `${first} ${last}`;              // derived: no useState, no sync code

// 2. Duplicated state: store the selected ID, not a copy of the selected object.
const [selectedItem, setSelectedItem] = useState(items[0]);     // BAD: a copy
const [selectedId, setSelectedId] = useState(0);                // GOOD: a reference by ID
const selected = storeObject ? selectedItem : items.find((i) => i.id === selectedId);

// 3. Mirroring a prop in state ignores later prop changes.
function MessageMirror({ color }) { const [c] = useState(color); return <p style={{ color: c }}>mirror: {c}</p>; }
function MessageDirect({ color }) { return <p style={{ color }}>direct: {color}</p>; }
```

Output:

```
derived fullName   : Ticket for: Ada King
store object copy  : You picked: pretzels
store selected id  : You picked: PRETZELS (renamed)
parent changed to red -> mirror: blue | direct: red
```

- A stored **copy** of the selected object kept the stale name after a rename; storing the **ID** and deriving always shows current data.
- **Mirroring props** (`useState(color)`) captures only the first value — the mirror stayed blue. Use the prop directly; if you really want only the initial value, name it `initialColor`.

Principles for structuring state

Principle	Example
Group related state that always changes together	`{ x, y }` position in one object
Avoid contradictions	Instead of `isSending` + `isSent` (both true = impossible), one `status: 'typing' | 'sending' | 'sent'`
Avoid redundant state	Compute `fullName`, `total`, `filteredList`, `remaining` in render
Avoid duplication	Store IDs, look objects up
Avoid deep nesting	Normalise: `{ byId: {…}, ids: […] }`

"Isn't recalculating slow?" Filtering hundreds of items takes microseconds. A genuinely expensive calculation can be cached with `useMemo` (Chapter 10) — still derived, never manually synced.

In the interview

30-second answer: "I keep the minimal state and derive everything else during render — full names, totals, filtered lists. Storing derived values creates two sources of truth that can drift. I store IDs instead of copied objects, avoid mirroring props into state, and replace contradictory booleans with a single status value. Expensive derivations can be memoised with `useMemo`."

- *Use `useEffect` to update derived state when inputs change?* — No; it causes an extra render and a moment of stale UI. Compute it in render (§5.6).

TL;DR

If it can be computed, compute it. Store IDs not copies, don't mirror props, prefer one status over several booleans.

3.9 Controlled inputs (basics)

What it is

A **controlled input**'s displayed value comes from React state: `value={state}` plus a state update in `onChange`; state is the **single source of truth**. An **uncontrolled input** keeps its value in the DOM (optional

`defaultValue`, read later) — §4.2.

Element	Controlled prop	Read new value from
`<input type="text">`, `<textarea>`	`value`	`e.target.value`
`<select>`	`value` (on the `<select>`, not `selected` on options)	`e.target.value`
`<input type="checkbox">` / `radio`	`checked`	`e.target.checked`

React's `onChange` fires on **every keystroke** (like the DOM `input` event), not only on **blur** (losing focus) like the native `change` event.

How it works

`code/09_controlled_inputs.jsx`:

```
function Profile() {
  const [name, setName] = useState('');
  const [bio, setBio] = useState('');
  const [plan, setPlan] = useState('free');
  const [agree, setAgree] = useState(false);
  return (
    <form onSubmit={(e) => { e.preventDefault(); console.log('  submit ->', JSON.stringify({ name, bio, plan, agree })); }}>
      <label>Username <input value={name} onChange={(e) => setName(e.target.value.toLowerCase().replace(/\s/g, ''))} /></label>
      <label>Bio <textarea value={bio} onChange={(e) => setBio(e.target.value.slice(0, 20))} /></label>
      <label>Plan
        <select value={plan} onChange={(e) => setPlan(e.target.value)}>
          <option value="free">Free</option><option value="pro">Pro</option>
        </select>
      </label>
      <label><input type="checkbox" checked={agree} onChange={(e) => setAgree(e.target.checked)} /> I agree</label>
      <p>{name ? `@${name}` : 'pick a username'} · {bio.length}/20</p>
      <button disabled={!agree || !name}>Create</button>
    </form>
  );
}
// user types "Ada Love Lace" and a 28-character bio, picks Pro, then ticks the box and submits
```

Output:

```
username field shows: adalovelace
bio field shows     : First programmer, 18
status line         : @adalovelace · 20/20
Create disabled?    : true
  submit -> {"name":"adalovelace","bio":"First programmer, 18","plan":"pro","agree":true}

value without onChange -> field shows: fixed
warning: You provided a `value` prop to a form field without an `onChange` handler. This will render a read-only
field....
```

Control buys you: **transforming input as typed** ("Ada Love Lace" → `adalovelace`), **enforcing limits** (bio stopped at 20), **live derived UI** (`@handle`, counter, disabled button), and a **submit** that reads state directly. The last lines show `value` without `onChange`: a **read-only** field plus a warning.

Common mistakes

Mistake	Symptom / fix
`value` without `onChange`	Read-only field + warning. Add `onChange`, use `defaultValue` for uncontrolled, or `readOnly` if intended
`value={undefined}` first, then a string	Warning "changing an uncontrolled input to be controlled". Initialise with `''`, never `undefined` / `null`
Reading `e.target.value` for a checkbox	Use `e.target.checked`
Setting `selected` on `<option>`	Put `value` on `<select>`
One `useState` per field in a big form	One object + `[e.target.name]: e.target.value`, a reducer (a function `(state, action) => newState`; §6.2), or a form library (Chapter 4)

Trade-offs

Controlled	Uncontrolled
Instant validation, formatting, dependent fields	Less code; the DOM holds the value
Re-renders on each keystroke (fine for most forms)	Read values on submit via `FormData` (browser object with all a form's field values) or refs
Easy to reset/prefill from state	React 19 form **Actions** (functions passed to `<form action>` that React runs on submit) make this style first-class (§4.4)

TL;DR

Controlled input = `value={state}` + `onChange={e => setState(e.target.value)}` (`checked`/`e.target.checked` for checkboxes). Initialise with `''`. Gives formatting, limits and live UI.

3.10 State is tied to a position in the tree

What it is

React stores state **not inside your component function** but in its tree, attached to a **position** (the path of element types from the root):

Situation	State
Same component type at the same position on the next render	**Kept** (even if props differ)
Different type at that position, or the component is removed	**Destroyed**; a new instance starts from initial state
Same type and position but a **different** `key`	**Destroyed and recreated**

How it works

`code/10_state_and_position.jsx`:

```
function Counter({ person }) {
  const [score, setScore] = useState(0);
  return <p>{person}'s score: {score} <button onClick={() => setScore(score + 1)}>+1 {person}</button></p>;
}

function Scoreboard({ mode }) {
  const [isTaylor, setIsTaylor] = useState(true);
  let counter;
  if (mode === 'same-position') counter = isTaylor ? <Counter person="Taylor" /> : <Counter person="Sarah" />;
  if (mode === 'with-key') counter = isTaylor ? <Counter key="Taylor" person="Taylor" /> : <Counter key="Sarah"
person="Sarah" />;
  return <div>{counter}<button onClick={() => setIsTaylor(!isTaylor)}>Next player</button></div>;
}
// Taylor scores 2, switch to Sarah, switch back
```

Output:

```
same-position: Taylor 2 -> switch -> Sarah's score: 2  -> switch back -> Taylor's score: 2
with-key     : Taylor 2 -> switch -> Sarah's score: 0  -> switch back -> Taylor's score: 0
```

- **Without keys** both branches are `<Counter>` at one position, so React sees *the same component with a new* `person` *prop* — Sarah "inherits" Taylor's score. Usually a bug.
- **With keys** they are different identities: switching destroys one and creates the other fresh. (To *remember* each score while hidden, lift scores into the parent, §3.7.)

Resetting with `key` in practice: `<Chat key={contact.id} contact={contact} />` clears the draft when switching contact (instead of sending your text to the wrong person); `<EditProfile key={userId} />` resets all fields for a different user — no Effect needed.

Common mistakes

- Defining a component inside another (new type every render → state reset every render; §1.7).
- Expecting different props to reset state — they don't; use a `key`.

TL;DR

State lives at a tree position. Same type + same position = kept; different type or different key = reset. Use `key` to reset on purpose.

3.11 Putting it together — a to-do app

`code/todo.jsx` uses every idea: a controlled input with `preventDefault` on submit, custom event props (`onAdd`, `onToggle`, `onDelete`), state lifted into `TodoApp`, immutable updates with updater functions, and derived `visible`/`remaining`. The core (full file in `code/todo.jsx`):

```jsx
function AddTodo({ onAdd }) {
  const [text, setText] = useState('');                    // controlled input
  function handleSubmit(e) {
    e.preventDefault();                                    // no page reload
    const title = text.trim();
    if (!title) return;
    onAdd(title);                                          // tell the parent
    setText('');
  }
  // <form onSubmit={handleSubmit}> <input value={text} onChange=…/> <button disabled={!text.trim()}>Add</button>
}
// TodoItem: checkbox calls onToggle(todo.id), ✕ button calls onDelete(todo.id)
// FilterBar: one button per filter with aria-pressed

const FILTERS = { all: () => true, active: (t) => !t.done, done: (t) => t.done };

export default function TodoApp({ initialTodos = [] }) {
  const [todos, setTodos] = useState(initialTodos);        // lifted state: shared by all children
  const [filter, setFilter] = useState('all');

  const visible = todos.filter(FILTERS[filter]);           // derived, not stored
  const remaining = todos.filter((t) => !t.done).length;   // derived, not stored

  const add = (title) => setTodos((ts) => [...ts, { id: nextId++, title, done: false }]);
  const toggle = (id) => setTodos((ts) => ts.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = (id) => setTodos((ts) => ts.filter((t) => t.id !== id));
  const clearDone = () => setTodos((ts) => ts.filter((t) => !t.done));

  return (
    <div className="todo">
      <h1>Today</h1>
      <AddTodo onAdd={add} />
      <FilterBar filter={filter} onChange={setFilter} />
      {visible.length === 0 ? <p className="empty">Nothing here.</p> : (
        <ul>{visible.map((t) => <TodoItem key={t.id} todo={t} onToggle={toggle} onDelete={remove} />)}</ul>
      )}
      <footer>
        <span>{remaining} {remaining === 1 ? 'task' : 'tasks'} left</span>
        {todos.length > remaining && <button onClick={clearDone}>Clear completed</button>}
      </footer>
    </div>
  );
}
```

`code/11_todo_app.jsx` drives it like a real user with `user-event` (e.g. `await user.type(screen.getByLabelText('New task'), 'Build the todo app{Enter}')`, then clicking checkboxes and filter buttons). Output (label | visible items | footer):

```
start                     | [x] Read chapter 3                                                    | 0 tasks left
added two                 | [x] Read chapter 3, [ ] Build the todo app, [ ] Explain batching | 2 tasks left
input cleared after add?   : true
ticked "Build the todo app" | [x] Read chapter 3, [x] Build the todo app, [ ] Explain batching | 1 task left
filter = active           | [ ] Explain batching                                                  | 1 task left
filter = done             | [x] Read chapter 3, [x] Build the todo app                            | 1 task left
clear completed           | Nothing here.                                                         | 1 task left
filter = all, delete last | Nothing here.                                                         | 0 tasks left
```

In the browser (`code/todo.html`, styles in `todo.css`):

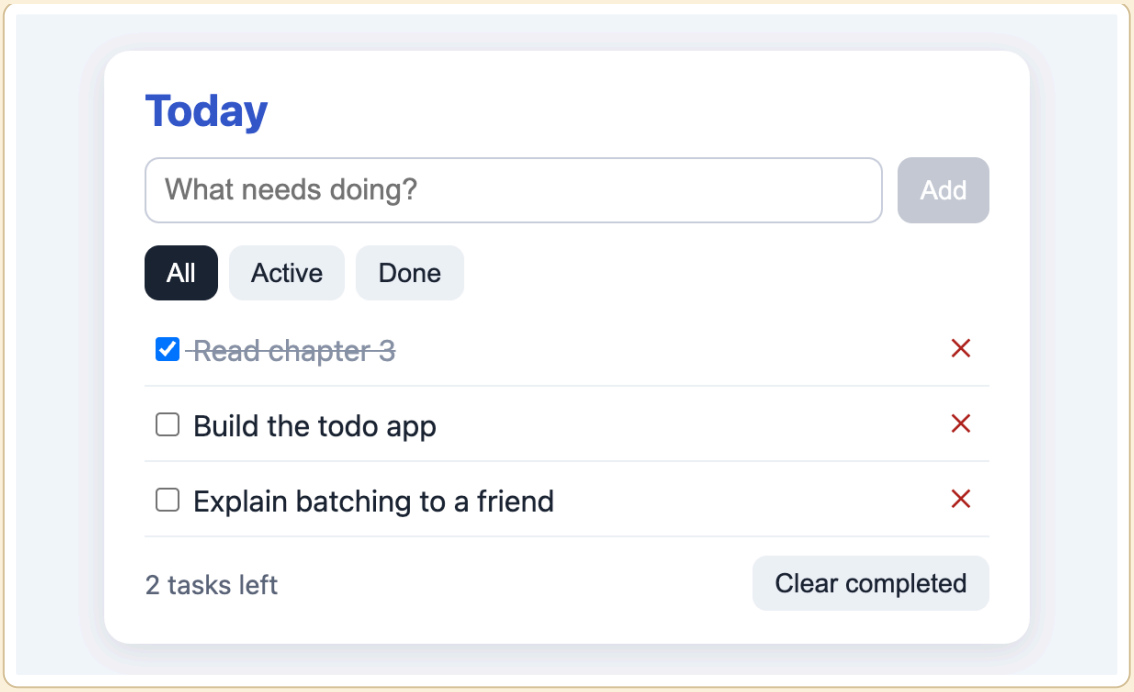


Figure 3.1 — `TodoApp` in Chrome. The struck-through item, the "2 tasks left" count and the "Clear completed" button are all derived from the one `todos` array.

Design notes:

- **One owner**: only `TodoApp` holds `todos`; children get data + callbacks.
- **Minimal state**: `todos` and `filter`; `visible`, `remaining` and "Clear completed" visibility are derived.
- **Updater functions** make `add`/`toggle`/`remove` safe even if called several times in one event.
- **Accessible queries**: the test finds elements by label and role, as screen-reader users do (Chapter 12).

When it grows (editing, undo, **persistence**), update logic moves into a **reducer** (§6.2) and saving to `localStorage` needs an **Effect** (Chapter 5).

Chapter 3 TL;DR

Idea	Remember
Events	`onClick={handle}` — pass, don't call; `e.stopPropagation()`, `e.preventDefault()`; custom `onX` props send events up
`useState`	`[value, setValue]`; persists between renders, per instance; top-level calls only
Snapshot	Inside a render state is constant; setters request a new render
Batching	All updates in one event → one render (everywhere since React 18)
Updaters	`setX(prev => …)` to build on queued updates
Immutability	New objects/arrays: spread, `map`, `filter`, `toSorted`, `with`; Immer for deep updates
Lifting state	Shared state goes to the closest common parent; one owner
Derived state	Compute, don't store; store IDs, don't mirror props
Controlled inputs	`value` + `onChange` (`checked` for checkboxes); start with `''`
Position & key	Same type + position keeps state; a new `key` resets it

Next: Chapter 4 — Forms and Actions.

Chapter 4 — Forms and Actions

§3.9 showed the basic **controlled input**. This chapter covers the whole topic: the two ways to own a field's value, helpful and accessible validation, and React 19's **Actions** — built-in form submission that tracks "pending" (request in flight), keeps the last result, resets the form, and supports **optimistic updates** (showing the expected result before the server confirms), with far less code than `onSubmit` + several `useState`s.

Code: `code/` — React 19.3 in jsdom, driven by `@testing-library/user-event`. Notes for reusing the tests: each program sets `globalThis.FormData = window.FormData` (Node's own `FormData` cannot read a `<form>` element), and after React resets a form, `user.clear()` is called before typing again (user-event keeps its own copy of the field's text and doesn't notice the reset).

4.1 How a form submission works

What it is

A **form** (`<form>`) groups input fields. On a submit-button click or Enter in a text field, the browser fires a **`submit` event** and then, by default, *navigates*: it sends the field values to the URL in the form's `action` attribute and loads a new page. (HTML side — input types, labels, fieldsets: `../../../html-css/chapters/05-tables-and-forms/README.md`.)

Only fields with a `name` are sent, as **name/value pairs**, exposed in JavaScript as `FormData`:

`FormData` method	Returns
`fd.get('email')`	The first value for that name (a string, or a `File` for file inputs), or `null`
`fd.getAll('topic')`	Every value for that name — several checkboxes sharing one name
`Object.fromEntries(fd)`	A plain object `{ name: value }` (keeps only the last value of repeated names)
`new FormData(formElement)`	Builds the pairs from a form's current field values

Memorise: an **unchecked** checkbox is **not included at all** (not `false`); a checked checkbox without `value` sends `"on"`; every value is a **string**; disabled fields are skipped.

In a single-page app

An SPA (§1.4) must not reload, or all React state is lost:

Classic (any React version)	React 19 Actions
`<form onSubmit={e => { e.preventDefault(); … }}>`	`<form action={fn}>`
You call `preventDefault()` and read values yourself	React prevents the navigation and calls `fn(formData)`
You track "submitting" and errors with `useState`	Pending state, result and reset are built in (§4.4–§4.7)

Both appear in real code bases.

TL;DR

Fields need `name` to be submitted. `FormData` reads them; unchecked boxes are missing and all values are strings. In React either `preventDefault()` in `onSubmit` or pass a function to `action`.

4.2 Controlled vs uncontrolled inputs

What it is

The question is **who owns** a field's current value:

- **Controlled** — React state owns it: `value={state}` (or `checked`) plus a state update in `onChange`. The DOM always shows exactly what state says.
- **Uncontrolled** — the DOM owns it, as in plain HTML. Optionally give a **starting** value with `defaultValue` (`defaultChecked`) and read the value **when needed**: from `FormData` on submit, or through a **ref** (a handle to the DOM element — §5.9).

How it works

`code/01_controlled_vs_uncontrolled.jsx`:

```
// 1. Uncontrolled: the DOM keeps the value. defaultValue/defaultChecked set only the START value.
function UncontrolledSignup() {
  renders++;
  function handleSubmit(e) {
    e.preventDefault();
    const data = new FormData(e.currentTarget);          // read every named field at once
    console.log('  submit ->', JSON.stringify(Object.fromEntries(data)));
  }
  return (
    <form onSubmit={handleSubmit}>
      <input name="city" aria-label="City" defaultValue="Oslo" />
      <select name="size" aria-label="Size" defaultValue="m">…</select>
      <label><input type="checkbox" name="gift" defaultChecked /> Gift wrap</label>
      <button>Order</button>
    </form>
  );
}

// 2. Uncontrolled + ref: read one field on demand without re-rendering.
function SearchBox() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} aria-label="Search" />
      <button onClick={() => console.log('  ref reads:', inputRef.current.value)}>Go</button>
    </>
  );
}

// 3. Controlled: React state owns the value, so every keystroke re-renders.
function ControlledCity() {
  const [city, setCity] = useState('Oslo');
  renders++;
  return <input aria-label="Controlled city" value={city} onChange={(e) => setCity(e.target.value.toUpperCase())} />;
}

// 4. The classic warning: value starts undefined (uncontrolled) and later becomes a string (controlled).
function Flip() {
  const [v, setV] = useState();                          // undefined!
  return <input aria-label="Flip" value={v} onChange={(e) => setV(e.target.value)} />;
}
// user: replaces "Oslo" with "Bergen", picks L, unticks Gift wrap, clicks Order; types "hooks" + Go;
// types "ville" into the controlled field; types "a" into Flip
```

Output:

```
  submit -> {"city":"Bergen","size":"l"}
uncontrolled renders while typing 6 letters: 1
  ref reads: hooks
controlled shows: OSLOVILLE | renders: 6
warning: A component is changing an uncontrolled input to be controlled. This is likely caused by t...
```

1. **Uncontrolled forms render once** — the browser handled typing; `FormData` collected everything at submit.
2. **The unticked checkbox vanished** (`gift` missing), as §4.1 said.
3. **A ref** reads one field on demand.
4. **Controlled inputs re-render per keystroke** (1 + 5 = 6) — the price of transforming text (upper-casing) as typed.
5. `value` going from `undefined` to a string triggers the mode-switch warning; initialise controlled state with `''`.

`defaultValue` is only a starting value

It is read when the element is created; afterwards the value belongs to the user. So:

- To **reset** an uncontrolled field to new defaults, **remount** it with a new `key` (§3.10) or call `form.reset()` (restores the *default* values).
- `<select>` **ignores a changed** `defaultValue` **after mount** (first appearance); §4.10 uses `key` to remount it.

`<input type="file">` is **always uncontrolled** — for security, JavaScript cannot set the chosen file. Read `input.files` or `formData.get('avatar')` (a `File`).

Trade-offs

	Controlled	Uncontrolled
Who owns the value	React state	The DOM
Re-renders	Every keystroke	None while typing
Instant formatting / limits (`toUpperCase`, max length)	Easy	Awkward
Live dependent UI (character counter, enable button)	Easy	Needs extra events
Code for a big form	One state + handler per field (or one object)	Just `name` attributes
Works with React 19 form Actions and auto-reset	Values come from state, not reset	Natural fit
Prefill / reset from data	Set state	`defaultValue` + `key` or `form.reset()`

When: uncontrolled + `FormData` (usually with a form Action) for most fill-and-submit forms (sign-up, contact, settings); controlled when the UI must react to every keystroke — live search, input masks (credit-card spacing), counters, interdependent fields, a **wizard** (multi-step form) keeping values across steps. Mixing is fine.

Common mistakes

Mistake	Fix
`value` without `onChange` (read-only field)	Add `onChange`, or use `defaultValue`
`useState()` / `useState(null)` for a text field	`useState('')`
Changing `defaultValue` and expecting the field to update	Remount with `key`, or make it controlled
Forgetting `name` on uncontrolled fields	`FormData` silently skips them
Treating `formData.get('age')` as a number	`Number(formData.get('age'))`

In the interview

30-second answer: "A controlled input's value lives in React state — `value` plus `onChange` — so React can validate or format on every keystroke, at the cost of a render per keystroke. An uncontrolled input keeps its value in the DOM; you set a starting `defaultValue` and read it on submit with `FormData` or a ref. File inputs are always uncontrolled. In React 19 I default to uncontrolled fields with a form Action and use controlled fields where the UI must react while typing."

- *What causes 'changing an uncontrolled input to be controlled'?* — `value` was `undefined`/`null` on one render and a string later.
- *How do you reset an uncontrolled form?* — `form.reset()` (a successful form Action does it automatically) or change the form's `key`.

TL;DR

Controlled = React owns the value (`value` + `onChange`), re-renders per keystroke, good for live behaviour. Uncontrolled = DOM owns it (`defaultValue`, read with `FormData`/ref), less code, pairs with Actions. File inputs are always uncontrolled.

4.3 Validation

What it is

Validation checks input before you act on it (required fields, email shape, password length, matching fields). Good validation says **what** is wrong, **where**, and **how to fix it**, at a moment that helps rather than nags. Real apps use three layers:

Layer	Where it runs	Purpose
Built-in HTML constraints (`required`, `type="email"`, `minLength`, `pattern` — a regex the value must match, `min`/`max`)	Browser	Free, instant, works without JavaScript
Client-side rules in React	Browser	Custom messages, cross-field rules ("passwords match"), consistent styling
Server-side validation	Server	The only one you can **trust** — anyone can bypass the browser

Client validation is for **user experience**; server validation is for **correctness and security**. Never skip the server.

How built-in validation works

If any constraint fails, the browser blocks the submit and shows its own bubble. Each field exposes a `validity` object (`valueMissing`, `typeMismatch`, `tooShort`, `patternMismatch`, `rangeOverflow`, …, summary `valid`) and `validationMessage`. `noValidate` on `<form>` turns off the blocking and bubbles so your UI takes over — `validity` still works.

How custom validation works

1. Write **one pure** `validate(values)` returning an `errors` object (`{ email: 'Enter a valid email' }`) — easy to test and reuse on the server.
2. Compute `errors` **during render** — derived state (§3.8), never stored.
3. Decide **when to show** errors. "Email is invalid" after one keystroke is hostile; the common rule: show once the user has **left** the field (**blur**, tracked in a `touched` object) or after the first **submit**, then update live as they fix it.
4. On a failed submit, **move focus to the first invalid field** so keyboard and screen-reader users land on the problem.
5. Make errors **accessible** with **ARIA** (Accessible Rich Internet Applications — attributes telling assistive technology what an element is or its state): `aria-invalid="true"` on the field, the message linked with `aria-describedby`, `role="alert"` so it is announced (`../../../html-css/chapters/03-semantic-html-and-accessibility/README.md`).

`code/02_validation.jsx`:

```jsx
// 1. Built-in constraint validation: the browser blocks submit while a field is invalid.
function Native() {
  return (
    <form onSubmit={(e) => { e.preventDefault(); console.log('  native form submitted'); }}>
      <input aria-label="Nick" name="nick" required minLength={3} pattern="[a-z]+" />
      <button>Save</button>
    </form>
  );
}

// 2. Custom validation in React: one pure validate() function, errors shown after blur or submit.
function validate({ email, password, confirm }) {
  const errors = {};
  if (!email) errors.email = 'Email is required';
  else if (!/^\S+@\S+\.\S+$/.test(email)) errors.email = 'Enter a valid email';
  if (password.length < 8) errors.password = 'Use at least 8 characters';
  if (confirm !== password) errors.confirm = 'Passwords do not match';   // cross-field rule
  return errors;
}

function Field({ name, label, type = 'text', value, error, show, onChange, onBlur }) {
  const errorId = `${name}-error`;
  return (
    <div>
      <label htmlFor={name}>{label}</label>
      <input id={name} name={name} type={type} value={value} onChange={onChange} onBlur={onBlur}
        aria-invalid={show && !!error} aria-describedby={show && error ? errorId : undefined} />
      {show && error && <p id={errorId} role="alert">{error}</p>}
    </div>
  );
}

function SignupForm() {
  const [values, setValues] = useState({ email: '', password: '', confirm: '' });
  const [touched, setTouched] = useState({});          // which fields the user has left
  const [submitted, setSubmitted] = useState(false);
  const errors = validate(values);                      // derived — never stored
  const show = (name) => submitted || touched[name];

  const handleChange = (e) => setValues({ ...values, [e.target.name]: e.target.value });
  const handleBlur = (e) => setTouched({ ...touched, [e.target.name]: true });
  function handleSubmit(e) {
    e.preventDefault();
    setSubmitted(true);
    const first = Object.keys(errors)[0];
    if (first) { document.getElementById(first).focus(); return; }   // move focus to the first problem
    console.log('  sending', values.email);
  }
  // ... <form noValidate onSubmit={handleSubmit}> with three <Field>s and a button
}
```

Output:

```
native: "Al" blocked | valid = false | patternMismatch = true
  native form submitted
while typing (not yet touched): []
after leaving Email           : [ 'Enter a valid email' ]
after submit                  : [ 'Enter a valid email', 'Use at least 8 characters' ]
focused field                 : email | aria-invalid = true
after fixing                  : []
  sending ada@example.com
```

- **Native**: "Al" broke `pattern="[a-z]+"` (capital A), so submit was blocked; "alan" went through.
- **Custom**: nothing shown while typing; leaving Email revealed its error; submit revealed *all* errors and focused the first; fixing cleared them live.
- One `handleChange` serves every field via the **computed property name** `[e.target.name]`.

Trade-offs — when to show errors

Strategy	Feels	Use for
On every change from the start	Nagging ("invalid" after one letter)	Only positive feedback like password-strength meters
On blur (touched)	Balanced	Most fields
On submit only	Calm but late	Short forms (login)
On blur, then live while fixing	Best of both	Default recommendation
Async (server) checks such as "username taken"	Needs debouncing (wait until typing pauses) and pending UI	Uniqueness checks — or return the error from the submit Action (§4.5)

Common mistakes

- Trusting client validation — attackers call your API directly. Validate on the server too.
- Storing `errors` in state and syncing it in an Effect — compute it (§5.6).
- Errors shown only by colour — also use text, `aria-invalid`, `aria-describedby`.
- Disabling submit until valid — users can't discover *why*. Allow submit and show errors.
- Over-strict regexes for emails and names; keep client checks simple and let the server (or a confirmation email) decide.

In the interview

30-second answer: "I layer validation: HTML constraints for free basics, a pure `validate` function in React for custom and cross-field rules, and always server-side validation because the client can be bypassed. Errors are derived from values, shown after blur or first submit, linked with `aria-describedby`, flagged with `aria-invalid`, and on a failed submit I focus the first invalid field. For big forms I'd use a schema library such as Zod so the same rules run on both sides." (A **schema** describes what valid data looks like — §4.9.)

- *What does `noValidate` do?* — Turns off the browser's blocking and bubbles; `validity` still works.
- *Why a pure `validate`?* — Testable, reusable on the server, and errors become derived state.

TL;DR

HTML constraints + a pure `validate()` + server checks. Derive errors; show them on blur or submit; announce and link them accessibly; focus the first invalid field.

4.4 Form Actions — `<form action={fn}>`

What it is

In React 19 `<form action>` accepts a **function**. On submit React prevents navigation, builds `FormData` from the form and calls your function with it. A function run inside a **Transition** (a non-urgent update React tracks as "pending" — §7.10) is called an **Action**; name them `…Action` or by what they do (`subscribe`, `saveNote`).

`formAction={fn}` on a `<button>` (or `<input type="submit">`) overrides the form's action for that button — e.g. "Publish" vs "Save draft".

Why it matters

- No `e.preventDefault()`, no reading fields by hand — you get `FormData`.
- The function may be **async** (returns a **promise**); React tracks it as pending, powering `useFormStatus` (§4.6), `useActionState`'s `isPending` (§4.5) and `useOptimistic` (§4.7).
- After the Action **finishes successfully**, React **resets the form**: uncontrolled fields return to their `defaultValue`s — a fresh form after "Send".
- With Server Functions (§4.8) the form can work **before JavaScript loads**.

How it works

1. User submits; React calls `preventDefault()`, creates `new FormData(form, submitter)` and starts a Transition.
2. Your function runs; while its promise is pending, the form is "pending".
3. On resolve, React commits state updates and **resets** uncontrolled fields.
4. If it **throws** (or rejects), the error goes to the nearest **error boundary** (catches crashes below it, shows a fallback; Chapter 14) — so catch expected errors and turn them into state (§4.5).

`code/03_form_action.jsx`:

```
function Newsletter() {
  const [log, setLog] = useState([]);
  async function subscribe(formData) {            // no event, no preventDefault
    const email = formData.get('email');
    const topics = formData.getAll('topic');       // several checkboxes with the same name
    await wait(50);                                // pretend network call
    setLog((l) => [...l, `${email} -> ${topics.join('+') || 'none'}`]);
  }
  function saveDraft(formData) {                   // a second button can run a different action
    setLog((l) => [...l, `draft saved for ${formData.get('email')}`]);
  }
  return (
    <form action={subscribe}>
      <input name="email" type="email" aria-label="Email" />
      <label><input type="checkbox" name="topic" value="react" /> React</label>
      <label><input type="checkbox" name="topic" value="css" /> CSS</label>
      <button type="submit">Subscribe</button>
      <button formAction={saveDraft}>Save draft</button>
      <ul>{log.map((line) => <li key={line}>{line}</li>)}</ul>
    </form>
  );
}
// user: types ada@example.com, ticks React and CSS, clicks Subscribe; later types bo@example.com, clicks Save draft
```

Output:

```
right after click, email field = "ada@example.com"
after the action finished:
  log          = [ 'ada@example.com -> react+css' ]
  email field  = "" (auto-reset)
  React ticked = false
  log          = [ 'ada@example.com -> react+css', 'draft saved for bo@example.com' ]
```

The field kept the email while the Action ran and was reset **after** it finished (boxes unticked too); `log` state was *not* reset — the reset touches only uncontrolled DOM values. `getAll('topic')` collected both boxes; "Save draft" ran `saveDraft` via `formAction`.

Details worth knowing

Detail	Behaviour
HTTP method (`GET`/`POST`)	Irrelevant with a function action — the browser sends nothing; React calls your function
Controlled fields	Not reset (value comes from state) — clear the state yourself
Keep values after an error	Return them from the Action and use as `defaultValue` (§4.5), or use controlled fields
Reset by hand	`requestFormReset(formElement)` from `react-dom`, called inside a Transition/Action
Mixing with `onSubmit`	If `onSubmit` calls `preventDefault()`, the function action does not run — use one or the other
Where it runs	In the browser. Only a Server Function (`'use server'`, §4.8) runs on the server

Trade-offs

`onSubmit` + `preventDefault`	`action={fn}`
Works in every React version	React 19+
Full access to the event	Receives `FormData` only
Pending/error/reset are manual	Pending state, reset, optimistic UI, progressive enhancement built in
Natural with controlled fields	Natural with uncontrolled fields

Common mistakes

- Losing the user's input after a failed validation because the form reset → return the values (§4.5) or keep them in state.
- Letting expected errors (validation, "email taken") throw — shows the error boundary. Return them.
- Forgetting `name` attributes — `FormData` is empty.

In the interview

30-second answer: "In React 19 `<form action>` accepts a function. React prevents the navigation, passes `FormData`, and runs it inside a Transition, so it can be async and its pending state is visible to `useFormStatus` and `useActionState`. After a successful Action, uncontrolled fields are reset. `formAction` on a button overrides the action per button. Uncaught errors go to an error boundary."

- *What's an "Action" in React 19?* — A function run inside a Transition, e.g. passed to `action`/`formAction` or called in `startTransition`.
- *Does the form reset if the Action throws?* — No; the reset follows successful completion; errors go to the error boundary.

TL;DR

`<form action={fn}>` → `fn(formData)` inside a Transition; async allowed; auto-reset of uncontrolled fields after success; `formAction` per button; return expected errors, don't throw them.

4.5 `useActionState` — the result of the last submission

What it is

Forms usually show **what happened last time** (validation errors, "Saved!", a server error). `useActionState` wraps your Action and stores its **return value** as state:

```
const [state, formAction, isPending] = useActionState(actionFn, initialState, permalink?);
//      last result  pass to action=   true while running
// actionFn(previousState, payload) -> new state (sync or async)
```

Piece	Meaning
`actionFn(prevState, formData)`	Your function. First argument is the **previous state**, then the form data. Its return value becomes the new `state`
`initialState`	The state before the first submission
`state`	The latest return value
`formAction`	A wrapped function for `<form action>` / `formAction` (or to call yourself)
`isPending`	`true` while the Action runs
`permalink` (optional)	For Server Functions: the URL the form posts to if JavaScript hasn't loaded

During the React 19 **canary** period (pre-release builds) it was `useFormState` in `react-dom`; the stable API is `useActionState` from `react`.

How it works

`code/04_use_action_state.jsx`:

```
// Pretend server call: validates, may fail, returns a result object.
async function registerUser(prevState, formData) {
  const username = String(formData.get('username') ?? '').trim();
  const age = Number(formData.get('age'));
  await wait(30);
  const errors = {};
  if (username.length < 3) errors.username = 'At least 3 characters';
  else if (taken.has(username)) errors.username = `"${username}" is taken`;
  if (!(age >= 13)) errors.age = 'You must be 13 or older';
  if (Object.keys(errors).length) return { errors, values: { username, age: formData.get('age') }, attempts:
prevState.attempts + 1 };
  taken.add(username);
  return { message: `Welcome, ${username}!`, errors: {}, values: { username: '', age: '' }, attempts: prevState.attempts
+ 1 };
}

const initialState = { errors: {}, values: { username: '', age: '' }, attempts: 0 };

function Register() {
  const [state, formAction, isPending] = useActionState(registerUser, initialState);
  return (
    <form action={formAction}>
      {/* defaultValue from the last result: after the automatic reset the fields show what the user typed */}
      <input name="username" aria-label="Username" defaultValue={state.values.username} />
      {state.errors.username && <p role="alert">{state.errors.username}</p>}
      <input name="age" aria-label="Age" inputMode="numeric" defaultValue={state.values.age} />
      {state.errors.age && <p role="alert">{state.errors.age}</p>}
      <button disabled={isPending}>{isPending ? 'Registering…' : 'Register'}</button>
      {state.message && <p role="status">{state.message}</p>}
      <small>attempts: {state.attempts}</small>
    </form>
  );
}

// Calling the dispatch function yourself: wrap it in startTransition. Calls are queued in order.
function Stock() {
  const [count, dispatch, isPending] = useActionState(async (prev, delta) => { await wait(20); return prev + delta; },
0);
  return (
    <p>
      <button onClick={() => startTransition(() => dispatch(+1))}>+1</button>
      <output>{count}</output>{isPending && ' (saving…)'}
    </p>
  );
}
```

Output:

```
pending   : button = "Registering…" | disabled = true
attempt 1 : [ '"ada" is taken', 'You must be 13 or older' ] | fields kept: ada 9
attempt 2 : [] | Welcome, grace! | fields: "" "" | attempts: 2

three quick clicks -> shows +10 (saving…)
after the queue ran  -> shows +13
```

1. **Pending**: while `registerUser` awaited, the button read "Registering…" and was disabled (no double submits).
2. **Attempt 1** returned errors *and the submitted values*. The form reset, but since `defaultValue` came from `state.values`, the reset restored the user's text — the standard "don't wipe the form on a validation error" pattern for uncontrolled fields.
3. **Attempt 2** succeeded and returned empty values → clean form + message; `prevState.attempts + 1` shows the previous state passed in.
4. **Manual dispatch**: three clicks queued three calls run **one after another**, each receiving the previous result (0→1→2→3). Dispatching outside a Transition logs a dev error and `isPending` doesn't update — wrap in `startTransition` (an `action` prop does it for you).

Rules and caveats

- The action receives `(prevState, payload)` — a common bug is `async (formData) => …`, which receives the previous state as "formData".
- **Queued and sequential**: dispatches never run in parallel — correct, but a slow request delays the next.
- **No built-in reset** of the state: return `initialState` from the action, or remount with a new `key`.
- **Throwing** cancels queued actions and shows the nearest error boundary — return errors instead.
- Setting *other* state after an `await` inside the action currently needs `startTransition` (§4.7).
- With Server Functions, `initialState` and the payload must be **serializable** (plain data that can be sent as text and rebuilt).

`useActionState` vs doing it yourself

Manual (`onSubmit`)	`useActionState`
`useState` for errors, message, isSubmitting	One Hook: `[state, action, isPending]`
`try/finally` to clear `isSubmitting`	Pending is automatic
Double submit possible unless guarded	Calls are queued; button can use `isPending`
Works with any event	Designed for Actions (forms, buttons, `startTransition`)

It is a **reducer for async work**: `(prevState, action) => newState`, like `useReducer` (§6.2) but allowed to be async and have side effects.

In the interview

30-second answer: "`useActionState(fn, initialState)` returns the latest result of an Action, a wrapped action to pass to a form, and `isPending`. The function gets the previous state and the form data and returns the next state, so it's the natural place to return validation errors or a success message. Calls are queued sequentially; when dispatching manually you wrap it in `startTransition`. With uncontrolled fields I return the submitted values and use them as `defaultValue`s so a failed submit doesn't wipe the form."

- *First argument of the action?* — The previous state; form data is second.
- *Where do server validation errors go?* — Returned from the action → `state.errors`.
- *How do you reset the state?* — Return `initialState` or remount with a `key`.

TL;DR

`const [state, action, isPending] = useActionState(async (prev, formData) => next, initial)`. Return errors/results, don't throw. Sequential queue. Manual calls go in `startTransition`.

4.6 `useFormStatus` — status of the parent form

What it is

`useFormStatus()` (from `react-dom`) lets any component **rendered inside a `<form>`** read that form's current submission without props:

Field	Meaning
`pending`	`true` while the parent form's Action is running
`data`	The `FormData` being submitted (or `null`)
`method`	The form's `method` attribute: `'get'` (default) or `'post'`
`action`	The function passed to the parent's `action` (or `null`)

How it works

`code/05_use_form_status.jsx`:

```jsx
// Reusable: works in any form, because it asks its parent <form> for the status.
function SubmitButton({ children }) {
  const { pending, data, method } = useFormStatus();
  return (
    <button disabled={pending}>
      {pending ? `Saving "${data.get('title')}" (${method})…` : children}
    </button>
  );
}

// Wrong place: the Hook is in the SAME component that renders the <form>, so it sees no parent form.
function WrongPlace() {
  const { pending } = useFormStatus();
  return (
    <form action={() => wait(30)}>
      <input name="title" aria-label="Title 2" defaultValue="x" />
      <button>{pending ? 'pending' : 'never pending'}</button>
    </form>
  );
}

async function saveNote(formData) { await wait(30); console.log('  saved:', formData.get('title')); }

function NoteForm() {
  return (
    <form action={saveNote}>
      <input name="title" aria-label="Title" />
      <SubmitButton>Save note</SubmitButton>
    </form>
  );
}
```

Output:

```
during  : "Saving \"Groceries\" (get)…" disabled = true | "never pending"
  saved: Groceries
after   : "Save note" disabled = false
```

`SubmitButton` saw pending and the submitted title via `data`; `method` just mirrors the form attribute (default `get` — no HTTP request is made with a function action). `WrongPlace` never saw `pending`: the Hook must be in a **child component of the form**.

`useFormStatus` vs `useActionState`'s `isPending`

`useFormStatus`	`isPending` from `useActionState`
Read by any **descendant** of the form	Available in the component that owns the action
Great for reusable design-system buttons/spinners	Great when the owner needs it (disable the whole form)
Also exposes the submitted `data`	Also gives the action's result `state`

Common mistakes

- Calling it in the same component as the `<form>` (always `pending: false`).
- Importing it from `react` — it's in **`react-dom`**.
- Expecting it to track sibling or nested forms — only the nearest **parent** `<form>`.

TL;DR

`const { pending, data } = useFormStatus()` (from `react-dom`) inside a child of the form. Perfect for a reusable Submit button.

4.7 `useOptimistic` — instant feedback, automatic rollback

What it is

An **optimistic update** shows the result you *expect* before the server confirms (a chat message appears immediately, a like count goes up); on failure you undo it. Most requests succeed, so the app feels fast.

```
const [optimisticState, addOptimistic] = useOptimistic(value, reducer?);
// value: the real (confirmed) state, e.g. from useState or props
// reducer(currentState, optimisticValue) -> the state to show while an Action runs (pure)
```

- With **no Action running**, `optimisticState` equals `value`.
- `addOptimistic(x)` **inside an Action** (form action or `startTransition`) immediately shows `reducer(value, x)` (or `x` with no reducer).
- When the Action **ends**, the optimistic value is dropped and `optimisticState` is `value` again — including the real result if you updated it, or the old data if it failed. **Rollback is automatic.**

How it works

`code/06_use_optimistic.jsx`:

```
function Chat() {
  const [messages, setMessages] = useState([{ id: 'm1', text: 'Hi!' }]);
  const [error, setError] = useState(null);
  // optimisticMessages === messages when no Action is running
  const [optimisticMessages, addOptimistic] = useOptimistic(
    messages,
    (current, text) => [...current, { id: 'temp-' + text, text, sending: true }],   // pure reducer
  );

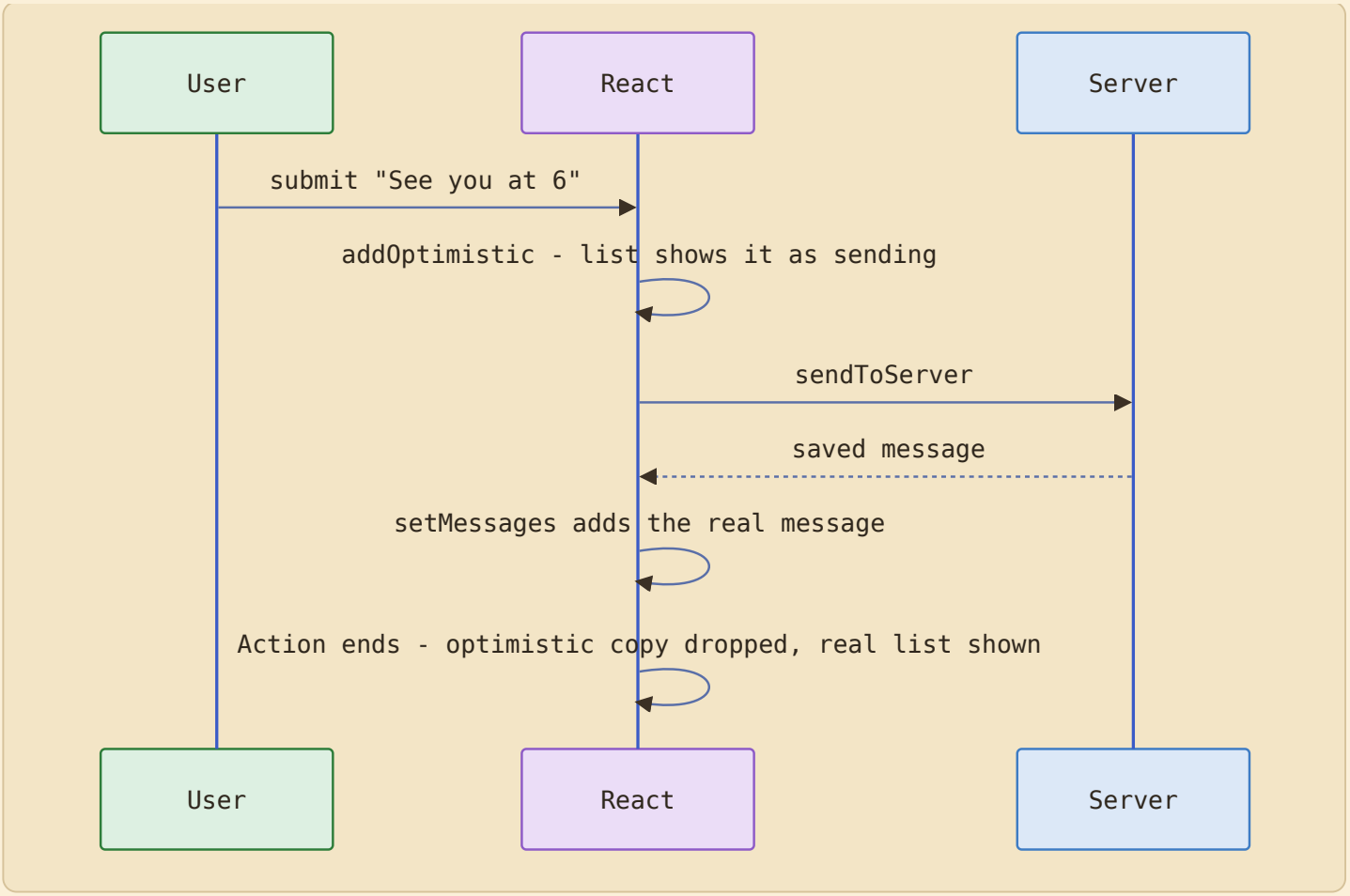
  async function sendAction(formData) {               // runs inside a Transition (form action)
    const text = formData.get('text');
    setError(null);
    addOptimistic(text);                              // 1. appears instantly, marked "sending"
    try {
      const saved = await sendToServer(text);         // 2. wait for the server
      startTransition(() => setMessages((ms) => [...ms, saved]));  // 3. real state (after await -> wrap)
    } catch (e) {
      setError(e.message);                            // optimistic item disappears by itself
    }
  }

  return (
    <>
      <ul>
        {optimisticMessages.map((m) => <li key={m.id}>{m.text}{m.sending && ' (sending…)'}</li>)}
      </ul>
      {error && <p role="alert">{error}</p>}
      <form action={sendAction}>
        <input name="text" aria-label="Message" />
        <button>Send</button>
      </form>
    </>
  );
}
```

Output:

```
instantly      : [ 'Hi!', 'See you at 6 (sending…)' ]
server said OK : [ 'Hi!', 'See you at 6' ]
instantly      : [ 'Hi!', 'See you at 6', 'buy spam now (sending…)' ]
server said NO : [ 'Hi!', 'See you at 6' ]
```

(the second run also shows the alert "Message rejected")



- The optimistic item had a temporary id and a `sending` flag so the UI could grey it out.
- On failure no "undo" code ran — `messages` never changed, so the optimistic line vanished; `catch` only showed a message.
- `setMessages` after `await` is wrapped in `startTransition` (a current React requirement for updates after `await` in an Action) so it lands with the end of the Action.

Outside forms

In a click handler, start the Action yourself:

```
// (not run separately — same rules as above)
function LikeButton({ likes, onSave }) {
  const [optimisticLikes, setOptimisticLikes] = useOptimistic(likes);
  return (
    <button onClick={() => startTransition(async () => {
      setOptimisticLikes((n) => n + 1);   // must be inside startTransition / an Action
      await onSave();                     // parent updates `likes` when the server confirms
    })}>♥ {optimisticLikes}</button>
  );
}
```

Calling the setter outside an Action gives a dev warning — no pending Action keeps the optimistic value alive.

When not to use it: when failure is common or costly to take back (payments, bookings, irreversible actions) — show a real pending state; or when the server's result is hard to predict (generated IDs are fine, complex server calculations are not).

Common mistakes

- Treating optimistic state as the source of truth — it's temporary; always update the real state (`setMessages`, refetch, or parent props).
- Side effects in the reducer — it must be pure (it re-runs if `value` changes while pending).
- Not showing that an item is pending — users should tell "sent" from "sending".

In the interview

> **30-second answer:** "`useOptimistic(value, reducer)` gives a state that equals the real value except while an Action is running, when it shows what I optimistically added. I call the setter inside the Action, then await the server and update the real state. When the Action ends React drops the optimistic value, so on success the real data takes over and on failure it rolls back automatically."

- *Where must* `addOptimistic` *be called?* — Inside an Action: a form/button action or `startTransition`.
- *How does rollback work?* — The optimistic value lives only for the Action's duration; if real state wasn't updated, the old value shows again.

TL;DR

`useOptimistic(realValue, reducer)` → show the expected result during an Action; update the real state on success; failure rolls back by itself. Only call the setter inside an Action.

4.8 Server Functions and progressive enhancement (overview)

What it is

In a framework supporting **React Server Components** (components that run only on the server and ship no JS — e.g. Next.js App Router, Chapter 13), the **directive** (a special string at the top of a file or function) `'use server'` turns an async function into a **Server Function**: its code stays on the server, and the client gets a reference that sends the arguments over the network and runs it there. Passed to a form's `action`, it is often called a **Server Action**.

```
// (framework-only — not run; needs a React Server Components bundler such as Next.js)
// app/actions.js
'use server';
export async function addToCart(prevState, formData) {
  const id = formData.get('productId');
  await db.cart.add(id);                        // runs on the server: database, secrets are safe here
  return { added: id };
}

// app/AddButton.jsx
'use client';
import { useActionState } from 'react';
import { addToCart } from './actions';
export function AddButton({ productId }) {
  const [state, action, isPending] = useActionState(addToCart, null, `/product/${productId}`);
  return (
    <form action={action}>
      <input type="hidden" name="productId" value={productId} />
      <button disabled={isPending}>Add to cart</button>
      {state?.added && <p>Added!</p>}
    </form>
  );
}
```

Why it matters — progressive enhancement

Progressive enhancement: the basic feature works with plain HTML; JavaScript makes it nicer. A Server Function has a URL, so a server-rendered form can be submitted **before the JS bundle has loaded** (slow phone, flaky network): the browser does a normal POST, the server runs the function and returns the page with the result (`useActionState`'s `permalink` says which page). Once **hydrated**, the same form behaves like the Actions above.

Rules to remember

- Arguments and return values must be **serializable** (strings, numbers, plain objects, arrays, `FormData`, …) — no functions or class instances.
- Treat every Server Function as a **public API endpoint** anyone can call: **authenticate** (who is calling), **authorise** (may they do this) and validate inside it.

- `'use server'` marks server-callable functions; it is **not** how you mark Server Components (they need no directive). Chapter 13.

TL;DR

`'use server'` functions run on the server but are called like local functions; used as form actions they enable forms that work before JS loads. Validate and authorise inside them.

4.9 Form libraries — React Hook Form and Zod (overview)

The problem they solve

Hand-written forms (§4.3) are fine for 3–5 fields. Big forms — dozens of fields, nested objects, arrays of rows ("add another address"), async checks, wizards — pile up `values` / `touched` / `errors` plumbing.

Library	Job
React Hook Form (RHF)	Field registration, values, touched/dirty state (dirty = changed from initial), errors, submit handling. Uses **uncontrolled inputs** + refs, so typing doesn't re-render the whole form
Zod	A **schema** library: describe data shape and rules once, then `parse` / `safeParse` any value. Also infers TypeScript types from the schema
`@hookform/resolvers`	Adapter letting RHF validate with a Zod (or Yup, Valibot…) schema

Sharing one Zod schema between browser form and server handler means the rules can't drift apart.

What the code looks like

```
// (React Hook Form 7 + Zod 4 + @hookform/resolvers — not run; these packages aren't installed for this book)
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const schema = z.object({
  email: z.email({ error: 'Enter a valid email' }),          // Zod 4: top-level z.email()
  password: z.string().min(8, { error: 'At least 8 characters' }),
  confirm: z.string(),
}).refine((d) => d.password === d.confirm, { error: 'Passwords do not match', path: ['confirm'] });

export function SignupForm({ onSignup }) {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm({ resolver: zodResolver(schema), mode: 'onTouched' });   // validate after blur, then live
  return (
    <form onSubmit={handleSubmit(onSignup)} noValidate>              {/* onSignup gets typed, valid data */}
      <input type="email" {...register('email')} aria-invalid={!!errors.email} />
      {errors.email && <p role="alert">{errors.email.message}</p>}
      {/* ... password and confirm the same way ... */}
      <button disabled={isSubmitting}>Create account</button>
    </form>
  );
}

// On the server, the same schema:
// const result = schema.safeParse(body);      // { success: true, data } | { success: false, error }
// if (!result.success) return { errors: z.flattenError(result.error).fieldErrors };
```

- `register('email')` returns `{ name, onChange, onBlur, ref }`, spread onto the input.
- `handleSubmit(fn)` prevents the default, validates, and calls `fn(data)` only when valid (focusing the first invalid field by default).
- Zod 4: string formats are top-level (`z.email()`); custom messages use the `error` option (`message` still works but is deprecated). `safeParse` never throws.

When to use what

Situation	Recommendation
Small form, submit → server	Plain form Action + `useActionState` (§4.5)
Needs live UI per keystroke	A few controlled fields
Large/complex client-side form, field arrays, wizards	React Hook Form (+ Zod)
Same rules on client and server, TypeScript types from rules	Zod (or Valibot) schema shared by both

Alternatives: Formik (older, controlled-input based, less active), TanStack Form, Conform (built around Actions/progressive enhancement), Yup/Valibot schemas.

> **TL;DR**
>
> RHF manages big forms efficiently with uncontrolled inputs; Zod describes validation rules once for client and server; `zodResolver` connects them. Don't add them for a 3-field form.

4.10 Putting it together — an RSVP form

`code/rsvp.jsx` combines the chapter: uncontrolled fields read with `FormData`, a pure `validateRsvp`, `useActionState` returning errors + submitted values, a `SubmitButton` using `useFormStatus`, and an optimistic guest list with `useOptimistic`. Core (full file in `code/rsvp.jsx`):

```
function SubmitButton() {
  const { pending } = useFormStatus();                // reads the parent <form>
  return <button className="primary" disabled={pending}>{pending ? 'Sending…' : 'Send RSVP'}</button>;
}

export default function Rsvp({ saveRsvp, initialGuests = [], initialResult = emptyResult }) {
  const [guests, setGuests] = useState(initialGuests);
  const [optimisticGuests, addOptimistic] = useOptimistic(guests, (list, g) => [...list, { ...g, id: 'pending', pending:
true }]);

  async function rsvpAction(prev, formData) {
    const values = Object.fromEntries(formData);        // { name, email, guests, diet }
    const rsvp = { name: values.name.trim(), email: values.email.trim(), guests: Number(values.guests), diet:
values.diet };
    const errors = validateRsvp(rsvp);
    if (Object.keys(errors).length) return { errors, values, message: '' };   // keep what they typed
    addOptimistic(rsvp);                                  // show in the list right away
    try {
      const saved = await saveRsvp(rsvp);
      startTransition(() => setGuests((gs) => [...gs, saved]));
      return { ...emptyResult, message: `Thanks ${saved.name}, see you there!` };
    } catch (e) {
      return { errors: { form: e.message }, values, message: '' };
    }
  }
  const [result, formAction] = useActionState(rsvpAction, initialResult);
  const { errors, values } = result;
  const total = optimisticGuests.reduce((n, g) => n + g.guests, 0);   // derived
  // JSX: <form action={formAction} noValidate>; inputs use defaultValue={values.name}, aria-invalid,
  // aria-describedby; <select key={values.guests} defaultValue={values.guests}> (key remounts it, since
  // React ignores a CHANGED defaultValue on <select>); radios use defaultChecked; <SubmitButton />;
  // then "Coming ({total})" and optimisticGuests.map(...) with "(saving…)" for pending rows.
}
```

`code/07_rsvp_app.jsx` drives an invalid submit (2 guests, "vegan"), a valid one, then a duplicate the fake server rejects:

```
1 invalid      : [ 'Please enter your name', 'Enter an email like ada@example.com' ] | kept: A 2 true
2 sending      : [ 'Host · 1 seat', 'Ada · 2 seats · vegan (saving…)' ] | button: Sending…
2 saved        : [ 'Host · 1 seat', 'Ada · 2 seats · vegan' ] | Thanks Ada, see you there! | heading: Coming (3)
3 server error : [ 'ada@example.com has already replied' ] | list: [ 'Host · 1 seat', 'Ada · 2 seats · vegan' ]
```

1. Invalid → two errors; name, guest count and diet survived the reset (returned `values` → `defaultValue`s; the select remounted via `key`).
2. Valid → the guest appeared instantly as "(saving…)", the button read "Sending…", the headcount already included the optimistic seats; then the real row replaced it, the message appeared and the form cleared.
3. Duplicate email → the server threw; the action **caught** it and returned a form-level error; the optimistic row rolled back by itself.

In the browser (`code/rsvp.html`, styles in `rsvp.css`; `rsvp-main.jsx` starts in the state after one invalid submit):

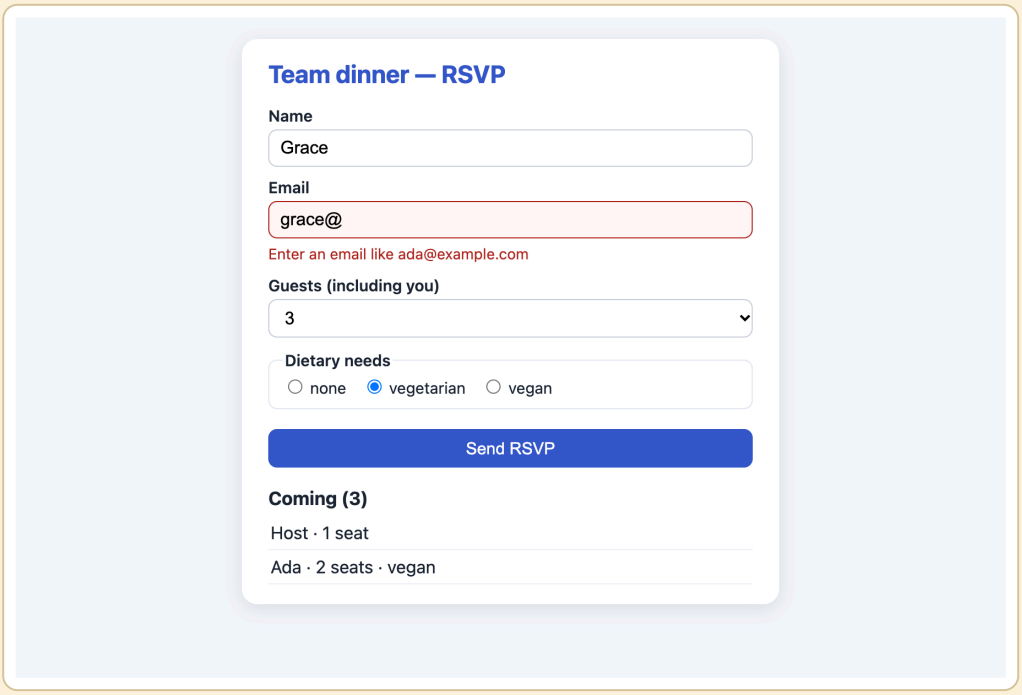


Figure 4.1 — The RSVP form after a failed submit: the typed values are kept, the invalid field is marked with `aria-invalid` (red border) and its message is linked by `aria-describedby`.

Design notes: **minimal state** (only `guests` is `useState`; the result lives in `useActionState`, pending in the form, headcount derived); **validation in one pure function** a real server would call too; **expected failures returned, not thrown**, so no error boundary is needed for "already replied".

Chapter 4 TL;DR

Idea	Remember
`FormData`	Needs `name`; unchecked boxes missing; values are strings; `getAll` for repeated names
Controlled	`value` + `onChange`; re-render per keystroke; live formatting/limits
Uncontrolled	`defaultValue` / `defaultChecked`; read via `FormData` or a ref; file inputs always uncontrolled
Validation	HTML constraints + pure `validate()` + **server**; show on blur/submit; `aria-invalid` + `aria-describedby`; focus first error
Form Action	`<form action={fn}>` → `fn(formData)` in a Transition; auto-reset after success; `formAction` per button
`useActionState`	`[state, action, isPending] = useActionState((prev, formData) => next, initial)`; return errors; queued calls; manual calls in `startTransition`
`useFormStatus`	From `react-dom`, in a **child** of the form: `{ pending, data, method, action }`
`useOptimistic`	Show expected result during an Action; update real state; auto rollback
Server Functions	`'use server'`; progressive enhancement; always validate/authorise inside
Libraries	React Hook Form + Zod for big forms; plain Actions for small ones

Next: Chapter 5 — Effects and Refs.

Chapter 5 — Effects and Refs

So far every component has been a pure function of props and state (§2.5) and every change came from an event handler (Chapter 3). Real apps also talk to things **outside React**: a chat connection, a timer, a browser API such as `document.title`, a **third-party** map widget, a DOM node to focus or measure. React's two **escape hatches** (deliberate ways to step outside render-from-state) are **Effects** (`useEffect`: "after rendering, synchronise with something outside") and **refs** (`useRef`: "remember a value or hold a DOM node without re-rendering").

They are also the most over-used Hooks, so half of this chapter is about **when not to use an Effect** — unnecessary Effects make apps slower and buggier.

Code: `code/` — React 19.3 in jsdom. jsdom has no **layout engine** (computes element sizes and positions), so examples that measure or scroll fake the numbers they need and say so.

5.1 Where side effects belong

What it is

A **side effect** is anything code does besides computing a return value: changing the DOM by hand, sending a request, starting a timer, writing to `localStorage`, logging analytics. React has three places for code:

Place	Runs when	Must be	Use it for
Rendering (the component body)	Every time React needs the UI	**Pure** — same inputs, same JSX, no side effects	Computing JSX and derived values
Event handlers	When a specific user action happens	Anything goes	Side effects **caused by a particular interaction**: submit a form, buy a product, start a timer on click
Effects (`useEffect`)	**After** a render is committed (changes applied to the real DOM)	Should be undoable (cleanup)	Side effects **caused by rendering itself** — keeping the component **synchronised** with an **external system** (network, timers, browser APIs, other libraries) while it's on screen

Ask **"why does this code run?"** Because the user clicked "Send" → handler. Because this component is visible and must stay connected / subscribed / in sync → Effect. Because I need a value computed from props or state → neither: compute it during render.

TL;DR

Pure calculations → render. Caused by a user action → handler. Needed because the component is on screen, to sync with something outside React → Effect.

5.2 `useEffect` basics

What it is

```
useEffect(setup, dependencies?)
// setup: function run after the render is committed; may return a cleanup function
// dependencies: array of every reactive value setup reads (props, state, values derived from them)
```

An **Effect** runs **after** React has updated the DOM (normally after the browser has **painted**), so it never blocks the new screen. A **reactive value** is anything declared in the component body that can change between renders: props, state, and values computed from them.

You write	Setup runs
`useEffect(fn)` — no array	After **every** render
`useEffect(fn, [])` — empty array	Once after the first render (**mount**); cleanup on **unmount** (removed from screen)
`useEffect(fn, [a, b])`	After mount, and after any render where `a` or `b` changed (compared with `Object.is`)

How it works

`code/01_effect_basics.jsx`:

```
function Demo() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('Ada');
  console.log(`render      count=${count} name=${name} | DOM says: ${document.querySelector('h1')?.textContent ??
'(nothing yet)'}`);

  useEffect(() => {
    console.log(`  effect A (no array)   -> after EVERY render; DOM says: ${document.querySelector('h1').textContent}`);
  });
  useEffect(() => {
    console.log('  effect B ([])         -> once, after the first render (mount)');
    return () => console.log('  cleanup B            -> on unmount');
  }, []);
  useEffect(() => {
    console.log(`  effect C ([name])     -> name is now ${name}`);
    return () => console.log(`  cleanup C            -> cleans up the "${name}" effect`);
  }, [name]);

  return (
    <>
      <h1>{name}: {count}</h1>
      <button onClick={() => setCount((c) => c + 1)}>count</button>
      <button onClick={() => setName('Grace')}>name</button>
    </>
  );
}
```

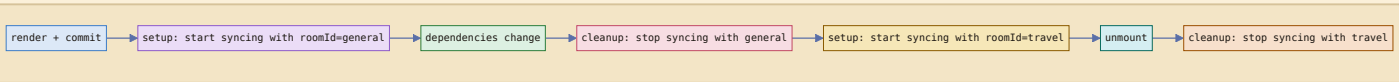
Output:

```
--- mount
render      count=0 name=Ada | DOM says: (nothing yet)
  effect A (no array)   -> after EVERY render; DOM says: Ada: 0
  effect B ([])         -> once, after the first render (mount)
  effect C ([name])     -> name is now Ada
--- click count (name unchanged)
render      count=1 name=Ada | DOM says: Ada: 0
  effect A (no array)   -> after EVERY render; DOM says: Ada: 1
--- click name
render      count=1 name=Grace | DOM says: Ada: 1
  cleanup C            -> cleans up the "Ada" effect
  effect A (no array)   -> after EVERY render; DOM says: Grace: 1
  effect C ([name])     -> name is now Grace
--- click name again (same value: nothing committed, so no effects)
render      count=1 name=Grace | DOM says: Grace: 1
--- unmount
  cleanup B            -> on unmount
  cleanup C            -> cleans up the "Grace" effect
```

1. **Effects run after the DOM is updated**: during render `<h1>` shows the old text, inside the Effect the new text.
2. **Dependencies are compared with the previous render**: "count" didn't re-run C; "name" did.
3. **Cleanup runs before the next setup** and sees the *old* values (`"Ada"`) — each render's Effect closes over that render's snapshot (a **closure**, §3.3).
4. **Same state value → nothing committed**: React may call the component once more to check, but no DOM update or Effects.
5. **Unmount** runs every remaining cleanup.

An Effect's life, from its own point of view

Don't think "mount/update/unmount" (class lifecycles). Think **"start synchronising with X; stop synchronising with X"** — React may start and stop as often as needed (on every dependency change, and once extra in development, §5.4). A symmetrical setup/cleanup pair makes the count irrelevant.



Rules

- Call `useEffect` at the **top level** (§7.2).
- The setup must not be `async` (it would return a **promise**, not a cleanup); define an async function inside and call it (§5.7).
- Effects **don't run on the server** (server rendering, Chapter 13).

In the interview

> **30-second answer:** "`useEffect` synchronises a component with an external system. The setup runs after the render is committed; the optional cleanup runs before the next setup and on unmount. The dependency array lists every reactive value the Effect reads: no array runs after every render, an empty array only on mount, and otherwise when a dependency changes by `Object.is`. I think of it as start-sync/stop-sync rather than lifecycle events."

- *Before or after paint?* — Normally after; `useLayoutEffect` runs before (§5.11).
- *Why can't the setup be `async`?* — It must return nothing or a cleanup function, not a promise.

TL;DR

`useEffect(() => { start; return () => stop; }, [deps])`. Runs after commit; re-runs when deps change; cleanup before the next run and on unmount.

5.3 Cleanup — undo what setup did

What it is

The **cleanup function** your setup returns must **undo** the setup: disconnect, unsubscribe, clear the timer, remove the listener, cancel the request. Without it each re-run *adds* another connection/timer/listener — a **leak** (resources never released, piling up memory and network use).

How it works

`code/02_cleanup_and_strict_mode.jsx`:

```jsx
function ChatRoom({ roomId }) {
  useEffect(() => {
    const connection = createConnection(roomId);   // setup
    connection.connect();
    return () => connection.disconnect();          // cleanup
  }, [roomId]);
  return <h2>Welcome to {roomId}</h2>;
}

function LeakyChatRoom({ roomId }) {
  useEffect(() => {
    createConnection(roomId).connect();            // no cleanup!
  }, [roomId]);
  return <h2>Leaky {roomId}</h2>;
}
// rendered inside <StrictMode>; the user switches room, then closes the chat

// Timers and listeners follow the same shape.
function Clock() {
  const [ticks, setTicks] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setTicks((t) => t + 1), 10);
    const onKey = (e) => console.log('  key pressed:', e.key);
    window.addEventListener('keydown', onKey);
    return () => { clearInterval(id); window.removeEventListener('keydown', onKey); };
  }, []);
  return <p>ticks: {ticks}</p>;
}
```

Output:

```
=== WITH cleanup, inside <StrictMode> (development)
mount:
  connect    general (open: 1)
  disconnect general (open: 0)
  connect    general (open: 1)
switch room:
  disconnect general (open: 0)
  connect    travel (open: 1)
close:
  disconnect travel (open: 0)
connections still open: 0

=== WITHOUT cleanup, inside <StrictMode> (development)
mount:
  connect    general (open: 1)
  connect    general (open: 2)
switch room:
  connect    travel (open: 3)
close:
connections still open: 3

  key pressed: Escape
clock shows ticks: 5 (one interval, one listener — the StrictMode duplicate was cleaned up)
after unmount the listener is gone (no "key pressed: Enter" above)
```

With cleanup there is always **exactly one** connection, to the right room. Without it, connections pile up — to rooms already left — and outlive the component.

Common setup/cleanup pairs

Setup	Cleanup
`connection.connect()`	`connection.disconnect()`
`const id = setInterval(…)` / `setTimeout(…)`	`clearInterval(id)` / `clearTimeout(id)`
`window.addEventListener('resize', fn)`	`window.removeEventListener('resize', fn)` — the **same** function reference
`const unsub = store.subscribe(fn)`	`unsub()`
`const controller = new AbortController(); fetch(url, { signal })`	`controller.abort()`
`const obs = new IntersectionObserver(…); obs.observe(el)` (reports when an element scrolls into view)	`obs.disconnect()`
`const map = new MapWidget(el)`	`map.destroy()`
`dialog.showModal()`	`dialog.close()`
Start an animation	Reset it to the initial state

Common mistakes

- Removing a listener with a *new* arrow function (`removeEventListener('x', () => fn())`) — nothing is removed. Keep the function in a variable.
- Forgetting that cleanup sees the old render's values — usually what you want (disconnect the *old* room).
- Writing cleanup for something that can't be undone (a one-off POST) — that code belongs in an event handler (§5.6).

TL;DR

Return a function that exactly undoes the setup. Then re-runs, room switches and unmounts never leak.

5.4 StrictMode's double run (development only)

What it is

Under `<StrictMode>` (Vite and most templates add it — §1.6), React in **development** mounts every component, then **immediately simulates unmount + remount**: setup → cleanup → setup (the `connect / disconnect / connect` above). Production runs setup once.

React 19 applies the same check to **ref callbacks** (§5.9), and React 19.3 also to components that were server-rendered and then **hydrated**.

Why React does this

Users *do* unmount and remount: navigating away and back, a tab hidden with `<Activity>` (React 19.2's component that hides a subtree while keeping its state) and shown again, a filtered list. Asymmetric setup/cleanup produces hard-to-reproduce bugs in those cases; StrictMode forces the situation on every dev mount so you see it immediately (the leaky version shows `open: 2` at mount).

How to handle it

The fix is **never** "stop React running it twice" — it is a correct cleanup:

Symptom in development	Wrong fix	Right fix
Two connections / subscriptions	A ref flag "already connected"	Return `disconnect` / `unsubscribe`
Two intervals ticking twice as fast	Disable StrictMode	`return () => clearInterval(id)`
Two fetches in the Network tab	Ref flag	Harmless if you ignore/abort the first (§5.7); or use a caching data library
Analytics event logged twice	Ref flag	Acceptable in dev (production logs once), or send analytics from the causing event
Code that must run once per **app load** (initialising a SDK — Software Development Kit, a vendor's client library)	Effect with a ref	Top-level module code, or a module-level `let didInit = false` guard

In the interview

30-second answer: "In development, StrictMode runs every Effect's setup, cleanup and setup again on mount to check the cleanup mirrors the setup. It doesn't happen in production. If the double run causes a visible problem, the Effect is missing a cleanup — the fix is to add one, not to suppress the second run."

TL;DR

Dev-only setup → cleanup → setup exposes missing cleanups. Fix the cleanup; don't fight the double run.

5.5 Dependencies in depth

The rule

Every **reactive value** the Effect reads must be in its dependency array. The `react-hooks/exhaustive-deps` rule of the **linter** (ESLint, via `eslint-plugin-react-hooks`, included in Vite's React template) checks this. Values that never change — module constants, `useRef` objects, state setters, `dispatch` from `useReducer`/`useActionState` — can be omitted.

You don't choose the dependencies; the code does. If an Effect re-runs too often, change the code so it reads fewer reactive values. Never silence the linter with `// eslint-disable-next-line` — it leads to stale values.

Techniques

`code/03_dependencies.jsx`:

```jsx
// 1. An object created during render is a NEW object every render -> Effect re-runs on every keystroke.
function ObjectDep({ roomId }) {
  const [draft, setDraft] = useState('');
  const options = { serverUrl: 'https://chat.example', roomId };     // new {} each render
  useEffect(() => connect(options), [options]);
  // ... controlled <input> for draft
}
// Fix: create the object INSIDE the Effect and depend on the primitive.
function PrimitiveDep({ roomId }) {
  const [draft, setDraft] = useState('');
  useEffect(() => {
    const options = { serverUrl: 'https://chat.example', roomId };
    return connect(options);
  }, [roomId]);
  // ...
}

// 2. Reading state only to compute the next state: use an updater, and the dependency disappears.
useEffect(() => {
  const id = setInterval(() => (useUpdater ? setCount((c) => c + 1) : setCount(count + 1)), 10);
  return () => clearInterval(id);
}, useUpdater ? [] : [count]);

// 3. useEffectEvent (React 19.2+): read the LATEST props/state inside an Effect without making them dependencies.
function ChatRoom({ roomId, theme }) {
  const onConnected = useEffectEvent(() => log.push(`connected to ${roomId}, toast in ${theme} theme`));
  useEffect(() => {
    log.push(`connect ${roomId}`);
    const t = setTimeout(() => onConnected(), 5);                // "the server said connected"
    return () => { clearTimeout(t); log.push(`disconnect ${roomId}`); };
  }, [roomId]);                                                   // theme is NOT a dependency
  return null;
}
// theme changes light -> dark while connecting; later roomId changes to travel
```

Output (the middle two lines depend on timer timing):

```
object dependency   : connects after typing 5 letters = 6
primitive dependency: connects after typing 5 letters = 1
count dep: count=5 interval restarts=6
updater: count=5 interval restarts=1
useEffectEvent: [
  'connect general',
  'connected to general, toast in dark theme',
  'disconnect general',
  'connect travel',
  'connected to travel, toast in dark theme'
]
```

1. **Objects/functions created during render** are new each render, so `Object.is` always says "changed": 6 connections for 5 keystrokes. Move them **inside** the Effect (or outside the component if they use no props/state) and depend on **primitives** (strings, numbers, booleans — compared by value).
2. **Updater functions** (§3.5) let the interval increment `count` without reading it, so it is created once instead of restarted every tick.
3. `useEffectEvent` splits the *reactive* part (connect to `roomId` — re-run on change) from the *non-reactive* part (show a **toast** — a small self-dismissing notification — in the *current* `theme`, without reconnecting when the theme changes). Theme change: no reconnect, toast still "dark". Room change: reconnect.

`useEffectEvent` rules (stable since React 19.2)

- Call it **only from inside Effects** (or other Effect Events), never during render or from event handlers.
- **Don't** list it in dependencies (the linter knows it's non-reactive).
- Declare it in the same component/Hook as the Effect; don't pass it to other components.
- Use it for genuinely event-like logic ("connected", "message arrived", "log a visit with the current cart size"), not to hide real dependencies.

Other techniques

Problem	Technique
Function prop from parent re-creates every render	Call it from an Effect Event, or have the parent memoise it (keep the same function) with `useCallback` (Chapter 10)
Effect reads a whole object prop but needs one field	Destructure the primitive outside: `const { id } = user;` → `[id]`
Two unrelated things in one Effect	Split into two Effects, each with its own dependencies
Mutable value such as `location.pathname` or a ref's `.current`	Not a valid dependency (changes don't re-render). Subscribe with `useSyncExternalStore` (§7.9)

TL;DR

List everything the Effect reads; reduce the list by changing code — move objects inside, use updaters, split Effects, and use `useEffectEvent` for non-reactive logic. Never lie to the linter.

5.6 You might not need an Effect

The problem it solves

Beginners reach for `useEffect` whenever "something should happen when X changes". Most such cases have **no external system** and are better without an Effect, which costs an **extra render**, can show **stale UI for a frame** (~16 ms at 60 fps), and creates hard-to-follow **chains**. Red flag: the Effect only (a) calls `setState` from props/state, or (b) reacts to a *user action* that just happened.

`code/04_you_might_not_need_an_effect.jsx` runs five before/after pairs:

```jsx
// 1. Derived data: computing in an Effect = one extra render + one stale frame.
function FullNameEffect({ first, last }) {
  const [fullName, setFullName] = useState('');
  useEffect(() => setFullName(`${first} ${last}`), [first, last]);   // 🔴
  return <p id="n">{fullName}</p>;
}
function FullNameRender({ first, last }) {
  const fullName = `${first} ${last}`;                               // ✅ just compute it
  return <p id="n">{fullName}</p>;
}

// 2. Resetting state when a prop changes: Effect shows the OLD comment for one frame; a key doesn't.
function CommentBoxEffect({ userId }) {
  const [comment, setComment] = useState('');
  useEffect(() => setComment(''), [userId]);                         // 🔴
  // ...
}
function Profile({ userId }) {
  return <CommentBox key={userId} userId={userId} />;                // ✅ new key = fresh state
}

// 3. Event-specific logic belongs in the event handler, not in an Effect watching state.
function ProductEffect({ initialInCart }) {
  const [inCart, setInCart] = useState(initialInCart);
  useEffect(() => { if (inCart) toasts.push('effect: "Added to cart!"'); }, [inCart]);      // 🔴 also fires on page
load
  return <button onClick={() => setInCart(true)}>Buy</button>;
}
function ProductHandler({ initialInCart }) {
  const [inCart, setInCart] = useState(initialInCart);
  function handleBuy() { setInCart(true); toasts.push('handler: "Added to cart!"'); }      // ✅ only when the user buys
  return <button onClick={handleBuy}>Buy</button>;
}

// 4. Notifying the parent: from an Effect = a second render pass; from the handler = one pass.
function Toggle({ onChange, viaEffect }) {
  const [on, setOn] = useState(false);
  useEffect(() => { if (viaEffect) onChange(on); }, [on]);           // 🔴 (and runs on mount too)
  function handleClick() { const next = !on; setOn(next); if (!viaEffect) onChange(next); }   // ✅ same event
  return <button onClick={handleClick}>{on ? 'On' : 'Off'}</button>;
}

// 5. Adjusting SOME state when a prop changes: best is to derive; else set state during render (no Effect).
function ListAdjust({ items }) {
  const [selection, setSelection] = useState(items[0]);
  const [prevItems, setPrevItems] = useState(items);
  if (items !== prevItems) {            // prop changed since last render -> fix up state right now
    setPrevItems(items);
    setSelection(null);                 // React re-runs this component immediately, before touching the DOM
  }
  return <p id="s">{selection ?? 'none'}</p>;
}
function ListDerived({ items }) {
  const [selectedId, setSelectedId] = useState('a');
  const selection = items.find((i) => i === selectedId) ?? null;      // ✅ derive: kept if still present
  return <p id="s">{selection ?? 'none'}</p>;
}
```

Output ("screen after each commit" is recorded right after every DOM update):

```
1 derived via Effect : renders=4 | screen after each commit: ["","Ada Lovelace","Ada Lovelace","Grace Hopper"]
1 derived via render : renders=2 | screen after each commit: ["Ada Lovelace","Grace Hopper"]
2 reset via Effect: screen after switching user: ["hi Ada","(empty)"]
2 reset via key   : screen after switching user: ["(empty)"]
3 ProductEffect : toasts on page load = ["effect: \"Added to cart!\""]
3 ProductHandler: toasts on page load = []
4 notify via Effect : component renders for one click = 3 | parent shows on
4 notify via handler: component renders for one click = 2 | parent shows on
5 ListAdjust : renders=3 | selection after items changed = none
5 ListDerived: renders=2 | selection after items changed = a
```

1. The Effect version rendered **twice as often**, committing an **empty name** first, then a **stale "Ada Lovelace"** while props already said Grace Hopper.
2. The Effect reset committed the previous user's draft ("hi Ada") before clearing it — in a chat app, a privacy bug. `key` never shows it.
3. The Effect toast fired when a page merely **loaded** with an item in the cart; the handler fires only on buying.
4. Notifying the parent from an Effect costs an extra render pass (and fires on mount); calling `onChange` in the handler batches both updates.
5. Setting state during render with a "previous value" avoids the Effect, but deriving from an ID is simpler — and keeps the selection when the item survives.

The full catalogue

You want to…	Instead of an Effect…
Compute something from props/state	Compute during render (§3.8)
Cache an expensive calculation	`useMemo` (Chapter 10)
Reset **all** state when a prop changes	Give the component a `key`
Adjust **some** state when a prop changes	Derive it; if you must, set state during render with a stored previous value
Share logic between two event handlers	Extract a function and call it from both
Send a request because the user did something (POST on submit)	Do it in the event handler / form Action (Chapter 4)
Run a chain of state updates (A changes → set B → set C…)	Calculate all next state in the one handler
Initialise the app once (read a token, init a SDK)	Top-level module code or a `didInit` module flag
Tell the parent about a change	Call the parent's callback in the same handler, or lift state up (§3.7)
Pass fetched data up to the parent	Fetch in the parent and pass it down
Subscribe to an external store	`useSyncExternalStore` (Chapter 7)
Fetch data	An Effect *with cleanup* is acceptable (§5.7), but prefer a framework loader (router function that fetches a page's data before showing it) or TanStack Query (§6.9, Chapter 9)

What *does* need an Effect

Connecting to a server or WebSocket (a connection kept open so the server can push messages), subscribing to browser events (`resize`, `online`, keyboard shortcuts), timers while visible, syncing a non-React widget (map, video player, `dialog.showModal()`), updating `document.title`, logging an **impression** (analytics record that something was seen) **because the component was shown**, and fetching (with cleanup).

In the interview

> **30-second answer:** "Effects are for synchronising with external systems. If there's no external system — just transforming props into state or reacting to a user event — I don't use one. Derived values are computed during render, expensive ones memoised; full resets use a `key`; event-caused logic goes in the handler; parents are notified in the same handler. Unneeded Effects add renders, show stale frames, and chain into hard-to-follow code."

- *`useEffect(() => setTotal(a + b), [a, b])`? —* `const total = a + b;` in render.
- *Reset a form when `userId` changes? —* `<Form key={userId} />`.

- *'POST analytics when the form is submitted'?* — Submit handler. 'Log that the page was viewed' → Effect.

TL;DR

No external system → no Effect. Compute in render, reset with `key`, act in handlers, notify parents in the same event, subscribe with `useSyncExternalStore`.

5.7 Fetching data in Effects and race conditions

What it is

Fetching in an Effect — when the component shows with some input, request the matching data and store it — is legitimate (syncing with a server) but has a classic trap: a **race condition** (the result depends on which overlapping operation finishes first). Two requests are **in flight** and the **slower, older** one finishes last, overwriting the correct result.

How it works — three versions

`code/05_fetch_race_conditions.jsx` (the fake API answers Alice in 60 ms and Bob in 10 ms; the user picks Alice, then quickly Bob):

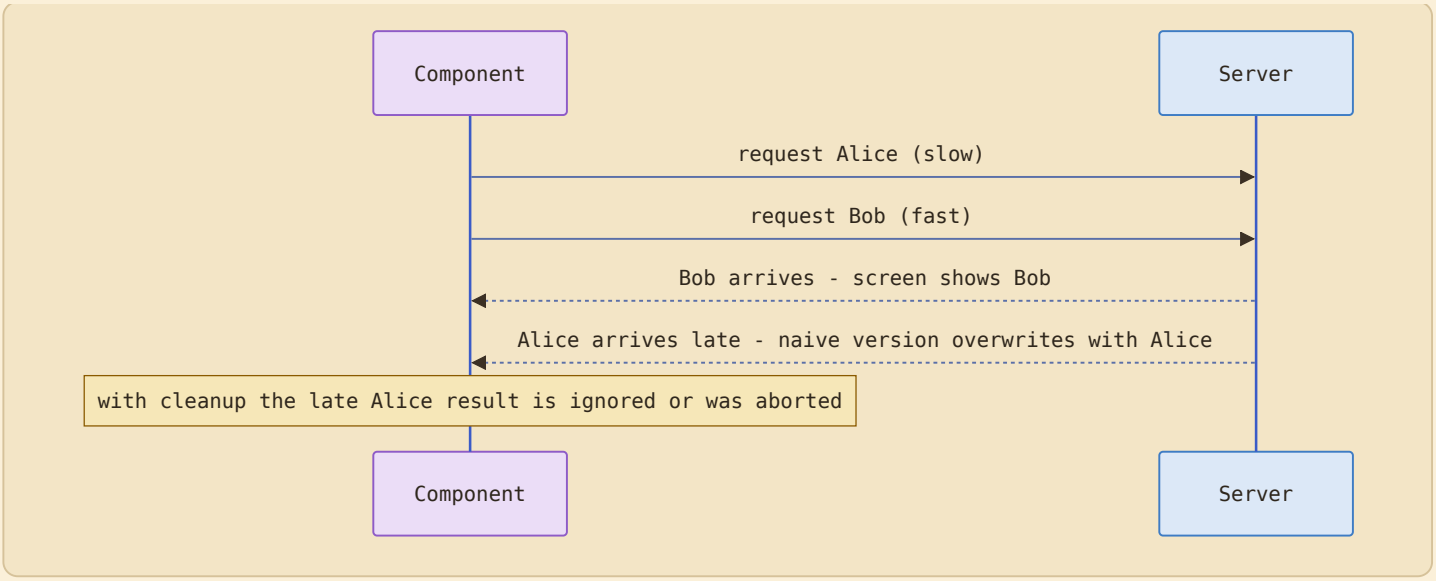
```
function BioNaive({ person }) {                              // 🔴 no cleanup
  const [bio, setBio] = useState(null);
  useEffect(() => { fetchBio(person).then(setBio); }, [person]);
  return <p>{bio ?? 'Loading…'}</p>;
}

function BioIgnore({ person }) {                             // ✅ ignore stale responses
  const [bio, setBio] = useState(null);
  useEffect(() => {
    let ignore = false;
    setBio(null);
    fetchBio(person).then((result) => { if (!ignore) setBio(result); });
    return () => { ignore = true; };                         // runs when person changes or on unmount
  }, [person]);
  return <p>{bio ?? 'Loading…'}</p>;
}

function BioAbort({ person }) {                              // ✅ cancel the request itself + loading/error states
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading' });
    fetchBio(person, { signal: controller.signal })
      .then((bio) => setState({ status: 'success', bio }))
      .catch((err) => { if (err.name !== 'AbortError') setState({ status: 'error', message: err.message }); });
    return () => controller.abort();
  }, [person]);
  if (state.status === 'loading') return <p>Loading…</p>;
  if (state.status === 'error') return <p role="alert">Could not load: {state.message}</p>;
  return <p>{state.bio}</p>;
}
```

Output:

```
BioNaive  selected Bob, screen shows: "This is Alice's bio."
BioIgnore selected Bob, screen shows: "This is Bob's bio."
BioAbort  selected Bob, screen shows: "This is Bob's bio." [ 'aborted Alice' ]
BioAbort  error state: Could not load: 404 Not Found
```



- **Naive**: the user selected Bob but sees Alice — the late response won.
- **Ignore flag**: each Effect run has its own `ignore`; the Alice run's cleanup sets *its* flag, so Alice is dropped.
- `AbortController` (`abort()` cancels requests given its `signal`): actually cancels the old request, saving **bandwidth**; `fetch` rejects with `AbortError`, which must not be shown as an error. It also models the three states you always need: **loading, error, success**.

With real `fetch`, check `response.ok` — `fetch` rejects only on network failure, not on HTTP 404/500 (**status codes**: 404 = not found, 500 = server error; HTTP basics: System Design Fundamentals):

```
// (pattern — not run)
const res = await fetch(`/api/bio/${person}`, { signal });
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();
```

Why fetching in Effects isn't the end of the story

Downside	Consequence
Starts only after render + paint	Slower than fetching during navigation or on the server
Parent fetches, then child renders and fetches	**Network waterfalls** (requests waiting on each other instead of running in parallel)
No cache	Going back and forth re-fetches everything
Boilerplate	Loading/error/abort/retry in every component
No deduplication (merging identical requests)	Two components needing the same data make two requests

Real apps use **framework data loading** (React Router loaders, Next.js Server Components — Chapters 9, 13) or a **client cache** such as **TanStack Query** (§6.9), which handle caching, deduplication, retries and races. React 19's `use()` with Suspense (shows a fallback while something loads) is another option for promises created outside render (Chapter 9).

Common mistakes

- `useEffect(async () => …)` — setup must not return a promise.
- No cleanup → race conditions and "state update on unmounted component" style bugs.
- Showing "Error: AbortError" after a cancellation — filter it out.
- Storing fetched data *and* a filtered copy in state — keep the data, derive the filtered list.

In the interview

> **30-second answer:** "Fetching in an Effect is fine if you handle races: either an `ignore` flag set in cleanup, or an `AbortController` aborted in cleanup, so a stale response can't overwrite a newer one. You also need loading and error states and to check `response.ok`. In production I'd rather use a data library like TanStack Query or framework loaders, which add caching, deduplication, retries and avoid waterfalls."

TL;DR

Fetch-in-Effect needs cleanup (ignore flag or `AbortController`) plus loading/error states. For real apps, prefer TanStack Query or framework loaders.

5.8 `useRef` for values

What it is

```
const ref = useRef(initialValue);   // -> { current: initialValue }, the same object on every render
```

A **ref** is a plain object with one `current` property that React keeps for the component's lifetime. You may read and change `ref.current` freely, and **changing it does not trigger a re-render**.

	`useState`	`useRef`
Returns	`[value, setValue]`	`{ current: value }`
Changing it	Via setter → **re-render**	Assign `ref.current = x` → **no re-render**
Value during a render	A fixed snapshot	Mutable; could change at any time
Read it during render?	Yes	**No** (except lazy initialisation)
Use for	Anything shown on screen	Timer IDs, DOM nodes, latest-value holders, instances of non-React objects

How it works

`code/06_use_ref_values.jsx`:

```jsx
// 1. Ref vs state: changing a ref never re-renders; the screen shows a stale value until something else renders.
function Compare() {
  const [stateClicks, setStateClicks] = useState(0);
  const refClicks = useRef(0);
  return (
    <>
      <button onClick={() => { refClicks.current++; console.log('  ref is now', refClicks.current); }}>ref +1</button>
      <button onClick={() => setStateClicks(stateClicks + 1)}>state +1</button>
      <p>state: {stateClicks} | ref (as of last render): {refClicks.current}</p>   {/* reading in render: a bug, shown on purpose */}
    </>
  );
}

// 2. Stopwatch: the interval ID is needed by a later event but never shown -> ref.
function Stopwatch() {
  const [startTime, setStartTime] = useState(null);
  const [now, setNow] = useState(null);
  const intervalRef = useRef(null);
  function handleStart() {
    setStartTime(Date.now()); setNow(Date.now());
    clearInterval(intervalRef.current);
    intervalRef.current = setInterval(() => setNow(Date.now()), 10);
  }
  function handleStop() { clearInterval(intervalRef.current); }
  const secondsPassed = startTime != null && now != null ? (now - startTime) / 1000 : 0;
  // ... <output>{secondsPassed.toFixed(3)} s</output> + Start / Stop buttons
}

// 3. Lazy-initialise an expensive object once (the only OK "write during render" pattern).
function Player() {
  const engine = useRef(null);
  if (engine.current === null) engine.current = new VideoPlayerEngine();   // runs on the first render only
  // ...
}
```

Output:

```
  ref is now 1
  ref is now 2
after 2 ref clicks  : state: 0 | ref (as of last render): 0
after 1 state click : state: 1 | ref (as of last render): 2
stopwatch ran       : true | still frozen 50 ms after Stop: true
engines created over 4 renders: 1
```

1. The ref really was 2, but the screen said 0 until an unrelated state change — so **anything displayed must be state**.
2. The interval ID lives in a ref: "Stop" needs it, the UI doesn't; state would cause a pointless re-render.
3. `useRef(new VideoPlayerEngine())` would construct an engine **every** render and discard it; `if (ref.current === null)` creates it once. (For state, `useState(() => …)` — §3.2.)

Rules

- **Don't read or write `ref.current` during rendering** (except lazy init): rendering must be pure, and the React Compiler (Chapter 10) assumes this rule.
- Read and write refs in **event handlers and Effects**.
- A ref is **per component instance**, like state; a module-level variable would be shared by all instances.
- The online "`usePrevious`" hook (writing the previous value into a ref in an Effect and reading it in render) breaks this rule; store the previous value in state as in §5.6 case 5.

In the interview

30-second answer: "`useRef` returns a stable `{ current }` object that persists across renders; mutating it doesn't re-render. I use it for values the UI doesn't display — timer IDs, DOM nodes, instances of non-React libraries, the latest value for a callback — and I only touch it in handlers and Effects, not during render. Anything shown on screen belongs in state."

- *Is `useRef(expensive())` efficient?* — No, `expensive()` runs every render; lazy-init with `if (r.current === null)`.

TL;DR

`useRef` = a persistent box whose changes don't re-render. Use it for non-visual values; use state for anything on screen; don't touch refs during render.

5.9 Refs to DOM nodes

What it is

Pass a ref to a built-in element — `<input ref={inputRef} />` — and React puts the **DOM node** in `inputRef.current` after creating it (and `null` when removed). Then call browser APIs React doesn't wrap: `focus()`, `scrollIntoView()`, `getBoundingClientRect()`, media `play()`/`pause()`, `<dialog>` `showModal()`.

How it works

`code/07_dom_refs.jsx`, parts 1 and 2:

```
// 1. The basic pattern: useRef(null) + ref={...}; React fills .current after the DOM node exists.
function FocusForm() {
  const inputRef = useRef(null);
  console.log('  during render, inputRef.current =', inputRef.current?.tagName ?? null);
  return (
    <>
      <input ref={inputRef} aria-label="name" />
      <button onClick={() => inputRef.current.focus()}>Focus the input</button>
    </>
  );
}

// 2. A list of refs: one ref holding a Map, filled by ref CALLBACKS. React 19: a callback may return a cleanup.
function CatGallery({ cats }) {
  const itemsRef = useRef(new Map());
  return (
    <>
      <nav>{cats.map((c) => <button key={c} onClick={() => itemsRef.current.get(c).scrollIntoView({ behavior: 'smooth'
})}>{c}</button>)}</nav>
      <ul>
        {cats.map((c) => (
          <li key={c}>
            <img alt={c} ref={(node) => {
              itemsRef.current.set(c, node);            // called with the node when attached
              return () => itemsRef.current.delete(c);  // cleanup when detached (React 19+)
            }} />
          </li>
        ))}
      </ul>
    </>
  );
}
```

Output:

```
1 focus:
  during render, inputRef.current = null
  focused element: name
2 list of refs:
  scrollIntoView -> Jellylorum
  after removing Jellylorum, the Map holds: Tom, Maru
```

- During the first render the ref is `null` — the node doesn't exist yet. React sets refs during **commit**, before layout Effects (§5.11). Use refs in handlers and Effects.
- `useRef` can't go in a loop (Hook rules), so a list uses **one ref holding a `Map`** plus a **ref callback** per item.

Ref callbacks

A **ref callback** is a function passed as `ref`, called with the DOM node when attached. In **React 19** it may **return a cleanup**, called on detach (removing the cat from the Map above). (Before 19, or if nothing is returned, React calls it again with `null` on detach.)

- An inline callback is a new function each render, so React detaches and re-attaches it every render — harmless for bookkeeping; wrap in `useCallback` if the work is expensive.
- Arrow functions with an **implicit return**, like `ref={(n) => (instance = n)}`, now look like "returning a cleanup"; TypeScript rejects them. Use a block body `{ instance = n; }`.

When to use DOM refs — and when not

Good uses	Avoid
Focus management (modals, forms, "press / to search")	Changing text, classes, children by hand — let React render them
Scrolling an item into view	Removing nodes React created (React will crash or be confused)
Measuring size/position (§5.11)	Reading input values you could keep in state/`FormData` (unless uncontrolled, §4.2)
Integrating non-React libraries (charts, maps, editors)	
Media: `play()`, `pause()`; `<dialog>`: `showModal()`	

To scroll to an item you *just* added with `setState`, the DOM doesn't have it yet inside the handler: wrap the update in `flushSync(() => setItems(…))` from `react-dom` to commit synchronously, then scroll (§3.4).

TL;DR

`ref={myRef}` → `myRef.current` is the DOM node after commit (`null` during the first render). Use ref callbacks + a `Map` for lists; React 19 callbacks may return a cleanup. Don't edit DOM that React manages.

5.10 Passing refs to your own components

What it is

A component's DOM nodes are private by default. When a parent legitimately needs a child's node — a design-system `<TextField>` the page wants to focus — the child must **pass the ref through**:

- **React 19**: `ref` is an **ordinary prop** for function components; destructure it and put it on the element.
- **React 18 and earlier**: wrap in **`forwardRef`**, which passes `ref` as a second argument. Still works in 19 but React has announced deprecation; new code uses the prop. (Class components still don't get `ref` as a prop.)
- **`useImperativeHandle(ref, () => api, deps)`** exposes a **custom, limited object** instead of the raw node — e.g. only `focus()` and `clear()`.

How it works

`code/07_dom_refs.jsx`, parts 3–5:

```jsx
// 3. React 19: `ref` is an ordinary prop for function components — no forwardRef needed.
function TextField({ label, ref }) {
  return <label>{label} <input ref={ref} /></label>;
}
// Legacy (React 18 and earlier) — still works in 19, but forwardRef is planned for deprecation.
const OldTextField = forwardRef(function OldTextField({ label }, ref) {
  return <label>{label} <input ref={ref} /></label>;
});

// 4. useImperativeHandle: expose a small, deliberate API instead of the raw DOM node.
function VideoLikeInput({ ref }) {
  const real = useRef(null);
  useImperativeHandle(ref, () => ({
    focus: () => real.current.focus(),
    clear: () => { real.current.value = ''; },
  }), []);
  return <input ref={real} aria-label="limited" defaultValue="draft" />;
}

// 5. React 19.3: a ref on a <Fragment> gives a FragmentInstance that acts on its children without a wrapper <div>.
function Toolbar() {
  const group = useRef(null);
  return (
    <>
      <Fragment ref={group}>
        <button>Bold</button><button>Italic</button>
      </Fragment>
      <button onClick={() => group.current.focus()}>focus toolbar</button>
    </>
  );
}
```

Output:

```
3 ref as a prop:
  new-style ref -> INPUT | old-style ref -> INPUT
  focused: New
4 useImperativeHandle:
  exposed keys: [ 'focus', 'clear' ]
  value now "" | focused: limited
5 Fragment ref:
  focused: Bold
```

- Both styles delivered the `<input>`; the React 19 version is just a prop.
- With `useImperativeHandle` the parent can only `focus` and `clear`, not restyle or remove the node — good **encapsulation** (exposing only a deliberate interface) for component libraries.
- **Fragment refs** (stable in React 19.3) give a `FragmentInstance` with `focus()` (first focusable child), `addEventListener`, `observeUsing(observer)` and `getClientRects()` — instead of a wrapper `<div>` just to hold a ref.

Migration

Old	New (React 19)
`const X = forwardRef((props, ref) => …)`	`function X({ ref, ...props }) { … }`
`X.displayName` needed for DevTools with anonymous `forwardRef`	Normal named function
`<Ctx.Provider value>` (related 19 change)	`<Ctx value>` (§6.3)

React provides a **codemod** (a script that rewrites source automatically) for the `forwardRef` migration.

In the interview

30-second answer: "To give a parent access to a child's DOM node, in React 19 the child just accepts `ref` as a prop and passes it to the element; `forwardRef` is the older way and is on the path to deprecation. If I don't want to expose the whole node, `useImperativeHandle` exposes a small API like

`focus`. Refs are an escape hatch — I use them for focus, scroll, measurement and third-party widgets, not for rendering."

TL;DR

React 19: `function Field({ ref }) { return <input ref={ref} /> }`. `forwardRef` = legacy. `useImperativeHandle` exposes a limited API. React 19.3 adds refs on `<Fragment>`.

5.11 `useLayoutEffect` — before the browser paints

What it is

Same signature as `useEffect`, but runs **synchronously after React updates the DOM and before the browser paints**; the browser waits for it. Its one job: **measure layout and adjust before the user sees anything** — position a **tooltip** (small pop-up label next to an element), or scroll a chat to the bottom without a jump.

`useInsertionEffect` runs even earlier (before layout effects) so CSS-in-JS libraries (§8.7) can insert `<style>` tags; app code rarely needs it.

How it works

`code/08_layout_effect.jsx` — the order of work, then a tooltip that must flip below its target when there's no room above:

```
function Order() {
  const [n, setN] = useState(0);
  console.log(`  render (n=${n})`);
  useInsertionEffect(() => { console.log('  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)'); });
  useLayoutEffect(() => { console.log('  useLayoutEffect     (DOM updated, not painted yet: measure here)'); return () => console.log('  layout cleanup'); });
  useEffect(() => { console.log('  useEffect           (after paint: subscriptions, fetching, logging)'); return () => console.log('  effect cleanup'); });
  return <button onClick={() => setN(n + 1)}>n={n}</button>;
}

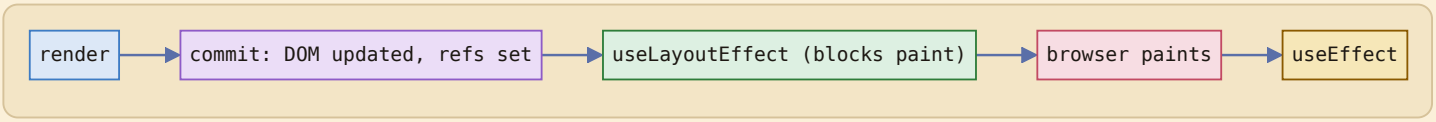
function Tooltip({ targetTop, useLayout }) {
  const ref = useRef(null);
  const [height, setHeight] = useState(0);                    // unknown until measured
  const measure = () => setHeight(ref.current.getBoundingClientRect().height);
  (useLayout ? useLayoutEffect : useEffect)(measure, []);
  const placeAbove = targetTop - height >= 0;                  // enough room above the target?
  const top = placeAbove ? targetTop - height : targetTop + 30;
  return <div ref={ref} data-h="80" role="tooltip" style={{ position: 'absolute', top }}>{placeAbove ? 'above' : 'below'} the button</div>;
}
// Rendered with createRoot + flushSync like a browser task; "first paint" = the DOM at the end of that task.
```

Output:

```
mount:
  render (n=0)
  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)
  useLayoutEffect     (DOM updated, not painted yet: measure here)
  useEffect           (after paint: subscriptions, fetching, logging)
click:
  render (n=1)
  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)
  layout cleanup
  useLayoutEffect     (DOM updated, not painted yet: measure here)
  effect cleanup
  useEffect           (after paint: subscriptions, fetching, logging)
  layout cleanup
  effect cleanup
useEffect      : first paint "above the button" top=50px | settled "below the button" top=80px
useLayoutEffect: first paint "below the button" top=80px | settled "below the button" top=80px
```

(The last two "cleanup" lines are the unmount between demos. The tooltip's 80 px height is faked because jsdom has no layout.)

The tooltip can't know its height until it's in the DOM, so the first render guesses (height 0 → "above"). With `useEffect` that wrong position reaches the screen and is fixed a frame later — a visible **flicker**. With `useLayoutEffect` React measures and re-renders **before** paint, so the user only sees the correct position.



Trade-offs

useEffect	useLayoutEffect
Doesn't block painting — **default choice**	Blocks painting — slow code here makes the app janky (stuttering)
May show one frame of the "before" state	No flicker for measure-then-adjust
Fine for 99% of Effects	Only for reading layout and synchronously fixing it
—	Does nothing during server rendering (HTML is sent before any layout exists)

TL;DR

`useLayoutEffect` = runs after DOM update, before paint; use it to measure and reposition without flicker. Everything else: `useEffect`.

5.12 Putting it together — a city search

`code/citysearch.jsx` uses each tool for its proper job:

Need	Tool	Why
Debounce the query (wait until typing pauses)	Effect with `setTimeout` + cleanup (in a tiny `useDebounced` Hook — custom Hooks are Chapter 7)	Timer = external system; each keystroke's cleanup cancels the previous timer
Focus the box when the page opens	DOM ref + Effect	Focus is a browser API
Press `/` anywhere to focus search	Window listener in an Effect + cleanup	Global subscription while mounted
Search the (fake) API	Effect with `AbortController`	Server sync, race-safe
Keep the tab title in sync	Effect writing `document.title`	Browser API
Tooltip that flips if there's no room	Refs + `useLayoutEffect`	Measure before paint
Result count	Plain variable	Derived — no Effect

Key parts (full file in `code/citysearch.jsx`):

```
// Effect 1 (timer): a debounced copy of a value — only updates after `delay` ms without changes.
function useDebounced(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);                  // a new keystroke cancels the pending update
  }, [value, delay]);
  return debounced;
}

export default function CitySearch({ searchCities, delay = 250, initialQuery = '', tipFor = null }) {
  const [query, setQuery] = useState(initialQuery);
  const [result, setResult] = useState({ status: 'idle', cities: [] });
  const inputRef = useRef(null);
  const debouncedQuery = useDebounced(query.trim(), delay);

  useEffect(() => { inputRef.current.focus(); }, []);          // Effect 2 (DOM): focus on open
  // Effect 3 (global listener): window keydown "/" -> inputRef.current.focus(); cleanup removes the listener

  // Effect 4 (network): fetch for the debounced query; abort the previous request on change.
  useEffect(() => {
    if (!debouncedQuery) { setResult({ status: 'idle', cities: [] }); return; }
    const controller = new AbortController();
    setResult((r) => ({ ...r, status: 'loading' }));  // keep old results visible while loading
    searchCities(debouncedQuery, { signal: controller.signal })
      .then((cities) => setResult({ status: 'done', cities }))
      .catch((e) => { if (e.name !== 'AbortError') setResult({ status: 'error', cities: [], message: e.message }); });
    return () => controller.abort();
  }, [debouncedQuery, searchCities]);

  // Effect 5 (browser API): keep the tab title in sync.
  useEffect(() => {
    document.title = debouncedQuery ? `“${debouncedQuery}” – City search` : 'City search';
  }, [debouncedQuery]);

  const count = result.cities.length;                  // derived
  // ... input, status line, <CityRow> list
}

// In CityRow: a useLayoutEffect([hover]) measures rowRef and tipRef with getBoundingClientRect() and
// flips the tooltip below the row when row.top - tipHeight - 6 < 0 (no room above).
// It lives in the row, not in the tooltip: a parent's layout effect runs after its children's refs are attached.
```

`code/09_city_search.jsx` drives it (debounce 50 ms; the fake API is slow for short queries so a race can happen):

```
on open, focused: true | Start typing to search.
typed "osa"  -> Searching… | requests so far: [ 'osa' ]
             -> 1 city for “osa” [ 'Osaka' ] | title: “osa” – City search
"to" then "tor" -> 1 city for “tor” [ 'Toronto' ] | requests: [ 'to', 'aborted "to"', 'tor' ]
"error"      -> Error: Service unavailable
pressed "/"  -> focused: true
```

Typing "o", "s", "a" made **one** request (debounce cleanups cancelled earlier timers); typing "r" while "to" was in flight **aborted** it, so only "tor" results showed; errors became visible state, aborts didn't; `/` worked because the listener was registered while mounted.

In the browser (`code/citysearch.html`, styles in `citysearch.css`; opens with query "o" and Tokyo's tooltip shown):

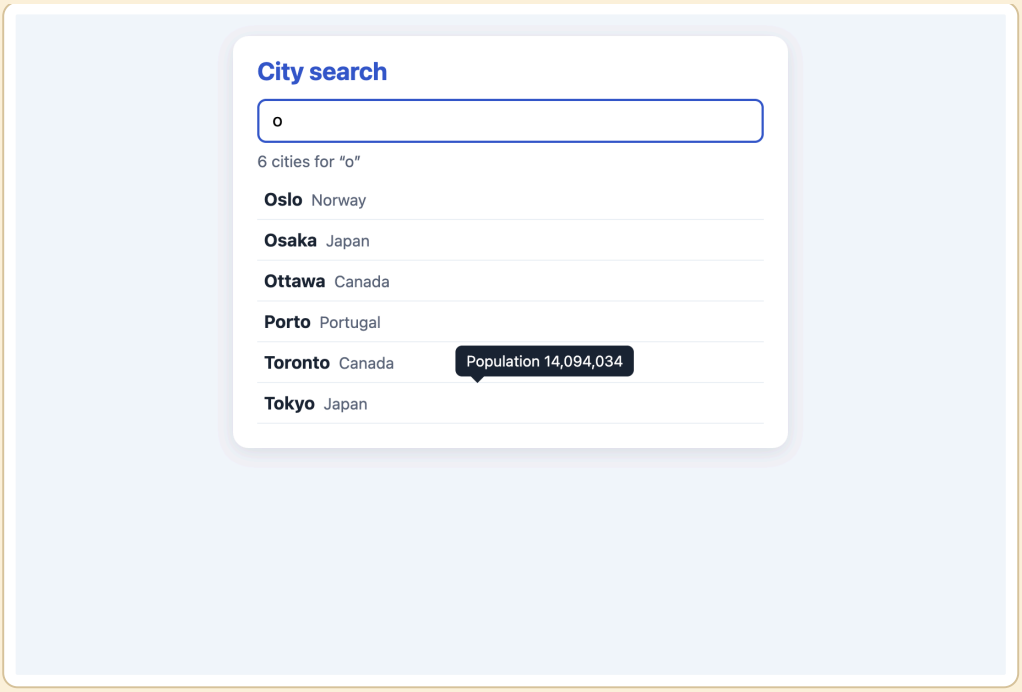


Figure 5.1 — The city search. The focus came from a ref + Effect, the list from an aborted-when-stale fetch Effect, and the tooltip's position from `useLayoutEffect` measuring before paint.

Design notes: **every Effect names its external system** (timer, DOM focus, window events, network, document title) — none merely copies props into state; **every Effect cleans up**, so StrictMode and fast typing are harmless; in production Effect 4 would be one line of TanStack Query (`useQuery({ queryKey: ['cities', q], queryFn })` — §6.9), which also caches.

Chapter 5 TL;DR

Idea	Remember
Where code goes	Pure calc → render; user action → handler; sync with outside while on screen → Effect
`useEffect`	Runs after commit; `[]` = mount only; `[a]` = when `a` changes; no array = every render
Cleanup	Return a function that undoes setup: disconnect, clear, remove, abort
StrictMode	Dev-only setup → cleanup → setup; fix cleanups, don't suppress
Dependencies	Everything reactive the Effect reads; reduce by moving objects inside, updaters, splitting, `useEffectEvent`
Not needed	Derived data, resets (`key`), event logic, notifying parents, chains, app init
Fetching	Ignore flag or `AbortController`; loading/error; prefer TanStack Query / loaders
`useRef`	`{ current }`, persists, no re-render; not for displayed data; not read in render
DOM refs	`ref={r}` → node after commit; ref callbacks (+ cleanup in 19) for lists
Refs to components	React 19 `ref` prop; `forwardRef` legacy; `useImperativeHandle` for limited APIs; Fragment refs in 19.3
`useLayoutEffect`	Before paint; measure and adjust without flicker; otherwise use `useEffect`

Next: Chapter 6 — State Management.

Chapter 6 — State Management

Chapter 3 gave components memory with `useState` and showed **lifting state up**. As apps grow, update logic scatters across handlers and data needed far away must pass through every layer. This chapter covers React's two built-ins for that — `useReducer` (organise *how* state changes) and **Context** (deliver values *far down* the tree) — then the libraries in most codebases: **Redux Toolkit** and **Zustand** for client state, **TanStack Query** for server state. Every library example runs with the real packages.

Code: `code/` — React 19.3, Redux Toolkit 2.12 + React Redux 9.3, Zustand 5.0, TanStack Query 5.104, in jsdom.

6.1 Kinds of state — and where each belongs

What it is

Apps have several **kinds** of state with different needs; most state-management pain comes from putting one kind in the wrong home — especially treating **server data** as ordinary client state.

Kind	Examples	Needs	Natural home
Local UI state	Is this dropdown open? Current tab? Text in this input?	Nothing special	`useState` in the component (Chapter 3)
Complex local state	A form wizard, an editor with many actions	Clear update rules	`useReducer` (§6.2)
Shared / global client state	Current user, theme, cart, feature flags (switches enabling a feature for some users), a multi-step draft	Reachable from far away; efficient updates	Lift up → Context (§6.3) → Zustand / Redux (§6.7–6.8)
Server state (a cached copy of data a server owns)	Product list, user profile, search results	Fetching, caching, de-duplication (merging identical requests), refetching, staleness (how out of date the copy may be), retries	TanStack Query (§6.9), RTK Query, or framework loaders (router functions that fetch a page's data; Chapter 9)
URL state	Filters, page number, selected item ID	Shareable, survives reload, back button	Route params (the `42` in `/books/42`) and search params (`?genre=sf&page=2`) (Chapter 9)
Form state	Field values, errors, pending	Validation, submission	Form Actions / `useActionState`, or a form library (Chapter 4)
Derived state	Totals, filtered lists, "is valid"	Must never be out of sync	**Not state** — compute during render (§3.8)

Why server state is different

Server data is **owned by someone else** — other users can change it any time, so your copy is always potentially **stale**. It needs loading/error states, caching, de-duplication, background refetching and **invalidation** (marking cached data out of date so it is refetched) after you change it (System Design Fundamentals on caching). Hand-writing that with `useEffect` + `useState` (§5.7) or storing it in Redux is a major source of complexity; a server-state library usually removes most of an app's "global state".

TL;DR

Classify first: local, complex local, shared client, server, URL, form, derived. Keep state as local as possible; put server data in a query cache; never store what you can derive.

6.2 `useReducer` — consolidate update logic

What it is

With `useState` each handler decides the next state; as actions multiply (add, edit, delete, reorder, clear…) that logic spreads. `useReducer` moves it into **one function outside the component**:

```
const [state, dispatch] = useReducer(reducer, initialArg, init?);
// reducer(state, action) -> nextState     (pure: no side effects, no mutation)
// dispatch(action)                        (action = a plain object describing what happened, e.g. { type: 'added', text
})
// init?: optional function; initial state = init(initialArg), computed once
```

An **action** is a plain object describing **what happened** (a `type` string like `'added'` plus data). **Dispatch** = send an action to the reducer. The **reducer** takes current state + action and returns the **next** state. Actions form a clear history of what happened.

How it works

The reducer (`code/tasks-reducer.js`) never mutates and throws on unknown actions:

```
export function tasksReducer(tasks, action) {
  switch (action.type) {
    case 'added':
      return [...tasks, { id: action.id, text: action.text, done: false }];
    case 'toggled':
      return tasks.map((t) => (t.id === action.id ? { ...t, done: !t.done } : t));
    case 'renamed':
      return tasks.map((t) => (t.id === action.id ? { ...t, text: action.text } : t));
    case 'deleted':
      return tasks.filter((t) => t.id !== action.id);
    case 'cleared_done':
      return tasks.filter((t) => !t.done);
    default:
      throw new Error(`Unknown action: ${action.type}`);   // catches typos early
  }
}
```

`code/01_use_reducer.jsx`:

```
// 1. Reducers are plain functions: test them without React.
const s0 = [{ id: 1, text: 'Visit Kafka Museum', done: true }];
const s1 = tasksReducer(s0, { type: 'added', id: 2, text: 'Watch a puppet show' });
const s2 = tasksReducer(s1, { type: 'toggled', id: 2 });
// ...
// 2. In a component: dispatch describes WHAT happened; the reducer decides HOW state changes.
function createInitialTasks(username) {                // lazy initialiser: runs once
  console.log(`  (init for ${username})`);
  return [{ id: 1, text: `${username}'s first task`, done: false }];
}
function TaskApp({ username }) {
  const [tasks, dispatch] = useReducer(tasksReducer, username, createInitialTasks);
  return (
    <>
      <button onClick={() => dispatch({ type: 'added', id: nextId++, text: 'Buy stamps' })}>add</button>
      <button onClick={() => dispatch({ type: 'cleared_done' })}>clear done</button>
      <ul>
        {tasks.map((t) => (
          <li key={t.id}>
            <label><input type="checkbox" checked={t.done} onChange={() => dispatch({ type: 'toggled', id: t.id })} />
{t.text}</label>
            <button onClick={() => dispatch({ type: 'deleted', id: t.id })}>delete {t.text}</button>
          </li>
        ))}
      </ul>
    </>
  );
}
```

Output:

```
pure reducer: ["[x] Visit Kafka Museum","[x] Watch a puppet show"] | s0 untouched: true
typo caught  : Unknown action: addded
  (init for Ada)
after 2 adds       : [ "[ ] Ada's first task", '[ ] Buy stamps', '[ ] Buy stamps' ]
toggle + clear done: [ '[ ] Buy stamps', '[ ] Buy stamps' ]
delete one         : [ '[ ] Buy stamps' ]
```

The reducer was **tested without rendering** and left the original array untouched (§3.6); a misspelt type failed loudly; the **lazy initialiser** ran once across several renders; handlers became one-liners saying *what happened*, with all the *how* in one place.

Rules for reducers

1. **Pure**: same state + action → same result. No requests, timers, random IDs or `Date.now()` inside — create those in the handler and put them in the action (`id: nextId++`). StrictMode calls reducers twice in development.
2. **Immutable**: return new objects/arrays. (Immer's `useImmerReducer` allows mutating-style code — the trick Redux Toolkit uses, §6.7.)
3. **Actions describe events, not setters**: `{ type: 'task_completed', id }` over `{ type: 'set_done', value: true }`. One user action = one dispatch, even if several fields change.
4. `dispatch` has a **stable identity** (same function every render) — safe to pass down or omit from Effect dependencies.
5. Like setters, `dispatch` affects only the **next** render; reading `state` right after gives the old snapshot (§3.3).

`useState` vs `useReducer`

	`useState`	`useReducer`
Code size	Small for simple values	More upfront (reducer + actions)
Many update types on one piece of state	Logic scattered in handlers	Centralised, readable
Testing	Needs a render	Test the reducer as a plain function
Debugging	Where did this change?	Log every action + state in one place
Next state depends on several fields	Several setters, risk of inconsistency	One transition computes everything

Start with `useState`; switch when several handlers modify the same state in related ways or updates get inconsistent. Mixing both is fine.

In the interview

30-second answer: "`useReducer` takes a reducer — a pure function from state and action to the next state — and returns the state plus a stable `dispatch`. Handlers dispatch actions describing what happened, and all update logic lives in the reducer, which I can **unit-test** (test one small piece of code on its own) without React. I switch from `useState` when many handlers update the same state or the transitions get complex, and I combine it with context for app-wide state."

- *Can a reducer fetch data?* — No; it must be pure. Fetch in a handler or Effect and dispatch the result.
- *Third argument?* — An initialiser: initial state = `init(initialArg)`, computed once.
- *Why throw on unknown types?* — Typos fail immediately instead of being silently ignored.

TL;DR

`const [state, dispatch] = useReducer(reducer, initial)`. Reducer = pure `(state, action) => next`. Dispatch events, not setters. Test reducers without React.

6.3 Context — pass data deeply without props

The problem it solves

Prop drilling — passing a prop through many components that don't use it — becomes noise when a theme, current user or **locale** (language and regional formats) is needed in dozens of places, and **couples** every layer to data it doesn't care about. **Context** lets a parent make a value available to **every component below it**, however deep; components outside the provider get the default.

How it works

Three steps: **create**, **provide**, **use**.

`code/02_context_basics.jsx`:

```jsx
// 1. Create: the argument is the DEFAULT, used only when there is no provider above.
const ThemeContext = createContext('light');
const LevelContext = createContext(1);

// 2. Read: useContext(Ctx) — or use(Ctx), which (unlike Hooks) may be called conditionally.
function Button({ children }) {
  const theme = useContext(ThemeContext);
  return <button className={`btn-${theme}`}>{children}</button>;
}
function Heading({ children }) {
  if (!children) return null;                  // early return...
  const level = use(LevelContext);             // ...then read context: allowed with use(), not with useContext
  const Tag = `h${Math.min(level, 6)}`;
  return <Tag>{children}</Tag>;
}
// 3. A component can read the nearest value and provide a new one for its subtree.
function Section({ children }) {
  const level = useContext(LevelContext);
  return <section><LevelContext value={level + 1}>{children}</LevelContext></section>;   // React 19: <Ctx value>
}

function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext value={theme}>
      <Heading>Title</Heading>
      <Section>
        <Heading>Chapter</Heading>
        <Section>
          <Heading>Sub-section</Heading>
          <Heading />
          <Button>Save</Button>
          <ThemeContext.Provider value="high-contrast">   {/* older syntax, still works */}
            <Button>Nested override</Button>
          </ThemeContext.Provider>
        </Section>
      </Section>
      <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>toggle theme</button>
    </ThemeContext>
  );
}
```

Output:

```
headings : H1:Title  H2:Chapter  H3:Sub-section
buttons  : Save -> btn-light | Nested override -> btn-high-contrast
toggled  : Save -> btn-dark | Nested override -> btn-high-contrast
no provider -> btn-light (the default from createContext)
```

1. Each `Section` read the level from the **nearest provider above** and provided `level + 1` — headings got the right tag with no `level` prop.
2. The inner `high-contrast` provider overrode the theme only for its **subtree**.
3. When the provided value changed, consumers **re-rendered automatically**.
4. With **no provider**, `useContext` returned the `createContext` **default**.

Syntax notes (React 19)

Topic	Detail
Providing	`<MyContext value={…}>` is the React 19 way; `<MyContext.Provider value={…}>` still works and is planned for deprecation
Reading	`useContext(MyContext)` (a Hook: top level only) or `use(MyContext)` (can be inside `if`/after early returns)
Default value	Used only when no provider is above — not when the provider's value is `undefined`
Scope	Context passes through any intermediate components, including library ones
Class components	`static contextType` (legacy)

Common uses: theme, locale/i18n (internationalisation — translating the UI), current user/auth, feature flags, a router's location, a form library's form state, design-system settings (density, direction), and **dependency injection** for tests (handing a component its services from outside, e.g. a fake API client).

Before you use context

- **Try props first** — explicit props through two or three levels are fine.
- **Try composition** — pass JSX as `children` so middle components never see the data (§2.4): `<Layout sidebar={<UserMenu user={user} />} />`.
- Context is a **delivery mechanism**, not a state manager: the value still lives in `useState`/`useReducer`/a store.

In the interview

> **30-second answer:** "Context lets a component provide a value to its entire subtree without prop drilling: `createContext(default)`, provide with `<Ctx value>` (or `.Provider` in older code), read with `useContext` or `use`. Consumers get the nearest provider's value and re-render when it changes. I use it for rarely-changing, widely-needed values like theme, locale or auth, and I try props or composition first."

- *No provider?* — The default from `createContext`.
- *`useContext` vs `use(Context)`?* — `use` can be called conditionally (after early returns, in loops); `useContext` follows Hook rules.

TL;DR

`createContext(default)` → `<Ctx value={v}>` → `useContext(Ctx)`/`use(Ctx)`. Nearest provider wins; consumers re-render on change. Props and composition first.

6.4 Context performance caveats

What it is

When a provider's `value` changes (`Object.is`), **every component reading that context re-renders** — even inside `memo` (skips re-render when props are unchanged — Chapter 10), and even if it uses one field. Fine for rarely-changing values (theme, locale); a problem for frequently-changing data or a new object created every render.

How it works — measuring renders

`code/03_context_performance.jsx`:

```
const AuthContext = createContext(null);
const UserBadge = memo(function UserBadge() {           // reads the user
  count('UserBadge'); const { user } = useContext(AuthContext); return <span>{user}</span>;
});
const LogoutButton = memo(function LogoutButton() {     // only needs the function
  count('LogoutButton'); const { logout } = useContext(AuthContext); return <button onClick={logout}>logout</button>;
});
const Sidebar = memo(function Sidebar() { count('Sidebar(no context)'); return <nav>links</nav>; });

function App({ memoValue }) {
  const [user, setUser] = useState('ada');
  const [clock, setClock] = useState(0);                // unrelated state in the same component
  const logout = useCallback(() => setUser(null), []);
  const literal = { user, logout };                      // 🔴 new object every render
  const memoised = useMemo(() => ({ user, logout }), [user, logout]);   // ✅ same object until user changes
  return (
    <AuthContext value={memoValue ? memoised : literal}>
      <UserBadge /><LogoutButton /><Sidebar />
      <button onClick={() => setClock(clock + 1)}>tick</button>
    </AuthContext>
  );
}

// Split state and actions into two contexts: action-only consumers stop re-rendering.
function AuthProvider({ children }) {
  const [user, setUser] = useState('ada');
  const actions = useMemo(() => ({ logout: () => setUser(null), login: (u) => setUser(u) }), []);  // never changes
  return <UserContext value={user}><AuthActionsContext value={actions}>{children}</AuthActionsContext></UserContext>;
}
// render(<AuthProvider><Badge2 /><Logout2 /><Sidebar /></AuthProvider>)
```

Output (renders per component for each step):

```
mount (literal value)                    {"UserBadge":1,"LogoutButton":1,"Sidebar(no context)":1}
  unrelated state change ->              {"UserBadge":1,"LogoutButton":1}
  user changed ->                        {"UserBadge":1,"LogoutButton":1}
mount (useMemo value)                    {"UserBadge":1,"LogoutButton":1,"Sidebar(no context)":1}
  unrelated state change ->              {}
  user changed ->                        {"UserBadge":1,"LogoutButton":1}
split contexts: mount                    {"Badge2":1,"Logout2":1,"Sidebar(no context)":1}
  user changed ->                        {"Badge2":1}
```

1. **Object literal as `value`** → new object every provider render → **all consumers re-render** on *unrelated* changes (the tick), defeating `memo`. `useMemo` for the value (returns the same cached value until inputs change) and `useCallback` for functions fixed it — the React Compiler does this automatically (Chapter 10).
2. **Consumers re-render when anything in the value changes**: `LogoutButton` needs only `logout` but re-rendered when `user` changed. **Splitting** state and actions contexts fixed it.
3. `memo` components that **don't read** the context (`Sidebar`) are unaffected; a provider receiving its subtree as `children` (like `AuthProvider`) doesn't re-render that subtree on its own state change — only the consumers.

Techniques summary

Problem	Fix
New object/function as `value` each render	`useMemo` for the value, `useCallback` for functions (or the React Compiler)
Some consumers only need actions/dispatch	Separate "state" and "dispatch/actions" contexts (§6.5)
One big context with unrelated values	Split per concern: `ThemeContext`, `AuthContext`, `CartContext`
High-frequency values (mouse position, typing, timers)	Keep local, or use a store with **selectors** (Zustand/Redux re-render only components whose selected slice changed)
Provider re-renders its whole subtree	Put state in a dedicated provider component that takes `children`

When not to use context: as a general global store for frequently changing data — it has no **selectors** ("re-render me only when `cart.count` changes"), which is exactly what external stores add.

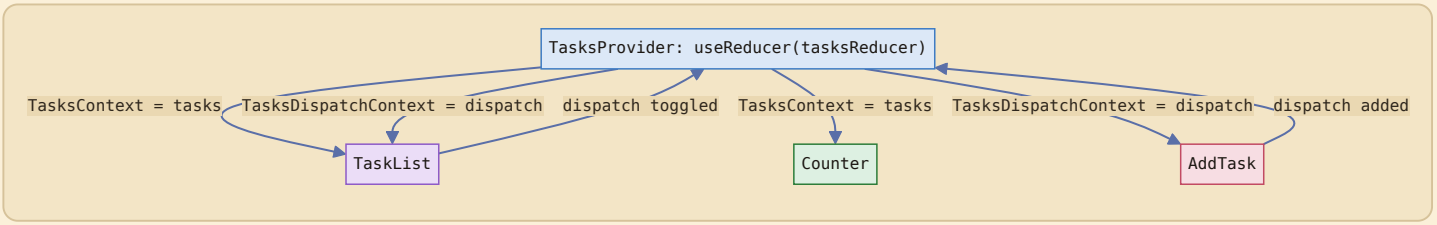
TL;DR

All consumers re-render when the value's identity changes. Memoise the value, split state from actions, split by concern, and use a selector-based store for fast-changing shared data.

6.5 Reducer + context — a store with only React

What it is

A **reducer** owns the state and update rules; **two contexts** deliver the state and `dispatch` to any component; a provider component plus two custom Hooks wrap it — a tidy, dependency-free store for a feature-sized slice of state.



How it works

`code/04_reducer_plus_context.jsx`:

```jsx
// ---- TasksContext.jsx (in a real app: its own file exporting the provider + two Hooks)
const TasksContext = createContext(null);
const TasksDispatchContext = createContext(null);

export function TasksProvider({ initialTasks, children }) {
  const [tasks, dispatch] = useReducer(tasksReducer, initialTasks);
  return (
    <TasksContext value={tasks}>
      <TasksDispatchContext value={dispatch}>{children}</TasksDispatchContext>
    </TasksContext>
  );
}
export function useTasks() {
  const tasks = useContext(TasksContext);
  if (tasks === null) throw new Error('useTasks must be used inside <TasksProvider>');
  return tasks;
}
export function useTasksDispatch() {
  const dispatch = useContext(TasksDispatchContext);
  if (dispatch === null) throw new Error('useTasksDispatch must be used inside <TasksProvider>');
  return dispatch;                                    // stable identity: components using only this rarely re-render
}

// ---- Components deep in the tree: no tasks/onAdd/onToggle props passed through the middle.
function AddTask() {
  const dispatch = useTasksDispatch();
  const [text, setText] = useState('');
  return (
    <form onSubmit={(e) => { e.preventDefault(); dispatch({ type: 'added', id: nextId++, text }); setText(''); }}>
      <input aria-label="New task" value={text} onChange={(e) => setText(e.target.value)} /><button>Add</button>
    </form>
  );
}
function TaskList() { const tasks = useTasks(); const dispatch = useTasksDispatch(); /* ... checkboxes dispatch
'toggled' */ }
function Counter() { const tasks = useTasks(); return <p>{tasks.filter((t) => !t.done).length} left</p>; }
function Layout() { return <main><AddTask /><section><TaskList /></section><footer><Counter /></footer></main>; }  //
passes nothing down
```

Output:

```
tasks  : [ '[x] Philosopher’s Path', '[ ] Visit the temple' ]
footer : 1 left
outside provider -> useTasks must be used inside <TasksProvider>
```

`Layout` passes **no props**. The **custom Hooks** hide the context objects and turn "forgot the provider" into a clear error (possible because the default is `null`). Since `dispatch` never changes, `AddTask` doesn't re-render when tasks change (§6.4 lesson 2).

Limits: every `useTasks()` consumer re-renders on every change; no selectors, no devtools/time travel (stepping back through previous states), no middleware; async flows are up to you. Fine for a feature or small app; large, frequently updated shared state is better in a library.

TL;DR

`useReducer` in a provider + separate state and dispatch contexts + `useX()` Hooks that throw outside the provider = a small, dependency-free store.

6.6 When to reach for a library

Built-ins (local state, lifting, `useReducer`, context) take most apps a long way. Signals a library pays off:

Signal	Likely answer
Lots of `useEffect` + `useState` for fetching; loading/error flags everywhere; duplicate requests; stale lists after edits	**TanStack Query** (or RTK Query / SWR / framework loaders) — this alone removes most "global state"
Shared client state that changes often and is read by many components (editor, canvas, dashboard filters, cart)	A store with selectors: **Zustand** (small) or **Redux Toolkit** (structured)
Large team, many features, strict conventions, action logs, time-travel debugging, middleware	**Redux Toolkit**
State updated from outside React (WebSocket messages, timers, non-React code)	A store usable outside React (Zustand's `getState/setState`, Redux `store.dispatch`)
Complex flows with explicit states and transitions (checkout, wizard, media player)	A reducer, or a **state-machine** library (fixed states + allowed transitions) such as XState
Fine-grained derived values (spreadsheet-like)	An **atom** library (state split into many tiny independent pieces) such as Jotai

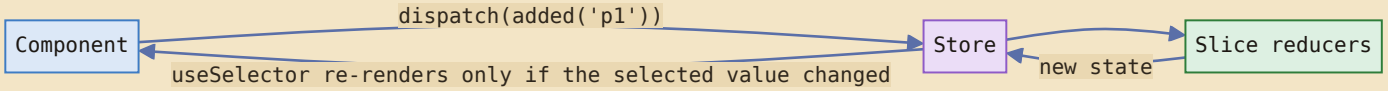
You *don't* need one for a few rarely-changing shared values (theme, user — context is enough) or for "global state" that is really server data (use a query library).

6.7 Redux Toolkit

What it is

Redux keeps app-wide state in **one store**, changed only by **dispatching actions** to **reducers** (`useReducer`'s idea, global, with a big ecosystem). **Redux Toolkit (RTK)** is the official way to write Redux today, removing the old **boilerplate** (hand-written action types, switch statements, spread-heavy updates, store setup).

Term	Meaning
Store	Holds the whole state tree; `getState()`, `dispatch(action)`, `subscribe(listener)`
Slice	One feature's state + reducers + generated action creators (`createSlice`)
Action creator	Function returning an action object: `added('p1')` → `{ type: 'cart/added', payload: 'p1' }`
Selector	Function that reads (and maybe derives) data from state: `state => state.cart.lines`
Thunk	A function you dispatch for async logic; `createAsyncThunk` dispatches `pending`/`fulfilled`/`rejected` (a promise's waiting/succeeded/failed states) for you
Middleware	Code between `dispatch` and the reducer (logging, thunks, analytics)



How it works

`code/05_redux_toolkit.jsx`:

```
// ---- features/products/productsSlice.js : async data with createAsyncThunk
export const fetchProducts = createAsyncThunk('products/fetch', () => fakeApi.fetchProducts());
const productsSlice = createSlice({
  name: 'products',
  initialState: { items: [], status: 'idle', error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchProducts.pending, (state) => { state.status = 'loading'; })
      .addCase(fetchProducts.fulfilled, (state, action) => { state.status = 'succeeded'; state.items = action.payload;
})
      .addCase(fetchProducts.rejected, (state, action) => { state.status = 'failed'; state.error = action.error.message;
});
  },
});

// ---- features/cart/cartSlice.js : "mutating" code is safe here — Immer produces a new immutable state
const cartSlice = createSlice({
  name: 'cart',
  initialState: { lines: {} },                        // { [productId]: quantity }
  reducers: {
    added(state, action) { const id = action.payload; state.lines[id] = (state.lines[id] ?? 0) + 1; },
    removed(state, action) { delete state.lines[action.payload]; },
    cleared() { return { lines: {} }; },              // or return a brand-new state
  },
});
export const { added, removed, cleared } = cartSlice.actions;   // action creators generated for you

// ---- selectors: read and derive; createSelector memoises derived data
const selectLines = (state) => state.cart.lines;
const selectProducts = (state) => state.products.items;
export const selectCartTotal = createSelector([selectLines, selectProducts], (lines, products) =>
  products.reduce((sum, p) => sum + (lines[p.id] ?? 0) * p.price, 0));

// ---- app/store.js
export const makeStore = () => configureStore({ reducer: { products: productsSlice.reducer, cart: cartSlice.reducer }
});

// ---- components
function ProductList() {
  const { items, status } = useSelector((s) => s.products);
  const dispatch = useDispatch();
  if (status === 'loading') return <p>Loading…</p>;
  return <ul>{items.map((p) => <li key={p.id}>{p.name} ${p.price} <button onClick={() => dispatch(added(p.id))}>add
{p.name}</button></li>)}</ul>;
}
function CartTotal() {
  const total = useSelector(selectCartTotal);          // re-renders only when THIS value changes
  return <p>Total: ${total}</p>;
}
// render(<Provider store={store}><ProductList /><CartTotal /></Provider>); store.dispatch(fetchProducts()); click "add"
×3
```

Output:

```
initial state : {"products":{"items":[],"status":"idle","error":null},"cart":{"lines":{}}}
after fetch   : Notebook, Pen | succeeded
cart lines    : {"p1":2,"p2":1} | Total: $10
renders       : {"CartTotal":3} (ProductList does not re-render: the products slice did not change)
memoised selector recomputed on repeat calls: 0 times
after cleared : Total: $0 | action object: {"type":"cart/added","payload":"p9"}
```

1. `createSlice` generated the reducer *and* action creators. Its reducers **look** mutating but run through **Immer**, producing new immutable state. (Mutate the draft *or* return a new value — not both.)
2. `createAsyncThunk` dispatched `pending` → `fulfilled`; `extraReducers` tracked `status`.
3. `configureStore` combined slices and by default added thunk middleware, Redux DevTools support, and dev checks warning about mutation outside reducers or non-serializable values (class instances, functions) in state.
4. `useSelector` subscribes to **just its selected value**: three cart adds re-rendered `CartTotal` three times, `ProductList` zero.

5. `createSelector` (Reselect, re-exported by RTK) **memoised** the derived total: repeat calls didn't recompute.

Rules and good practices

- One store per app; **slices per feature** (`features/cart/cartSlice.js`). Only put state there if other parts of the app care (form input text usually doesn't).
- Keep state **serializable** (plain objects/arrays/primitives) — needed for DevTools, persistence and SSR.
- **Selectors must not return new objects/arrays each call** (`useSelector(s => s.items.filter(...))`) — the component re-renders on every dispatch (React Redux warns in dev). Use `createSelector` or select primitives.
- Model actions as **events** (`cart/added`), not setters (`cart/setLines`).
- For server data prefer **RTK Query** (in RTK: `createApi`, generated hooks like `useGetProductsQuery`, caching and invalidation by tags — labels linking queries to the mutations that make them stale) over hand-written thunks — same ideas as §6.9.
- TypeScript: infer `RootState` and `AppDispatch` from the store and use pre-typed hooks (Chapter 11).

Trade-offs

Pros	Cons
Predictable one-way data flow; every change is a loggable action	More concepts and files than Zustand
Excellent DevTools: action log, state diff, time travel	Overkill for small apps
Conventions scale to large teams	Needs a `<Provider>`
Middleware ecosystem; RTK Query built in	Global by default — easy to over-centralise

In the interview

30-second answer: "Redux keeps app state in one store changed only by dispatching actions to pure reducers. Redux Toolkit is the modern way: `createSlice` generates reducers and action creators and uses Immer so I can write mutating-style updates; `configureStore` sets up thunks, DevTools and safety checks; `createAsyncThunk` or RTK Query handle async data. Components read with `useSelector`, which re-renders only when the selected value changes, and write with `useDispatch`. I'd pick it for large apps with lots of shared client state and a need for strict conventions and tooling."

- *Why can `state.count++` appear in an RTK reducer?* — Immer gives a draft and produces new immutable state from the changes.
- *Why might a component re-render on every dispatch?* — Its selector returns a new object/array each time; memoise with `createSelector`.
- *Redux vs Context?* — Context only delivers a value; Redux is a store with selectors, middleware and tooling, so consumers re-render selectively.

TL;DR

RTK: `createSlice` (reducers + actions, Immer), `configureStore`, `<Provider>`, `useSelector` (fine-grained), `useDispatch`, `createAsyncThunk`/RTK Query, `createSelector`. Structured and tool-rich; best for large apps.

6.8 Zustand

What it is

Zustand ("state" in German) is a small store library: one function returns the state *and* the actions, and `create` gives back a **Hook**. Components pass a **selector** and re-render only when that selection changes. No provider, action types or reducers required.

How it works

`code/06_zustand.jsx`:

```
// ---- stores/useBearStore.js : state and the actions that change it live together
const useBearStore = create((set, get) => ({
  bears: 0,
  fish: 10,
  addBear: () => set((state) => ({ bears: state.bears + 1 })),     // set merges (shallowly) into the state
  eatFish: () => set((state) => ({ fish: Math.max(0, state.fish - get().bears) })),
  reset: () => set({ bears: 0, fish: 10 }),
}));

function BearCounter() { const bears = useBearStore((s) => s.bears); return <p>{bears} bears</p>; }
function FishCounter() { const fish = useBearStore((s) => s.fish); return <p>{fish} fish</p>; }
function Controls() {
  // several values at once: useShallow compares the picked object's fields, not its identity
  const { addBear, eatFish } = useBearStore(useShallow((s) => ({ addBear: s.addBear, eatFish: s.eatFish })));
  return <><button onClick={addBear}>add bear</button><button onClick={eatFish}>eat</button></>;
}
function Everything() { const s = useBearStore(); return <small>{s.bears}/{s.fish}</small>; }   // no selector

// Outside React: read, write, subscribe (tests, WebSocket handlers, other modules).
const unsub = useBearStore.subscribe((state, prev) => console.log(`  subscriber: bears ${prev.bears} ->
${state.bears}`));
useBearStore.getState().addBear();
useBearStore.setState({ fish: 99 });

// ---- Middleware: persist to localStorage.
const useSettings = create(persist(
  (set) => ({ theme: 'light', toggle: () => set((s) => ({ theme: s.theme === 'light' ? 'dark' : 'light' })) }),
  { name: 'settings', storage: createJSONStorage(() => localStorage) },
));

// ---- Common mistake: a selector that builds a NEW object each call, without useShallow.
function Broken() { const { bears, fish } = useBearStore((s) => ({ bears: s.bears, fish: s.fish })); return <p>
{bears}/{fish}</p>; }
```

Output:

```
after 2 bears: 2 bears | 10 fish | renders: {"BearCounter":2,"Everything(no selector)":2}
after eating : 8 fish | renders: {"FishCounter":1,"Everything(no selector)":1}
  subscriber: bears 2 -> 3
  subscriber: bears 3 -> 3
getState()   : {"bears":3,"fish":99} | screen: 99 fish
persisted    : {"state":{"theme":"dark"},"version":0}
new-object selector -> Maximum update depth exceeded. This can …  (fix: useShallow)
```

- **Selective re-renders**: bears → only `BearCounter`; fish → only `FishCounter`; `Controls` (stable action functions) never; `Everything` (**no selector**) subscribed to the whole store and re-rendered every time — avoid.
- `set` **merges** the returned object one level deep (like the old class `setState`); pass `true` as a second argument to replace. `get()` reads current state inside actions.
- **Outside React**: `getState`, `setState`, `subscribe` work anywhere (the subscriber fired for both changes, including fish-only).
- **Middleware**: `persist` saved to `localStorage` and **rehydrates** (loads it back) on load; `devtools` connects Redux DevTools; `immer` allows mutating-style `set`.
- **Classic Zustand 5 bug**: a selector returning a **new object** every call makes React see a change on every check → **infinite loop** ("Maximum update depth exceeded"). Select primitives separately, or wrap object/array selectors in `useShallow`.

Trade-offs

Pros	Cons
Tiny API; almost no boilerplate	Fewer conventions — large teams must agree on structure
No provider; usable outside React	Stores are module **singletons** (one shared instance per loaded file): SSR per-request state or tests may need a store-per-provider pattern (`createStore` + context)
Selector-based re-renders	Less tooling than Redux (DevTools via middleware)
Middleware for persist/devtools/immer	Easy to grow one giant store — split into slices/stores

In the interview

> **30-second answer:** "Zustand creates a store with `create((set, get) => ({ state…, actions… }))` and returns a Hook. Components select what they need — `useStore(s => s.count)` — and re-render only when that value changes; for multiple values I wrap the selector in `useShallow`. It needs no provider, works outside React via `getState/setState`, and has middleware like `persist` and `devtools`. I choose it for shared client state when Redux's structure would be overkill."

TL;DR

`const useStore = create((set, get) => ({ …state, …actions }))`; read with selectors; `useShallow` for objects; `getState/setState/subscribe` outside React; `persist`/`devtools` middleware.

6.9 TanStack Query — server state

What it is

TanStack Query (formerly React Query) is a **cache for server data**. Describe *what* data with a **query key** and *how* to fetch it with a **query function**; it handles loading/error states, caching, de-duplication, background refetching, retries, and staleness after changes.

Concept	Meaning	Default
`queryKey`	Array identifying the data, e.g. `['todos']`, `['user', id]`. Include every variable the fetch depends on	—
`queryFn`	Async function returning the data (or throwing)	—
`staleTime`	How long fetched data counts as **fresh** (no automatic refetch)	`0` (stale immediately)
`gcTime`	How long **unused** cached data is kept before garbage collection (freeing unused memory)	5 minutes
Retries	Failed queries retried with exponential backoff (waits 1 s, 2 s, 4 s…)	3 retries in the browser
Refetch triggers	While stale: component mount, window refocus (user returns to the tab), network reconnect	On
`status`	`'pending'` (no data yet), `'error'`, `'success'`	—
`fetchStatus`	`'fetching'`, `'paused'` (offline), `'idle'` — can be fetching while `success` (background refresh)	—

How it works

`code/07_tanstack_query.jsx`:

```
function useTodos() {                                   // wrap each query in a small custom Hook
  return useQuery({ queryKey: ['todos'], queryFn: api.getTodos, staleTime: 30_000 });
}
function TodoCount() { const { data } = useTodos(); return <p>count: {data?.length ?? '…'}</p>; }
function TodoList() {
  const { data, isPending, isError, error, isFetching } = useTodos();
  if (isPending) return <p>Loading…</p>;                // no data yet
  if (isError) return <p role="alert">{error.message}</p>;
  return <ul aria-busy={isFetching}>{data.map((t) => <li key={t.id}>{t.title}</li>)}</ul>;
}
function AddTodo() {
  const queryClient = useQueryClient();
  const mutation = useMutation({
    mutationFn: api.addTodo,
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['todos'] }),   // mark stale + refetch
  });
  return <button disabled={mutation.isPending} onClick={() => mutation.mutate('Review chapter 6')}>{mutation.isPending ?
'Adding…' : 'Add'}</button>;
}
function UserCard({ id }) {                              // the key includes the variable -> one cache entry per id
  const { data, status, error } = useQuery({ queryKey: ['user', id], queryFn: () => api.getUser(id) });
  return <p>user {id}: {status === 'success' ? data.name : status === 'error' ? `error (${error.message})` : 'loading'}
</p>;
}

// Test/script settings: no retries (default is 3) and no 5-minute garbage-collection timers keeping Node alive.
const queryClient = new QueryClient({ defaultOptions: { queries: { retry: false, gcTime: Infinity }, mutations: {
gcTime: Infinity } } });
const App = ({ children }) => <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>;
```

Output:

```
first render  : Loading… | count: …
loaded        : [ 'Write chapter 6' ] | count: 1 | requests: [ 'GET /todos' ]
remount       : shows 1 item(s) instantly from cache | requests: []
mutating      : button = Adding…
after mutation: [ 'Write chapter 6', 'Review chapter 6' ] | requests: [ 'POST /todos', 'GET /todos' ]
user switch   : user 1: User 1 (from cache at once; refetched in the background because staleTime is 0)
missing user  : user 404: error (User not found) | requests: [ 'GET /users/1', 'GET /users/2', 'GET /users/1', 'GET
/users/404' ]
cache keys    : ["todos"] ["user",1] ["user",2] ["user",404]
```

1. **De-duplication**: `TodoList` and `TodoCount` both asked for `['todos']` — **one** request.
2. **Caching**: remounting within `staleTime` (30 s) showed the list **instantly with zero requests**.
3. **Mutations + invalidation**: `useMutation` gave `isPending` for the button; `invalidateQueries(['todos'])` marked the list stale and refetched — no manual "push the item into state".
4. **Keys with variables**: one cache entry per user ID. Back to user 1: cached name at once, then a background refetch (default `staleTime` 0) — **stale-while-revalidate**.
5. **Errors** are a state (`status === 'error'`, `error.message`), not a crash. A browser would first retry 3 times; retries are off here.

Compared with §5.7: no `useEffect`, `ignore` flag, loading/error `useState`s, cache or refetch-after-save code; races are handled because results are stored per key.

More features (brief)

- **Optimistic updates** with `onMutate` (update the cache, return a rollback) or by rendering `mutation.variables` while pending.
- `enabled: false` for dependent queries (fetch B only when A's result exists).
- `placeholderData: keepPreviousData` for **pagination** (one page of results at a time) without flashing a spinner.
- `useInfiniteQuery` for "load more"/infinite scroll; `useSuspenseQuery` for Suspense (Chapter 9).
- **Prefetching** (`queryClient.prefetchQuery` before data is needed) on hover or in route loaders; a **DevTools** panel.

Common mistakes

- A variable left out of the key (`['user']` for every id) — users overwrite each other's cache.

- Copying query `data` into `useState`/Redux — you lose updates; read from the query, derive filtered views in render.
- Creating `QueryClient` in a component body without `useState` — a new cache every render. Create once (module level or `useState(() => new QueryClient())`).
- Expecting `isPending` during background refetches — use `isFetching`.

In the interview

> **30-second answer:** "Server state is a cache of data someone else owns, so it needs fetching, caching, de-duplication, refetching and invalidation — TanStack Query does that. `useQuery({ queryKey, queryFn })` returns status, data and error; the key identifies the cache entry and must include every variable. `staleTime` controls freshness, `gcTime` how long unused data stays. Writes go through `useMutation`, and on success I invalidate the affected keys so they refetch. With it, most 'global state' disappears from Redux or context."

- *`isPending` vs `isFetching`?* — `isPending`: no data yet. `isFetching`: any request in flight, including background refetches.
- *`staleTime` vs `gcTime`?* — Fresh vs stale (drives refetch triggers) vs how long unused data stays in memory.

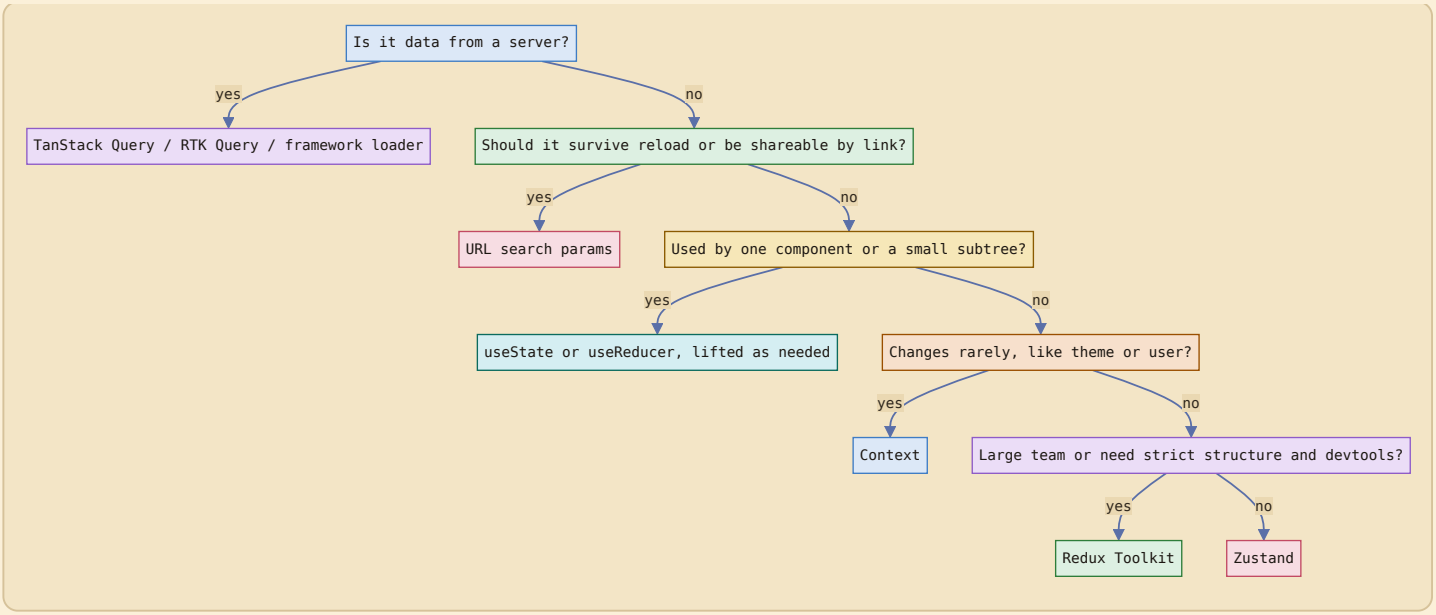
TL;DR

TanStack Query = server-state cache. `useQuery({ queryKey: [...vars], queryFn })`, `useMutation` + `invalidateQueries`. Defaults: `staleTime` 0, `gcTime` 5 min, 3 retries, refetch on focus/reconnect/mount.

6.10 Choosing — comparison and decision flow

	Context (+ reducer)	Redux Toolkit	Zustand	TanStack Query
For	Rarely-changing shared values; small feature stores	Large shared client state, strict conventions	Shared client state with little ceremony	**Server** state
Provider needed	Yes	Yes	No	Yes (`QueryClientProvider`)
Selective re-renders	No (all consumers)	Yes (`useSelector`)	Yes (selectors)	Per query/key
Async	Your code	Thunks, RTK Query	Plain async actions	Built in (it's the whole point)
DevTools	React DevTools only	Redux DevTools (time travel)	Via `devtools` middleware	TanStack Query DevTools
Outside React	No	`store.dispatch/getState`	`getState/setState`	`queryClient` methods
Extra bundle size	None	Moderate (RTK + React Redux)	Very small (about 1 kB gzipped)	Moderate

Other names: **Jotai** (atoms composed together), **MobX** (observable objects that notify listeners, with automatic tracking), **Valtio** (proxy-based mutable state), **XState** (state machines), **SWR** (Vercel's server-state library, similar to TanStack Query), **Recoil** (Meta's atom library — its repository was archived, i.e. made read-only and unmaintained, in January 2025; don't start new projects with it).



In the interview

30-second answer: "I start by classifying the state. Server data goes into a query cache like TanStack Query; shareable UI state like filters goes in the URL; local state stays in components with `useState` / `useReducer`; rarely-changing app-wide values use context. Only frequently-changing shared client state needs a store — Zustand for simplicity, Redux Toolkit when the team benefits from its structure and tooling."

6.11 Putting it together — a bookshop

`code/shop.jsx` puts each kind of state in its natural home:

State	Home	Why
Book catalogue	TanStack Query (`['books']`)	Server data: cached, shared by `Catalog` and `Cart` with one request
Cart lines	Zustand store	Shared client state, updated often, needed in several places, cleared from outside React after checkout
Currency	Context	App-wide, changes rarely
Genre filter	`useState` in `Catalog`	Local UI state
Visible books, cart items, totals	Computed during render	Derived

Key parts (full file in `code/shop.jsx`):

```
// ---- Zustand: cart store (usable from any component, no provider)
export const useCart = create((set) => ({
  lines: {},                                                  // { [bookId]: qty }
  add: (id) => set((s) => ({ lines: { ...s.lines, [id]: (s.lines[id] ?? 0) + 1 } })),
  remove: (id) => set((s) => { const { [id]: _, ...rest } = s.lines; return { lines: rest }; }),
  clear: () => set({ lines: {} }),
}));

// ---- Context: currency formatter chosen once at the top
const CurrencyContext = createContext({ code: 'USD', locale: 'en-US' });
function useMoney() {
  const { code, locale } = useContext(CurrencyContext);
  return (n) => new Intl.NumberFormat(locale, { style: 'currency', currency: code }).format(n);
}

// ---- TanStack Query: the catalog
function useBooks(fetchBooks) {
  return useQuery({ queryKey: ['books'], queryFn: fetchBooks, staleTime: 60_000 });
}

function Catalog({ fetchBooks }) {
  const [genre, setGenre] = useState('all');                  // local UI state
  const { data: books, isPending, isError, error } = useBooks(fetchBooks);
  const add = useCart((s) => s.add);                          // select just the action -> never re-renders for cart
changes
  const money = useMoney();
  if (isPending) return <p className="muted">Loading catalog…</p>;
  if (isError) return <p role="alert">{error.message}</p>;
  const visible = genre === 'all' ? books : books.filter((b) => b.genre === genre);   // derived
  // ... genre buttons + list with "Add" buttons
}

function Cart({ fetchBooks }) {
  const { lines, remove, clear } = useCart(useShallow((s) => ({ lines: s.lines, remove: s.remove, clear: s.clear })));
  const { data: books = [] } = useBooks(fetchBooks);            // same key -> same cache, no extra request
  const money = useMoney();
  const items = books.filter((b) => lines[b.id]).map((b) => ({ ...b, qty: lines[b.id] }));
  const total = items.reduce((sum, i) => sum + i.qty * i.price, 0);   // derived
  // ... list with remove buttons, total, "Empty cart"
}

// Shop = <QueryClientProvider client={queryClient}><CurrencyContext value={currency}> <Catalog/> <Cart/> …
```

(`Shop` takes `queryClient = new QueryClient()` as a default parameter — fine for an app root that renders once; in a re-rendering component use `useState(() => new QueryClient())`.)

`code/08_bookshop.jsx` drives it with a euro/German-locale currency from context:

```
start      : Loading catalog…
catalog    : 4 books | catalog requests: 1 (Catalog and Cart share one query)
fiction    : [ 'The Hobbit', 'Dune' ]
cart       : Cart (3) -> 1 × The Pragmatic Programmer; 2 × Dune | Total 72,00 €
store      : {"b4":2,"b1":1}
removed    : 1 × The Pragmatic Programmer | Total 42,00 €
cleared    : Your cart is empty.
```

One catalogue request served both panels; the filter was purely local; money formatting followed the context (`72,00 €`); the cart was cleared from outside React, as a checkout callback would.

In the browser (`code/shop.html`, styles in `shop.css`; the entry seeds the cart store with two titles, default USD):

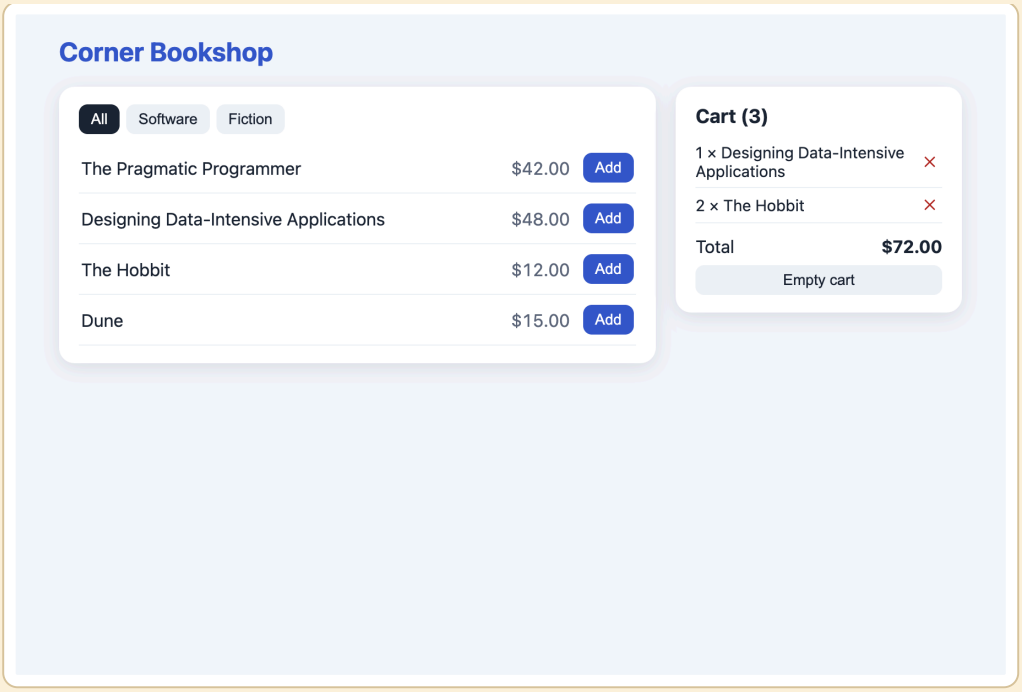


Figure 6.1 — The bookshop. The catalogue comes from the TanStack Query cache, the cart from a Zustand store seeded with `useCart.setState` before rendering, and the prices are formatted using the currency context.

Design notes: **no Redux, no Effects** (server data in the query cache; the store holds only client-owned data); **selectors keep renders narrow** (`Catalog` selects only `add`); **everything countable is derived**.

Chapter 6 TL;DR

Idea	Remember
Kinds of state	Local, complex local, shared client, server, URL, form, derived — each has a home
`useReducer`	Pure `(state, action) => next`; dispatch events; test without React
Context	`createContext(default)`, `<Ctx value>`, `useContext` / `use`; nearest provider wins
Context performance	All consumers re-render on value change; memoise value, split state/actions, split by concern
Reducer + context	Provider + two contexts + `useX()` Hooks that throw outside the provider
Redux Toolkit	`createSlice` (Immer), `configureStore`, `useSelector` / `useDispatch`, thunks, `createSelector`, RTK Query
Zustand	`create((set, get) => …)`; selectors; `useShallow`; no provider; `persist` / `devtools`
TanStack Query	`useQuery({ queryKey, queryFn })`, `useMutation` + `invalidateQueries`; `staleTime` vs `gcTime`
Choosing	Server → query cache; shareable → URL; local → `useState`; rare global → context; busy global → Zustand/RTK

Next: Chapter 7 — Custom Hooks and the Rules of Hooks.

Chapter 7 — Custom Hooks and the Rules of Hooks

You have used `useState` (Chapter 3), `useActionState` (Chapter 4), `useEffect` and `useRef` (Chapter 5), `useReducer` and `useContext` (Chapter 6). This chapter explains the **Rules of Hooks** properly — what they are, *why* React needs them, what breaks when ignored — then shows how to write your **own** Hooks: reusable **stateful logic** (code that remembers values between renders) such as `useLocalStorage`, `useDebounce` and `useFetch`. It finishes with built-ins that are easiest to grasp once you think in Hooks: `useId`, `useSyncExternalStore`, `useTransition`, `useDeferredValue` (plus `useEffectEvent` for callback-taking Hooks).

Code: `code/` — React 19.3 in jsdom with `@testing-library/react`. Fetching examples start a tiny `node:http` server on `127.0.0.1` (localhost) inside the script, so no internet is needed.

7.1 What a Hook really is

What it is

A **Hook** is a function whose name starts with `use` that lets a component "hook into" a React feature: state (`useState`), **side effects** (`useEffect`), **context** (`useContext`), and so on. Hooks only work **while React is rendering a component** — the only time React knows which component the call belongs to.

How it works — state is matched by call order

A component's local variables vanish when it returns, so React stores Hook state *outside* your function, in an internal record per component instance (a **fiber**) holding a **list of Hook slots**:

1. Before calling your component, React points a **cursor** at slot 0.
2. Each Hook call reads the current slot and advances the cursor.
3. The **only** thing identifying "this `useState` is the `name` one" is its **position**. Hooks have no names or keys.

A 15-line toy model in plain JavaScript reproduces this.

`code/01_hooks_by_call_order.js`:

```
let slots = [];      // this component instance's remembered values
let index = 0;       // which slot the next useState call reads

function useState(initial) {
  const i = index++;                      // position = identity
  if (!(i in slots)) slots[i] = initial;   // first render: store the initial value
  const setState = (v) => { slots[i] = v; };
  return [slots[i], setState];
}
function render(Component, props) { index = 0; return Component(props); }

// A "component" that breaks the rule: it calls a Hook only when a condition is true.
function Profile({ showNickname }) {
  const [name] = useState('Ada');
  if (showNickname) {
    const [nick] = useState('Countess');   // slot 1 ONLY on some renders
    void nick;
  }
  const [age] = useState(36);              // slot 1 or 2 depending on the condition!
  return `name=${name} age=${age}`;
}

console.log('render 1 (showNickname=true): ', render(Profile, { showNickname: true }));
console.log('slots now:', JSON.stringify(slots));
console.log('render 2 (showNickname=false):', render(Profile, { showNickname: false }));
```

Output:

```
render 1 (showNickname=true):  name=Ada age=36
slots now: ["Ada","Countess",36]
render 2 (showNickname=false): name=Ada age=Countess
-> age read slot 1, which holds the nickname. State got mixed up.
```

The skipped conditional call made `useState(36)` the *second* call, reading the nickname's slot — the whole reason for the Rules of Hooks.

Why React chose call order

Names or keys were considered. Position won because it keeps Hooks cheap (no string keys), makes custom Hooks **composable** (two custom Hooks can each call `useState` without name clashes) and allows the same Hook several times in one component. The price: *call Hooks in the same order every render*.

> **TL;DR**
>
> React stores Hook state in a per-component list and matches calls by **position**. Change the number or order of calls between renders and values land in the wrong slots.

7.2 The Rules of Hooks

> **What it is**

From react.dev:

1. **Only call Hooks at the top level** of a function component or custom Hook — never inside conditions, loops or nested functions.
2. **Only call Hooks from React functions** — components and custom Hooks, not plain JavaScript functions or class components.

The allowed and forbidden places

✅ Allowed	🔴 Not allowed
Top level of a function component body	Inside `if`, `&&`, `? :`, `switch` branches
Top level of a custom Hook body	Inside `for` / `while` loops
	After an early `return` (the Hook is skipped on some renders)
	Inside event handlers (`onClick={() => useState()}`)
	Inside the callbacks you pass to `useMemo`, `useReducer`, `useEffect`
	Inside `try` / `catch` / `finally` blocks
	In class components, or in plain helper functions

One exception: the `use` API (§9.11 of Chapter 9) *may* be called inside `if` and loops because it keeps no slot — but still not inside `try/catch`.

What React does when you break rule 1

`code/02_rules_of_hooks_errors.jsx`:

```jsx
// BAD: a Hook after an early return
function Greeting({ user }) {
  const [lang] = useState('en');
  if (!user) return <p>Please sign in ({lang})</p>;
  const [greeting] = useState('Hello');     // skipped when user is missing
  return <p>{greeting}, {user}</p>;
}

// BAD: a Hook inside a condition
function Tracker({ tracking }) {
  const [clicks] = useState(0);
  if (tracking) useEffect(() => {});
  return <p>clicks: {clicks}</p>;
}
// ... tryRender(label, firstUi, nextUi) renders, re-renders, prints the error ...

// GOOD: call every Hook unconditionally at the top, put the condition inside.
function GreetingFixed({ user }) {
  const [greeting] = useState('Hello');
  if (!user) return <p>Please sign in</p>;
  return <p>{greeting}, {user}</p>;
}
function TrackerFixed({ tracking }) {
  const [clicks] = useState(0);
  useEffect(() => { if (!tracking) return; /* start tracking */ }, [tracking]);
  return <p>clicks: {clicks}</p>;
}
```

Output:

```
  [dev warning] React has detected a change in the order of Hooks called by Greeting. This will lead to bugs and errors if not fixed. For more information, read the Rules of Hooks: https://react.dev/link/rules-of-hooks
Greeting: signed-out then signed-in -> Rendered more hooks than during the previous render.
Greeting: signed-in then signed-out -> Rendered fewer hooks than expected. This may be caused by an accidental early return statement.
  [dev warning] React has detected a change in the order of Hooks called by Tracker. This will lead to bugs and errors if not fixed. For more information, read the Rules of Hooks: https://react.dev/link/rules-of-hooks
Tracker : off then on               -> Rendered more hooks than during the previous render.
GreetingFixed     -> Hello, Ada
TrackerFixed      -> clicks: 0
```

A changed count makes React **throw** ("Rendered more/fewer hooks…"), crashing that part of the tree. If the count is the same but the *kind* of Hook changes, development builds warn about "a change in the order of Hooks" and you get §7.1's mixed-up state. The fix is always: **call the Hook unconditionally, move the condition inside it** (Effect body, returned JSX, or arguments).

Why rule 2 exists

A Hook needs a "currently rendering component" for its slot. A plain function called from a handler, a timer or **module scope** (top level of a file) has none → *"Invalid hook call. Hooks can only be called inside of the body of a function component."* The same message appears when the app **bundles two copies of React** (e.g. a linked library with its own `react`) — a classic **monorepo** (one repository holding several packages) bug.

The linter: `eslint-plugin-react-hooks`

Never find these bugs at **runtime**: the official **ESLint** plugin (a **linter** reads code without running it and flags likely bugs) checks as you type:

Rule	What it catches
`react-hooks/rules-of-hooks`	Hooks in conditions, loops, after early returns, in non-`use` functions
`react-hooks/exhaustive-deps`	Missing or unnecessary values in `useEffect` / `useMemo` / `useCallback` dependency arrays (§5.5)
Compiler-based rules (purity, refs, set-state-in-effect, …)	Code the React Compiler (auto-memoising build tool) cannot safely optimise — in the plugin's recommended preset in recent versions

Setup with ESLint 9's flat config (one `eslint.config.js` exporting an array; not run — Vite's React template already includes it):

```
// eslint.config.js
import reactHooks from 'eslint-plugin-react-hooks';
import { defineConfig } from 'eslint/config';

export default defineConfig([
  reactHooks.configs.flat.recommended,
]);
```

The linter recognises Hooks **by name**: `useSomething` (capital after `use`) is checked as a Hook; a capitalised function is treated as a component. Hence naming matters (§7.3).

Common mistakes

Mistake	Fix
`if (!data) return <Spinner/>;` above a `useMemo`	Move all Hooks above the first `return`
Calling a Hook in a `.map()` to make one per item	Extract an `<Item>` component and call the Hook inside it
Calling `useContext` in an event handler "to get the latest value"	Read it at top level; the handler closes over it
Silencing `exhaustive-deps` with `// eslint-disable-next-line`	Almost always hides a stale-value bug; restructure (Chapter 5, §7.7)
Naming a Hook `getUser()`	Call it `useUser()` so the linter checks it and its callers

In the interview

30-second answer: "Two rules: call Hooks only at the top level of components or custom Hooks, and only from React functions. React stores Hook state in a list per component and matches calls by order, so the order must be identical every render — a Hook inside an `if` or after an early return shifts every later slot. React throws 'Rendered more/fewer hooks' when the count changes, and `eslint-plugin-react-hooks` catches it **statically** (by reading the code, before it runs)."

- *Why can `use` be conditional but `useState` can't?* — `use` stores nothing in a slot; it reads a promise or context each time.
- *Conditionally run an effect?* — Always call `useEffect`; put the `if` inside and include the condition in dependencies.
- *Is `const [a] = cond ? useState(1) : [0]` allowed?* — No — the Hook is conditional.
- *What else causes "Invalid hook call"?* — Calling a Hook outside a component, or two copies of React in the bundle.

TL;DR

Top level only, React functions only. The rule protects call order; the linter enforces it.

7.3 Custom Hooks — reusing stateful logic

What it is

A **custom Hook** is a function whose name starts with `use` and that **calls other Hooks**, extracting stateful logic for reuse — like a helper function, but for logic needing state or effects. Callers share the logic, each with its own state.

The problem it solves

Before Hooks, sharing stateful logic needed **higher-order components** (HOCs: functions taking a component and returning a wrapped one with extra props) and **render props** (a function prop that returns what to render) (Chapter 14), which nested the tree deeply ("wrapper hell"). A custom Hook is a plain call: no extra components or nesting, and you can call several.

How it works

1. Spot duplicated Hook logic (a toggle, a subscription, a fetch) and move it into `function useSomething(args) { … }`.
2. Return what callers need — a value, a **tuple** (fixed-length array with meaningful positions) `[value, setter]`, or an object `{ data, error, status }`.
3. Every call site gets its **own** state: `useToggle()` in two components runs `useState` twice, in two fibers.
4. The Hook re-runs on each render of its caller, so it always sees the latest arguments.

`code/03_custom_hook_basics.jsx`:

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = () => setOn(v => !v);
  return [on, toggle];                    // return whatever shape is convenient
}

function Spoiler({ title, children }) {
  const [open, toggleOpen] = useToggle();
  return (
    <section>
      <button onClick={toggleOpen}>{open ? 'Hide' : 'Show'} {title}</button>
      {open && <p>{children}</p>}
    </section>
  );
}

function DarkSwitch() {
  const [dark, toggleDark] = useToggle(true);
  return <button onClick={toggleDark}>theme: {dark ? 'dark' : 'light'}</button>;
}
// render two <Spoiler>s and a <DarkSwitch>; click "Show ending", then "theme"

// Not every helper is a Hook: this one calls no Hooks, so it should NOT start with "use".
function formatCount(n) { return n === 1 ? '1 item' : `${n} items`; }
```

Output:

```
start          : Show ending | Show twist | theme: dark
open "ending"  : Hide ending | The butler did it. | Show twist | theme: dark
toggle theme   : Hide ending | The butler did it. | Show twist | theme: light
plain helper   : 3 items
```

Opening one spoiler didn't open the other; the theme switch has its own value: **custom Hooks share logic, never state**. For shared state, lift it, use context, or an external store (Chapter 6, §7.9).

Designing a good custom Hook

Guideline	Why
Name it `use` + CapitalLetter (`useOnlineStatus`)	The linter and React Compiler treat it as a Hook; readers know it may hold state
If it calls **no** Hooks, don't prefix it with `use`	It's a normal function; the prefix would wrongly restrict where it can be called
Name it after its **purpose**, not its mechanism	`useChatRoom(roomId)` beats `useEffectWithWebSocket`
Mirror built-in shapes where it helps	`useLocalStorage` returning `[value, setValue]` feels like `useState`
Return objects when there are 3+ values	`{ data, error, status }` is self-documenting and order-free
Keep it pure during render	Same rule as components: side effects go in Effects or handlers
Avoid "lifecycle" Hooks like `useMount(fn)`	They hide dependencies from the linter; write the Effect you mean
Add `useDebugValue(label)` for library Hooks	Shows a label next to the Hook in React DevTools (used in `code/hooks.js`)

Don't write one when the logic is short and used once (inline is clearer), when it is **server data** (fetch + cache + refetch — TanStack Query does it better, Chapter 9), or when you only share a *value* (props or context). **Testing:** through a tiny component (as here) or `renderHook` from `@testing-library/react` (Chapter 12).

In the interview

> **30-second answer:** "A custom Hook is a function starting with `use` that calls other Hooks. It's how you reuse stateful logic — like a subscription or a debounced value — without wrapper components. Each call has independent state, it re-runs on every render so it sees fresh arguments, and it must follow the Rules of Hooks. The `use` prefix is what lets the linter check it."

- *Do two components using the same custom Hook share state?* — No; share via lifting, context, or a store.
- *Custom Hooks vs HOCs/render props?* — Hooks compose without tree nesting and make data flow explicit; HOCs and render props remain valid but are rarely needed.

TL;DR

`useX` = a function that calls Hooks. It shares logic, not state. Name by purpose; don't prefix non-Hooks with `use`.

7.4 `useLocalStorage` — state that survives a reload

What it is

A Hook with `useState`'s API whose value is saved in `localStorage` (a small per-site key–value store keeping strings across reloads and restarts) and read back next visit.

How it works

1. **Lazy initial state**: pass a function to `useState` so storage is read once, on mount.
2. `JSON.parse` / `JSON.stringify` (**JSON** = JavaScript Object Notation, the standard text data format) because storage holds only strings.
3. `try/catch` around every storage call: it can throw (Safari private mode, **quota** — size limit — full, disabled cookies) and stored JSON can be corrupted.
4. An Effect writes the value when it changes — writing is a side effect, not render work.
5. A second Effect listens for the `storage` **event**, fired in *other* tabs of the same site when one tab changes a key — keeping tabs in sync.

`code/04_use_local_storage.jsx`:

```jsx
export function useLocalStorage(key, initialValue) {
  // 1. Lazy initial state: read storage once, on the first render only.
  const [value, setValue] = useState(() => {
    try {
      const raw = window.localStorage.getItem(key);
      return raw !== null ? JSON.parse(raw) : initialValue;
    } catch {
      return initialValue;                  // corrupted JSON or storage blocked
    }
  });

  // 2. Write whenever the value (or key) changes.
  useEffect(() => {
    try { window.localStorage.setItem(key, JSON.stringify(value)); } catch { /* quota / private mode */ }
  }, [key, value]);

  // 3. Other tabs fire a "storage" event when they change the same key.
  useEffect(() => {
    function onStorage(e) {
      if (e.key === key && e.newValue !== null) setValue(JSON.parse(e.newValue));
    }
    window.addEventListener('storage', onStorage);
    return () => window.removeEventListener('storage', onStorage);
  }, [key]);

  return [value, setValue];                 // same shape as useState, so updaters work too
}

function Settings() {
  const [fontSize, setFontSize] = useLocalStorage('fontSize', 16);
  return (
    <div>
      <p>font size: {fontSize}px</p>
      <button onClick={() => setFontSize(s => s + 2)}>bigger</button>
    </div>
  );
}
```

Output:

```
first visit     : font size: 16px
after 2 clicks  : font size: 20px | stored = 20
after reload    : font size: 20px
other tab -> 24 : font size: 24px
corrupted JSON  : font size: 16px (fell back to the default)
```

Trade-offs and limits

Concern	Note
Server rendering	No `localStorage` on the server. Reading it during render makes server HTML (default) differ from the client (stored value) → **hydration mismatch** (React warns and redraws). With SSR, render the default first and load the stored value in an Effect, or use `useSyncExternalStore` with `getServerSnapshot` (§7.9).
Two components, same key, same tab	Each has its own `useState` and doesn't see the other's changes (`storage` fires only in *other* tabs). If that matters, make storage an external store read with `useSyncExternalStore`.
Size	About 5 MB per **origin** (protocol + domain + port, e.g. `https://shop.example`); strings only; synchronous (blocks the **main thread** that runs JS and draws the page). Large data → IndexedDB (asynchronous browser database).
Security	Readable by any script on the page — never store tokens (session/access credentials) or personal data (see the JS book's storage chapter).

TL;DR

Lazy-read once, write in an Effect, `try/catch` everything, listen to `storage` for other tabs. Beware SSR mismatches.

7.5 `useDebounce` — wait until the user stops

What it is

Debouncing delays an action until input has been quiet for a set time: a search box firing per keystroke sends 5 requests for "react"; debounced by 300 ms it sends 1, after the pause.

How it works

1. Keep a second state, `debounced`, starting equal to `value`.
2. Each time `value` changes, an Effect schedules `setDebounced(value)` after `delayMs`.
3. The Effect's **cleanup** cancels the previous timer, so each keystroke cancels the last pending update — only the final value survives the pause.

`code/05_use_debounce.jsx`:

```
export function useDebounce(value, delayMs = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);  // schedule
    return () => clearTimeout(id);           // a newer value cancels the pending one
  }, [value, delayMs]);
  return debounced;
}

function Search() {
  const [text, setText] = useState('');
  const query = useDebounce(text, 100);
  useEffect(() => { if (query) console.log(`  -> would search for "${query}"`); }, [query]);
  return <input aria-label="search" value={text} onChange={e => setText(e.target.value)} />;
}
// types r, re, rea, reac, react (one key every 30 ms), pauses, then types "react hooks"
```

Output:

```
typed "r"
typed "re"
typed "rea"
typed "reac"
typed "react"
(pause 150 ms)
  -> would search for "react"
typed "react hooks"
(pause 150 ms)
  -> would search for "react hooks"
```

Five keystrokes, one search. The input itself is **not** debounced — `text` updates immediately so typing feels instant; only the *derived* `query` waits.

Debounce vs throttle vs `useDeferredValue`

Technique	Behaviour	Use for
Debounce	Fire once after N ms of silence	Network requests while typing, autosave
Throttle	Fire at most once every N ms	Scroll/resize handlers, analytics
`useDeferredValue` (§7.11)	Re-render a slow part as soon as React has time; no fixed delay, interruptible	Expensive *rendering* (filtering a big list) — not network calls

`useDebouncedCallback(fn, ms)` debounces a *function* instead (autosave on change); libraries such as `use-debounce` provide both.

Common mistakes

- No cleanup → every keystroke eventually fires.
- Debouncing the controlled input's own value → the **caret** (text cursor) lags and characters appear late.

- Creating the debounced function inline each render (`debounce(fn, 300)` in the body) → a new timer per render, nothing debounced. Keep it in a ref or use the value-based Hook.

TL;DR

Timer in an Effect, cleared by the cleanup. Debounce the derived value, never the input.

7.6 `useFetch` — loading, error, success, and no races

What it is

A Hook that fetches a URL and returns `{ status, data, error }`, packaging the Effect-based fetching pattern and its **race-condition** fix from §5.7: when the URL changes while a request is in flight, the last response to arrive would otherwise win, even for the old URL.

`code/06_use_fetch.jsx` — the local server answers user 1 slowly (150 ms) and user 2 quickly (20 ms):

```
// The naive version: no cleanup, so whichever response arrives LAST wins.
function useFetchNaive(url) {
  const [data, setData] = useState(null);
  useEffect(() => { fetch(url).then(r => r.json()).then(setData); }, [url]);
  return data;
}

export function useFetch(url) {
  const [state, setState] = useState({ status: 'idle', data: null, error: null });
  useEffect(() => {
    if (!url) return;
    const controller = new AbortController();          // lets cleanup cancel the request
    setState(s => ({ ...s, status: 'loading', error: null }));
    fetch(url, { signal: controller.signal })
      .then(res => { if (!res.ok) throw new Error(`HTTP ${res.status}`); return res.json(); })
      .then(data => setState({ status: 'success', data, error: null }))
      .catch(error => {
        if (error.name === 'AbortError') return;         // we cancelled it: not a real error
        setState({ status: 'error', data: null, error });
      });
    return () => controller.abort();                    // url changed or unmounted
  }, [url]);
  return state;
}

function Profile({ id }) {
  const { status, data, error } = useFetch(`${BASE}/api/users/${id}`);
  if (status === 'loading' && !data) return <p>Loading…</p>;
  if (status === 'error') return <p role="alert">Could not load user {id}: {error.message}</p>;
  if (!data) return null;
  return <p>User {id}: {data.name}{status === 'loading' ? ' (refreshing…)' : ''}</p>;
}
// render id=1, immediately switch to id=2, wait; then id=99 (does not exist)
```

Output:

```
naive         : Asked for user 2, showing: Ada <- WRONG user (race condition)
initial       : Loading…
switched to 2 : User 2: Grace
200 ms later  : User 2: Grace (the slow Ada response was aborted, not shown)
missing user  : Could not load user 99: HTTP 404
```

How it works

1. Each Effect run creates an `AbortController`; its `signal` goes to `fetch`. When `url` changes (or on unmount), cleanup calls `abort()` → the old `fetch` rejects with `AbortError`, which is ignored. (An `ignore` flag also prevents the stale `setState` but lets the request finish; aborting saves bandwidth.)
2. `fetch` rejects only on *network* failure, so `res.ok` is checked to turn 4xx/5xx **status codes** (e.g. 404 Not Found, 500 server error) into errors.
3. One state object with a `status` field makes impossible combinations (loading *and* error) impossible.

4. Keeping old `data` while `status === 'loading'` shows "refreshing…" instead of flashing a spinner.

When not to use it: fine as an exercise or one-off widget, but real apps need **caching**, **deduplication**, **refetch on focus**, **retries**, **pagination** and **mutations with invalidation** — exactly what TanStack Query or a router loader provides (Chapter 9). Effects also start fetching only *after* render, causing request **waterfalls** in nested components.

In the interview

> **30-second answer:** "I'd keep `{status, data, error}` in one state object, start the fetch in an Effect keyed on the URL, pass an AbortController signal, and abort in the cleanup so a stale response can't overwrite a newer one. I'd check `res.ok` because fetch doesn't reject on 404. For production I'd use TanStack Query or router loaders to get caching, dedupe and retries."

TL;DR

Fetch in an Effect, abort in cleanup, check `res.ok`, one status field. For real apps, reach for a data library.

7.7 Hooks that take callbacks — `useEffectEvent`

What it is

Some custom Hooks accept a function — `useInterval(callback, ms)`, `useEventListener('keydown', handler)`, `useWebSocket(url, onMessage)`. The inner Effect must call the **latest** callback (seeing the latest props/state) without **restarting** the timer or connection each time the parent passes a new function.

`useEffectEvent` (stable since React 19.2) wraps a function into an **Effect Event**: a stable function that, called from an Effect, always runs the newest callback version. It is *not* reactive, so it stays out of the dependency array.

The problem it solves

Before, both options were bad: `callback` in the dependencies → the interval is torn down and recreated every render; leave it out (`[]`) → the Effect keeps the **first** callback forever (a **stale closure** — a function holding variables from an old render) and the linter complains.

`code/11_use_effect_event.jsx`:

```jsx
export function useInterval(callback, delayMs) {
  const onTick = useEffectEvent(callback);        // always calls the newest callback
  useEffect(() => {
    if (delayMs === null) return;                  // null = paused
    timersStarted++;
    const id = setInterval(() => onTick(), delayMs);
    return () => clearInterval(id);
  }, [delayMs]);                                   // onTick is deliberately NOT a dependency
}

// The tempting-but-wrong version: an empty dependency array freezes the FIRST callback.
function useIntervalStale(callback, delayMs) {
  useEffect(() => {
    const id = setInterval(() => callback(), delayMs);
    return () => clearInterval(id);
  }, []);                                          // the linter would warn: missing 'callback', 'delayMs'
}

// Stopwatch: useIntervalImpl(() => stepsSeen.add(step), running ? 20 : null)
// wait 70 ms, click "step" twice (step becomes 3), wait 70 ms
```

Output:

```
useInterval (useEffectEvent): ticks used step values 1, 3
useIntervalStale ([] deps)  : ticks used step values 1
intervals started: 2 (mount + resume; changing step never restarted it)
```

The correct Hook saw `step = 3` without restarting; the stale one kept `1`. Changing `delayMs` (pause → `null`, resume → `20`) *does* restart it, because the delay is a real dependency.

Rules (full list in §5.5): call Effect Events only from inside Effects (not during render or from handlers, which already see latest values); don't pass them to other components or Hooks; never list them as dependencies; use them for the *non-reactive* part of an Effect, not to hide real dependencies.

Before 19.2 the "latest ref" pattern did this — `const saved = useRef(callback); useEffect(() => { saved.current = callback; });` then `saved.current()` in the interval — still common in older code and libraries.

TL;DR

Callback-taking Hooks: wrap the callback in `useEffectEvent`, call it from the Effect, keep only truly reactive values in the dependencies.

7.8 `useId` — IDs for accessibility

What it is

`useId()` returns a **unique string ID** for the calling component instance, stable across re-renders and identical between server HTML and the hydrated client. It is for attributes linking elements by ID, including **ARIA** (`aria-*` attributes describing elements to screen readers): `<label htmlFor>`, `aria-describedby`, `aria-labelledby`, `aria-controls`.

The problem it solves

A reusable `<PasswordField>` needs an input `id` for its `<label>`. Hard-coded `id="password"` breaks when the field appears twice (duplicate IDs; the second label points at the first input). A global counter (`id = nextId++`) breaks under server rendering, where server and client may count in different orders → hydration mismatches.

`code/07_use_id.jsx`:

```
function PasswordField({ label }) {
  const id = useId();                       // one ID per component instance
  return (
    <div>
      <label htmlFor={`${id}-input`}>{label}</label>
      <input id={`${id}-input`} type="password" aria-describedby={`${id}-hint`} />
      <p id={`${id}-hint`}>At least 12 characters.</p>
    </div>
  );
}
function SignupForm() {
  return (<form><PasswordField label="Password" /><PasswordField label="Repeat password" /></form>);
}
// ... render, then renderToString + hydrateRoot, then an identifierPrefix
```

Output:

```
input id=_r_0_-input  aria-describedby=_r_0_-hint
input id=_r_1_-input  aria-describedby=_r_1_-hint
label lookup finds the 2nd input: true
its description: _r_1_-hint
server ids   : _R_1_-input, _R_1_-hint, _R_2_-input, _R_2_-hint
after hydrate: _R_1_-input, _R_1_-hint, _R_2_-input, _R_2_-hint
with identifierPrefix: _widget-R_0_-input
```

How it works

- The ID derives from the component's **position in the tree** (parent path), not a counter, so server and client agree when they render the same tree.
- Call it **once** per component; derive related IDs with suffixes (`${id}-input`, `${id}-hint`).
- `identifierPrefix` (option of `createRoot`, `hydrateRoot` and server renderers) stops IDs from two React **roots** on one page colliding.

- The format is an implementation detail that has changed between versions; never parse it or hard-code it in CSS or tests (query by role/label instead — which works *because* the IDs are wired correctly).

Not for list keys (keys come from data, `todo.id`; `useId` per item defeats the purpose), and not for database IDs or anything globally unique — use `crypto.randomUUID()` when *creating* data.

TL;DR

`useId()` → unique, SSR-safe ID per component for `htmlFor`/`aria-*`. One call, suffixes for related IDs. Never for keys.

7.9 `useSyncExternalStore` — subscribing to outside data

What it is

An **external store** is data living outside React state that can change on its own: browser APIs (`navigator.onLine` — online?, `matchMedia` — does a CSS media query match?, `window.innerWidth`, `localStorage`), third-party state libraries, a WebSocket cache. `useSyncExternalStore` lets a component **read** it and **re-render** on change, without the subtle bugs of `useState` + `useEffect`.

```
const snapshot = useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?);
```

Argument	Contract
`subscribe(callback)`	Start listening; call `callback` whenever the store changes; **return an unsubscribe function**. Should be the same function each render (define it outside the component) or React re-subscribes every render.
`getSnapshot()`	Return the current value. Must return the **same value (by `Object.is`)** if the store hasn't changed.
`getServerSnapshot()`	Optional: the value to use during server rendering and hydration. Required if the component renders on the server.

Why not `useState` + `useEffect`?

The Effect version subscribes *after* the first paint (the first render may be wrong), and under **concurrent rendering** (React can pause a render and continue later, §7.10) different components can read the store at different moments in one render and show inconsistent values — **tearing**. `useSyncExternalStore` reads snapshots in a way React can check for consistency, forcing a synchronous re-render if the store changes mid-render.

`code/08_use_sync_external_store.jsx`:

```
// ---- 1. A browser API: navigator.onLine + 'online'/'offline' events ----
function subscribeOnline(callback) {          // React passes a callback; call it on every change
  window.addEventListener('online', callback);
  window.addEventListener('offline', callback);
  return () => {                              // and return an unsubscribe function
    window.removeEventListener('online', callback);
    window.removeEventListener('offline', callback);
  };
}
function useOnlineStatus() {
  return useSyncExternalStore(
    subscribeOnline,                          // defined OUTSIDE the component: same function every render
    () => navigator.onLine,                   // getSnapshot (client)
    () => true                                // getServerSnapshot (used during SSR and hydration)
  );
}
function StatusBar() { return <p>{useOnlineStatus() ? 'Online' : 'Offline - changes will sync later'}</p>; }

// ---- 2. Your own tiny store (the pattern Redux/Zustand use internally) ----
function createStore(initial) {
  let state = initial;
  const listeners = new Set();
  return {
    getState: () => state,
    setState: (update) => { state = { ...state, ...update }; listeners.forEach(l => l()); },
    subscribe: (l) => { listeners.add(l); return () => listeners.delete(l); },
  };
}
const cart = createStore({ items: 0 });
function useCartCount() { return useSyncExternalStore(cart.subscribe, () => cart.getState().items); }

// ---- 3. The classic bug: getSnapshot returns a NEW object every time ----
function BadCart() {
  const snap = useSyncExternalStore(cart.subscribe, () => ({ ...cart.getState() })); // new object each call
  return <p>{snap.items}</p>;
}
```

Output:

```
start        : Online
wifi drops   : Offline - changes will sync later
wifi returns : Online
store update : header says 3 items | badge: 3
dev warning  : The result of getSnapshot should be cached to avoid an infinite loop
bad snapshot : Maximum update depth exceeded. This can happen when a component repeatedly calls setState inside
componentWillUpdate or componentDidUpdate. React limits the number of nested updates to prevent infinite loops.
```

How it works

1. On render React calls `getSnapshot()`; after commit it calls `subscribe(callback)`.
2. When the store changes, your code calls `callback`; React re-reads `getSnapshot()` and re-renders only if the value differs (`Object.is`).
3. On unmount (or if `subscribe` changes) React calls the unsubscribe function.

Part 3 shows why the snapshot must be *stable*: a fresh object is "different" every time → infinite re-renders. Return the same immutable object until it changes (as `cart.getState()` does), or select a primitive (`.items`).

Real world: Zustand and **React Redux** v8+ are built on it (Chapter 6); `useMediaQuery('(prefers-color-scheme: dark)')` subscribes to `matchMedia(...).addEventListener('change', …)` for theming (§8.8); routers subscribe to the history/location store this way.

When not to use it: if data can live in React state, keep it there. External store mutations can't be non-blocking **transitions** (§7.10) — a store update during a transition falls back to a blocking render — so prefer React state for UI that benefits from `useTransition`.

In the interview

30-second answer: "`useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot)` is the official way to read mutable data from outside React. React subscribes after commit and re-reads the snapshot on each change,

re-rendering when it differs. It prevents tearing in concurrent rendering. The snapshot must be cached — return the same reference while nothing changed — and `subscribe` should be stable. Zustand and React Redux use it internally."

- *What is tearing?* — Different parts of the UI showing different versions of the same store during one render.
- `getSnapshot` *returns a new array each time?* — Infinite re-render loop; React warns "should be cached".

TL;DR

External data → `useSyncExternalStore`. Stable `subscribe`, cached `getSnapshot`, `getServerSnapshot` for SSR.

7.10 `useTransition` — non-urgent updates

What it is

React 18 introduced **concurrent rendering**: React can prepare new UI in the background, pause it for something more urgent, and discard it if outdated. You opt in by marking updates as **transitions**:

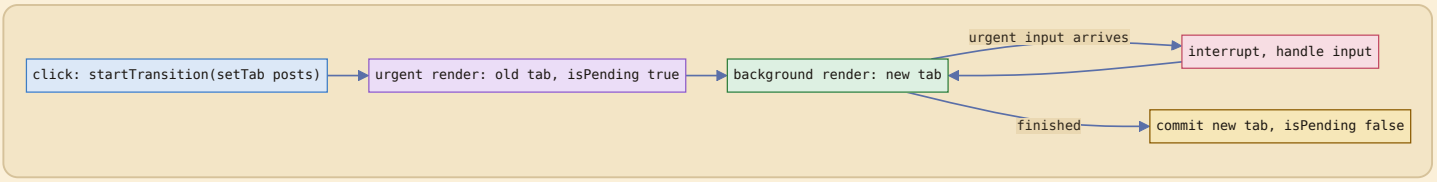
- **Urgent updates**: direct responses to input (typing, clicking) — expected instantly.
- **Transition updates**: moving between views (switching tabs, showing results) — a short delay is fine; freezing is not.

```
const [isPending, startTransition] = useTransition();
startTransition(() => setTab('posts'));          // this update is a transition
```

`isPending` is `true` while the transition is in progress — dim the old UI or show a small spinner instead of a blank screen.

How it works

1. `startTransition(fn)` runs `fn` immediately; updates inside it are tagged as transitions.
2. React first re-renders **urgently** with `isPending = true` and the *old* state — the current screen stays visible and interactive.
3. It renders the new state in the background; an urgent update arriving meanwhile interrupts it, is handled, and the transition restarts with the newest state.
4. When done, React commits and `isPending` becomes `false`.



`code/09_use_transition.jsx`:

```
function SlowPosts() {                              // pretend this tab is expensive to render
  const start = performance.now(); while (performance.now() - start < 30) { /* busy 30 ms */ }
  return <p>500 posts</p>;
}
const TABS = { about: () => <p>About me</p>, posts: SlowPosts };

function TabApp() {
  const [tab, setTab] = useState('about');
  const [isPending, startTransition] = useTransition();
  log.push(`render tab=${tab} isPending=${isPending}`);
  const Tab = TABS[tab];
  return (
    <div style={{ opacity: isPending ? 0.6 : 1 }}>
      <button onClick={() => startTransition(() => setTab('posts'))}>Posts</button>
      <Tab />
    </div>
  );
}

// React 19: the function passed to startTransition may be ASYNC (an "Action").
function RenameForm() {
  const [name, setName] = useState('ada');
  const [isPending, startTransition] = useTransition();
  seen.push(`${isPending ? 'Saving…' : 'Idle   '} name=${name}`);
  function handleSave() {
    startTransition(async () => {
      const saved = await saveToServer('grace');
      startTransition(() => setName(saved));      // updates AFTER an await need their own startTransition
    });
  }
  return <button onClick={handleSave}>Save</button>;
}
```

Output:

```
--- click "Posts" (update inside startTransition) ---
render tab=about isPending=true
render tab=posts isPending=false
--- async action ---
Idle    name=ada
Saving… name=ada
Saving… name=ada
Idle    name=GRACE
```

The first render after the click still shows **about** (dimmed, `isPending=true`); the expensive posts tab replaces it when ready. In the async example `isPending` stays true from click until the whole async function — network wait included — finishes and the new name applies (React may render more than once while pending).

React 19: async transitions are "Actions"

Since React 19, `startTransition` accepts an **async function**, called an **Action**: React tracks its pending state, and it's the mechanism behind form `action` props, `useActionState` and `useOptimistic` (Chapter 4). Current limitation: state updates **after an** `await` aren't automatically in the transition — wrap them in another `startTransition`, as above.

Transitions and Suspense

If a transition makes a component **suspend** (pause rendering because something isn't ready, e.g. route data — Chapter 9), React **keeps showing the old screen** instead of replacing visible content with a `<Suspense>` fallback (placeholder UI such as a spinner). That's why routers wrap navigations in transitions: no spinner flashes between pages.

Trade-offs and caveats

Point	Detail
Can't control text inputs	Updates to an input's `value` must be urgent; a transition would make the caret lag. Keep two states (urgent text + transitioned results) or use `useDeferredValue`.
The callback runs synchronously	`startTransition` doesn't delay anything like `setTimeout`; it only lowers the priority of the updates.
Not for making slow code fast	It keeps the UI *responsive* while slow rendering happens; to reduce the work itself, see Chapter 10.
No `isPending` needed?	Import the standalone `startTransition` from `react` (usable outside components, e.g. in a store).

In the interview

> **30-second answer:** "`useTransition` marks state updates as non-urgent. React renders them in the background and can interrupt them for urgent input, while `isPending` lets me show the old UI dimmed instead of freezing or flashing a fallback. In React 19 the callback can be async — an Action — and `isPending` covers the whole async operation. I can't use it for the value of a controlled input."

- *`useTransition` vs `useDeferredValue`?* — `useTransition` wraps the *update* (you own the setter); `useDeferredValue` wraps a *value* (useful when you only receive it, e.g. as a prop).
- *Is it a debounce?* — No: no timer; it runs as soon as possible and yields to urgent work.

TL;DR

`startTransition(() => setX(...))` = "update when you can, keep the old UI meanwhile". `isPending` for feedback. Async callbacks = Actions (React 19).

7.11 `useDeferredValue` — let part of the UI lag behind

What it is

`useDeferredValue(value)` returns a **deferred copy** of `value`: on an urgent update it first returns the **old** value (so fast parts update immediately), then schedules a background re-render with the new value — the same interruptible, low-priority render a transition uses.

```
const deferredQuery = useDeferredValue(query);                // React 18+
const deferredQuery = useDeferredValue(query, initialValue);  // React 19: value for the very first render
```

How it works

1. The user types; `text` updates urgently and the component renders with `text = "re"` but `deferred = ""`.
2. A `memo`-wrapped child (skips re-rendering when props are unchanged; Chapter 10) receiving `deferred` sees the **same prop** and is skipped — the input stays snappy.
3. React re-renders in the background with `deferred = "re"`; now the slow child renders.
4. If another key arrives first, React abandons that render and restarts with the newest value.

`code/10_use_deferred_value.jsx`:

```
// memo is essential: it lets React skip SlowList when only the urgent `text` changed.
const SlowList = memo(function SlowList({ query }) {
  log.push(`  SlowList renders with query="${query}"`);
  const start = performance.now(); while (performance.now() - start < 20) { /* expensive */ }
  return <ul>{WORDS.filter(w => w.startsWith(query)).map(w => <li key={w}>{w}</li>)}</ul>;
});

function Search() {
  const [text, setText] = useState('');
  const deferred = useDeferredValue(text);           // lags behind `text` during typing
  const isStale = text !== deferred;
  log.push(`render text="${text}" deferred="${deferred}"${isStale ? '  (list is stale)' : ''}`);
  return (
    <>
      <input aria-label="q" value={text} onChange={e => setText(e.target.value)} />
      <div style={{ opacity: isStale ? 0.5 : 1 }}><SlowList query={deferred} /></div>
    </>
  );
}

// React 19: an initial value lets the FIRST render be cheap too.
function Preview({ text }) {
  const shown = useDeferredValue(text, '');          // first render uses '' then catches up
  log.push(`Preview render shown="${shown}"`);
  return <p>{shown}</p>;
}
```

Output:

```
render text="re" deferred=""  (list is stale)
render text="re" deferred="re"
  SlowList renders with query="re"
list now: react, redux, reducer, ref, render
Preview render shown=""
Preview render shown="hello"
```

The first render after the keystroke didn't run `SlowList` (memo saw the same `query`), so the input updated instantly; the list caught up in the background render. `text !== deferred` is a free "stale" flag for dimming.

Common mistakes

Mistake	Fix
No `memo` on the slow child	Without it the child re-renders on the urgent pass too, and nothing is gained
Passing a new object each render (`useDeferredValue({ q })`)	Defer primitives or memoised values; a new object is "new" every time
Using it to reduce network requests	It has no delay; it doesn't limit requests — debounce instead (§7.5)
Expecting it to help with slow *effects*	It only defers rendering

With Suspense: if the deferred value makes a child suspend (loading data for the new query), React keeps showing the **previous results** instead of a fallback — "stale-while-loading" search for free.

TL;DR

`useDeferredValue(v)` → urgent render with the old value, background render with the new one. Pair with `memo`. Not a debounce.

7.12 Putting it together — a user search

Four Hooks compose into one feature: `useId` wires the label, `useDebounce` limits requests, `useFetch` loads results without races, `useLocalStorage` remembers recent searches. Final Hook versions are in `code/hooks.js` (with a `useDebugValue` label on `useFetch`); the component is `code/UserSearch.jsx`:

```jsx
export function UserSearch({ apiBase = '', initialQuery = '' }) {
  const id = useId();
  const [text, setText] = useState(initialQuery);
  const query = useDebounce(text.trim(), 250);
  const { status, data, error } = useFetch(query ? `${apiBase}/api/users?q=${encodeURIComponent(query)}` : null);
  const [recent, setRecent] = useLocalStorage('recent-searches', []);

  function remember(term) { setRecent(r => [term, ...r.filter(t => t !== term)].slice(0, 3)); }

  return (
    <div className="search">
      <label htmlFor={id}>Find a person</label>
      <input id={id} value={text} placeholder="Type a name…" onChange={e => setText(e.target.value)}
             onKeyDown={e => { if (e.key === 'Enter' && text.trim()) remember(text.trim()); }} />
      {recent.length > 0 && <p className="recent">Recent: {recent.join(', ')}</p>}
      {status === 'loading' && <p className="muted">Searching…</p>}
      {status === 'error' && <p role="alert">Search failed: {error.message}</p>}
      {status === 'success' && (data.length === 0
        ? <p className="muted">No one called “{query}”.</p>
        : <ul>{data.map(u => <li key={u.id}><strong>{u.name}</strong> <span>{u.role}</span></li>)}</ul>)}
    </div>
  );
}
```

`code/12_user_search.jsx` runs it against a local server that records every request:

```
typed a, ad, ada quickly -> requests sent: ["ada"]
results: Ada Lovelace Analyst
after "zed": No one called “zed”.
Recent: zed, ada | stored: ["zed","ada"]
```

In the browser (`code/search.html`; `fetch` is mocked in `search-main.jsx` so it works offline):

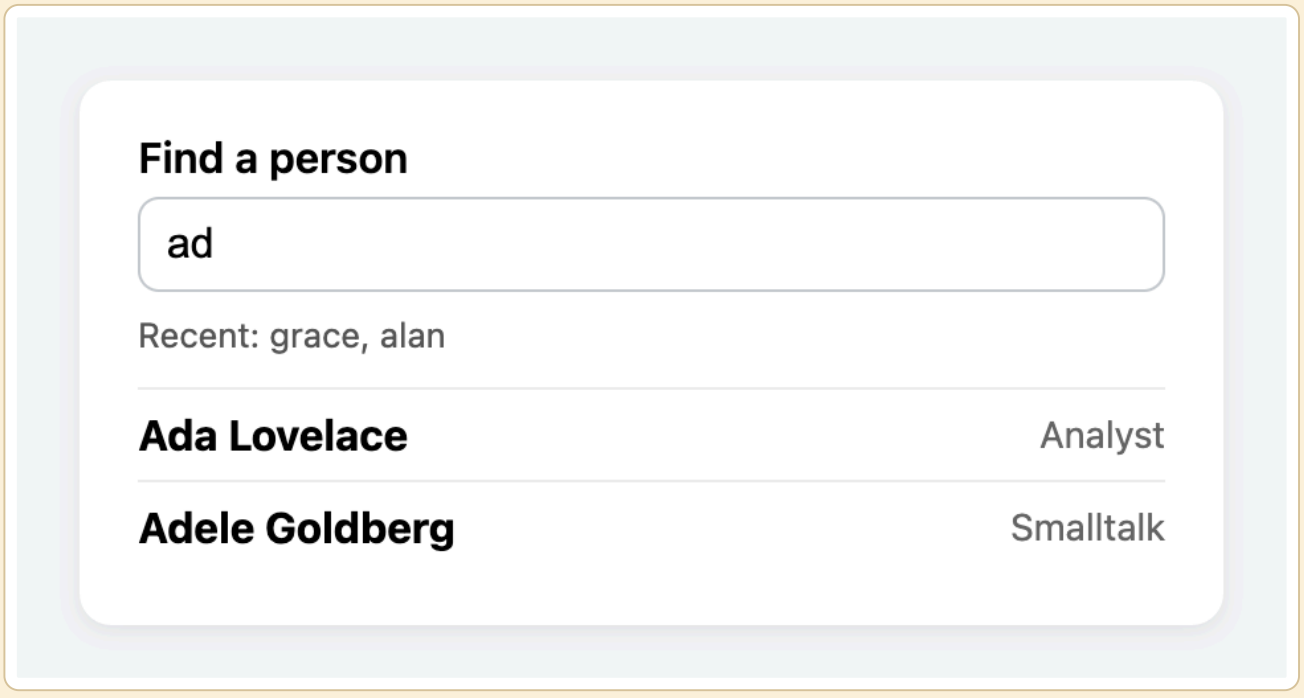


Figure 7.1 — `UserSearch` in Chrome. The component itself contains no timers, `fetch` calls or storage code — each concern is one Hook call.

Design notes: the component reads like a description of the feature, and each Hook is testable and reusable alone; swapping `useFetch` for TanStack Query's `useQuery` (Chapter 9) changes one line and adds caching; three typed characters made one request — debounce + abort keep the server quiet and results correct.

Chapter 7 TL;DR

Idea	Remember
How Hooks work	State lives in a per-component list, matched by **call order**
Rules of Hooks	Top level only (no `if`/loops/early-return/`try`/handlers); only in components and custom Hooks; enforced by `eslint-plugin-react-hooks`
Custom Hooks	`useX` functions that call Hooks; share **logic**, not state; name by purpose
`useLocalStorage`	Lazy read, write in Effect, `try/catch`, `storage` event; beware SSR mismatch
`useDebounce`	Timer in Effect + cleanup; debounce the derived value, not the input
`useFetch`	AbortController in cleanup, check `res.ok`, one status field — then prefer TanStack Query
`useEffectEvent`	Latest callback inside an Effect without re-subscribing (React 19.2)
`useId`	SSR-safe unique IDs for `htmlFor`/`aria-*`; never for keys
`useSyncExternalStore`	Read outside data; stable `subscribe`, cached `getSnapshot`, `getServerSnapshot`
`useTransition`	Mark updates non-urgent; `isPending`; async callbacks = Actions (19)
`useDeferredValue`	Deferred copy of a value; needs `memo`; `initialValue` in 19; not a debounce

Next: Chapter 8 — Styling.

Chapter 8 — Styling React Apps

React has no built-in styling system: it renders HTML elements and you style them with CSS. The question is *how you write, **scope** (limit which elements a rule can affect) and ship* that CSS. This chapter compares the approaches in real codebases, runs each, and helps you choose. It assumes CSS basics (selectors; the cascade — which conflicting declaration wins; specificity — selector "strength"; custom properties — CSS variables such as `--accent`); otherwise read the HTML & CSS book first — CSS fundamentals and modern CSS & architecture.

Code: `code/`. Node programs print what React puts in the DOM; visuals are screenshots of real pages in **headless** Chrome (no window, script-driven), bundled with esbuild, with Tailwind compiled by the Tailwind v4.3 **CLI** (command-line interface).

8.1 The styling landscape

The problem every approach solves

CSS was designed for documents: every rule is **global**, and conflicts are settled by the **cascade** (origin — browser default vs your stylesheet, specificity, order). In an app of hundreds of components that causes three pains:

1. **Name collisions** — two teams both write `.title`; the last-loaded file wins.
2. **Dead CSS** — nobody dares delete a rule because nobody knows who still uses it.
3. **Dynamic styles** — colours or sizes depending on props and state.

Family	How you write styles	Scoping	Runtime cost (extra browser work while the app runs)	Typical tools
Global CSS files	`.css` files, `className="card"`	Naming conventions (BEM, §8.2)	None	Plain CSS, Sass (a CSS preprocessor adding variables, nesting and mixins)
Inline styles	`style={{ color: 'red' }}`	Per element	Tiny	Built into React
CSS Modules	`.module.css`, `className={styles.card}`	Automatic, per file	None	Vite, Next.js, webpack
Utility-first CSS	`className="p-4 rounded-xl bg-white"`	Nothing to scope — classes are single-purpose	None	**Tailwind CSS**, UnoCSS
CSS-in-JS	Styles in JS (`styled.div\`…\``)	Automatic, generated names	Runtime libs: yes; zero-runtime: none	styled-components, Emotion; vanilla-extract, StyleX, Panda

The same card styled five ways (`code/gallery.html`, components in `code/cards.jsx`):

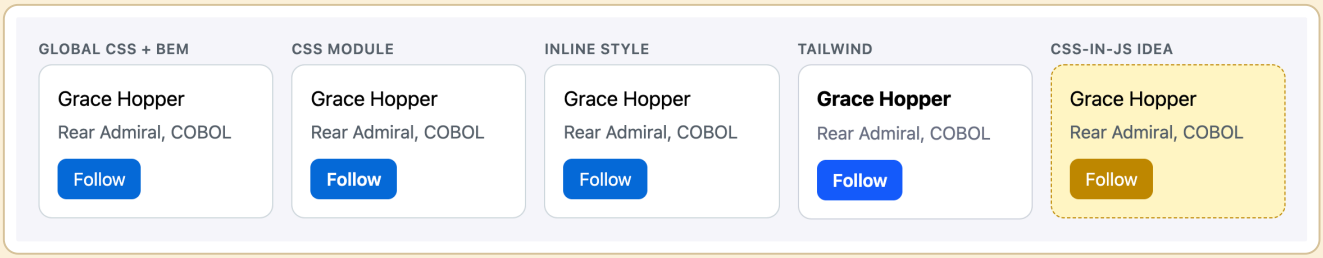


*Figure 8.1 — One component, five styling approaches. The output is ordinary HTML + CSS in every case; they differ in how the CSS is authored, scoped and delivered. (The Tailwind heading uses an explicit `font-bold` — Tailwind's **reset** (a stylesheet that wipes the browser's default styles), "Preflight", removes default heading styles on the whole page, which is why the other headings look lighter than browser defaults.)*

8.2 Plain CSS files

What it is

Write rules in a `.css` file and apply them with `className` (not `class`, a reserved word in JavaScript — §1.8). With a **bundler** (combines source files and their imports into browser files) such as Vite you can `import './Card.css'` from a component; the bundler adds it to the page.

```jsx
// cards.jsx
import './global.css';
export function GlobalCssCard({ name, role }) {
  const [following, setFollowing] = useState(false);
  return (
    <div className="profile-card">
      <h3 className="profile-card__name">{name}</h3>
      <p className="profile-card__role">{role}</p>
      <button className={cx('profile-card__button', following && 'profile-card__button--following')}
              onClick={() => setFollowing(f => !f)}>{following ? 'Following' : 'Follow'}</button>
    </div>
  );
}
```

```css
/* global.css — BEM naming keeps selectors from colliding by convention only. */
.profile-card { border: 1px solid #d0d7de; border-radius: 12px; padding: 16px; background: #fff; }
.profile-card__name { margin: 0 0 4px; font-size: 18px; }
.profile-card__button:hover { background: #0550ae; }
.profile-card__button--following { background: #eaeef2; color: #24292f; }
```

How it works

1. `import './global.css'` is a **bundler feature**, not JavaScript: imported CSS is collected into the page (a `<link>` in production, injected `<style>` in dev).
2. **Importing CSS in a component does not scope it** — the rules apply to the *whole page*, even to components that never imported the file.
3. Teams avoid collisions with naming conventions such as **BEM** (Block__Element--Modifier): `profile-card` (block), `profile-card__name` (element), `profile-card__button--following` (modifier).

Trade-offs

✅ Strengths	❌ Weaknesses
Zero tooling, zero runtime, full CSS power (`:hover`, media/container queries — rules for certain screen or container sizes, animations, nesting, `@layer`)	Global by default — scoping is only as good as your discipline
Easy for designers and CSS experts	Hard to know if a rule is still used; dead CSS accumulates
Browser caches the stylesheet separately from JS	Dynamic values need CSS variables or inline styles

Modern CSS removed much of the old pain: **native nesting**, **cascade layers** (`@layer` — decide which group of rules wins regardless of specificity) and **custom properties** work in current browsers (HTML & CSS book, Chapter 13). Sass is still common but no longer needed for variables or nesting.

React 19: stylesheets as components

Render `<link rel="stylesheet" href="/card.css" precedence="default" />` anywhere and React moves it into `<head>`, loads it once even if many components render it, orders it by `precedence`, and — inside a Suspense boundary (`<Suspense>` shows a fallback while content isn't ready) — waits for it before revealing the content. Mostly used by frameworks and libraries; a Vite app normally just `import`s CSS.

TL;DR

`className` + CSS file. Imports are global — use a naming convention or move to CSS Modules.

8.3 Inline styles — the `style` prop

What it is

Every HTML element accepts a **`style` prop**: an object whose keys are CSS properties in **camelCase** (`backgroundColor`, not `background-color`). React writes it to the element's `style` attribute. In `style={{ … }}` the outer braces are a JSX expression, the inner an object literal.

`code/01_inline_styles.jsx`:

```
function ProgressBar({ percent, color = 'seagreen' }) {
  return (
    <div style={{ background: '#eee', borderRadius: 4, height: 8 }}>            {/* numbers -> px */}
      <div style={{ width: `${percent}%`, height: '100%', backgroundColor: color, borderRadius: 4 }} />
    </div>
  );
}

function Examples() {
  return (
    <>
      <ProgressBar percent={40} />
      <p style={{ lineHeight: 1.5, zIndex: 2, opacity: 0.8, flexGrow: 1 }}>unitless properties stay unitless</p>
      <p style={{ '--accent': 'tomato', color: 'var(--accent)' }}>CSS custom property set inline</p>
      <p style={{ WebkitLineClamp: 2, msTransform: 'none' }}>vendor prefixes</p>
      <p style={{ ':hover': { color: 'red' }, '@media (max-width: 600px)': { display: 'none' } }}>no pseudo-classes or
media queries</p>
    </>
  );
}
```

Output (real `style` attributes; the first line is React's dev warning):

```
Unsupported style property @media (max-width: 600px). Did you mean @media (maxWidth: 600px)?
<div style="background: rgb(238, 238, 238); border-radius: 4px; height: 8px;">
<div style="width: 40%; height: 100%; background-color: seagreen; border-radius: 4px;">
<p style="line-height: 1.5; z-index: 2; opacity: 0.8; flex-grow: 1;">
<p style="--accent: tomato; color: var(--accent);">
<p style="-webkit-line-clamp: 2;">
```

How it works — the rules the output shows

1. **Numbers get `px`** for lengths (`borderRadius: 4` → `4px`), but **unitless properties** (`lineHeight`, `zIndex`, `opacity`, `flexGrow`, …) stay plain. Other units: strings (`'50%'`, `'2rem'`).
2. **Custom properties** (`'--accent'`) pass through unchanged — the cleanest way to feed a dynamic value into a stylesheet.
3. **Vendor prefixes** (browser-specific prefixes such as `-webkit-` from when features were experimental) start with a capital (`WebkitLineClamp`), except `ms` (`msTransform`). (jsdom, like modern browsers, dropped the obsolete `-ms-transform`.)
4. **Pseudo-classes (`:hover`), pseudo-elements (`::before`), media queries and keyframes are impossible**: a `style` attribute can't hold selectors or at-rules (`@…` statements). The last `<p>` got no style, and React warned.

When to use it (and when not)

Use inline styles for	Don't use them for
Values computed at runtime: progress width, drag position, a user-chosen colour	Whole-component styling (no hover/focus/responsive; hard to override)
Passing a value to CSS: `style={{ '--progress': ${p}% }}` with the rest in a stylesheet	Anything a designer will want to theme
Quick prototypes	Large lists where every row repeats the same object (use a class)

Inline styles have the **highest specificity** of normal declarations — a stylesheet can override them only with `!important`, another reason to reserve them for truly dynamic values.

TL;DR

`style={{ camelCase: value }}`; numbers → px except unitless properties; no `:hover` or media queries. Best for dynamic values, ideally via CSS variables.

8.4 CSS Modules

What it is

A **CSS Module** (`*.module.css`) has class names **local to the file that imports it**: the build tool renames every class uniquely and gives JavaScript an object mapping original to generated names. Normal CSS, no collisions. Supported out of the box by Vite, Next.js, Parcel, webpack's `css-loader` and esbuild.

`code/Card.module.css`:

```
.card { border: 1px solid #d0d7de; border-radius: 12px; padding: 16px; background: #fff; }
.title { margin: 0 0 4px; font-size: 18px; }
.role { margin: 0 0 12px; color: #57606a; }
.button { font: inherit; padding: 6px 14px; border-radius: 8px; border: 0; background: #0969da; color: #fff; cursor:
pointer; }
.button:hover { background: #0550ae; }
.primary { composes: button; font-weight: 600; }       /* reuse another class */
:global(.sr-only) { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); } /* escape
hatch */
```

Using it (`code/cards.jsx`):

```
import styles from './Card.module.css';
export function ModuleCard({ name, role }) {
  return (
    <div className={styles.card}>
      <h3 className={styles.title}>{name}</h3>
      <p className={styles.role}>{role}</p>
      <button className={styles.primary}>Follow</button>
    </div>
  );
}
```

`code/02_css_modules.mjs` runs esbuild on that file (bundling `import styles from './Card.module.css'` and printing the object and emitted CSS):

```
--- the JS object you get from `import styles from "./Card.module.css"` ---
{ card: "Card_card", title: "Card_title", role: "Card_role", button: "Card_button", primary: "Card_button Card_primary"
}
--- the CSS the bundler emits (renamed, so it cannot clash) ---
.Card_card {
  border: 1px solid #d0d7de;
  border-radius: 12px;
  padding: 16px;
  background: #fff;
}
.Card_title {
  margin: 0 0 4px;
  font-size: 18px;
}
... (role, button and button:hover renamed the same way)
.Card_primary {
  font-weight: 600;
}
.sr-only {
  position: absolute;
  ...
}
```

How it works

1. Each class selector is renamed: `.card` → `.Card_card` (esbuild's scheme; Vite and webpack add a short **hash** computed from the content, e.g. `_card_1x9ab_1`, so names are unique even across same-named files).
2. `import styles from …` returns the mapping; `styles.card` is the generated string.
3. `composes: button` makes `styles.primary` *both* names — `"Card_button Card_primary"` — reuse without duplicated rules.
4. `:global(.sr-only)` opts a selector out of renaming.
5. Only class names are scoped — element selectors (`h3 { … }`) and IDs are not — so keep module files to classes.

Trade-offs

✅	❌
Real CSS: hover, media/container queries, animations, nesting	Styles live in a second file (some like this, some don't)
Zero runtime, works with Server Components (Chapter 13)	Dynamic values still need CSS variables or inline styles
Unused-class detection with TypeScript plugins / linters	Class names with dashes need bracket access: `styles['is-active']` (or configure camelCase conversion)
Easy migration from plain CSS	Theming across modules relies on CSS variables (§8.8)

Common mistakes

- Naming the file `Card.css` (no `.module`) → nothing scoped, `styles` empty.
- `className="card"` instead of `className={styles.card}` → no style (the real name was changed).
- A non-existent class → `styles.tittle` is `undefined` and React silently renders `class` without it. TypeScript typings for modules catch this.

TL;DR

`*.module.css` + `styles.x`. Local class names, real CSS, no runtime. A safe default for teams that like writing CSS.

8.5 Composing class names from props

What it is

Components choose classes from props and state (a Button's `variant`, `size`, `loading`, plus an extra `className` from its parent). Building the string with `+` and ternaries is error-prone; the tiny `clsx` package (and older `classnames`) does it. `code/cx.js` is a 10-line version — no magic:

```
// cx('a', cond && 'b', { c: true, d: false }, ['e', null]) -> 'a b c e'
export function cx(...args) {
  const out = [];
  for (const a of args) {
    if (!a) continue;                                   // false, null, undefined, 0, ''
    if (typeof a === 'string' || typeof a === 'number') out.push(a);
    else if (Array.isArray(a)) { const inner = cx(...a); if (inner) out.push(inner); }
    else if (typeof a === 'object') for (const [k, v] of Object.entries(a)) if (v) out.push(k);
  }
  return out.join(' ');
}
```

`code/03_class_names.jsx`:

```
function Button({ variant = 'primary', size = 'md', loading = false, className, ...rest }) {
  return (
    <button
      className={cx('btn', `btn--${variant}`, `btn--${size}`, { 'btn--loading': loading }, className)}
      disabled={loading || rest.disabled}
      {...rest}
    />
  );
}
// <Button>Save</Button>  <Button variant="danger" size="sm">Delete</Button>
// <Button loading>Uploading</Button>  <Button variant="ghost" className="ml-auto">Cancel</Button>
```

Output:

```
"btn active lg"
Save      class="btn btn--primary btn--md"
Delete    class="btn btn--danger btn--sm"
Uploading class="btn btn--primary btn--md btn--loading" disabled
Cancel    class="btn btn--ghost btn--md ml-auto"
concatenated: "btn false undefined"
```

The last line is the bug `clsx` prevents: naive concatenation leaks `"false"` and `"undefined"` into the DOM.

Patterns worth knowing

- **Always accept and merge a `className` prop** in reusable components, so parents can add spacing/layout (`ml-auto`) without **forking** (copying and modifying) the component.
- **Variant maps** beat nested ternaries: `const VARIANTS = { primary: '…', danger: '…' }` then `VARIANTS[variant]`; `cva` (class-variance-authority) formalises this for Tailwind.
- **With CSS Modules**: `cx(styles.btn, styles[variant], loading && styles.loading)`.
- **With Tailwind**, conflicting utilities (`px-2` from the component, `px-4` from the parent) are *not* resolved by string order; use `tailwind-merge` (`twMerge(clsx(...))`) so the last wins (§8.6).

In a real project: `npm install clsx` and `import { clsx } from 'clsx'` — same API for these cases.

> **TL;DR**
>
> Build class strings with `clsx`/`cx`; accept a `className` prop; use variant maps instead of ternary chains.

8.6 Tailwind CSS

What it is

Tailwind CSS is a **utility-first** framework: instead of inventing class names and writing rules, you compose tiny single-purpose classes in your **markup** — `p-4` (padding), `rounded-xl`, `bg-white`, `text-gray-500`, `hover:bg-blue-700`. A build step scans your source and generates CSS for **only the classes you use**. Current major version: **v4** (released January 2025; v4.3 at the time of writing), configured in CSS rather than a JavaScript config file.

The Tailwind card from Figure 8.1:

```
export function TailwindCard({ name, role }) {
  return (
    <div className="rounded-xl border border-gray-300 bg-white p-4">
      <h3 className="mb-1 text-lg font-bold">{name}</h3>
      <p className="mb-3 text-gray-500">{role}</p>
      <button className="rounded-lg bg-blue-600 px-3.5 py-1.5 font-semibold text-white hover:bg-blue-
700">Follow</button>
    </div>
  );
}
```

How it works

1. One line in your main CSS, `@import "tailwindcss";`, brings in the default **theme** (colours, spacing, fonts as CSS variables), **Preflight** (a CSS reset) and the **utilities** layer.

2. The build tool (Vite plugin or CLI) scans source files for anything that looks like a class name; `@source "./path"` adds places it wouldn't find.
3. Each recognised class generates one rule. Variants are prefixes: `hover:`, `focus-visible:`, `md:` (screen ≥ 48rem = 768px; `rem` is relative to the root font size, usually 16px), `dark:`, `disabled:`, `group-hover:` …
4. Values come from the theme (`px-2` = 2 × `--spacing`), or an **arbitrary value** in brackets: `w-[37px]`, `grid-cols-[1fr_2fr]`.

`code/05_tailwind_build.mjs` runs the real Tailwind CLI on:

```
<span className="px-2 font-semibold hover:bg-blue-700 md:text-lg dark:text-white w-[37px]">new</span>
```

Output (only these utilities out of thousands possible):

```
@layer utilities {
  .w-\[37px\] {
    width: 37px;
  }
  .px-2 {
    padding-inline: calc(var(--spacing) * 2);
  }
  .font-semibold {
    --tw-font-weight: var(--font-weight-semibold);
    font-weight: var(--font-weight-semibold);
  }
  @media (hover: hover) {
    .hover\:bg-blue-700:hover {
      background-color: var(--color-blue-700);
    }
  }
  @media (width >= 48rem) {
    .md\:text-lg {
      font-size: var(--text-lg);
      line-height: var(--tw-leading, var(--text-lg--line-height));
    }
  }
  @media (prefers-color-scheme: dark) {
    .dark\:text-white {
      color: var(--color-white);
    }
  }
}
```

Variants become ordinary CSS (media queries, `:hover`); `hover:` applies only on devices that can hover (v4 behaviour); every value points at a theme variable — change `--color-blue-700` once and every use follows.

Setting it up (Vite — not run here)

```
npm install tailwindcss @tailwindcss/vite
```

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';
export default defineConfig({ plugins: [react(), tailwindcss()] });
```

```
/* src/index.css */
@import "tailwindcss";

@theme {                                  /* your design tokens -> new utilities */
  --color-brand: oklch(0.55 0.2 260);     /* enables bg-brand, text-brand, border-brand… */
  --font-display: "Inter", sans-serif;    /* enables font-display */
}
@custom-variant dark (&:where(.dark, .dark *));   /* optional: dark: follows a .dark class instead of the OS */
```

(Here the CLI did the same job: `tailwindcss -i tailwind.input.css -o tailwind.css` produced `code/tailwind.css` for the screenshot.)

Reusing styles: components, not `@apply`

Reuse long class lists by making a **component** (`<Button>`, `<Card>`) — the intended way in React. `@apply` (copying utilities into a custom class) brings back naming and dead-CSS problems; use it sparingly (e.g. styling Markdown output you don't control).

Common mistakes

Mistake	Why it fails	Fix
`className={`bg-${color}-500`}`	The scanner reads source *text*; it never sees `bg-red-500`, so no CSS is generated	Map to complete class names: `{ red: 'bg-red-500', green: 'bg-green-500' }[color]`
`className="px-2 px-4"` expecting px-4 to win	Winner is decided by order *in the generated CSS*, not in the attribute	Don't pass conflicting classes; merge with `tailwind-merge`
Forgetting Preflight resets headings, lists, buttons	`<h1>` looks like body text	Style explicitly (`text-3xl font-bold`) — intended
Huge unreadable class strings	Hard to review	Extract components; order with the official Prettier (code formatter) plugin

Trade-offs

✅	❌
No naming, no switching files, no dead CSS (unused classes are never generated)	Markup gets long and noisy
Consistent design tokens (named design values such as colours and spacing; §8.8) out of the box	A new vocabulary to learn; team must agree to use it
Tiny production CSS that stops growing with the app	Dynamic values need arbitrary values or CSS variables
Zero runtime; works with Server Components	Styles are coupled to markup — harder to restyle a whole app by editing one stylesheet (use theme variables)

Tailwind is the default in many starter kits (`create-next-app` offers it; shadcn/ui is built on it) and widely used by teams valuing iteration speed.

In the interview

30-second answer: "Tailwind is utility-first: I compose small classes like `p-4 rounded-xl hover:bg-blue-700` in JSX, and the build scans my files and generates only the CSS I used, so there's no naming, no dead CSS and no runtime. Reuse happens through React components. The traps are dynamically-built class names, which the scanner can't see, and conflicting utilities, which need `tailwind-merge`. v4 configures the theme in CSS with `@theme`."

- *Why won't `` `text-${size}` `` work?* — The scanner only sees complete class names in source text.

TL;DR

Utilities in JSX, CSS generated from what you use. Write complete class names, reuse via components, configure with `@theme`.

8.7 CSS-in-JS

What it is

CSS-in-JS = styles written in JavaScript, usually **colocated** with the component and able to use props:

```
// styled-components / Emotion style (not run — not installed)
import styled from 'styled-components';
const Button = styled.button`
  padding: 6px 14px;
  border-radius: 8px;
  background: ${props => (props.$danger ? 'crimson' : '#0969da')};
  &:hover { filter: brightness(0.9); }
`;
<Button $danger>Delete</Button>
```

Kind	How it works	Examples
Runtime	In the browser, the library turns the template (the **tagged template literal** after `styled.button`) into CSS, hashes it into a class name, and injects a `<style>` tag while rendering	styled-components, Emotion
Zero-runtime (build-time)	A compiler extracts the styles into static `.css` files during the build; only class names remain in JS	vanilla-extract, Linaria, Panda CSS, StyleX (Meta)

How runtime CSS-in-JS works — a tiny version

`code/cards.jsx` has a 5-line imitation using a React 19 feature: a `<style>` with `href` and `precedence` is **hoisted into `<head>` and de-duplicated** by React, however many components render it.

```
function css(strings) { /* hash the text into a class name, replace & with .className */ }
const cardStyle = css`& { border: 1px dashed #bf8700; border-radius: 12px; padding: 16px; background: #fff8c5; }
  & h3 { margin: 0 0 4px; font-size: 18px; } & p { margin: 0 0 12px; color: #57606a; }
  & button { font: inherit; padding: 6px 14px; border-radius: 8px; border: 0; background: #bf8700; color: #fff; }`;

export function CssInJsCard({ name, role }) {
  return (
    <div className={cardStyle.id}>
      <style href={cardStyle.id} precedence="medium">{cardStyle.text}</style>
      <h3>{name}</h3><p>{role}</p><button>Follow</button>
    </div>
  );
}
```

That is the yellow card in Figure 8.1. Real libraries add prop interpolation, theming, vendor prefixing and server-side extraction — and must insert styles *before* layout effects read the DOM, hence React's library-author Hook `useInsertionEffect` (§5.11).

Why the trend moved away from runtime CSS-in-JS

1. **Runtime cost**: **serialising** style objects to CSS text and hashing during render adds work every render, and injected rules force style recalculation.
2. **Server Components** (Chapter 13): runtime libraries rely on context and client-side rendering, so styled components must live in `'use client'` files.
3. **Streaming SSR** (HTML sent in chunks as ready) needs extra setup to collect styles per request.

In **March 2025 styled-components entered maintenance mode** (bug and security fixes only); its maintainer advised against adopting it for new projects. Existing apps keep working, and Emotion remains widely used (e.g. inside MUI). New projects mostly choose CSS Modules, Tailwind, or a **zero-runtime** library keeping colocated, typed authoring without the runtime.

Trade-offs

✅	❌
Styles next to the component; dynamic styles from props	Runtime libs: slower renders, bigger bundles, RSC (React Server Components) friction
Automatic scoping; dead styles die with the component	Another abstraction to learn; debugging generated class names
Zero-runtime libs: type-safe tokens, static CSS output	Zero-runtime libs need build-tool plugins

TL;DR

Runtime CSS-in-JS (styled-components, Emotion) is convenient but costs render time and clashes with Server Components; zero-runtime tools (vanilla-extract, StyleX, Panda) or CSS Modules/Tailwind are the usual picks for new work.

8.8 Theming and dark mode

What it is

Theming switches a whole set of design values (colours, radii, fonts) at once. The modern, framework-independent way: **CSS custom properties** as **design tokens** — components only reference `var(--bg)`, `var(--accent)`; a theme is a different set of values.

How it works

1. Define tokens on `:root`; override them under an attribute selector such as `[data-theme="dark"]`.
2. A small React **provider** (supplies a value to its subtree via context) stores the choice (`light` / `dark` / `system`), resolves `system` via the OS preference (`prefers-color-scheme`), and writes `<html data-theme="…">`.
3. CSS does the rest — no component re-renders to recolour.
4. Save the choice in `localStorage`, and apply it with a tiny inline script **before paint** to avoid a **flash of the wrong theme**.
5. Set CSS `color-scheme` so browser-drawn parts (form controls, scrollbars) match.

`code/theme.css` (tokens):

```
:root {
  --bg: #ffffff; --surface: #f6f8fa; --text: #1f2328; --muted: #59636e;
  --border: #d0d7de; --accent: #0969da; --accent-text: #ffffff;
}
[data-theme="dark"] {
  --bg: #0d1117; --surface: #161b22; --text: #e6edf3; --muted: #9198a1;
  --border: #3d444d; --accent: #4493f8; --accent-text: #0d1117;
}
body  { background: var(--bg); color: var(--text); }
.panel { background: var(--surface); border: 1px solid var(--border); border-radius: 12px; padding: 16px; }
.cta  { background: var(--accent); color: var(--accent-text); }
```

`code/theme.jsx` (the provider — the OS setting is an external store, so it uses `useSyncExternalStore`, §7.9):

```
const ThemeContext = createContext(null);
const QUERY = '(prefers-color-scheme: dark)';

function subscribeSystem(cb) { const m = window.matchMedia(QUERY); m.addEventListener('change', cb); return () =>
m.removeEventListener('change', cb); }
function useSystemDark() { return useSyncExternalStore(subscribeSystem, () => window.matchMedia(QUERY).matches, () =>
false); }

export function ThemeProvider({ children }) {
  const [choice, setChoice] = useState(() => localStorage.getItem('theme') ?? 'system'); // 'light' | 'dark' | 'system'
  const systemDark = useSystemDark();
  const resolved = choice === 'system' ? (systemDark ? 'dark' : 'light') : choice;

  useEffect(() => {                       // sync to the <html> element: CSS does the rest
    document.documentElement.dataset.theme = resolved;
    document.documentElement.style.colorScheme = resolved;   // native controls + scrollbars follow
    localStorage.setItem('theme', choice);
  }, [choice, resolved]);

  return <ThemeContext.Provider value={{ choice, resolved, setChoice }}>{children}</ThemeContext.Provider>;
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used inside <ThemeProvider>');
  return ctx;
}
// ThemeSwitcher renders three role="radio" buttons: light / dark / system
```

`code/04_theme_provider.jsx` drives it with a fake OS setting:

```
first visit (OS light)     choice=system resolved=light | <html data-theme="light"> | saved=system
OS switches to dark        choice=system resolved=dark | <html data-theme="dark"> | saved=system
user picks "light"         choice=light resolved=light | <html data-theme="light"> | saved=light
OS changes again (ignored) choice=light resolved=light | <html data-theme="light"> | saved=light
reload                     choice=light resolved=light | <html data-theme="light"> | saved=light
outside provider -> useTheme must be used inside <ThemeProvider>
```

The same page in Chrome with the OS in light and dark mode (`code/theme.html`, "system" selected):

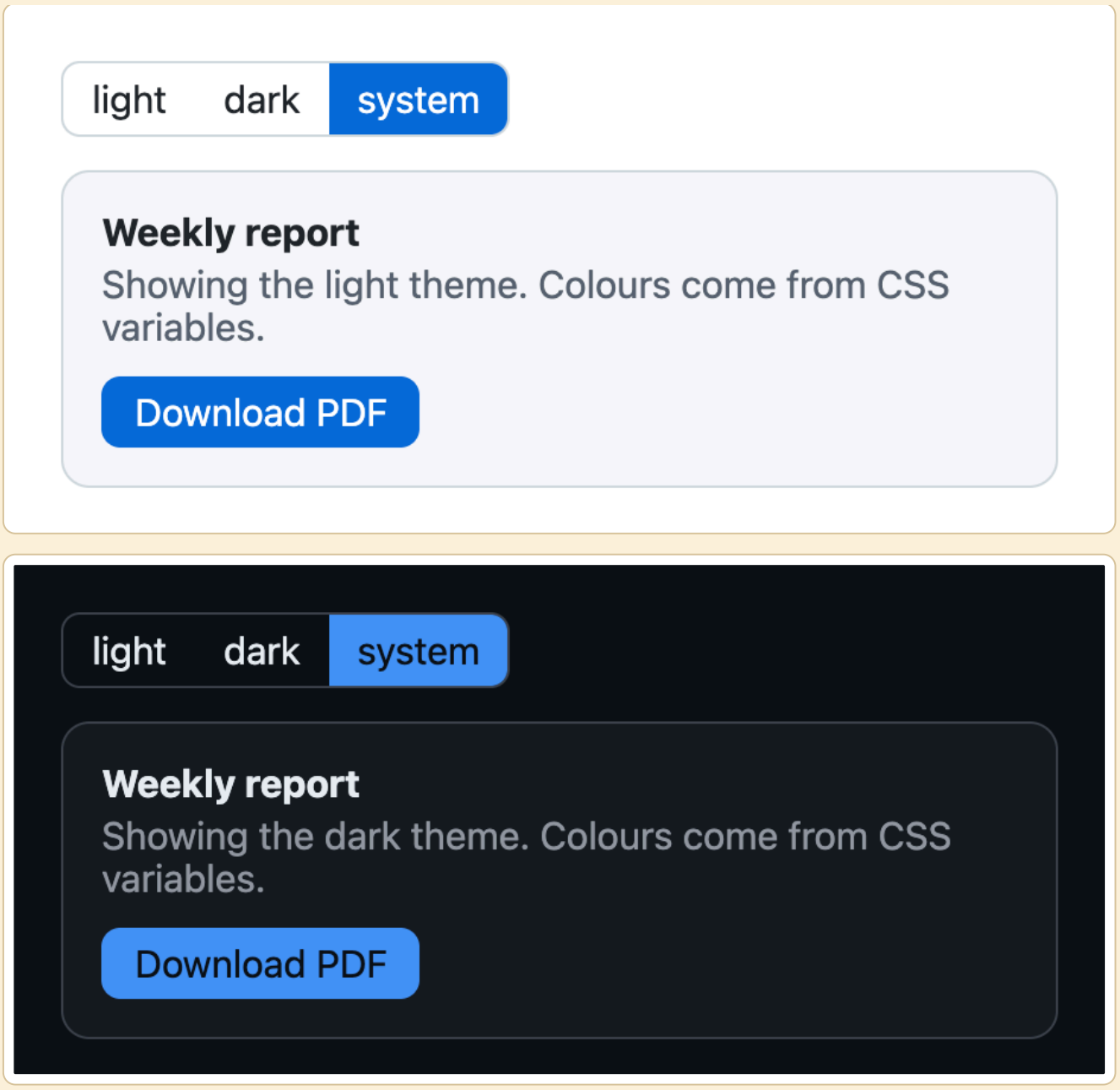


Figure 8.2 — One set of components, two token sets. Only the `data-theme` attribute on `<html>` differs.

Avoiding the flash

React runs after HTML and CSS are parsed, so a theme applied only in an Effect briefly shows dark-mode users the light page. `code/theme.html` uses a blocking script in `<head>`:

```
<script>
  try {
    const saved = localStorage.getItem('theme') || 'system';
    const dark = saved === 'dark' || (saved === 'system' && matchMedia('(prefers-color-scheme: dark)').matches);
    document.documentElement.dataset.theme = dark ? 'dark' : 'light';
  } catch {}
</script>
```

SSR frameworks need the same trick (`next-themes` injects it); a **cookie** (sent to the server with every request) lets the server render the right theme directly.

Alternatives and extras

- **CSS only**: without a manual toggle, `@media (prefers-color-scheme: dark) { :root { … } }` suffices — no JavaScript.
- `light-dark()`: with `color-scheme: light dark`, `color: light-dark(#111, #eee)` picks a value per scheme (Baseline since 2024 — supported in all major current browsers).

- **Tailwind**: `dark:` follows the OS by default; `@custom-variant dark (&:where(.dark, .dark *))` or a `[data-theme=dark]` selector makes it follow your toggle.
- **Component libraries** expose a `ThemeProvider` filling the same role (§8.9).
- **Why not colours in React context?** Every theme change would re-render every consumer; CSS variables recolour without rendering.

Common mistakes

- Hard-coded colours (`color: '#333'`) that ignore the theme — always use tokens.
- Reading `localStorage` during server rendering (it doesn't exist there) — read on the client or use a cookie.
- Only inverting colours: dark themes also need adjusted shadows, borders and less saturated accents, and must meet contrast ratios (WCAG 2.2 — Web Content Accessibility Guidelines — requires 4.5:1 for normal text).

TL;DR

Tokens as CSS variables, overridden by `[data-theme]`; a provider resolves light/dark/system and writes the attribute; a head script prevents the flash; `color-scheme` for native controls.

8.9 Component libraries

What it is

A **component library** provides ready-made, accessible components — buttons, dialogs, menus, date pickers, tables. Accessibility is the main reason: a correct dropdown menu needs focus management, keyboard navigation and ARIA roles (e.g. `role="menu"`, telling screen readers what a custom element is) that take weeks to get right (Chapter 14; HTML & CSS accessibility chapter).

Three kinds

Kind	What you get	Examples	Best when
Styled kits	Complete components with a design system (shared visual rules and components) and theming API	MUI (Material UI), Mantine, Chakra UI, Ant Design, React Bootstrap	Internal tools, admin panels, fast delivery, you accept their look
Headless (unstyled) primitives	Behaviour + accessibility, no styles — you bring CSS/Tailwind	Radix UI Primitives, React Aria (Adobe), Headless UI (Tailwind Labs), Base UI	A custom brand design that must still be accessible
Copy-in components	Source code copied into *your* repo (via a CLI), built on headless primitives + Tailwind	shadcn/ui	You want to own and edit the code, not depend on a package's API

How to choose

Question	Leans towards
Is there a designer with a custom design system?	Headless or copy-in
Do we need 50 components next week?	Styled kit
Are we using Tailwind?	shadcn/ui, Headless UI, Radix + Tailwind
Server Components / Next.js App Router?	Check the library's RSC guidance — many interactive components need `'use client'`
Bundle-size sensitive (public site)?	Headless + your CSS; import only what you use

Common mistakes

- Fighting a styled kit's design with overrides everywhere — pick headless instead.

- Mixing two styled kits (two resets, two themes, two button styles).
- Wrapping a library component so thinly that its API leaks everywhere — wrap it once in your own `<Button>` so you can swap later.

TL;DR

Styled kit for speed, headless for custom design with accessibility, copy-in (shadcn/ui) to own the code. Never hand-roll complex widgets without a reason.

8.10 Choosing an approach

Situation	Sensible default
New app, team likes utility classes and fast iteration	**Tailwind** + `clsx` / `tailwind-merge`, headless or shadcn/ui components
New app, team prefers writing CSS	**CSS Modules** + CSS variables for tokens
Content site with little interactivity	Plain CSS (with `@layer` and nesting) or CSS Modules
Existing styled-components/Emotion codebase	Keep it; consider zero-runtime or CSS Modules for new areas, especially if moving to Server Components
Design system shared across apps	Tokens as CSS variables + headless primitives; publish compiled CSS
Truly dynamic values (drag, progress, user colours)	Inline `style`, ideally setting a CSS variable

Whatever you choose: **one approach per codebase**, **tokens in CSS variables**, **components as the unit of reuse**, and **accessibility first** — never remove focus outlines without replacing them.

In the interview

30-second answer: "React doesn't prescribe styling. Global CSS is simple but needs naming conventions; CSS Modules scope classes per file with zero runtime; Tailwind uses utility classes and generates only what I use; runtime CSS-in-JS like styled-components is convenient but adds render cost and doesn't fit Server Components, so zero-runtime tools are preferred now. Inline styles are for truly dynamic values. For theming I use CSS variables switched by a data attribute."

- *Dark mode?* — Tokens as CSS variables, `[data-theme=dark]` overrides, a provider for the toggle, a head script against the flash, `color-scheme` for native controls; no component re-renders needed.
- *CSS Modules vs Tailwind?* — Both scoped and zero-runtime; Modules keep CSS in CSS files with semantic names (`composes` reuses another class), Tailwind keeps styling in markup with a token scale. Team preference decides.
- *Does `import './Card.css'` scope the rules?* — No; imported CSS is global.
- *`style={{ zIndex: 2, width: 100 }}`?* — `z-index: 2; width: 100px`.
- *Why `className`, not `class`?* — JSX compiles to JavaScript, where `class` is reserved, and it matches the DOM property `element.className`.

Chapter 8 TL;DR

Approach	Remember
Plain CSS	`className`; imports are global; BEM or layers for order
Inline `style`	camelCase object; numbers → px (except unitless); no `:hover`/media; for dynamic values
CSS Modules	`*.module.css`, `styles.x`; local names, `composes`, `:global`; zero runtime
Class composition	`clsx`/`cx`; accept `className`; variant maps
Tailwind v4	Utilities in JSX; only used classes generated; `@theme` tokens; no dynamic class names; `tailwind-merge` for conflicts
CSS-in-JS	Runtime (styled-components, Emotion) vs zero-runtime (vanilla-extract, StyleX, Panda); runtime is out of favour for new RSC apps
Theming	CSS variables + `data-theme`; provider for choice; head script against flash; `color-scheme`; `light-dark()`
Libraries	Styled kits vs headless vs copy-in (shadcn/ui); accessibility is the reason

Next: Chapter 9 — Routing and Data Fetching.

Chapter 9 — Routing and Data Fetching

Real apps have many screens and live on server data. This chapter covers **React Router** (which screen for which URL and, in data mode, loading that screen's data) and **TanStack Query** (caching — keeping a reusable local copy of — and synchronising server data), then React's own async tools **Suspense** (a fallback while something loads) and `use()` (reads a promise's value during render), and ends with loading/error-state patterns. The manual way — fetching in an Effect with cleanup — is in Chapter 5 and the `useFetch` Hook in Chapter 7; why server data is a different kind of state is in Chapter 6.

Versions: **React 19.3**, **React Router 8.4**, **TanStack Query 5.104**. Code in `code/` runs in Node with jsdom (a browser-like DOM for Node) against a tiny in-memory API (`code/api-server.js`, started on `127.0.0.1` per script); `[server]` lines in outputs are its log. Listings below show the essential lines; the files are complete.

9.1 Client-side routing

What it is

Routing chooses what to show from the **URL** (the web address, e.g. `/products/3?sort=price`). In a multi-page site (MPA) the *server* routes: every click requests a new HTML page and the browser reloads. In a **single-page application (SPA)** the page loads once and a **client-side router** in JavaScript:

1. Intercepts clicks on its `<Link>`s (`event.preventDefault()`) and updates the address bar with the **History API** (the browser's functions for changing the URL and history, e.g. `history.pushState`) — no request, no reload.
2. Listens for Back/Forward (the `popstate` event).
3. Matches the new URL against your **route definitions** and renders the matching components. Your JS and state stay loaded.

Why the URL matters

The URL *is* state — the most shareable state you have. Anything a user might bookmark, share, refresh or go "back" to (page, product, tab, filters, sort, page number) belongs in the URL, not only in `useState`.

Trade-offs

Client-side routing	Server routing (MPA)
Instant page changes, state survives navigation	Every page is a fresh load
Needs JS; first load downloads the app	Works without JS
Server must return `index.html` for *every* app path (else a refresh on `/products/3` gets 404 Not Found)	Each URL maps to a real page
SEO (search-engine optimisation) and first paint (first time content appears) need SSR/pre-rendering (HTML produced on the server per request or ahead of time) for public content (Chapter 13)	Good by default

---

9.2 React Router v8 and its three modes

What it is

React Router is the most widely used React router. **v8** (June 2026) is a small upgrade from v7: requires React 19.2.7+ and Node 22.22+, is ESM-only (only `import`/`export` modules, no `require`), removes the `react-router-dom` package, and turns v7's "future flags" (opt-in switches, notably **middleware**, §9.7) on by default.

```
npm install react-router          # one package; there is no react-router-dom in v8
```

```
import { Link, useParams, createBrowserRouter } from 'react-router';   // everything...
import { RouterProvider } from 'react-router/dom';                      // ...except the DOM-specific pieces
```

Three **modes** — pick one per app; each includes everything above it:

Mode	You write	Router provides	Use when
Declarative	`<BrowserRouter>` + `<Routes>`/`<Route>` JSX	URL matching, links, params, navigation	Simple SPAs, adding routing to an existing app, fetching your own way (e.g. TanStack Query)
Data	Route **objects** to `createBrowserRouter`, rendered with `<RouterProvider>`	+ **loaders**, **actions**, pending states, route error boundaries, fetchers, middleware	SPAs that want data loading tied to navigation
Framework	Route **modules** (files) + the React Router Vite plugin	+ server rendering, per-route code splitting, type generation, pre-rendering	Full-stack apps (successor of Remix)

§9.3–9.4 use declarative mode, §9.5–9.7 data mode. Examples use the **memory** variants (`<MemoryRouter>`, `createMemoryRouter`), which keep history in memory; in a browser use `<BrowserRouter>`/`createBrowserRouter` with identical routes.

Alternatives: **TanStack Router** (type-safe paths/params, built-in search-param validation) and file-based routers (**Next.js** etc.).

9.3 Routes, links and URL parameters (declarative mode)

How it works

1. Wrap the app in `<BrowserRouter>` (tests: `<MemoryRouter>`).
2. `<Routes>` holds `<Route path element>` entries. React Router picks the **best** match by specificity — static **segments** (pieces between slashes) beat dynamic `:params`, which beat `*` — so write order doesn't matter.
3. **Dynamic segments**: `/products/:productId` matches `/products/3`; `useParams()` returns `{ productId: '3' }` (always strings).
4. `path="*"` matches anything left: your **404** page.
5. `<Link to>` navigates; **`<NavLink>`** knows if it's **active** and passes `isActive` to `className`/`style` (`end` = match exactly).
6. **Search params** (`?sort=price`): `useSearchParams()`, like `useState` but stored in the URL.
7. **`useNavigate()`** navigates in code: `navigate('/products')`, `navigate(-1)` (back), `navigate('/login', { replace: true })` (replace the history entry). Declaratively: `<Navigate to="/login" />`.

`code/01_declarative_routing.jsx` (abridged):

```
function Nav() {
  const cls = ({ isActive }) => (isActive ? 'active' : undefined);   // NavLink knows if it matches the URL
  return <nav><NavLink to="/" end className={cls}>Home</NavLink> <NavLink to="/products" className=
{cls}>Products</NavLink></nav>;
}
function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();       // ?sort=price
  const sort = searchParams.get('sort') ?? 'name';
  // ...sorted list of <Link to={`/products/${p.id}`}>, plus:
  // <button onClick={() => setSearchParams({ sort: 'price' })}>Sort by price</button>
}
function ProductPage() {
  const { productId } = useParams();                                 // from the :productId segment
  const navigate = useNavigate();
  // ...<button onClick={() => navigate('/products')}>  <button onClick={() => navigate(-1)}>
}
function NotFound() { return <h1>404 - nothing at {useLocation().pathname}</h1>; }

function App() {
  return (
    <>
      <Nav />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<ProductList />} />
        <Route path="/products/:productId" element={<ProductPage />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </>
  );
}
```

Output:

```
start                   url=/                    active=Home     h1="Welcome"
click Products          url=/products            active=Products h1="Products (by name)"
sort by price           url=/products?sort=price active=Products h1="Products (by price)"
   list order: Mouse, Keyboard, Monitor
click Monitor           url=/products/3          active=Products h1="Monitor - $199"
navigate(-1)            url=/products?sort=price active=Products h1="Products (by price)"
click Mouse             url=/products/2          active=Products h1="Mouse - $19"
navigate("/products")   url=/products            active=Products h1="Products (by name)"
deep link /nope -> 404 - nothing at /nope
```

- "Products" stays **active** on `/products/3` (NavLink matches descendants); "Home" uses `end`.
- The sort lives in the URL, so **Back** returned to `?sort=price`; `useState` would have lost it.

Common mistakes

Mistake	Fix
`<a href="/products">` inside the app	Full reload, state lost — use `<Link to>` (keep `<a>` for external sites)
Treating `useParams()` values as numbers	They're strings: `Number(productId)`
Filters/tabs/page in `useState`	Put shareable UI state in search params
Calling `navigate()` during render	Navigate in handlers/Effects, or render `<Navigate>`
Server 404s on refresh of `/products/3`	Fallback to `index.html` (Vite preview, Netlify `_redirects`, nginx `try_files`)

9.4 Nested routes and layouts

What it is

Layouts (a header on every page, a sidebar on account pages) are modelled by **nesting** routes: the parent renders shared UI and an `<Outlet />` where the matching child appears. URL segments nest too: `users` + `:userId` = `/users/42`.

Route kind	Written as	Matches	Use
Index route	`<Route index element=…>`	The parent's own URL exactly	Default child ("Pick a user" at `/users`)
Layout (pathless) route	`<Route element={<Layout/>}>`, no `path`	Nothing itself; wraps children	Shared UI without a URL segment
Static vs dynamic siblings	`new` and `:userId` side by side	`/users/new` → static wins	Ranking, not order

`code/02_nested_routes.jsx` (abridged):

```
function AppLayout() { return <div><header>MyShop</header><main><Outlet /></main><footer>(c) 2026</footer></div>; }
function AccountLayout() {                          // pathless layout: groups routes, adds no URL segment
  const user = { name: 'Ada' };
  return <section><aside>Account menu</aside><Outlet context={user} /></section>;
}
function Settings() { const user = useOutletContext(); return <h1>Settings for {user.name}</h1>; }
function UsersLayout() {
  return <div><nav><Link to="new">New user</Link> <Link to="42">User 42</Link> <Link to="..">Up</Link></nav><Outlet />
</div>;
}
<Routes>
  <Route element={<AppLayout />}>                          {/* layout route: no path */}
    <Route index element={<h1>Home</h1>} />                {/* "/" exactly */}
    <Route path="users" element={<UsersLayout />}>
      <Route index element={<UsersIndex />} />             {/* /users */}
      <Route path="new" element={<NewUser />} />           {/* static beats dynamic */}
      <Route path=":userId" element={<User />} />          {/* /users/42 */}
    </Route>
    <Route element={<AccountLayout />}><Route path="settings" element={<Settings />} /></Route>
  </Route>
</Routes>
```

Output:

```
/           -> header  h1="Home"
/users      -> header+nav  h1="Pick a user"
/users/new  -> header+nav  h1="Create a user"
/users/42   -> header+nav  h1="User #42"
/settings   -> header+aside  h1="Settings for Ada"
from /users/42 click "new" -> header+nav  h1="Create a user"
click ".."              -> header  h1="Home"
```

How it works

1. The router finds the **chain** of matching routes (`AppLayout → UsersLayout → User`), renders the outermost, and each `<Outlet />` renders the next.
2. **Relative links** (no leading `/`) resolve against the **route** rendering them: `to="new"` in `UsersLayout` is `/users/new` even at `/users/42`; `..` goes to the parent *route* (here the pathless layout at `/`).
3. `<Outlet context={value}>` + `useOutletContext()` pass data from a layout to its rendered child.

Why it matters

Layouts don't re-mount when moving between their children — header state, sidebar scroll and open menus survive.

9.5 Loaders — data before render (data mode)

What it is

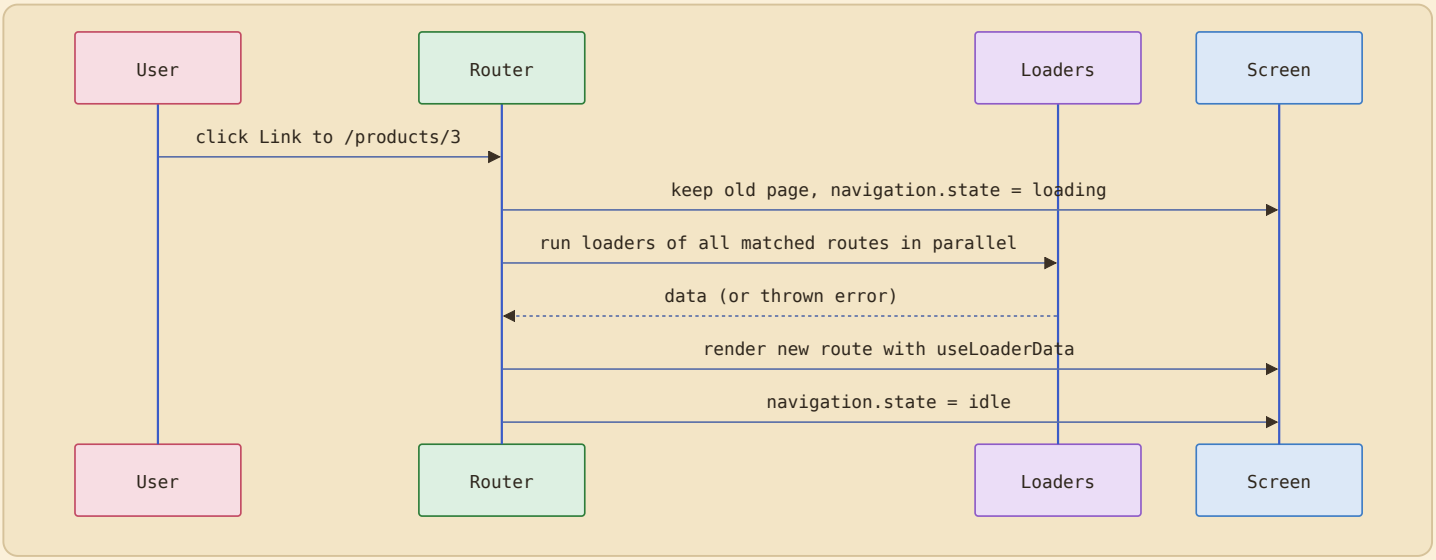
A route's `loader` is an async function that runs **when navigation to the route starts, before it renders**; the component reads the result with `useLoaderData()`.

The problem it solves

Effect fetching (Chapter 5) starts only *after* render, and nested components wait for their parent before starting their own request — a **waterfall** (request → render → request…). Loaders start **all** requests for **all** matched routes **in parallel** on click; the component renders once, with data.

How it works

1. Define route objects `{ path, Component, loader, ErrorBoundary, children }`; `createBrowserRouter(routes)` (tests: `createMemoryRouter(routes, { initialEntries })`); render `<RouterProvider router={router} />`.
2. On navigation, matched loaders run in parallel with `{ params, request, context }` (`request.signal` aborts if the user navigates away).
3. Meanwhile the **current** screen stays and `useNavigation().state` is `'loading'` — drive a progress bar from it.
4. On resolve, the new route renders with `useLoaderData()`.
5. If a loader **throws**, the nearest route `ErrorBoundary` renders. Throw `data(message, { status })` for expected HTTP-like errors; check with `isRouteErrorResponse(error)`. (No `ErrorBoundary` at all → React Router's default error screen.)
6. On first load there's no current screen, so the root route's `HydrateFallback` shows until the app has **hydrated** (React started in the browser).



`code/03_loaders_and_errors.jsx` (abridged):

```
async function getJson(path, request) {
  const res = await fetch(api.base + path, { signal: request.signal });   // aborted if the user navigates away
  if (res.status === 404) throw data((await res.json()).message, { status: 404 });
  if (!res.ok) throw new Error(`API error ${res.status}`);
  return res.json();
}
function Root() {
  const navigation = useNavigation();               // 'idle' | 'loading' | 'submitting'
  return <div><nav>{/* links */}</nav>{navigation.state === 'loading' && <p role="status">Loading…</p>}<Outlet /></div>;
}
function Product() { const product = useLoaderData(); return <h1>{product.name} costs ${product.price}</h1>; }
function ProductError() {
  const error = useRouteError();                    // whatever the loader threw
  if (isRouteErrorResponse(error)) return <h1>{error.status}: {error.data}</h1>;
  return <h1>Unexpected: {error.message}</h1>;
}
const router = createMemoryRouter([{
  path: '/', Component: Root, HydrateFallback: () => <p>Starting…</p>,
  children: [
    { index: true, Component: ProductList,
      loader: async ({ request }) => {              // two requests in PARALLEL, not a waterfall
        const [products, categories] = await Promise.all([getJson('/api/products', request), getJson('/api/categories',
request)]);
        return { products, categories };
      } },
    { path: 'products/:id', Component: Product, ErrorBoundary: ProductError,
      loader: ({ params, request }) => getJson(`/api/products/${params.id}`, request) },
  ],
}], { initialEntries: ['/'] });
```

Output:

```
first paint: Starting…
  [server] start GET /api/products
  [server] start GET /api/categories
  [server] end   GET /api/products -> 200
  [server] end   GET /api/categories -> 200
loaded     : 3 products in 2 categories
  [server] start GET /api/products/3
during nav : Loading… | still showing: 3 products in 2 categories
  [server] end   GET /api/products/3 -> 200
after nav  : Monitor costs $199
  [server] start GET /api/products/9
  [server] end   GET /api/products/9 -> 404
missing    : 404: No product 9 | nav still rendered: true
```

The two list requests ran **in parallel**; the **old** page stayed during navigation (no blank flash); the 404 was contained by the product route's `ErrorBoundary`, so the root nav kept working.

Details worth knowing

- **Revalidation**: after an action (§9.6) or `useRevalidator().revalidate()`, loaders on the page re-run; a route's `shouldRevalidate` can opt out.
- **Streaming non-critical data**: return an object with an *un-awaited* promise (`return { product: await p, reviews: reviewsPromise }`) and render it with `<Await resolve={reviews}>` inside `<Suspense>`, or with `use()` (§9.11). The page shows once critical data is ready.
- **Loaders are not components**: no Hooks; plain async functions, easy to unit-test.
- **Loaders don't cache**: going back re-runs them. Pair with a cache (TanStack Query, §9.13) when that matters.

9.6 Actions, `<Form>` and fetchers

What it is

An **action** is a route's function for **writes**. `<Form method="post">` works like an HTML form but, instead of reloading, **serialises** the fields into a `Request`, calls the route's `action`, then **automatically re-runs the page's loaders**.

How it works

1. `<Form method="post">` submits to its own route's action (or `action="/path"`).
2. The action reads `await request.formData()`, validates, writes.
3. It returns **validation errors** — `return data({ errors }, { status: 400 })`, read via `useActionData()` — or `redirect('/somewhere')` (*Post/Redirect/Get*: after a POST, send the browser to a GET page so Refresh doesn't resubmit).
4. `useNavigation().state === 'submitting'` while it runs — disable the button.
5. Afterwards the router **revalidates**: every `useLoaderData()` gets fresh data.
6. `useFetcher()` does the same *without navigating* (like buttons, inline edits, quick-add, several concurrent submissions), with its own `fetcher.state` and `fetcher.data`.

`code/04_actions_and_forms.jsx` (abridged):

```
async function newTodoAction({ request }) {
  const form = await request.formData();
  const title = String(form.get('title') ?? '').trim();
  if (title.length < 3) return data({ errors: { title: 'At least 3 characters' } }, { status: 400 });
  await postTodo(title);
  return redirect('/todos');                          // Post/Redirect/Get
}
function TodoList() {
  const todos = useLoaderData();
  const fetcher = useFetcher();                       // submit WITHOUT navigating
  return (/* list, then */
    <fetcher.Form method="post" action="/todos/new">
      <input name="title" aria-label="quick add" defaultValue="Buy milk" />
      <button>{fetcher.state === 'idle' ? 'Quick add' : 'Adding…'}</button>
    </fetcher.Form>);
}
function NewTodo() {
  const actionData = useActionData();                 // validation errors from the action
  const navigation = useNavigation();
  return (
    <Form method="post">
      <input name="title" aria-label="title" />
      {actionData?.errors?.title && <p role="alert">{actionData.errors.title}</p>}
      <button disabled={navigation.state === 'submitting'}>{navigation.state === 'submitting' ? 'Saving…' : 'Save'}
</button>
    </Form>);
}
// routes: /todos (index: TodoList + todosLoader), /todos/new (NewTodo + newTodoAction)
```

Output:

```
list      : 1 todos: Read chapter 9
too short : At least 3 characters | url still /todos/new
  [server] start POST /api/todos
submitting: button says "Saving…"
  [server] end   POST /api/todos -> 201
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
redirected: /todos | 2 todos: Read chapter 9, Write the router demo
  [server] … same POST + GET pair
fetcher   : /todos | 3 todos: Read chapter 9, Write the router demo, Buy milk
```

The short title never reached the server; after each POST the router re-fetched `GET /api/todos` **by itself**.

Relation to React 19 form actions

React's `<form action={fn}>`, `useActionState`, `useFormStatus` (Chapter 4) solve this at component level. The router's `<Form>` adds URL-addressed actions, redirects and automatic revalidation of route data — prefer it (and fetchers) for route data in data mode.

9.7 Middleware, lazy routes and framework mode

Middleware (v8: on by default)

Middleware runs **before** a route's loaders/actions, parent to child, and can act before and after the rest of the chain by awaiting `next()`. Uses: **authentication guards** (redirect logged-out users to login), logging/timing, putting shared data (current user) into a typed **context** for loaders.

`code/05_middleware.jsx` (abridged):

```
const userContext = createContext(null);                    // typed slot in the router context (from react-router)
async function logMiddleware({ request }, next) {
  const path = new URL(request.url).pathname;
  console.log(`  -> ${path}`);
  await next();                                             // child middleware + loaders
  console.log(`  <- ${path} done`);
}
async function authMiddleware({ request, context }) {
  if (!session) {
    console.log('  auth: no session, redirecting');
    throw redirect(`/login?next=${encodeURIComponent(new URL(request.url).pathname)}`);
  }
  context.set(userContext, session);                        // loaders below can read it
}
const routes = [{ path: '/', Component: Outlet, middleware: [logMiddleware], HydrateFallback: () => null, children: [
  { path: 'login', Component: Login },
  { path: 'dashboard', Component: Dashboard, middleware: [authMiddleware],
    loader: ({ context }) => ({ user: context.get(userContext) }) },
] }];
```

Output:

```
signed out, open /dashboard:
  -> /dashboard
  auth: no session, redirecting
  <- /dashboard done
  -> /login
  <- /login done
  shows "Please log in" at /login?next=%2Fdashboard
signed in, open /dashboard:
  -> /dashboard
  <- /dashboard done
  shows "Dashboard for Ada"
```

One guard on a parent protects every child; loaders stop re-checking the session. Client-side guards are **UX**, not security — the server must still reject unauthorised API calls. Declarative mode has no middleware: use a wrapper component rendering `<Navigate to="/login" replace />`.

Lazy routes (code splitting)

Code splitting breaks the bundle into files loaded on demand. A route's `lazy` loads its module on first visit:

```
// not run
{ path: 'reports', lazy: () => import('./routes/reports.jsx') }   // the module exports Component, loader, …
```

React's `lazy()` + `<Suspense>` does this for any component (Chapter 10).

Framework mode in one page

React Router as a full-stack framework (successor of Remix). Routes in `app/routes.ts` point at **route modules** that export their pieces:

```
// app/routes.ts (not run)
import { type RouteConfig, index, route } from '@react-router/dev/routes';
export default [index('routes/home.tsx'), route('products/:id', 'routes/product.tsx')] satisfies RouteConfig;
```

```tsx
// app/routes/product.tsx (not run)
import type { Route } from './+types/product';          // generated types for THIS route
export async function loader({ params }: Route.LoaderArgs) {   // runs on the server
  return { product: await db.product.find(params.id) };
}
export default function Product({ loaderData }: Route.ComponentProps) {
  return <h1>{loaderData.product.name}</h1>;             // loaderData is fully typed
}
export function ErrorBoundary({ error }: Route.ErrorBoundaryProps) { /* … */ }
```

You get server rendering (or `ssr: false` for an SPA, or pre-rendering to static HTML), `loader` on the server and `clientLoader` in the browser, automatic per-route code splitting, and generated types for `params`/`loaderData`. Chapter 13 compares it with Next.js.

9.8 TanStack Query basics

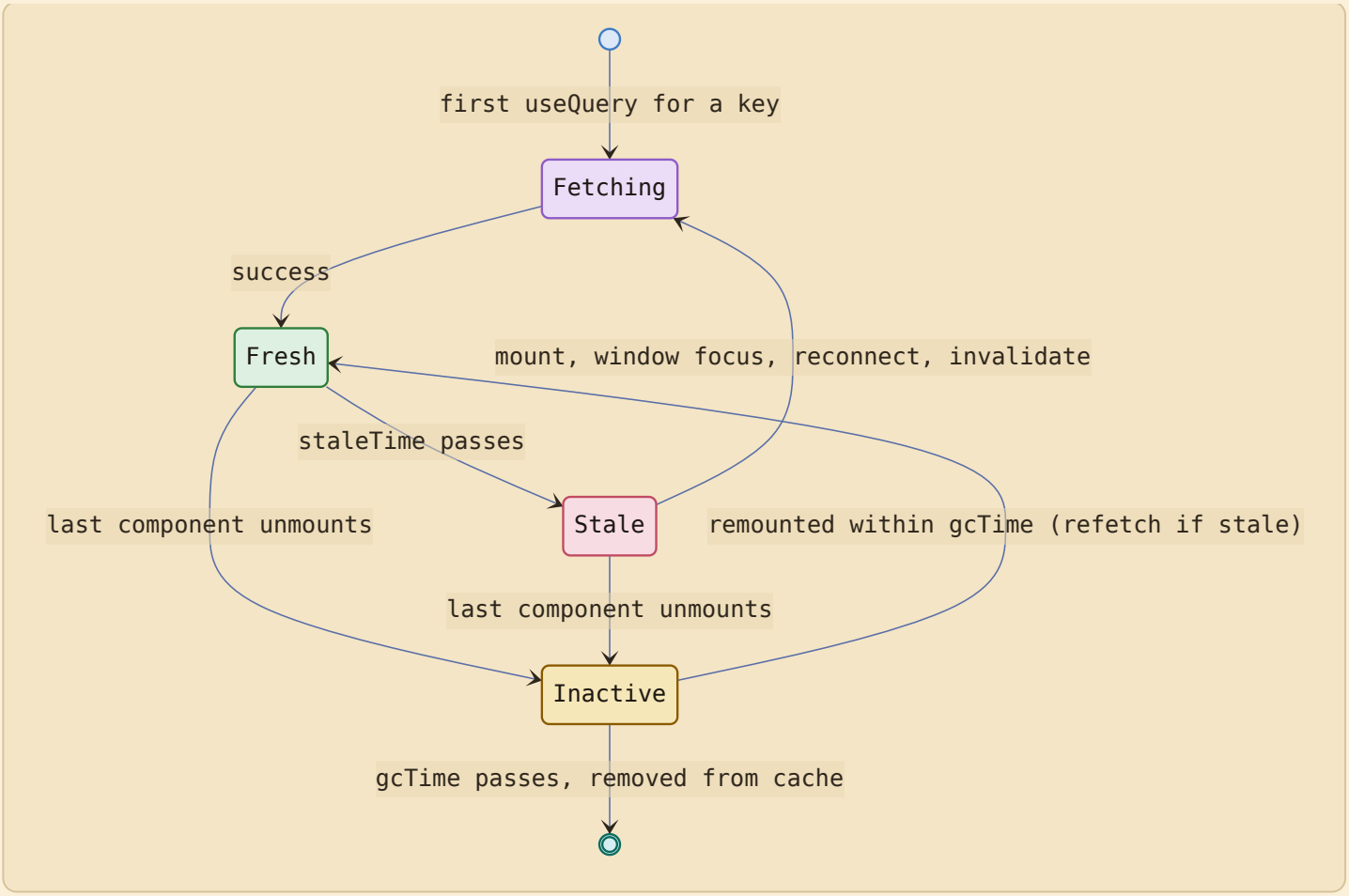
What it is

TanStack Query (formerly React Query) is a **server-state** library: it fetches, **caches**, de-duplicates (merges identical simultaneous requests), refreshes and invalidates (marks out of date) server data. (Caching in general: System Design Fundamentals.) You say *what* (a **query key**) and *how* (a **query function**); it handles *when*. Server data can be changed by others, go stale, and be needed by many components (Chapter 6) — doing that with Effects means re-implementing a cache.

How it works

1. Create one `QueryClient` (the cache) and provide it: `<QueryClientProvider client={queryClient}>`.
2. `useQuery({ queryKey: ['product', id], queryFn: () => fetchProduct(id) })`.
3. The first asker triggers the fetch; simultaneous askers for the same key **share** it.
4. Later askers get cached data **immediately**; if it is **stale**, a background refetch updates it.
5. The query function must **throw** on failure (`fetch` doesn't throw on 404 — check `res.ok`); failures are **retried** (3 times by default in the browser, with **exponential backoff** — doubling the wait each time).

Option	Default	Meaning
`staleTime`	`0`	How long data is **fresh** (served with **no** refetch). With `0`, every mount/focus triggers a background refetch (cached data still shows instantly).
`gcTime`	5 minutes	How long **unused** data (no component using the key) stays before garbage collection.



Stale queries refetch on **mount**, window **focus** and network **reconnect** (each switchable, e.g. `refetchOnWindowFocus: false`).

Field	Meaning
`status` / `isPending`, `isError`, `isSuccess`	Do we **have** data? `pending` = not yet
`fetchStatus` / `isFetching`	Is a request **in flight** (including background refetches)?
`isLoading`	`isPending && isFetching` — the first load
`data`, `error`, `failureCount`, `refetch()`	As named

`code/06_query_basics.jsx` (abridged):

```
function ProductName({ id, staleTime = 0 }) {
  const { data, status, fetchStatus, isPending, isFetching, error } = useQuery({
    queryKey: ['product', id],                          // cache key
    queryFn: () => fetchJson(`/api/products/${id}`),  // must return a promise
    staleTime,
  });
  // logs status/fetchStatus each render
  if (isPending) return <p>Loading…</p>;
  if (error) return <p role="alert">Error: {error.message}</p>;
  return <p>{data.name}</p>;
}
function Flaky() {
  const q = useQuery({ queryKey: ['flaky'], queryFn: () => fetchJson('/api/flaky'), retry: 3, retryDelay: 10 });
  return <p>{q.isSuccess ? 'flaky ok' : `attempts failed so far: ${q.failureCount}`}</p>;
}
// 1) render once  2) two at once  3) unmount + remount (staleTime 0, then 60 s)  4) flaky endpoint  5) a 404 with
retry: false
```

Output:

```
1) first load:
   status=pending fetchStatus=fetching isPending=true isFetching=true data=undefined
   status=success fetchStatus=idle     isPending=false isFetching=false data=Keyboard
2) two <ProductName id={2}/> mounted together -> requests for /api/products/2: 1
3) remount id=1 (staleTime 0): first render -> status=success fetchStatus=fetching isPending=false isFetching=true
data=Keyboard
   background refetch happened: requests for /api/products/1 = 2
   remount with staleTime 60s -> requests still 2 (fresh data, no refetch)
4) retry: 3 -> succeeded after 3 requests
5) missing product -> Error: HTTP 404
```

(2) **De-duplication**. (3) **Cache**: the remount rendered data at once while `isFetching` refreshed it — no spinner; with `staleTime: 60s`, no refetch. (4) Retries hid two transient 503s. (5) Errors arrive as `status: 'error'` with the thrown `Error`.

Choosing `staleTime`

`0` is safe but chatty. For rarely changing data (catalogue, profile) 30 s–5 min removes most refetches. Set a default — `new QueryClient({ defaultOptions: { queries: { staleTime: 60_000 } } })` — and override per query.

Common mistakes

Mistake	Fix
Copying query data into `useState`	Use `data` directly; derive during render
`new QueryClient()` inside a component	Create once (module scope or `useState(() => new QueryClient())`)
Query function that doesn't throw on HTTP errors	Check `res.ok` and throw
Using `isLoading` to decide whether `data` exists	Use `isPending` (or check `data`)
Turning off all refetching "because it's noisy"	Raise `staleTime`

In the interview

30-second answer: "TanStack Query manages server state. `useQuery` takes a key and a fetch function; it caches by key, de-dupes concurrent requests, retries failures, and refetches stale data in the background on mount, focus and reconnect. `staleTime` (default 0) is how long data is fresh; `gcTime` (default 5 minutes) how long unused data stays cached. `isPending` = no data yet; `isFetching` = request in flight."

- *Why not Redux for server data?* — You'd hand-write caching, dedupe, refetching and invalidation; RTK Query/TanStack Query exist because that's hard.
- *What happens on window focus?* — Stale queries with mounted observers (components using them) refetch.

9.9 Query keys, pagination and dependent queries

Query keys

A **query key** is an array identifying the data: `['products']`, `['products', 7]`, `['products', { page: 2, sort: 'price' }]`.

- **Every variable the query function uses belongs in the key** — new variable value → new key → new cache entry → fetch. Forget one and stale data keeps showing.
- Compared by value (objects hashed with sorted keys): `{ page: 1, sort: 'x' }` equals `{ sort: 'x', page: 1 }`.
- **Hierarchical**: invalidating `['products']` matches `['products', 7]` and `['products', 7, 'reviews']` — design general → specific.

`queryOptions()` bundles key + function into one reusable definition for `useQuery`, `useSuspenseQuery`, `prefetchQuery`, `ensureQueryData`, `getQueryData` (in TypeScript the data type flows through).

`code/08_query_keys_pagination.jsx` (abridged):

```
const productsPage = (page) => queryOptions({
  queryKey: ['products', { page }],              // variables IN the key -> each page cached separately
  queryFn: () => getJson(`/api/products?page=${page}`),
});
const reviewsOf = (id) => queryOptions({ queryKey: ['products', id, 'reviews'], queryFn: () =>
getJson(`/api/products/${id}/reviews`) });

function Catalog() {
  const [page, setPage] = useState(1);
  const [selected, setSelected] = useState(null);
  const list = useQuery({ ...productsPage(page), placeholderData: keepPreviousData });  // keep old page while next
loads
  const reviews = useQuery({ ...reviewsOf(selected), enabled: selected !== null });     // waits until something is
picked
  // dim while list.isPlaceholderData; Next disabled={!list.data.hasMore || list.isPlaceholderData}
}
```

Output (one line per render):

```
load page 1:
   page=1 showing=[] placeholder=false reviews=idle/pending
   page=1 showing=[Keyboard,Mouse] placeholder=false reviews=idle/pending
click Next (old page stays visible, dimmed, until page 2 arrives):
   page=2 showing=[Keyboard,Mouse] placeholder=true reviews=idle/pending
   page=2 showing=[Monitor] placeholder=false reviews=idle/pending
click Prev (page 1 is cached -> instant):
   page=1 showing=[Keyboard,Mouse] placeholder=false reviews=idle/pending
pick Mouse (the dependent reviews query is now enabled):
   page=1 showing=[Keyboard,Mouse] placeholder=false reviews=fetching/pending
   page=1 showing=[Keyboard,Mouse] placeholder=false reviews=idle/success
cache keys: ["products",{"page":1}]  ["products",null,"reviews"]  ["products",{"page":2}]  ["products",2,"reviews"]
```

- **Pagination without flicker**: a new key normally means `isPending`; `placeholderData: keepPreviousData` keeps page 1 (flagged `isPlaceholderData`) until page 2 arrives. Back to page 1 was instant (cached).
- **Dependent queries**: `enabled: false` keeps a query idle (`fetchStatus: 'idle'`, `status: 'pending'`) until its input exists — the `null` entry exists but never fetched.
- "Load more"/infinite scroll: `useInfiniteQuery` stores pages in one entry with `fetchNextPage` and `hasNextPage`.

9.10 Mutations, invalidation and optimistic updates

What it is

A **mutation** changes server data (POST/PUT/PATCH/DELETE — create, replace, partial update, delete). `useMutation({ mutationFn })` gives `mutate(variables)` plus `isPending`, `isError`, `error`, `data`; mutations don't run automatically and aren't cached by key. Afterwards, **invalidate** affected queries: `queryClient.invalidateQueries({ queryKey: ['todos'] })` marks matches stale and refetches those on screen.

How it works — two strategies

Strategy	Steps	Feels	Use when
Invalidate (pessimistic: wait for the server)	`mutate` → server → invalidate → refetch → UI	Correct, slight delay	Default; complex server-side effects
Optimistic update	`onMutate`: cancel refetches, snapshot cache, write expected result → server → `onError`: restore snapshot → `onSettled`: invalidate	Instant	Likes, toggles, reordering, adding to lists — server almost always agrees

`code/07_mutations.jsx` (abridged):

```
const mutation = useMutation({
  mutationFn: addTodo,
  onMutate: optimistic ? async (newTitle) => {
    await queryClient.cancelQueries({ queryKey: ['todos'] });          // don't let a refetch overwrite us
    const previous = queryClient.getQueryData(['todos']);
    queryClient.setQueryData(['todos'], old => [...old, { id: 'temp', title: newTitle + ' (saving)' }]);
    return { previous };                                               // 3rd argument of onError
  } : undefined,
  onError: (err, newTitle, ctx) => { if (ctx?.previous) queryClient.setQueryData(['todos'], ctx.previous); },
  onSettled: () => queryClient.invalidateQueries({ queryKey: ['todos'] }),  // refetch the truth either way
});
// <button disabled={mutation.isPending}>; {mutation.isError && <p role="alert">Could not add: {mutation.error.message}
</p>}
```

Output (server log abridged to the requests that matter):

```
A) plain mutation + invalidate
   right after submit : Read chapter 9 | button: Adding…
  [server] start POST /api/todos
  [server] end   POST /api/todos -> 201
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
   after refetch      : Read chapter 9 | Learn mutations
B) optimistic update
  [server] start GET /api/todos
  [server] start POST /api/todos
   right after submit : Read chapter 9 | Learn mutations | Ship it (saving)
  [server] end   GET /api/todos -> 200
  [server] end   POST /api/todos -> 201
   after server + refetch: Read chapter 9 | Learn mutations | Ship it
C) optimistic update that fails
  [server] start POST /api/todos
   right after submit : Read chapter 9 | Learn mutations | Ship it | This will fail (saving)
  [server] end   POST /api/todos -> 500
   after the 500      : Read chapter 9 | Learn mutations | Ship it | Could not add: Server refused
```

A updated only after POST + refetch. **B** showed the item **immediately**; `cancelQueries` made the client ignore the in-flight `GET` so it couldn't overwrite the optimistic list; `onSettled` swapped in the server's version. **C**: `onError` restored the snapshot — always pair optimistic updates with a visible error.

Other tools

- `queryClient.setQueryData(key, data)` — write the mutation's response into the cache (skip a refetch).
- `mutateAsync` — returns a promise for `await` (wrap in `try/catch`).
- `useOptimistic` (React 19, Chapter 4) — optimistic UI for one component's form action; the cache approach is better when many components show the data.

9.11 Suspense and `use()`

What it is

`<Suspense fallback={…}>` shows a placeholder while something inside isn't ready (lazy code or data). A component **suspends** when it asks for an unavailable value; React shows the nearest boundary's fallback and retries when the value arrives.

`use(resource)` (React 19) reads such a value during render:

- `use(promise)` returns the resolved value, **suspends** while pending, **throws** a rejection to the nearest **error boundary**.
- `use(context)` reads context like `useContext`.
- Unlike Hooks, `use` **may be called inside** `if` **and loops** (§7.2); not inside `try/catch`.

The one rule: don't create the promise during render

`use(fetch(url))` in a component creates a **new** promise each render, so React never finishes (warning: "suspended by an uncached promise"). The promise must be **created once and reused**: a prop from a Server

Component (runs only on the server, Chapter 13) or loader, a cache, or a library (TanStack Query, the router).

`code/09_suspense_and_use.jsx` (abridged):

```
const cache = new Map();                                  // create each promise ONCE, reuse across renders
function fetchProduct(id) {
  if (!cache.has(id)) cache.set(id, fetch(`${api.base}/api/products/${id}`).then(async r => {
    if (!r.ok) throw new Error(`HTTP ${r.status}`);
    return r.json();
  }));
  return cache.get(id);
}
const CurrencyContext = createContext('$');
function Product({ id, showPrice }) {
  const product = use(fetchProduct(id));                  // suspends until resolved; throws to the boundary if rejected
  if (!showPrice) return <h2>{product.name}</h2>;
  const currency = use(CurrencyContext);                  // use() may sit inside an if
  return <h2>{product.name} {currency}{product.price}</h2>;
}
function Page() {
  const [id, setId] = useState(1);
  const [isPending, startTransition] = useTransition();
  return (<>
    <button onClick={() => setId(2)}>plain setId(2)</button>
    <button onClick={() => startTransition(() => setId(3))}>transition setId(3)</button>
    <button onClick={() => setId(9)}>missing</button>
    {isPending && <p>(switching…)</p>}
    <ErrorBoundary fallback={({ error, reset }) => <p role="alert">Failed: {error.message} <button onClick={() => {
reset(); setId(1); }}>Retry</button></p>}>
      <Suspense fallback={<p>Loading product…</p>}><Product id={id} showPrice /></Suspense>
    </ErrorBoundary>
  </>);
}
// TanStack Query: useSuspenseQuery never returns "pending" - data is always defined.
function SuspenseName({ id }) {
  const { data } = useSuspenseQuery({ queryKey: ['product', id], queryFn: () => fetchProduct(id) });
  return <p>useSuspenseQuery: {data.name}</p>;
}
```

The error boundary (`code/ErrorBoundary.jsx`) is a class component — React has no Hook equivalent of `getDerivedStateFromError`; most apps use the `react-error-boundary` package. Full anatomy: §14.9.

Output (visible text after each step):

```
first render      : Loading product…
resolved          : Keyboard $49
plain update      : Loading product… <- content replaced by the fallback
transition update : (switching…) Mouse $19 <- old content kept, isPending shown
then              : Monitor $199
rejected promise  : Failed: HTTP 404 Retry
after Retry       : Keyboard $49
useSuspenseQuery: Mouse
```

How it works

1. `use(promise)` on a pending promise stops rendering `Product`; the **closest** `<Suspense>` fallback shows.
2. When it settles, React re-renders; `use` returns the value or throws to the **closest error boundary**.
3. A **plain update** to an unloaded id **hides** visible content behind the fallback (a jarring flash).
4. A **transition** (`startTransition`, Chapter 7) **keeps the old content** and exposes `isPending`. Routers wrap navigations in transitions for this reason.
5. The boundary's `reset()` clears the error and retries (a real cache should also **evict** failed promises so a retry refetches).

Placing boundaries

- Put `<Suspense>` where a placeholder makes sense **visually** — a card, panel, route — not around every component.
- Siblings in one boundary reveal **together** (no "popcorn" pop-in); separate boundaries let fast parts show first.

- Nest boundaries to stream progressively: shell → main content → comments.
- Pair each important Suspense boundary with an **error boundary** outside it.

9.12 Loading and error states that users like

Loading states — pick by situation

Situation	Show	Tool
First load of a page/section	**Skeleton** in the final layout — avoids **layout shift** (content jumping)	Suspense fallback, `isPending`
Navigating between pages	**Old page + progress indicator**	`useNavigation`, transitions
Refreshing data already shown	Keep data; small hint or nothing	`isFetching` with data
Paginating / filtering	Previous results dimmed until new arrive	`keepPreviousData`, `useDeferredValue`
User action (save, like)	Disabled button + "Saving…", or optimistic UI	mutation/action `isPending`, `useOptimistic`
Very fast responses	**Nothing** — a 50 ms spinner is a flicker	Delay spinners ~200–300 ms, or transitions

Error states — every failure needs a home

Error kind	Where	UI
Expected not found / 403 Forbidden	Route `ErrorBoundary` via `throw data(msg, { status })`, or query `error`	Friendly page inside the layout; navigation still works
Transient network error	Retries (Query: 3 by default), then error UI	Message + **Retry** (`refetch()` / boundary `reset`)
Validation error on submit	Action return / mutation error	Inline by the field (`role="alert"`)
Bug (exception during render)	Error boundary per route + one at the root	"Something went wrong" + report to monitoring (e.g. Sentry) in `componentDidCatch`
Failure after optimistic update	`onError` rollback	Revert + toast explaining what didn't save

Error boundaries **don't** catch errors in event handlers, async code outside render (`setTimeout`), or the boundary itself — use `try/catch` + state, or route them into render (e.g. `mutation.error`).

Common mistakes

One app-wide spinner per fetch (users lose context); an error page for a failed *background* refetch when good cached data exists (show data + subtle warning); swallowed errors (`catch {}`) leaving "Loading…" forever; no empty state ("No results") distinct from loading and error.

9.13 Putting it together — loaders + TanStack Query

- The **router** knows *when* data is needed (on click), so loaders **start** fetches early and in parallel.
- **TanStack Query** knows *whether* it's cached and fresh, and keeps it updated.

Pattern: in the loader, `queryClient.ensureQueryData(options)` (return cached or fetch) for critical data and `prefetchQuery` (start, don't wait) for non-critical; read both in the component with the **same** `queryOptions` via `useSuspenseQuery`/`useQuery`.

`code/shop.jsx` (key parts):

```
const fresh = 30_000;                                          // fresh for 30 s
const productsQuery = queryOptions({ queryKey: ['products'], queryFn: () => getJson('/api/products'), staleTime: fresh
});
const productQuery = (id) => queryOptions({ queryKey: ['products', id], queryFn: () => getJson(`/api/products/${id}`),
staleTime: fresh });
const reviewsQuery = (id) => queryOptions({ queryKey: ['products', id, 'reviews'], queryFn: () =>
getJson(`/api/products/${id}/reviews`), staleTime: fresh });

function Product() {
  const { id } = useParams();
  const { data: product } = useSuspenseQuery(productQuery(id));   // already cached thanks to the loader
  const reviews = useQuery(reviewsQuery(id));                      // not awaited by the loader: streams in after
  // ... render product, reviews (or "Loading reviews…")
}

return [{
  path: '/', Component: Layout, ErrorBoundary: RouteError, HydrateFallback: () => <p>Loading shop…</p>,
  loader: () => queryClient.ensureQueryData(productsQuery),        // fetch if missing, reuse if cached
  children: [
    { index: true, Component: Home },
    { path: 'products/:id', Component: Product, ErrorBoundary: RouteError,
      loader: async ({ params }) => {
        queryClient.prefetchQuery(reviewsQuery(params.id));          // start, don't wait (non-critical)
        await queryClient.ensureQueryData(productQuery(params.id));  // wait (critical)
        return null;
      } },
  ],
}];
```

`code/10_router_with_query.jsx` deep-links to `/products/1`, clicks around, then opens a missing product:

```
  [server] start GET /api/products
  [server] start GET /api/products/1/reviews
  [server] start GET /api/products/1
  [server] end   GET /api/products -> 200
  [server] end   GET /api/products/1/reviews -> 200
  [server] end   GET /api/products/1 -> 200
deep link /products/1 -> Keyboard | reviews: 2
  [server] … GET /api/products/3/reviews and /api/products/3 in parallel
click Monitor         -> Monitor
back to Keyboard      -> Keyboard (served from the cache: no requests)
  [server] start GET /api/products/7/reviews
  [server] start GET /api/products/7
  [server] end   GET /api/products/7/reviews -> 200
  [server] end   GET /api/products/7 -> 404
/products/7           -> 404 - This product does not exist
```

The deep link started **all three** requests at once (parent and child loaders in parallel); back to Keyboard made **zero** requests (`ensureQueryData` found fresh data — plain loaders would refetch); the 404 was thrown as `data(…, { status: 404 })` by the query function and rendered by the route's `ErrorBoundary`.

In the browser (`code/shop.html`; `fetch` is mocked in `shop-main.jsx`):

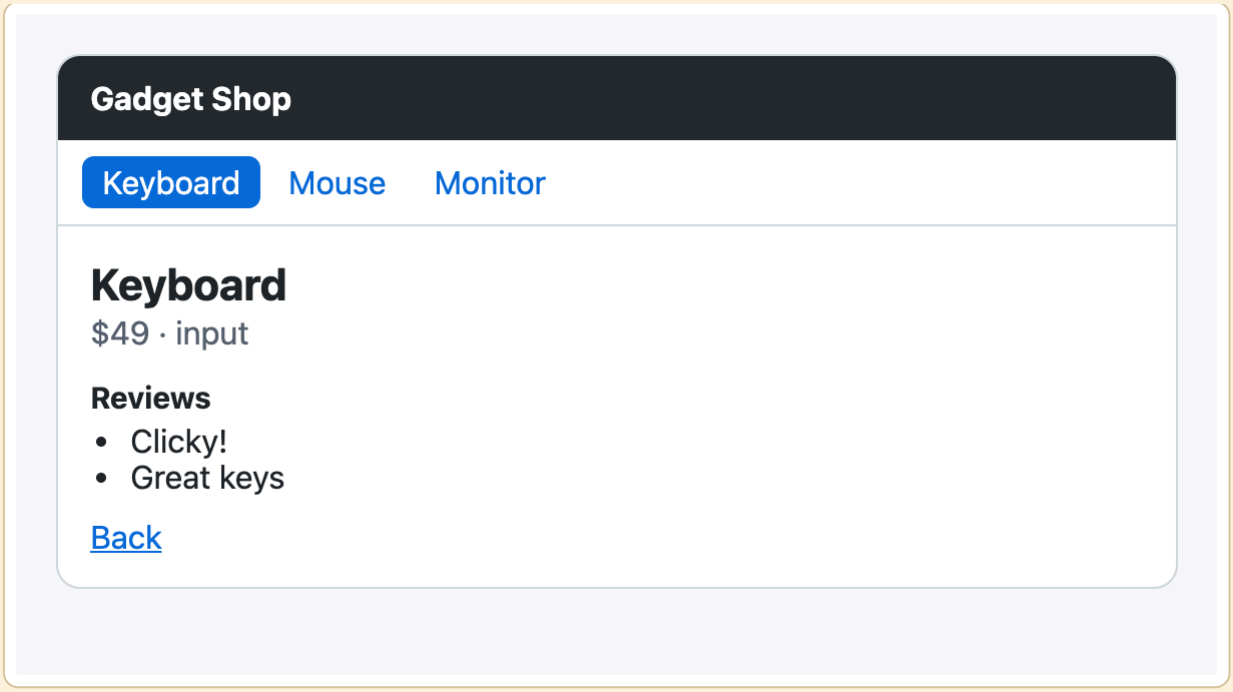


Figure 9.1 — The shop at `/products/1`. `NavLink` highlighted the active product; the product came from the loader-filled cache, the reviews from a prefetch.

In the interview

30-second answer: "React Router's data mode so loaders start fetching in parallel as navigation begins, and TanStack Query as the cache: the loader calls `ensureQueryData` with shared `queryOptions`, the component reads the same options with `useSuspenseQuery`. Critical data is awaited, non-critical prefetched. Mutations go through actions or `useMutation` and invalidate affected keys. Route error boundaries handle errors; navigations are transitions so the old page stays visible."

- *Why both a loader and a query?* — The loader removes the render-then-fetch waterfall; the cache removes repeated requests and keeps data fresh.
- *Where does the `QueryClient` live?* — Created once outside components, provided at the root, passed to route loaders.

Chapter 9 TL;DR

Idea	Remember
Client-side routing	History API + route matching; URL is shareable state
React Router v8	`npm i react-router`; `RouterProvider` from `react-router/dom`; declarative / data / framework modes
Routes	`path`, `:params` (`useParams`, strings), `?search` (`useSearchParams`), `*` 404, `<Link>`/`<NavLink>`, `useNavigate`
Nesting	`<Outlet/>`, `index`, pathless layouts, relative links resolve against the route tree, `useOutletContext`
Loaders	Run before render, in parallel; `useLoaderData`; `useNavigation` for pending; `throw data(msg, {status})` + `ErrorBoundary`; no caching
Actions	`<Form method="post">` → `action` → errors (`useActionData`) or `redirect`; automatic revalidation; `useFetcher`
Middleware / lazy / framework	Before/after loaders (auth, logging, `createContext` + `context.get/set`); `lazy` splits route code; framework mode = route modules + SSR + typegen
TanStack Query	Key + fn; cache, dedupe, retries, background refetch; `staleTime` (0) / `gcTime` (5 min); `isPending` vs `isFetching`
Keys & pagination	Every input in the key; `queryOptions`; `keepPreviousData`; `enabled`; `useInfiniteQuery`
Mutations	`useMutation` + `invalidateQueries`; optimistic: cancel → snapshot → set → rollback → invalidate
Suspense & `use()`	Cached promise only; fallback while pending; error boundary for rejections; transitions keep old UI
Loading/error UX	Skeletons first, old content during navigation, inline validation, Retry buttons, visible rollbacks

Next: Chapter 10 — Performance.

Chapter 10 — Performance

Most React apps are fast enough without special effort. When one isn't, the cause is almost always one of: components rendering far more often than needed, an expensive calculation repeated every render, too many DOM nodes, or too much JavaScript downloaded up front. This chapter teaches you to **see** each problem (render counters, the `<Profiler>` API, React DevTools — the browser extension for inspecting components) and fix it with the right tool: `memo`, `useMemo`, `useCallback` (all **memoization** — reusing a previous result when inputs are unchanged), React Compiler, code splitting with `lazy` + `<Suspense>`, and list virtualization. Every demo prints **numbers** (render counts, DOM nodes, milliseconds) so you watch a fix work.

Code: `code/` — React 19.3 in jsdom (a browser-like DOM for Node), driven with `@testing-library/react`. jsdom timings are only indicative (no real **layout** — computing sizes/positions — or **paint**), so demos focus on exact, repeatable **counts**. Background: state and batching (Chapter 3), Effects (Chapter 5), Context (Chapter 6), `useTransition`/`useDeferredValue` (Chapter 7).

10.1 The render cycle: trigger → render → commit

What it is

1. **Trigger** — the first `root.render(...)`, or a state setter somewhere in the tree.
2. **Render** — React **calls your component functions** to get a tree of React elements (plain objects). The screen doesn't change yet.
3. **Commit** — React compares the new tree with the previous one (**reconciliation**, Chapter 1) and applies **only the differences** to the DOM, runs layout effects (`useLayoutEffect`, after DOM changes but before paint), the browser paints, then ordinary Effects (`useEffect`) run.



Why it matters

- **"Re-render" ≠ "DOM update".** A re-render is a function call; identical output changes nothing in the DOM. Re-renders are cheap *unless* a component does heavy work or renders a huge subtree.
- **Render must be pure** (Chapter 2). React may call components many times (twice in development under StrictMode), so render must not have **side effects** (network calls, mutating shared variables). Purity lets React skip, repeat or interrupt renders safely.

Seeing it

`code/01_render_cycle.jsx` logs from render, a layout effect and an effect, and counts DOM changes with a `MutationObserver` (browser API reporting every DOM change):

```
function Clock({ label }) {
  log(`  render  <Clock label="${label}">`);              // RENDER phase
  useLayoutEffect(() => log('  commit  React applied changes, then ran layout effects'));
  useEffect(() => log('  after   passive effects run (after paint)'));
  return <p>{label}</p>;
}
// App: <Clock label={label} /> + buttons: setLabel('world') | setClicks(c => c + 1) | setLabel(label)
```

Output:

```
TRIGGER 1: initial render (createRoot().render)
  render  <Clock label="hello">
  commit  React applied changes, then ran layout effects
  after   passive effects run (after paint)
  DOM mutations: many (whole tree inserted)

TRIGGER 2: state change that changes Clock output
  render  <Clock label="world">
  commit  React applied changes, then ran layout effects
  after   passive effects run (after paint)
  DOM mutations: 1

TRIGGER 3: state change elsewhere - Clock re-renders, but its output is identical
  render  <Clock label="world">
  commit  React applied changes, then ran layout effects
  after   passive effects run (after paint)
  DOM mutations: 1  (only the button text changed; <p> untouched)

TRIGGER 4: setState with the SAME value (Object.is equal)
  DOM mutations: 0  (React bailed out: nothing rendered)
```

- Trigger 3: `Clock` re-rendered with unchanged props, but the DOM got one change (the button text) — the diff protected the DOM.
- Trigger 4: an `Object.is`-equal value (JavaScript's "same value/same object" check) lets React **bail out**, skipping the component and its children. (React may occasionally call the component once before bailing out; the result is discarded.)
- Effects ran every commit only because they have no dependency array.

Common mistakes

- Treating every re-render as a bug — a 0.1 ms re-render isn't worth optimizing. Measure first (§10.3).
- Side effects (server logging, mutating globals) in the render body — they run more often than you expect.

10.2 When components re-render

What it is

Cause	Example	Notes
Its own state changed	`setOpen(true)` inside it	Unless the new value is `Object.is`-equal
Its parent re-rendered	parent's state changed	Default: *all* children re-render, props changed or not
A context it reads changed	provider `value` changed	Even if the parent didn't re-render, and through `memo`
A subscribed external store changed (state outside React)	Redux/Zustand selector result changed	Via `useSyncExternalStore` (Chapter 7)

Not causes: mutating a prop object in place, changing `ref.current`, changing a plain variable (why you never mutate state — React wouldn't notice). Myth: "re-renders when props change". Accurate: **re-renders when its parent re-renders**; props aren't even checked without `memo` (§10.4).

Seeing it

`code/02_when_rerenders.jsx` gives each component a render counter:

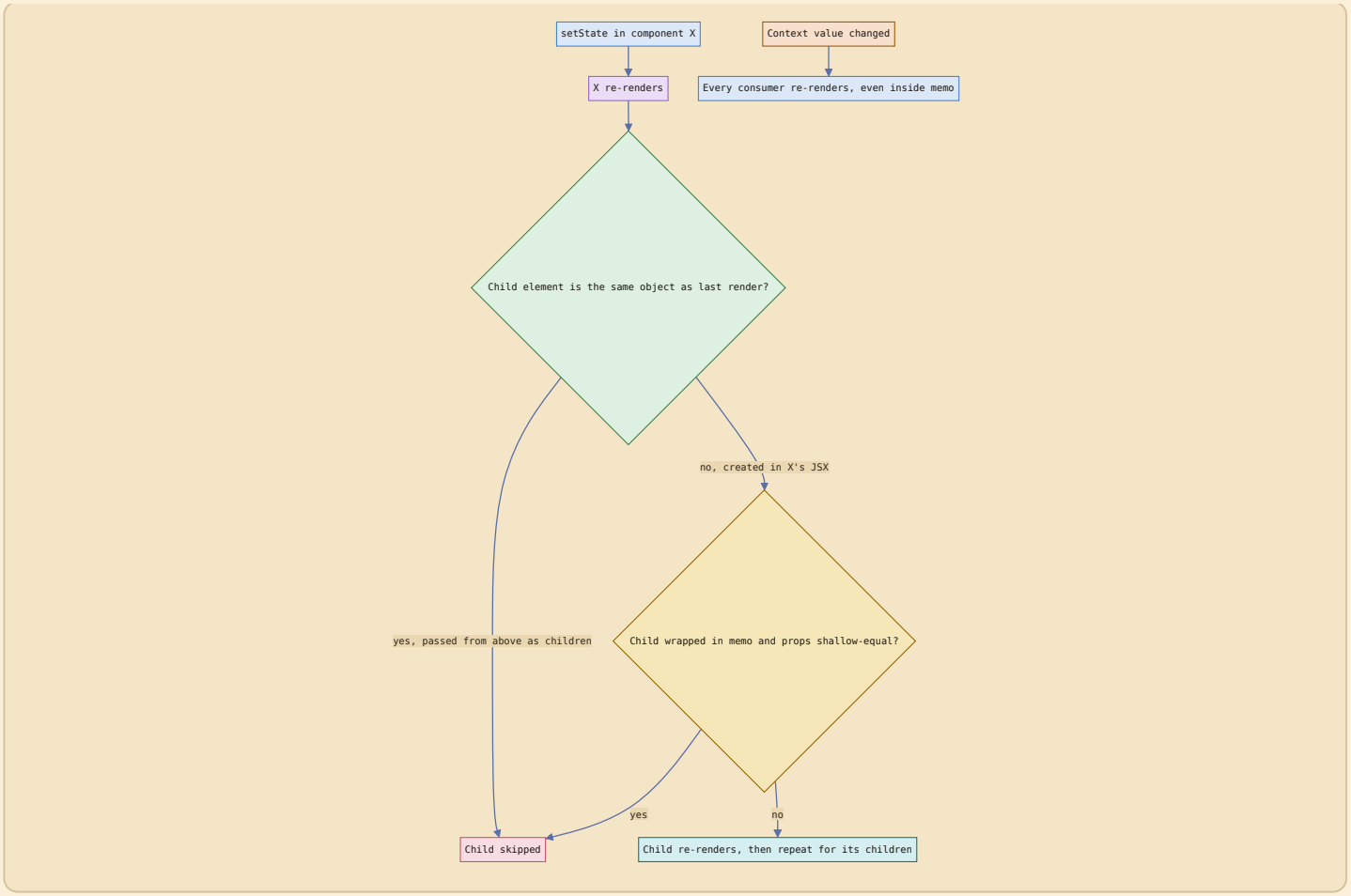
```
// Case A: a parent's state change re-renders ALL of its children (props or not)
function Parent() {
  count('Parent');
  const [n, setN] = useState(0);
  return <div><button onClick={() => setN(n + 1)}>parent clicks {n}</button><ul><Child name="ChildWithProps" /><Static
/></ul></div>;
}
// Case B: the same children passed in as JSX from ABOVE do NOT re-render
function Wrapper({ children }) {
  count('Wrapper');
  const [n, setN] = useState(0);
  return <div><button onClick={() => setN(n + 1)}>wrapper clicks {n}</button>{children}</div>;
}
// Case C: a context change re-renders every consumer, wherever it is
function ThemeProvider({ children }) {         // state lives here; children come from above
  const [theme, setTheme] = useState('light');
  return <ThemeCtx value={theme}><button onClick={() => setTheme(t => (t === 'light' ? 'dark' : 'light'))}>toggle
theme</button>{children}</ThemeCtx>;
}
function Middle() { count('Middle'); return <><Consumer /><NotConsumer /></>; }   // Consumer reads ThemeCtx
render(<><Parent /><Wrapper><Child name="ChildAsChildren" /></Wrapper><ThemeProvider><Middle /></ThemeProvider></>);
```

Output:

```
mount
{"Parent":1,"ChildWithProps":1,"StaticNoProps":1,"Wrapper":1,"ChildAsChildren":1,"Middle":1,"ContextConsumer":1,"NotAConsumer":1}
Parent setState   {"Parent":1,"ChildWithProps":1,"StaticNoProps":1}
Wrapper setState  {"Wrapper":1}
theme change      {"ContextConsumer":1}
```

How it works

1. **A.** Re-running `Parent` created **new** elements for `<Child>` and `<Static>`, so both rendered — even prop-less `Static`.
2. **B.** `<Child name="ChildAsChildren">` was created by *Wrapper's parent*, which didn't re-render; React got the **same element object** in `children` and skipped it. "Pass JSX as children" is the cheapest optimization in React — no `memo` needed.
3. **C.** React re-rendered **only** context readers: `Consumer` yes; `Middle` and `NotConsumer` (came in as `children`) no.



Real-world examples

- A layout owning "sidebar open" state that receives the page as `children` doesn't re-render the page on toggle.
- Redux/Zustand let a component subscribe to a *slice* via a selector, re-rendering only when that slice changes — why they scale better than one big context (Chapter 6).

In the interview

30-second answer: "A component re-renders when its own state changes, its parent re-renders, a context it reads changes, or a subscribed external store changes. Unchanged props don't stop a re-render by default — only `memo` checks props. Setting state to an identical value bails out. Elements passed as `children` from above keep their identity, so they're skipped when the wrapper re-renders."

- *Does changing a ref cause a re-render?* — No; refs are invisible to rendering.
- *Does `memo` stop context updates?* — No.
- *Parent re-renders; `Child` has no props — does it re-render?* — Yes, unless wrapped in `memo` or passed from above as `children`.

10.3 Measure first: DevTools Profiler and `<Profiler>`

Why it matters

Optimizing without measuring usually fixes the wrong thing and adds complexity.

Tool	Where	What it shows
React DevTools → Profiler tab	browser extension	Recorded commits; *flamegraph* (bar width = render time), *ranked* chart (slowest first); "why did this render?"
React Performance Tracks (React 19.2+)	Chrome DevTools → Performance panel	React's **scheduler** work (blocking vs transition — low-priority, interruptible — updates) and a component flamegraph on the same timeline as network and JS
`<Profiler>` component	your code	Timings for every commit of a subtree — for tests, logging, monitoring

Using React DevTools (step by step)

1. Install **React Developer Tools**; open the app in **development mode** (profiling needs a development or profiling build).
2. Settings (gear) → **General** → **"Highlight updates when components render"**: every re-render flashes an outline.
3. Settings → **Profiler** → **"Record why each component rendered while profiling"**.
4. **Record**, do the slow interaction once, **Stop**.
5. Pick the tallest commit bar. In the **flamegraph**, grey = didn't render this commit; coloured = rendered, warmer = slower. Click a bar for *why* ("Props changed: onAdd", "Hook 2 changed", "The parent component rendered").
6. **Ranked** lists the slowest first. Fix the top one, record again, compare.

Also profile a **production build** (profiling build, below) with **CPU throttling** (Performance panel → 4× slowdown) so your laptop behaves like an average phone.

The `<Profiler>` API

`<Profiler id="..." onRender={fn}>` calls, after each commit involving the subtree:

```
onRender(id, phase, actualDuration, baseDuration, startTime, commitTime)
```

Parameter	Meaning
`id`	the `id` prop
`phase`	`"mount"`, `"update"`, or `"nested-update"` (update scheduled during a layout effect)
`actualDuration`	ms rendering this subtree **in this commit** — falls when memoization works
`baseDuration`	estimated ms for a *full* re-render (sum of each component's latest render time) — the worst case
`startTime`, `commitTime`	timestamps; profilers in one commit share `commitTime`

`code/09_profiler_api.jsx` (abridged):

```jsx
const Sidebar = memo(function Sidebar() { busy(3); return <nav>links</nav>; });   // memoized, 3 ms
function Feed({ posts }) { busy(5); return <section>{posts} posts</section>; }    // 5 ms every render
function App() {
  const [posts, setPosts] = useState(10);
  return (
    <Profiler id="app" onRender={onRender}>
      <button onClick={() => setPosts(p => p + 1)}>load more</button>
      <Profiler id="sidebar" onRender={onRender}><Sidebar /></Profiler>
      <Profiler id="feed" onRender={onRender}><Feed posts={posts} /></Profiler>
    </Profiler>
  );
}
// render, click "load more" twice, print samples
```

Output (ms vary per run):

```
id       phase    actual  base  (ms)
sidebar  mount         3     3
feed     mount         5     5
app      mount        10     9
sidebar  update        0     3
feed     update        5     5
app      update        5     8
sidebar  update        0     3
feed     update        5     5
app      update        5     8

9 onRender calls in 3 commits (profilers in one commit share commitTime)
```

On updates `sidebar` has `actual 0`, `base 3` — memoization saved 3 ms. `feed` has `actual = base = 5` — it fully re-renders each time, so look there next. `app`'s actual (5) < base (8): the tree is partly skipped.

Profiling in production

The normal production build **disables** profiling (`onRender` not called). To profile production code, **alias** (have the bundler substitute one import path for another) `react-dom/client` to `react-dom/profiling`; some frameworks have a flag. Don't ship it to all users without reason.

Real-world examples

- Teams send `<Profiler>` samples from a small share of sessions to metrics to catch **regressions**.
- Tests assert render counts for critical components so an "everything re-renders" change fails **CI** (the server running tests on every push).
- Web Vitals such as **INP** (Interaction to Next Paint: how fast the page visually responds to input) are what users feel; React profiling explains *why* an interaction is slow.

Common mistakes

- Profiling only a dev build on a fast laptop — dev React is much slower than production, and users' phones are slower than your laptop.
- Leaving `<Profiler>` everywhere permanently — each has a cost.

10.4 `memo` — skipping a child's re-render

What it is

`memo(Component)` returns a component React **skips re-rendering** when its parent re-renders and every prop is **shallow**-equal to last time (each prop compared with `Object.is` — like `===` except `NaN` equals `NaN` and `+0` differs from `-0`).

```
import { memo } from 'react';
const Row = memo(function Row({ product, onAdd }) { /* ... */ });
// optional second argument: arePropsEqual(prevProps, nextProps) => true to skip
```

How it works

1. Parent re-renders, producing `<Row product={p} onAdd={fn} />`.
2. React compares new vs previous props key by key with `Object.is`.
3. All equal → reuse the previous result; `Row` (and its children) aren't called. Any different → normal render.

`memo` only filters parent-caused re-renders: own state and read contexts still re-render it.

The catch: primitives compare by value, objects by identity

`Object.is(3, 3)` is `true`; `Object.is({}, {})` and `Object.is(() => {}, () => {})` are `false`. Inline props (`style={{ padding: 0 }}`, `onAdd={id => ...}`) are new every parent render, so a memoized child re-renders anyway and `memo` only adds a comparison. §10.5 fixes that.

Measuring it: before and after

`code/03_memo_before_after.jsx`: a search box plus 300 product rows; type five characters (irrelevant to the list, so ideally zero row renders). A `<Profiler>` wraps the list; each `Row` counts renders.

```
function Row({ product, onAdd }) {
  rowRenders++;
  let x = 0; for (let i = 0; i < 2000; i++) x += i;          // pretend work
  return <li>{product.name} ${product.price} <button onClick={() => onAdd(product.id)}>add</button></li>;
}
const MemoRow = memo(Row);
const MemoList = memo(List);   // List renders <RowComponent key product onAdd /> for each item
// V1 - no memo: <List items={PRODUCTS} onAdd={id => setCart(c => [...c, id])} RowComponent={Row} style={{ padding: 0 }}
/>
// V2 - <MemoList ... RowComponent={MemoRow} ...> but new onAdd and new style each render
// V3 - memo + stable props:
//   const onAdd = useCallback(id => setCart(c => [...c, id]), []);
//   const style = useMemo(() => ({ padding: 0 }), []);
```

Output:

```
V1 no memo                        typing 5 chars -> Row renders: 1500, list profiler commits: 5 | click add -> Row
renders: 300
V2 memo, unstable props           typing 5 chars -> Row renders: 1500, list profiler commits: 5 | click add -> Row
renders: 300
V3 memo + useCallback/useMemo     typing 5 chars -> Row renders:    0, list profiler commits: 5 | click add -> Row
renders: 0

actualDuration while typing (ms, varies per run): V1 32.6, V2 26.5, V3 0.0
```

- **V1:** 5 keystrokes × 300 rows = 1,500 wasted row renders.
- **V2:** `memo` changed **nothing** — new `onAdd`/`style` each render fail the shallow compare. The most common misuse.
- **V3:** zero rows render while typing or on **add** (the list doesn't receive `cart`). The profiler still reports 5 commits: `onRender` fires whenever the surrounding tree commits, even if the subtree was skipped (`actualDuration` ≈ 0).

When `memo` is worth it — and when not

Use `memo` when…	Skip it when…
The component often re-renders with the **same props**	It's cheap (button, label), or props change nearly every render
Its render is noticeably expensive (large subtree, heavy calculation)	Moving state down or passing JSX as `children` (§10.10) removes the re-render
You can keep props stable (primitives, memoized objects/functions)	The project uses **React Compiler** (§10.7)

Custom comparison

`memo(Chart, (prev, next) => prev.data.version === next.data.version)` defines equality yourself. Returning `true` while a callback prop changed leaves the child calling a **stale closure** (a function that captured old state). Compare every prop, or make props stable.

Common mistakes

- `memo` plus inline objects/functions/JSX — see V2.
- `<MemoCard><p>hi</p></MemoCard>` — `children` is a new element every render, so `MemoCard` always re-renders.
- Deep-equality in `arePropsEqual` on big data — can cost more than the render.

In the interview

30-second answer: "`memo` makes a component skip re-rendering when its parent re-renders with shallow-equal props (`Object.is`). It doesn't stop re-renders from its own state or context. It only helps with

stable props, so it goes with `useCallback` / `useMemo`. I use it for expensive components that re-render often with the same props, after profiling — and with React Compiler I mostly don't need it."

- *Why can `memo` make things slower?* — A props comparison on every parent render; if props always differ you pay for both.
- *Is `memo` a guarantee?* — No, a performance hint; code must be correct if the component re-renders.

10.5 `useCallback` and `useMemo` for stable props

What it is

Both **cache a value between renders** and return the *same* reference until a dependency changes:

```
const onAdd = useCallback(id => setCart(c => [...c, id]), []);   // caches the FUNCTION
const style = useMemo(() => ({ padding: 0 }), []);                 // caches the RESULT of calling the function
```

`useCallback(fn, deps)` is exactly `useMemo(() => fn, deps)`. On first render React stores `[value, deps]` in the component's Hook slot (Chapter 7); on later renders it compares each dep with `Object.is` — all equal → stored value (same reference, so a memoized child sees an unchanged prop); else store and return the new one.

Keeping dependencies small

- Use an **updater function** (`setCart(c => [...c, id])`) so the callback needn't depend on `cart` (Chapter 3).
- Move constants **out of the component** (`const STYLE = { padding: 0 }` at module level) — no Hook needed.
- Pass **primitives** (`userId`) rather than objects (`user`) when the child needs one field.

When they help — and when they don't

Situation	Worth it?
Passing a function/object to a `memo` child	Yes — else `memo` is useless (V2)
Value is a `useEffect` dependency that mustn't re-trigger every render	Yes (or restructure the Effect, Chapter 5)
A custom Hook returns functions callers may put in dependency arrays	Yes
Callback on a plain DOM element (`<button onClick>`)	No — DOM elements aren't memoized
Child not wrapped in `memo`	No — it re-renders anyway
Everything, "just in case"	No — noise and memory; hides real problems

Common mistakes

- **Missing dependencies**: `useCallback(() => save(draft), [])` keeps the first `draft` forever (stale closure). Let `eslint-plugin-react-hooks` check.
- **Unstable dependencies**: `useMemo(() => filter(items, opts), [items, opts])` with a new `opts` each render never hits the cache.
- Assuming the cache survives — React may discard cached values (e.g. during dev file edits); code must be correct without it.

10.6 `useMemo` for expensive calculations

What it is

`useMemo`'s second job: skip **re-doing a slow calculation** when its inputs didn't change.

```
const visible = useMemo(() => filterTodos(todos, tab), [todos, tab]);
```

Seeing it

`code/04_usememo_expensive.jsx` filters 5,000 todos (~2 ms) with 4 unrelated re-renders (theme toggles) and 1 relevant one (tab change):

```
const visible = filterTodos(TODOS, tab);                          // Plain: recomputed on EVERY render
const visible = useMemo(() => filterTodos(TODOS, tab), [tab]);    // Memoized: cached until `tab` changes
```

Output:

```
without useMemo: 6 renders, filterTodos ran 6x, shows "1667 visible"
with useMemo   : 6 renders, filterTodos ran 2x, shows "1667 visible"
```

How do I know a calculation is "expensive"?

Time it with `console.time`/`console.timeEnd`, ideally CPU-throttled. React's docs suggest memoizing at around **1 ms or more** — sorting/filtering thousands of items, building chart data, parsing large text; not `a + b` or mapping 20 items.

Alternatives to consider first

- **Do it outside render**: once when data arrives (fetch handler, reducer, server).
- **Keep less state**: cheap derived values (Chapter 3) can just be recomputed.
- **Don't block typing**: `useDeferredValue`/`useTransition` mark rendering as low-priority so input stays instant (Chapter 7); combine with `memo` on the slow list.

Common mistakes

- Side effects in the `useMemo` function — it must be pure (StrictMode calls it twice in dev to surface this).
- Expecting a faster **first** render — it only helps later renders.

10.7 React Compiler

What it is

React Compiler is a build-time Babel plugin (`babel-plugin-react-compiler`; Babel transforms JS source) that **automatically adds memoization** — the equivalent of well-placed `useMemo`, `useCallback` and `memo`. **v1.0** went stable on **7 October 2025**. Best with React 19; also supports React 17 and 18 (via the extra `react-compiler-runtime` package and a `target` option).

How it works

1. At build time it analyses which values in each component/Hook depend on which props and state.
2. It rewrites the function to keep a per-instance **cache array** (`useMemoCache`, from `react/compiler-runtime`).
3. Each computed value and JSX element is recomputed only when its inputs changed. Returning the *same* JSX element object lets React skip that child (as with "children from above", §10.2) — so `memo` is typically unnecessary.
4. Components that break the Rules of React (mutating props, reading refs during render, side effects in render) are **skipped**, not risked — they still run, unoptimised.

Seeing it — what the compiler produces (by hand)

This book's toolchain doesn't run the plugin, so `code/05_compiler_output.jsx` has a **hand-simplified** version of compiler-style output using the real `c` from `react/compiler-runtime` (real output differs in naming and detail):

```
// ---- what you write ----
function ProductPage({ products, theme }) {
  const visible = products.filter(p => p.inStock);
  const onBuy = id => console.log('buy', id);
  return <div className={theme}><ProductList items={visible} onBuy={onBuy} /></div>;
}

// ---- roughly what the compiler turns it into ----
import { c as _c } from 'react/compiler-runtime';
function ProductPageCompiled({ products, theme }) {
  const $ = _c(6);                                          // 6 cache slots for this component instance
  let visible;
  if ($[0] !== products) {                                  // input changed -> recompute
    visible = products.filter(p => p.inStock);
    $[0] = products; $[1] = visible;
  } else visible = $[1];                                    // unchanged -> reuse the same array
  let onBuy;
  if ($[2] === Symbol.for('react.memo_cache_sentinel')) {   // first render only: the function uses no props
    onBuy = id => console.log('buy', id);
    $[2] = onBuy;
  } else onBuy = $[2];
  let list;
  if ($[3] !== visible || $[4] !== onBuy) {                 // the JSX element itself is cached
    list = <ProductList items={visible} onBuy={onBuy} />;
    $[3] = visible; $[4] = onBuy; $[5] = list;
  } else list = $[5];
  return <div className={theme}>{list}</div>;
}
```

`ProductList` is **not** wrapped in `memo`. Changing `theme` three times:

```
hand-written code  : 4 renders of the page -> filter ran 4x, ProductList rendered 4x
compiler-style code: 4 renders of the page -> filter ran 1x, ProductList rendered 1x
```

The online **React Compiler Playground** (linked from react.dev) shows real output for any component.

Setting it up (not run here)

Install with an exact version; it must run **first** among Babel plugins:

```
npm install --save-dev --save-exact babel-plugin-react-compiler@latest
```

```
// babel.config.js
module.exports = {
  plugins: ['babel-plugin-react-compiler'],   // must be first
};
```

```
// vite.config.js — @vitejs/plugin-react v6 and later (also: npm install -D @rolldown/plugin-babel)
import { defineConfig } from 'vite';
import react, { reactCompilerPreset } from '@vitejs/plugin-react';
import babel from '@rolldown/plugin-babel';
export default defineConfig({
  plugins: [react(), babel({ presets: [reactCompilerPreset()] })],
});

// older @vitejs/plugin-react versions:
//   plugins: [react({ babel: { plugins: ['babel-plugin-react-compiler'] } })]
```

- **Next.js** and **Vite** templates offer it at creation (`create-next-app`, `create-vite`); **Expo SDK 54+** enables it by default.
- Its lint rules ship in `eslint-plugin-react-hooks` (`recommended`), replacing `eslint-plugin-react-compiler`; they flag code it can't optimise (setting state or reading `ref.current` during render).
- **Verify**: compiled components show a **"Memo ✨"** badge in React DevTools.
- **Opt out** one component/file temporarily with the **directive** (a special string statement tools read) `"use no memo"`; fix the issue and remove it.
- Adopt **incrementally** in large codebases (one directory first, or behind a flag — react.dev has a guide), and pin the exact version if test coverage is thin, since new versions can memoize differently.

What happens to `useMemo`, `useCallback` and `memo`?

- They still work and are respected; no need to delete them.
- In new compiled code, use them only as **escape hatches** — e.g. to guarantee an Effect dependency keeps its identity.
- The compiler can memoize things Hooks can't, such as values computed **after an early return**.
- It doesn't fix slow algorithms (an $O(n^2)$ loop is still slow), huge DOM trees (virtualize) or large bundles (split).

Real-world examples

The v1.0 announcement (react.dev, October 2025) cites the Meta Quest Store: initial loads and cross-page navigations up to 12% faster, some interactions over 2.5× faster, memory unchanged. Examples, not a promise — measure.

In the interview

> **30-second answer:** "React Compiler is a build-time Babel plugin, stable since v1.0 in October 2025, that automatically memoizes components and Hooks — values, callbacks and JSX — from static analysis of their dependencies. It relies on the Rules of React and skips components that break them. With it, manual `memo` / `useMemo` / `useCallback` become mostly escape hatches. It doesn't solve slow algorithms, big DOM or bundle size."

- *How do you check it's working?* — "Memo ✨" badge in DevTools; the playground shows transformed code.
- *A component misbehaves after compiling?* — Add `"use no memo"`, find the Rules-of-React violation (lint helps), fix, remove the directive.

10.8 Code splitting with `lazy` and `<Suspense>`

What it is

By default the bundler emits one JS file the browser must download, **parse** and run before the first screen works — including a chart library for a page the user may never open. **Code splitting** breaks it into chunks loaded **on demand**. The building block is the **dynamic import** `import('./chart.jsx')`, which returns a **Promise** of the **module**; bundlers (Vite, webpack, esbuild, Rollup) emit each as a separate file. `lazy` turns that Promise into a component:

```jsx
import { lazy, Suspense } from 'react';
const HeavyChart = lazy(() => import('./heavy_chart.jsx'));   // at MODULE level

<Suspense fallback={<p>Loading chart...</p>}>
  <HeavyChart points={data} />
</Suspense>
```

How it works

1. `lazy(load)` returns a placeholder component; nothing downloads yet.
2. On first render React calls `load()`; the Promise is pending, so the component **suspends** and the **nearest** `<Suspense>` shows its `fallback`.
3. On resolve, React renders the module's `default` **export** in its place and caches it — `load` runs once.
4. On reject (network failure), the error is thrown during render and caught by the nearest **error boundary** (a component that catches children's render errors and shows fallback UI).

Seeing it — real chunk files

esbuild with code splitting (`--splitting --format=esm --minify`; a large data array lives in `chart.jsx`):

```jsx
// main.jsx
import { lazy } from 'react';
export const HeavyChart = lazy(() => import('./chart.jsx'));
export const Settings = lazy(() => import('./settings.jsx'));
```

Output (sizes/hashes depend on input):

```
  dist/chart-GX2X5FGR.js     22.9kb
  dist/main.js                153b
  dist/settings-7FL34NKO.js   114b
```

`dist/main.js` contains `import("./chart-GX2X5FGR.js")` — the 22.9 kB chunk is fetched only when `HeavyChart` first renders. The **hash** (content fingerprint) changes with content, so chunks can be cached forever.

Seeing it — loading states and failures

`code/06_lazy_suspense.jsx` (chunk load delayed 100 ms to imitate the network):

```
// 1. Declare lazy components at MODULE level (never inside a component).
const HeavyChart = lazy(() => delay(100).then(() => import('./heavy_chart.jsx')));
// lazy() needs a default export; for a named export, map it to { default }.
const Legend = lazy(() => import('./heavy_chart.jsx').then(m => ({ default: m.Legend })));
// A chunk that fails to load (e.g. a deploy replaced the file, or the user went offline).
const Broken = lazy(() => delay(10).then(() => { throw new Error('Failed to fetch dynamically imported module'); }));

function Dashboard() {
  const [tab, setTab] = useState('summary');
  return (
    <main>
      {/* buttons: chart tab, broken tab */}
      {tab === 'summary' && <p>summary text</p>}
      <Suspense fallback={<p>Loading chart...</p>}>
        {tab === 'chart' && <><HeavyChart points={[1, 2, 3]} /><Legend /></>}
      </Suspense>
      <ChunkErrorBoundary>   {/* class with getDerivedStateFromError, renders "Could not load: …" */}
        <Suspense fallback={<p>Loading...</p>}>{tab === 'broken' && <Broken />}</Suspense>
      </ChunkErrorBoundary>
    </main>
  );
}
```

Output (`heavy_chart.jsx` prints when evaluated):

```
first render             -> summary text
clicked chart tab        -> Loading chart...
    [module] heavy_chart.jsx evaluated (its code was just loaded)
after the chunk loaded   -> chart with 3 points | legend
broken tab               -> Could not load: Failed to fetch dynamically imported module
```

The module didn't run until the tab opened; one boundary covered both lazy components, so one loading message.

Where to split

Split point	Why
Routes / pages	Users view one page at a time; React Router's route `lazy`, frameworks split per route automatically (Chapter 9, Chapter 13)
Heavy, rarely used widgets	Rich-text editors, charts, maps, PDF viewers, admin panels
Below-the-fold (not visible until the user scrolls) **or behind-a-click UI**	Modals, settings drawers, "advanced" tabs
Not worth it	Small components; anything on the first screen (adds a request and a spinner)

Making it feel fast

- **Preload on intent** — calling the same `import()` early is enough; the module system caches it:

  ```
  const loadChart = () => import('./heavy_chart.jsx');
  const HeavyChart = lazy(loadChart);
  <button onMouseEnter={loadChart} onFocus={loadChart} onClick={() => setTab('chart')}>Chart</button>
  ```

- **Place boundaries deliberately**: one around the page blanks everything; one per widget keeps the rest usable. Skeletons avoid layout jumps.
- **Don't hide visible content**: wrap the update in `startTransition` so React keeps the old tab until the new one is ready (Chapter 7).
- **Handle chunk failures**: after a new deployment old chunk files may be gone — an error boundary with "Reload" is the usual fix.

Common mistakes

- **`lazy` inside a component** — a new component type each render, so it remounts and loses state. Declare at module level.
- **Named export without mapping** — `lazy` needs `default`.
- **No `<Suspense>` above a lazy component**.
- **Splitting everything** — many tiny chunks mean many requests and waterfalls.

How to find what to split

A bundle analyser (`rollup-plugin-visualizer`, `webpack-bundle-analyzer`, `source-map-explorer`) shows what takes space; Chrome DevTools' **Coverage** tab shows downloaded JS that never ran.

In the interview

> **30-second answer:** "Code splitting moves code not needed immediately into chunks loaded on demand. `const Page = lazy(() => import('./Page'))` at module level, rendered inside `<Suspense fallback>`; it suspends until the chunk loads. Split at routes and heavy widgets, preload on hover, catch failed loads with an error boundary, and use `startTransition` so navigation doesn't flash a spinner over existing content."

- *Why must the module have a default export?* — `lazy` reads the resolved module's `default`; map named exports to `{ default }`.

10.9 List virtualization

What it is

Virtualization (**windowing**) renders only the rows visible in a scrollable area (plus a few extra); rows leaving the window are removed and entering rows added. A tall spacer keeps the scrollbar correct so the list *looks* complete.

Why it matters

Every row costs DOM nodes, memory, style and layout; 10,000 rows mean tens of thousands of nodes and slow rendering, scrolling, even typing. Memoization can't help — the problem is node count, not re-renders.

How it works (fixed-height rows)

1. Inputs: `rowHeight`, **viewport** `height` (the visible scrolling box), current `scrollTop`.
2. First visible row = `floor(scrollTop / rowHeight)`; visible count = `ceil(height / rowHeight)`.
3. Add **overscan** rows above/below so fast scrolling shows no gaps.
4. Render a spacer of `items.length × rowHeight` px with only in-range rows, **absolutely positioned** at `top = index × rowHeight`.
5. Each `scroll` event stores `scrollTop` → re-render → new slice.

`code/virtual_list.jsx`:

```jsx
export function VirtualList({ items, rowHeight, height, overscan = 3, renderRow }) {
  const [scrollTop, setScrollTop] = useState(0);
  const first = Math.max(0, Math.floor(scrollTop / rowHeight) - overscan);
  const visibleCount = Math.ceil(height / rowHeight) + 2 * overscan;
  const last = Math.min(items.length - 1, first + visibleCount - 1);
  const rows = [];
  for (let i = first; i <= last; i++) {
    rows.push(
      <div key={i} role="listitem" className="row"
           style={{ position: 'absolute', top: i * rowHeight, height: rowHeight, left: 0, right: 0 }}>
        {renderRow(items[i], i)}
      </div>
    );
  }
  return (
    <div role="list" aria-label="virtual list" className="viewport"
         style={{ height, overflowY: 'auto', position: 'relative' }}
         onScroll={e => setScrollTop(e.currentTarget.scrollTop)}>
      <div style={{ height: items.length * rowHeight, position: 'relative' }}>{rows}</div>
    </div>
  );
}
```

`code/07_virtualization.jsx` renders 10,000 customers both ways, then simulates scrolling (jsdom has no layout, so the test sets `scrollTop` and fires `scroll`):

```
naive list    : 10000 row nodes, 10006 elements in document
virtual list  : 16 row nodes, 23 elements in document
  at top          -> rows Customer #0 .. Customer #15
  scrollTop=3000   -> rows Customer #97 .. Customer #112  (16 in DOM)
  scrollTop=150000 -> rows Customer #4997 .. Customer #5012  (16 in DOM)
  scrollTop=299700 -> rows Customer #9987 .. Customer #9999  (13 in DOM)

mount time (varies per run): naive 130 ms, virtual 6 ms
```

At `scrollTop = 3000`: 3000 / 30 = 100, minus overscan 3 = 97. The window is always 300 / 30 = 10 + 2 × 3 = 16 rows, however long the list; at the end it shrinks to 13 (no rows after #9999).

In Chrome (`code/virtual.html`, scrolled half-way; rows 32 px, viewport 320 px):

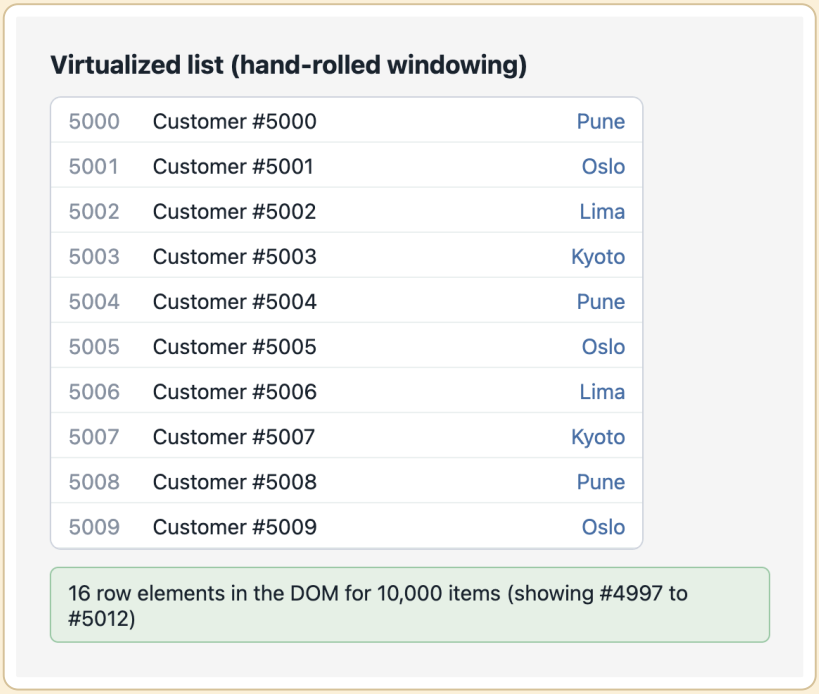


Figure 10.1 — The hand-rolled `VirtualList` in Chrome. Ten rows are visible; only sixteen exist in the DOM.

Use a library in real projects

Production needs **variable/unknown row heights** (measured after render), grids, sticky headers, stable scroll when prepending (chat), infinite loading:

Library	Notes
TanStack Virtual	Headless (logic only — you write the markup); lists, grids, variable sizes
react-window	Small and simple; fixed/variable lists and grids
react-virtuoso	Automatic variable heights, grouped lists, chat-style reverse scrolling

CSS-only alternative for moderately long pages: `content-visibility: auto` lets the browser skip layout/paint for off-screen sections while keeping them in the DOM (find-in-page still works) — cuts rendering cost, not React component count.

Trade-offs

	Plain list	Virtualized list
DOM size	grows with data	constant (~visible rows)
Browser find (Ctrl+F)	finds any row	only rendered rows
Accessibility	screen readers see all rows	need `aria-setsize` / `aria-posinset` ("item 5 of 10,000") on rows
Complexity	none	scroll math, measuring, focus management
Good for	< a few hundred rows	thousands of rows, feeds, logs, tables

Don't virtualize short lists (pagination/"load more" is often better UX) or content that must be printable/fully searchable.

Common mistakes

- Index `key`s when items are inserted/removed — rows show wrong state; use stable IDs (the demo list never changes, so index keys are safe there).
- No overscan → blank flashes when scrolling fast.
- No fixed viewport height → it never scrolls.

In the interview

> **30-second answer:** "Virtualization renders only visible rows plus overscan inside a spacer the height of the full list, recomputing the slice on scroll, so the DOM stays small. Fixed heights need only `floor(scrollTop / rowHeight)`; variable heights need measuring, so in production I'd use TanStack Virtual or react-window. Downsides: browser find and screen-reader counts need care."

- *Why doesn't `memo` fix a 10,000-row list?* — The cost is DOM node count and initial render, not re-renders.
- *Infinite scroll?* — Virtualize; when the last rendered index nears the end, fetch the next page (an `IntersectionObserver` — reports when an element becomes visible — on a **sentinel** row at the end, or the library's range callback).

10.10 Common performance bugs

What it is

Four mistakes cause a large share of slow screens. `code/08_perf_bugs.jsx` measures each before/after, counting **renders** (function ran) and **mounts** (brand-new instance: new DOM, state reset, effects re-run — far costlier).

```jsx
function Tracked({ label }) { renders++; useEffect(() => { mounts++; }, []); return <span>{label}</span>; }

// BUG 1: a component defined INSIDE another component
function Bad1() {
  const [n, setN] = useState(0);
  function Inner() { return <Tracked label="inner" />; }        // new type every render!
  return <><button onClick={() => setN(n + 1)}>bump</button><Inner /></>;
}
// BUG 2: unstable keys
<ul>{ITEMS.map(it => <li key={Math.random()}><Tracked label={it} /></li>)}</ul>
// BUG 3: a new context object every render (Consumer is memo'd and reads Ctx)
const fresh = { user };                                  // new object each render
const cached = useMemo(() => ({ user }), [user]);        // same object until user changes
// BUG 4: state kept too high
function AppHigh() {
  const [q, setQ] = useState('');
  return <><input aria-label="q" value={q} onChange={e => setQ(e.target.value)} /><Expensive /><Expensive /></>;
}
const AppLow = () => <><SearchBox /><Expensive /><Expensive /></>;   // SearchBox owns q: state moved DOWN
```

Output:

```
1. component defined inside a component (3 parent re-renders)
  bad : Inner declared in render           renders=3, mounts=3
  good: Inner declared at top level        renders=3, mounts=0
2. unstable keys on a 3-item list (3 parent re-renders)
  bad : key={Math.random()}                renders=9, mounts=9
  good: key={item.id}                      renders=9, mounts=0
3. new context object each render (3 unrelated provider re-renders)
  bad : value={{ user }}                   renders=3, mounts=0
  good: value={useMemo(...)}               renders=0, mounts=0
4. input state kept too high (typing 5 characters)
  bad : query state in <App>               renders=10, mounts=0
  good: query state in <SearchBox>         renders=0, mounts=0
```

The four bugs explained

#	Bug	Why it's slow	Fix
1	Component declared inside another	Each render makes a **new component type** (a different function): old subtree unmounted, new one mounted — DOM rebuilt, state lost, effects re-run	Declare components at module top level; pass data as props
2	`key={Math.random()}` / changing keys	Every item looks new → unmount + mount every render	Stable IDs from the data (Chapter 2)
3	`value={{ user, setUser }}` on a provider	New object each render → **every** consumer re-renders; `memo` can't stop it	`useMemo` the value, split contexts (state vs setters), or a store with selectors
4	State owned higher than needed	Each keystroke re-renders everything under the owner	**Colocate** state in the smallest component that uses it; or pass expensive parts as `children`

More perf traps worth knowing

- **Effects that set state every render** (render → effect → setState → render) — usually a derived value to compute during render (Chapter 5).
- **Fetch waterfalls** (parent fetches, then child fetches…) — fetch in parallel, in route loaders or with TanStack Query (Chapter 9).
- **Huge images, heavy libraries** (a whole date library for one format) — modern image formats, `Intl.DateTimeFormat`, **tree-shakable** imports (unused code trimmed by the bundler).
- **Expensive event handlers** block the next paint — split the work, `startTransition`, or a **Web Worker** (background JS thread).
- **Development build in production** — deploy `vite build` / `next build` output.

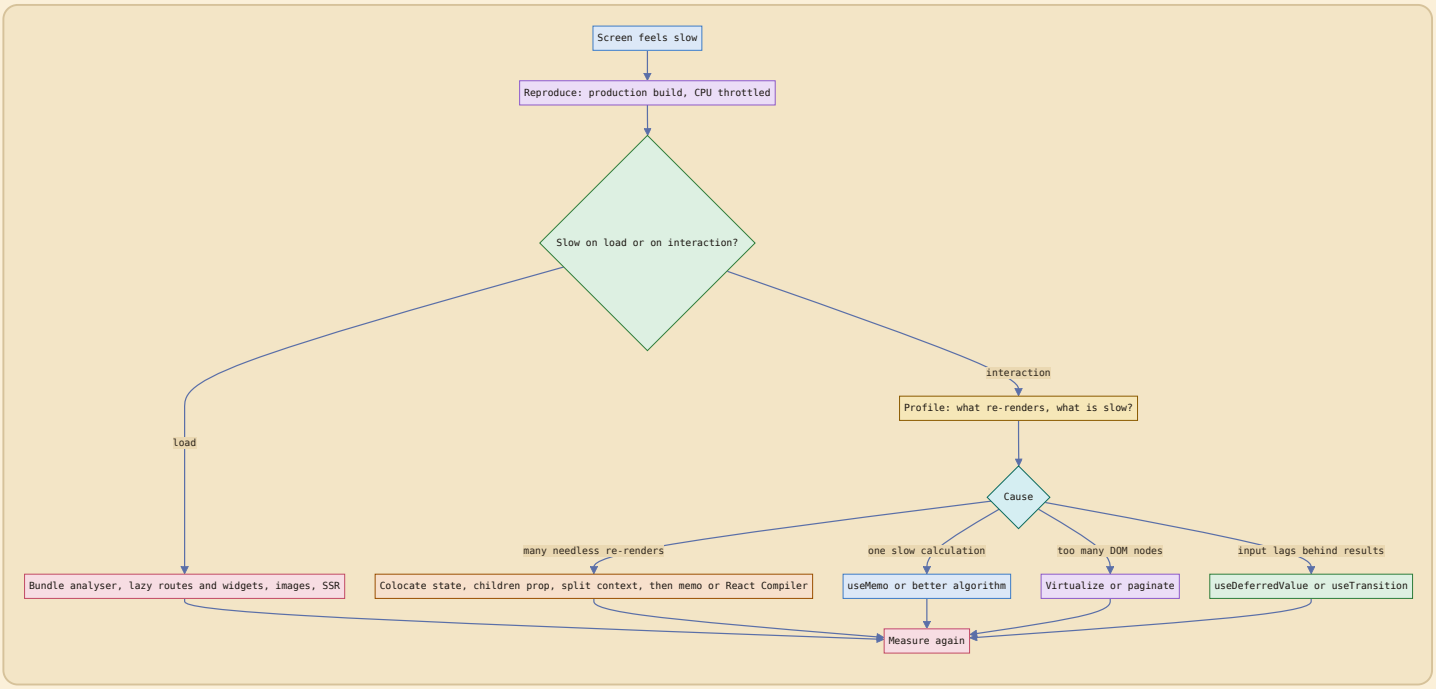
In the interview

> **30-second answer:** "Components defined inside components (remount every render), unstable keys, context values recreated each render, state kept too high, and effect-driven render loops. Fix structure first — colocate state, pass JSX as children, stable keys, memoized or split context — then `memo` / `useMemo`."

10.11 A performance checklist

Work through in order; stop when it's fast enough:

1. **Reproduce with a production build**, CPU throttled, realistic data.
2. **Find the slow interaction** (Performance panel with React tracks, or the DevTools Profiler): initial load or a specific interaction?
3. **Slow load?** Bundle analyser, `lazy` routes/heavy widgets (§10.8), images, server rendering (Chapter 13).
4. **Slow interaction?** "Highlight updates" and see what re-renders:
   - Too much re-renders → colocate state, `children`, split context (§10.10); then `memo` + stable props (§10.4–10.5) or React Compiler (§10.7).
   - One component slow on its own → `useMemo` (§10.6) or a better algorithm.
   - Too many DOM nodes → virtualize (§10.9) or paginate.
   - Typing lags while results render → `useDeferredValue` / `useTransition`.
5. **Measure again**; keep the change only if it measurably helped.



In the interview — putting it together

"How would you speed up a slow React page?" — "Measure first: production build, CPU throttling, the Profiler to find the slowest commit and why things rendered. For load time, check bundle size and split routes and heavy widgets with `lazy` + `Suspense`. For interactions, reduce re-renders structurally (colocate state, pass children, stable context values), then `memo` with stable props or React Compiler. Virtualize long lists, memoize expensive calculations, mark non-urgent updates as transitions — and measure again."

Self-check (whole chapter)

1. Render vs commit? → Render calls components to compute UI; commit applies differences to the DOM.
2. Why didn't `ChildAsChildren` re-render (§10.2 B)? → Same element object, created by a component that didn't re-render.
3. Which components re-render on a context change? → Those reading it (plus what they render).
4. Why did V2 render as much as V1? → New function/object props fail the shallow comparison.

5. What does `baseDuration` tell you? → Estimated full re-render time of the subtree — compare with `actualDuration`.
6. DOM rows for a 300 px viewport, 30 px rows, overscan 3? → 10 + 6 = 16.
7. Why does declaring `Inner` inside `Bad1` reset its state? → Each render yields a different function = different type, so React mounts a fresh instance.
8. "Colocating state"? → Keeping state in the lowest component that needs it.
9. A memoized consumer re-renders on unrelated provider state — why? → The provider's `value` is a new object each render; context changes bypass `memo`.

Chapter 10 TL;DR

Idea	Remember
Render cycle	Trigger → render (call components) → commit (DOM diff) → effects; a re-render is not a DOM update
Re-render causes	Own state, parent render, context, external store — props aren't compared without `memo`
Measure	DevTools highlight + Profiler ("why did this render?"), Performance tracks (19.2+), `<Profiler onRender>` (`actualDuration` vs `baseDuration`); production profiling needs the profiling build
`memo`	Skip re-render on shallow-equal props; useless with new objects/functions as props
`useCallback` / `useMemo`	Stable references for `memo` children and effect deps; cache calculations ≥ ~1 ms
React Compiler	v1.0 (Oct 2025); build-time auto-memoization; follow the Rules of React; `"use no memo"` escape hatch
Code splitting	`lazy(() => import())` at module level + `<Suspense>` + error boundary; split by route/heavy widget; preload on intent
Virtualization	Visible rows + overscan in a full-height spacer; TanStack Virtual / react-window in production
Perf bugs	Nested component definitions, unstable keys, new context objects, state too high, effect loops
Order of fixes	Structure first (colocate, children, keys) → memoization → specialised tools

Next: Chapter 11 — TypeScript with React.